Machine Learning Reference

A guide from first principles to research: theory, code, experiments, and projects

Personal curriculum

2026-10-25

Table of contents

1 Machine Learning Reference 37

2 Machine Learning Reference 38
- 2.1 The progression . . . 38
- 2.2 Four layers . . . 40
- 2.3 Formats . . . 40

3 How to study this reference 41

4 How to study this reference 42
- 4.1 Who it is for . . . 42
- 4.2 How a session works . . . 42
- 4.3 What “comprehensive” means here . . . 43
- 4.4 Suggested order . . . 43
- 4.5 What this book will not do . . . 44

5 Glossary 45

6 Glossary 46

7 Upstream books 48

8 Upstream books 49
- 8.1 Where they sit on the path . . . 49

I Volume 01. Mathematical language 51

9 Volume 01. Mathematical language 52

10 Volume 01. Mathematical language 53
- 10.1 How to use the four layers . . . 53
- 10.2 Depth tracks . . . 53
- 10.3 Chapters . . . 54
- 10.4 Lab note . . . 54

11 How to use this reference 55

12 How to use this reference 56
- 12.1 Reference card . . . 56

12.2 Theory . . . 56
12.3 Derivation . . . 57
12.4 Code . . . 57
12.5 Experiment . . . 58
12.6 Project . . . 58
12.7 Depth tracks . . . 58
12.8 Failure modes . . . 58
12.9 See also . . . 59

13 Sets, functions, and notation **60**

14 Sets, functions, and notation **61**
14.1 Reference card . . . 61
14.2 Theory . . . 61
14.3 Derivation . . . 62
14.4 Code . . . 62
14.5 Experiment . . . 62
14.6 Project . . . 63
14.7 Depth tracks . . . 63
14.8 Failure modes . . . 63
14.9 See also . . . 63

15 Linear algebra for models **64**

16 Linear algebra for models **65**
16.1 Reference card . . . 65
16.2 Theory . . . 65
16.3 Derivation . . . 66
16.4 Code . . . 66
16.5 Experiment . . . 66
16.6 Project . . . 67
16.7 Depth tracks . . . 67
16.8 Failure modes . . . 67
16.9 See also . . . 67

17 Matrix calculus **68**

18 Matrix calculus **69**
18.1 Reference card . . . 69
18.2 Theory . . . 69
18.3 Derivation . . . 70
18.4 Code . . . 70
18.5 Experiment . . . 70
18.6 Project . . . 71
18.7 Depth tracks . . . 71
18.8 Failure modes . . . 71
18.9 See also . . . 71

19 Numerical linear algebra 72

20 Numerical linear algebra 73
20.1 Reference card . . . 73
20.2 Theory . . . 73
20.3 Derivation . . . 74
20.4 Code . . . 74
20.5 Experiment . . . 74
20.6 Project . . . 75
20.7 Depth tracks . . . 75
20.8 Failure modes . . . 75
20.9 See also . . . 75

21 Project: least squares from the axioms 76

22 Project: least squares from the axioms 77
22.1 Reference card . . . 77
22.2 Theory . . . 77
22.3 Derivation . . . 78
22.4 Code . . . 78
22.5 Experiment . . . 78
22.6 Project . . . 79
22.7 Depth tracks . . . 79
22.8 Failure modes . . . 79
22.9 See also . . . 79

II Volume 02. Calculus and optimization 80

23 Volume 02. Calculus and optimization 81

24 Volume 02. Calculus and optimization 82
24.1 How to use the four layers . . . 82
24.2 Depth tracks . . . 82
24.3 Chapters . . . 83
24.4 Lab note . . . 83

25 Derivatives, gradients, and Jacobians 84

26 Derivatives, gradients, and Jacobians 85
26.1 Reference card . . . 85
26.2 Theory . . . 85
26.3 Derivation . . . 86
26.4 Code . . . 86
26.5 Experiment . . . 86
26.6 Project . . . 87
26.7 Depth tracks . . . 87

26.8 Failure modes . . . 87
26.9 See also . . . 87

27 Taylor expansions and local models **88**

28 Taylor expansions and local models **89**
28.1 Reference card . . . 89
28.2 Theory . . . 89
28.3 Derivation . . . 90
28.4 Code . . . 90
28.5 Experiment . . . 90
28.6 Project . . . 91
28.7 Depth tracks . . . 91
28.8 Failure modes . . . 91
28.9 See also . . . 91

29 Convex sets and convex functions **92**

30 Convex sets and convex functions **93**
30.1 Reference card . . . 93
30.2 Theory . . . 93
30.3 Derivation . . . 94
30.4 Code . . . 94
30.5 Experiment . . . 94
30.6 Project . . . 95
30.7 Depth tracks . . . 95
30.8 Failure modes . . . 95
30.9 See also . . . 95

31 Constraints and optimality **96**

32 Constraints and optimality **97**
32.1 Reference card . . . 97
32.2 Theory . . . 97
32.3 Derivation . . . 98
32.4 Code . . . 98
32.5 Experiment . . . 98
32.6 Project . . . 98
32.7 Depth tracks . . . 99
32.8 Failure modes . . . 99
32.9 See also . . . 99

33 Stochastic gradients **100**

34 Stochastic gradients **101**
34.1 Reference card . . . 101
34.2 Theory . . . 101

34.3 Derivation . . . 102
34.4 Code . . . 102
34.5 Experiment . . . 102
34.6 Project . . . 103
34.7 Depth tracks . . . 103
34.8 Failure modes . . . 103
34.9 See also . . . 103

35 Project: a gradient-checking harness 104

36 Project: a gradient-checking harness 105
36.1 Reference card . . . 105
36.2 Theory . . . 105
36.3 Derivation . . . 106
36.4 Code . . . 106
36.5 Experiment . . . 106
36.6 Project . . . 107
36.7 Depth tracks . . . 107
36.8 Failure modes . . . 107
36.9 See also . . . 107

III Volume 03. Computation for experiments 108

37 Volume 03. Computation for experiments 109

38 Volume 03. Computation for experiments 110
38.1 How to use the four layers . . . 110
38.2 Depth tracks . . . 110
38.3 Chapters . . . 111
38.4 Lab note . . . 111

39 Python for reference implementations 112

40 Python for reference implementations 113
40.1 Reference card . . . 113
40.2 Theory . . . 113
40.3 Derivation . . . 114
40.4 Code . . . 114
40.5 Experiment . . . 114
40.6 Project . . . 115
40.7 Depth tracks . . . 115
40.8 Failure modes . . . 115
40.9 See also . . . 115

41 NumPy as a mathematical instrument 116

42 NumPy as a mathematical instrument **117**
42.1 Reference card . . . 117
42.2 Theory . . . 117
42.3 Derivation . . . 118
42.4 Code . . . 118
42.5 Experiment . . . 118
42.6 Project . . . 118
42.7 Depth tracks . . . 119
42.8 Failure modes . . . 119
42.9 See also . . . 119

43 Data contracts **120**

44 Data contracts **121**
44.1 Reference card . . . 121
44.2 Theory . . . 121
44.3 Derivation . . . 122
44.4 Code . . . 122
44.5 Experiment . . . 122
44.6 Project . . . 123
44.7 Depth tracks . . . 123
44.8 Failure modes . . . 123
44.9 See also . . . 123

45 Figures, tables, and claims **124**

46 Figures, tables, and claims **125**
46.1 Reference card . . . 125
46.2 Theory . . . 125
46.3 Derivation . . . 126
46.4 Code . . . 126
46.5 Experiment . . . 126
46.6 Project . . . 126
46.7 Depth tracks . . . 127
46.8 Failure modes . . . 127
46.9 See also . . . 127

47 Reproducible experiment folders **128**

48 Reproducible experiment folders **129**
48.1 Reference card . . . 129
48.2 Theory . . . 129
48.3 Derivation . . . 130
48.4 Code . . . 130
48.5 Experiment . . . 130
48.6 Project . . . 130
48.7 Depth tracks . . . 131

48.8 Failure modes . . . 131
48.9 See also . . . 131

49 Project: audit a dataset **132**

50 Project: audit a dataset **133**
50.1 Reference card . . . 133
50.2 Theory . . . 133
50.3 Derivation . . . 134
50.4 Code . . . 134
50.5 Experiment . . . 134
50.6 Project . . . 134
50.7 Depth tracks . . . 135
50.8 Failure modes . . . 135
50.9 See also . . . 135

IV Volume 04. Probability **136**

51 Volume 04. Probability **137**

52 Volume 04. Probability **138**
52.1 How to use the four layers . . . 138
52.2 Depth tracks . . . 138
52.3 Chapters . . . 139
52.4 Lab note . . . 139

53 Probability, events, and random variables **140**

54 Probability, events, and random variables **141**
54.1 Reference card . . . 141
54.2 Theory . . . 141
54.3 Derivation . . . 142
54.4 Code . . . 142
54.5 Experiment . . . 142
54.6 Project . . . 143
54.7 Depth tracks . . . 143
54.8 Failure modes . . . 143
54.9 See also . . . 143

55 Distributions you will actually use **144**

56 Distributions you will actually use **145**
56.1 Reference card . . . 145
56.2 Theory . . . 145
56.3 Derivation . . . 146
56.4 Code . . . 146

56.5 Experiment . . . 146
56.6 Project . . . 147
56.7 Depth tracks . . . 147
56.8 Failure modes . . . 147
56.9 See also . . . 147

57 Expectation, variance, and concentration **148**

58 Expectation, variance, and concentration **149**
58.1 Reference card . . . 149
58.2 Theory . . . 149
58.3 Derivation . . . 150
58.4 Code . . . 150
58.5 Experiment . . . 150
58.6 Project . . . 150
58.7 Depth tracks . . . 151
58.8 Failure modes . . . 151
58.9 See also . . . 151

59 Multivariate Gaussians **152**

60 Multivariate Gaussians **153**
60.1 Reference card . . . 153
60.2 Theory . . . 153
60.3 Derivation . . . 154
60.4 Code . . . 154
60.5 Experiment . . . 154
60.6 Project . . . 155
60.7 Depth tracks . . . 155
60.8 Failure modes . . . 155
60.9 See also . . . 155

61 Conditional independence **156**

62 Conditional independence **157**
62.1 Reference card . . . 157
62.2 Theory . . . 157
62.3 Derivation . . . 158
62.4 Code . . . 158
62.5 Experiment . . . 158
62.6 Project . . . 159
62.7 Depth tracks . . . 159
62.8 Failure modes . . . 159
62.9 See also . . . 159

63 Project: fit a law and show the misfit **160**

64 Project: fit a law and show the misfit **161**
64.1 Reference card . . . 161
64.2 Theory . . . 161
64.3 Derivation . . . 162
64.4 Code . . . 162
64.5 Experiment . . . 162
64.6 Project . . . 163
64.7 Depth tracks . . . 163
64.8 Failure modes . . . 163
64.9 See also . . . 163

V Volume 05. Statistical inference **164**

65 Volume 05. Statistical inference **165**

66 Volume 05. Statistical inference **166**
66.1 How to use the four layers . . . 166
66.2 Depth tracks . . . 166
66.3 Chapters . . . 167
66.4 Lab note . . . 167

67 Estimators and sampling distributions **168**

68 Estimators and sampling distributions **169**
68.1 Reference card . . . 169
68.2 Theory . . . 169
68.3 Derivation . . . 170
68.4 Code . . . 170
68.5 Experiment . . . 170
68.6 Project . . . 171
68.7 Depth tracks . . . 171
68.8 Failure modes . . . 171
68.9 See also . . . 171

69 Maximum likelihood **172**

70 Maximum likelihood **173**
70.1 Reference card . . . 173
70.2 Theory . . . 173
70.3 Derivation . . . 174
70.4 Code . . . 174
70.5 Experiment . . . 174
70.6 Project . . . 175
70.7 Depth tracks . . . 175
70.8 Failure modes . . . 175
70.9 See also . . . 175

71 Tests and intervals **176**

72 Tests and intervals **177**
72.1 Reference card . . . 177
72.2 Theory . . . 177
72.3 Derivation . . . 178
72.4 Code . . . 178
72.5 Experiment . . . 178
72.6 Project . . . 179
72.7 Depth tracks . . . 179
72.8 Failure modes . . . 179
72.9 See also . . . 180

73 Bayesian updating **181**

74 Bayesian updating **182**
74.1 Reference card . . . 182
74.2 Theory . . . 182
74.3 Derivation . . . 183
74.4 Code . . . 183
74.5 Experiment . . . 183
74.6 Project . . . 183
74.7 Depth tracks . . . 184
74.8 Failure modes . . . 184
74.9 See also . . . 184

75 Experimental design and power **185**

76 Experimental design and power **186**
76.1 Reference card . . . 186
76.2 Theory . . . 186
76.3 Derivation . . . 187
76.4 Code . . . 187
76.5 Experiment . . . 187
76.6 Project . . . 188
76.7 Depth tracks . . . 188
76.8 Failure modes . . . 188
76.9 See also . . . 188

77 Project: analyze a comparison without fooling yourself **189**

78 Project: analyze a comparison without fooling yourself **190**
78.1 Reference card . . . 190
78.2 Theory . . . 190
78.3 Derivation . . . 191
78.4 Code . . . 191
78.5 Experiment . . . 191

78.6 Project . . . 191
78.7 Depth tracks . . . 192
78.8 Failure modes . . . 192
78.9 See also . . . 192

VI Volume 06. Supervised learning **193**

79 Volume 06. Supervised learning **194**

80 Volume 06. Supervised learning **195**
80.1 How to use the four layers . . . 195
80.2 Depth tracks . . . 195
80.3 Chapters . . . 196
80.4 Lab note . . . 196

81 The supervised problem **197**

82 The supervised problem **198**
82.1 Reference card . . . 198
82.2 Theory . . . 198
82.3 Derivation . . . 199
82.4 Code . . . 199
82.5 Experiment . . . 199
82.6 Project . . . 200
82.7 Depth tracks . . . 200
82.8 Failure modes . . . 200
82.9 See also . . . 200

83 Linear and logistic regression **201**

84 Linear and logistic regression **202**
84.1 Reference card . . . 202
84.2 Theory . . . 202
84.3 Derivation . . . 203
84.4 Code . . . 203
84.5 Experiment . . . 203
84.6 Project . . . 204
84.7 Depth tracks . . . 204
84.8 Failure modes . . . 204
84.9 See also . . . 204

85 Trees and ensembles **205**

86 Trees and ensembles **206**
86.1 Reference card . . . 206
86.2 Theory . . . 206

86.3 Derivation . . . 207
86.4 Code . . . 207
86.5 Experiment . . . 208
86.6 Project . . . 208
86.7 Depth tracks . . . 208
86.8 Failure modes . . . 208
86.9 See also . . . 209

87 Kernel methods **210**

88 Kernel methods **211**
88.1 Reference card . . . 211
88.2 Theory . . . 211
88.3 Derivation . . . 212
88.4 Code . . . 212
88.5 Experiment . . . 212
88.6 Project . . . 213
88.7 Depth tracks . . . 213
88.8 Failure modes . . . 213
88.9 See also . . . 213

89 Calibration and proper scoring **214**

90 Calibration and proper scoring **215**
90.1 Reference card . . . 215
90.2 Theory . . . 215
90.3 Derivation . . . 216
90.4 Code . . . 216
90.5 Experiment . . . 217
90.6 Project . . . 217
90.7 Depth tracks . . . 217
90.8 Failure modes . . . 217
90.9 See also . . . 218

91 Project: a baseline bake-off **219**

92 Project: a baseline bake-off **220**
92.1 Reference card . . . 220
92.2 Theory . . . 220
92.3 Derivation . . . 221
92.4 Code . . . 221
92.5 Experiment . . . 221
92.6 Project . . . 221
92.7 Depth tracks . . . 222
92.8 Failure modes . . . 222
92.9 See also . . . 222

VII Volume 07. Evaluation and model selection 223

93 Volume 07. Evaluation and model selection 224

94 Volume 07. Evaluation and model selection 225
94.1 How to use the four layers . . . 225
94.2 Depth tracks . . . 225
94.3 Chapters . . . 226
94.4 Lab note . . . 226

95 Train, validation, test, and leakage 227

96 Train, validation, test, and leakage 228
96.1 Reference card . . . 228
96.2 Theory . . . 228
96.3 Derivation . . . 229
96.4 Code . . . 229
96.5 Experiment . . . 229
96.6 Project . . . 229
96.7 Depth tracks . . . 230
96.8 Failure modes . . . 230
96.9 See also . . . 230

97 Metrics by decision 231

98 Metrics by decision 232
98.1 Reference card . . . 232
98.2 Theory . . . 232
98.3 Derivation . . . 233
98.4 Code . . . 233
98.5 Experiment . . . 233
98.6 Project . . . 234
98.7 Depth tracks . . . 234
98.8 Failure modes . . . 234
98.9 See also . . . 234

99 Error analysis 235

100 Error analysis 236
100.1 Reference card . . . 236
100.2 Theory . . . 236
100.3 Derivation . . . 237
100.4 Code . . . 237
100.5 Experiment . . . 237
100.6 Project . . . 238
100.7 Depth tracks . . . 238
100.8 Failure modes . . . 238

100.9See also . . . 238

101 Cross-validation **239**

102 Cross-validation **240**
102.1 Reference card . . . 240
102.2 Theory . . . 240
102.3 Derivation . . . 241
102.4 Code . . . 241
102.5 Experiment . . . 241
102.6 Project . . . 242
102.7 Depth tracks . . . 242
102.8 Failure modes . . . 242
102.9 See also . . . 242

103 Comparing models under uncertainty **243**

104 Comparing models under uncertainty **244**
104.1 Reference card . . . 244
104.2 Theory . . . 244
104.3 Derivation . . . 245
104.4 Code . . . 245
104.5 Experiment . . . 245
104.6 Project . . . 246
104.7 Depth tracks . . . 246
104.8 Failure modes . . . 246
104.9 See also . . . 246

105 Project: a model card and an error taxonomy **247**

106 Project: a model card and an error taxonomy **248**
106.1 Reference card . . . 248
106.2 Theory . . . 248
106.3 Derivation . . . 249
106.4 Code . . . 249
106.5 Experiment . . . 249
106.6 Project . . . 250
106.7 Depth tracks . . . 250
106.8 Failure modes . . . 250
106.9 See also . . . 250

VIII Volume 08. Unsupervised learning **251**

107 Volume 08. Unsupervised learning **252**

108 Volume 08. Unsupervised learning **253**
108.1 How to use the four layers . . . 253
108.2 Depth tracks . . . 253
108.3 Chapters . . . 254
108.4 Lab note . . . 254

109 Clustering **255**

110 Clustering **256**
110.1 Reference card . . . 256
110.2 Theory . . . 256
110.3 Derivation . . . 257
110.4 Code . . . 257
110.5 Experiment . . . 258
110.6 Project . . . 258
110.7 Depth tracks . . . 258
110.8 Failure modes . . . 258
110.9 See also . . . 259

111 Dimensionality reduction **260**

112 Dimensionality reduction **261**
112.1 Reference card . . . 261
112.2 Theory . . . 261
112.3 Derivation . . . 262
112.4 Code . . . 262
112.5 Experiment . . . 262
112.6 Project . . . 263
112.7 Depth tracks . . . 263
112.8 Failure modes . . . 263
112.9 See also . . . 263

113 Density estimation and anomalies **264**

114 Density estimation and anomalies **265**
114.1 Reference card . . . 265
114.2 Theory . . . 265
114.3 Derivation . . . 266
114.4 Code . . . 266
114.5 Experiment . . . 266
114.6 Project . . . 267
114.7 Depth tracks . . . 267
114.8 Failure modes . . . 267
114.9 See also . . . 267

115 Mixture models **268**

116 Mixture models **269**
116.1 Reference card . . . 269
116.2 Theory . . . 269
116.3 Derivation . . . 270
116.4 Code . . . 270
116.5 Experiment . . . 270
116.6 Project . . . 271
116.7 Depth tracks . . . 271
116.8 Failure modes . . . 271
116.9 See also . . . 271

117 Self-supervised representations **272**

118 Self-supervised representations **273**
118.1 Reference card . . . 273
118.2 Theory . . . 273
118.3 Derivation . . . 274
118.4 Code . . . 274
118.5 Experiment . . . 274
118.6 Project . . . 275
118.7 Depth tracks . . . 275
118.8 Failure modes . . . 275
118.9 See also . . . 275

119 Project: evaluate a representation **276**

120 Project: evaluate a representation **277**
120.1 Reference card . . . 277
120.2 Theory . . . 277
120.3 Derivation . . . 278
120.4 Code . . . 278
120.5 Experiment . . . 278
120.6 Project . . . 279
120.7 Depth tracks . . . 279
120.8 Failure modes . . . 279
120.9 See also . . . 279

IX Volume 09. Neural networks **280**

121 Volume 09. Neural networks **281**

122 Volume 09. Neural networks **282**
122.1 How to use the four layers . . . 282
122.2 Depth tracks . . . 282
122.3 Chapters . . . 283
122.4 Lab note . . . 283

123 Computation graphs **284**

124 Computation graphs **285**
124.1 Reference card . . . 285
124.2 Theory . . . 285
124.3 Derivation . . . 286
124.4 Code . . . 286
124.5 Experiment . . . 286
124.6 Project . . . 287
124.7 Depth tracks . . . 287
124.8 Failure modes . . . 287
124.9 See also . . . 287

125 Backpropagation **288**

126 Backpropagation **289**
126.1 Reference card . . . 289
126.2 Theory . . . 289
126.3 Derivation . . . 290
126.4 Code . . . 290
126.5 Experiment . . . 290
126.6 Project . . . 291
126.7 Depth tracks . . . 291
126.8 Failure modes . . . 291
126.9 See also . . . 291

127 Architectures and inductive bias **292**

128 Architectures and inductive bias **293**
128.1 Reference card . . . 293
128.2 Theory . . . 293
128.3 Derivation . . . 294
128.4 Code . . . 294
128.5 Experiment . . . 294
128.6 Project . . . 295
128.7 Depth tracks . . . 295
128.8 Failure modes . . . 295
128.9 See also . . . 295

129 Initialization and activations **296**

130 Initialization and activations **297**
130.1 Reference card . . . 297
130.2 Theory . . . 297
130.3 Derivation . . . 298
130.4 Code . . . 298
130.5 Experiment . . . 298

130.6Project . . . 298
130.7Depth tracks . . . 299
130.8Failure modes . . . 299
130.9See also . . . 299

131What universal approximation does not give you 300

132What universal approximation does not give you 301
132.1Reference card . . . 301
132.2Theory . . . 301
132.3Derivation . . . 302
132.4Code . . . 302
132.5Experiment . . . 302
132.6Project . . . 303
132.7Depth tracks . . . 303
132.8Failure modes . . . 303
132.9See also . . . 303

133Project: a network in NumPy 304

134Project: a network in NumPy 305
134.1Reference card . . . 305
134.2Theory . . . 305
134.3Derivation . . . 306
134.4Code . . . 306
134.5Experiment . . . 306
134.6Project . . . 306
134.7Depth tracks . . . 307
134.8Failure modes . . . 307
134.9See also . . . 307

X Volume 10. Training deep models 308

135Volume 10. Training deep models 309

136Volume 10. Training deep models 310
136.1How to use the four layers . . . 310
136.2Depth tracks . . . 310
136.3Chapters . . . 311
136.4Lab note . . . 311

137Losses for modern models 312

138Losses for modern models 313
138.1Reference card . . . 313
138.2Theory . . . 313

138.3Derivation . . . 314
138.4Code . . . 314
138.5Experiment . . . 314
138.6Project . . . 315
138.7Depth tracks . . . 315
138.8Failure modes . . . 315
138.9See also . . . 315

139Optimizers **316**

140Optimizers **317**
140.1Reference card . . . 317
140.2Theory . . . 317
140.3Derivation . . . 318
140.4Code . . . 318
140.5Experiment . . . 318
140.6Project . . . 319
140.7Depth tracks . . . 319
140.8Failure modes . . . 319
140.9See also . . . 319

141Schedules and normalization **320**

142Schedules and normalization **321**
142.1Reference card . . . 321
142.2Theory . . . 321
142.3Derivation . . . 322
142.4Code . . . 322
142.5Experiment . . . 322
142.6Project . . . 323
142.7Depth tracks . . . 323
142.8Failure modes . . . 323
142.9See also . . . 323

143Regularization and augmentation **324**

144Regularization and augmentation **325**
144.1Reference card . . . 325
144.2Theory . . . 325
144.3Derivation . . . 326
144.4Code . . . 326
144.5Experiment . . . 326
144.6Project . . . 327
144.7Depth tracks . . . 327
144.8Failure modes . . . 327
144.9See also . . . 327

145 Debugging a training run 328

146 Debugging a training run 329
146.1 Reference card . . . 329
146.2 Theory . . . 329
146.3 Derivation . . . 330
146.4 Code . . . 330
146.5 Experiment . . . 330
146.6 Project . . . 331
146.7 Depth tracks . . . 331
146.8 Failure modes . . . 331
146.9 See also . . . 331

147 Project: a controlled training ablation 332

148 Project: a controlled training ablation 333
148.1 Reference card . . . 333
148.2 Theory . . . 333
148.3 Derivation . . . 334
148.4 Code . . . 334
148.5 Experiment . . . 334
148.6 Project . . . 334
148.7 Depth tracks . . . 335
148.8 Failure modes . . . 335
148.9 See also . . . 335

XI Volume 11. Vision 336

149 Volume 11. Vision 337

150 Volume 11. Vision 338
150.1 How to use the four layers . . . 338
150.2 Depth tracks . . . 338
150.3 Chapters . . . 339
150.4 Lab note . . . 339

151 Images as tensors 340

152 Images as tensors 341
152.1 Reference card . . . 341
152.2 Theory . . . 341
152.3 Derivation . . . 342
152.4 Code . . . 342
152.5 Experiment . . . 342
152.6 Project . . . 343
152.7 Depth tracks . . . 343

152.8 Failure modes . . . 343
152.9 See also . . . 343

153 Convolution and equivariance **344**

154 Convolution and equivariance **345**
154.1 Reference card . . . 345
154.2 Theory . . . 345
154.3 Derivation . . . 346
154.4 Code . . . 346
154.5 Experiment . . . 346
154.6 Project . . . 347
154.7 Depth tracks . . . 347
154.8 Failure modes . . . 347
154.9 See also . . . 347

155 Detection and segmentation **348**

156 Detection and segmentation **349**
156.1 Reference card . . . 349
156.2 Theory . . . 349
156.3 Derivation . . . 350
156.4 Code . . . 350
156.5 Experiment . . . 350
156.6 Project . . . 351
156.7 Depth tracks . . . 351
156.8 Failure modes . . . 351
156.9 See also . . . 351

157 Modern vision architectures **352**

158 Modern vision architectures **353**
158.1 Reference card . . . 353
158.2 Theory . . . 353
158.3 Derivation . . . 354
158.4 Code . . . 354
158.5 Experiment . . . 354
158.6 Project . . . 355
158.7 Depth tracks . . . 355
158.8 Failure modes . . . 355
158.9 See also . . . 355

159 Evaluation for vision **356**

160 Evaluation for vision **357**
160.1 Reference card . . . 357
160.2 Theory . . . 357

160.3Derivation . . . 358
160.4Code . . . 358
160.5Experiment . . . 358
160.6Project . . . 358
160.7Depth tracks . . . 359
160.8Failure modes . . . 359
160.9See also . . . 359

161Project: a small vision study **360**

162Project: a small vision study **361**
162.1Reference card . . . 361
162.2Theory . . . 361
162.3Derivation . . . 362
162.4Code . . . 362
162.5Experiment . . . 362
162.6Project . . . 362
162.7Depth tracks . . . 363
162.8Failure modes . . . 363
162.9See also . . . 363

XII Volume 12. Sequences and language **364**

163Volume 12. Sequences and language **365**

164Volume 12. Sequences and language **366**
164.1How to use the four layers . . . 366
164.2Depth tracks . . . 366
164.3Chapters . . . 367
164.4Lab note . . . 367

165Tokens and sequences **368**

166Tokens and sequences **369**
166.1Reference card . . . 369
166.2Theory . . . 369
166.3Derivation . . . 370
166.4Code . . . 370
166.5Experiment . . . 370
166.6Project . . . 370
166.7Depth tracks . . . 371
166.8Failure modes . . . 371
166.9See also . . . 371

167Recurrent models **372**

168 Recurrent models **373**
168.1 Reference card . . . 373
168.2 Theory . . . 373
168.3 Derivation . . . 374
168.4 Code . . . 374
168.5 Experiment . . . 374
168.6 Project . . . 375
168.7 Depth tracks . . . 375
168.8 Failure modes . . . 375
168.9 See also . . . 375

169 Attention and transformers **376**

170 Attention and transformers **377**
170.1 Reference card . . . 377
170.2 Theory . . . 377
170.3 Derivation . . . 378
170.4 Code . . . 378
170.5 Experiment . . . 378
170.6 Project . . . 379
170.7 Depth tracks . . . 379
170.8 Failure modes . . . 379
170.9 See also . . . 379

171 Training and adapting language models **380**

172 Training and adapting language models **381**
172.1 Reference card . . . 381
172.2 Theory . . . 381
172.3 Derivation . . . 382
172.4 Code . . . 382
172.5 Experiment . . . 382
172.6 Project . . . 383
172.7 Depth tracks . . . 383
172.8 Failure modes . . . 383
172.9 See also . . . 383

173 Evaluation of generation **384**

174 Evaluation of generation **385**
174.1 Reference card . . . 385
174.2 Theory . . . 385
174.3 Derivation . . . 386
174.4 Code . . . 386
174.5 Experiment . . . 386
174.6 Project . . . 387
174.7 Depth tracks . . . 387

174.8Failure modes . . . 387
174.9See also . . . 387

175 Project: a small language pipeline **388**

176 Project: a small language pipeline **389**
176.1Reference card . . . 389
176.2Theory . . . 389
176.3Derivation . . . 390
176.4Code . . . 390
176.5Experiment . . . 390
176.6Project . . . 390
176.7Depth tracks . . . 391
176.8Failure modes . . . 391
176.9See also . . . 391

XIII Volume 13. Generative models **392**

177 Volume 13. Generative models **393**

178 Volume 13. Generative models **394**
178.1How to use the four layers . . . 394
178.2Depth tracks . . . 394
178.3Chapters . . . 395
178.4Lab note . . . 395

179 Autoregressive models **396**

180 Autoregressive models **397**
180.1Reference card . . . 397
180.2Theory . . . 397
180.3Derivation . . . 398
180.4Code . . . 398
180.5Experiment . . . 398
180.6Project . . . 399
180.7Depth tracks . . . 399
180.8Failure modes . . . 399
180.9See also . . . 399

181 Variational autoencoders **400**

182 Variational autoencoders **401**
182.1Reference card . . . 401
182.2Theory . . . 401
182.3Derivation . . . 402
182.4Code . . . 402

182.5Experiment . . . 402
182.6Project . . . 403
182.7Depth tracks . . . 403
182.8Failure modes . . . 403
182.9See also . . . 403

183Adversarial generators **404**

184Adversarial generators **405**
184.1Reference card . . . 405
184.2Theory . . . 405
184.3Derivation . . . 406
184.4Code . . . 406
184.5Experiment . . . 406
184.6Project . . . 407
184.7Depth tracks . . . 407
184.8Failure modes . . . 407
184.9See also . . . 407

185Diffusion models **408**

186Diffusion models **409**
186.1Reference card . . . 409
186.2Theory . . . 409
186.3Derivation . . . 410
186.4Code . . . 410
186.5Experiment . . . 410
186.6Project . . . 411
186.7Depth tracks . . . 411
186.8Failure modes . . . 411
186.9See also . . . 411

187Evaluating generators **412**

188Evaluating generators **413**
188.1Reference card . . . 413
188.2Theory . . . 413
188.3Derivation . . . 414
188.4Code . . . 414
188.5Experiment . . . 414
188.6Project . . . 415
188.7Depth tracks . . . 415
188.8Failure modes . . . 415
188.9See also . . . 415

189Project: a small generator with an evaluation **416**

190 Project: a small generator with an evaluation **417**
190.1 Reference card . . . 417
190.2 Theory . . . 417
190.3 Derivation . . . 418
190.4 Code . . . 418
190.5 Experiment . . . 418
190.6 Project . . . 418
190.7 Depth tracks . . . 419
190.8 Failure modes . . . 419
190.9 See also . . . 419

XIV Volume 14. Reinforcement learning **420**

191 Volume 14. Reinforcement learning **421**

192 Volume 14. Reinforcement learning **422**
192.1 How to use the four layers . . . 422
192.2 Depth tracks . . . 422
192.3 Chapters . . . 423
192.4 Lab note . . . 423

193 Markov decision processes **424**

194 Markov decision processes **425**
194.1 Reference card . . . 425
194.2 Theory . . . 425
194.3 Derivation . . . 426
194.4 Code . . . 426
194.5 Experiment . . . 426
194.6 Project . . . 427
194.7 Depth tracks . . . 427
194.8 Failure modes . . . 427
194.9 See also . . . 427

195 Dynamic programming and Monte Carlo **428**

196 Dynamic programming and Monte Carlo **429**
196.1 Reference card . . . 429
196.2 Theory . . . 429
196.3 Derivation . . . 430
196.4 Code . . . 430
196.5 Experiment . . . 430
196.6 Project . . . 431
196.7 Depth tracks . . . 431
196.8 Failure modes . . . 431
196.9 See also . . . 431

197 Temporal difference and Q-learning **432**

198 Temporal difference and Q-learning **433**
198.1 Reference card . . . 433
198.2 Theory . . . 433
198.3 Derivation . . . 434
198.4 Code . . . 434
198.5 Experiment . . . 434
198.6 Project . . . 435
198.7 Depth tracks . . . 435
198.8 Failure modes . . . 435
198.9 See also . . . 435

199 Policy gradients and actor-critic **436**

200 Policy gradients and actor-critic **437**
200.1 Reference card . . . 437
200.2 Theory . . . 437
200.3 Derivation . . . 438
200.4 Code . . . 438
200.5 Experiment . . . 438
200.6 Project . . . 439
200.7 Depth tracks . . . 439
200.8 Failure modes . . . 439
200.9 See also . . . 439

201 Offline data and partial observations **440**

202 Offline data and partial observations **441**
202.1 Reference card . . . 441
202.2 Theory . . . 441
202.3 Derivation . . . 442
202.4 Code . . . 442
202.5 Experiment . . . 442
202.6 Project . . . 443
202.7 Depth tracks . . . 443
202.8 Failure modes . . . 443
202.9 See also . . . 443

203 Project: a tabular agent with ablations **444**

204 Project: a tabular agent with ablations **445**
204.1 Reference card . . . 445
204.2 Theory . . . 445
204.3 Derivation . . . 446
204.4 Code . . . 446
204.5 Experiment . . . 446

204.6Project . . . 446
204.7Depth tracks . . . 447
204.8Failure modes . . . 447
204.9See also . . . 447

XV Volume 15. Systems and production **448**

205Volume 15. Systems and production **449**

206Volume 15. Systems and production **450**
206.1How to use the four layers . . . 450
206.2Depth tracks . . . 450
206.3Chapters . . . 451
206.4Lab note . . . 451

207The lifecycle as a system **452**

208The lifecycle as a system **453**
208.1Reference card . . . 453
208.2Theory . . . 453
208.3Derivation . . . 454
208.4Code . . . 454
208.5Experiment . . . 454
208.6Project . . . 455
208.7Depth tracks . . . 455
208.8Failure modes . . . 455
208.9See also . . . 455

209Compute, memory, and batching **456**

210Compute, memory, and batching **457**
210.1Reference card . . . 457
210.2Theory . . . 457
210.3Derivation . . . 458
210.4Code . . . 458
210.5Experiment . . . 458
210.6Project . . . 459
210.7Depth tracks . . . 459
210.8Failure modes . . . 459
210.9See also . . . 459

211Data pipelines **460**

212Data pipelines **461**
212.1Reference card . . . 461
212.2Theory . . . 461

212.3Derivation . . . 462
212.4Code . . . 462
212.5Experiment . . . 462
212.6Project . . . 462
212.7Depth tracks . . . 463
212.8Failure modes . . . 463
212.9See also . . . 463

213 Serving and monitoring **464**

214 Serving and monitoring **465**
214.1Reference card . . . 465
214.2Theory . . . 465
214.3Derivation . . . 466
214.4Code . . . 466
214.5Experiment . . . 466
214.6Project . . . 466
214.7Depth tracks . . . 467
214.8Failure modes . . . 467
214.9See also . . . 467

215 Reproducibility in production **468**

216 Reproducibility in production **469**
216.1Reference card . . . 469
216.2Theory . . . 469
216.3Derivation . . . 470
216.4Code . . . 470
216.5Experiment . . . 470
216.6Project . . . 470
216.7Depth tracks . . . 471
216.8Failure modes . . . 471
216.9See also . . . 471

217 Project: a batch-scoring service **472**

218 Project: a batch-scoring service **473**
218.1Reference card . . . 473
218.2Theory . . . 473
218.3Derivation . . . 474
218.4Code . . . 474
218.5Experiment . . . 474
218.6Project . . . 474
218.7Depth tracks . . . 475
218.8Failure modes . . . 475
218.9See also . . . 475

XVI Volume 16. Reliability, privacy, and harm **476**

219 Volume 16. Reliability, privacy, and harm **477**

220 Volume 16. Reliability, privacy, and harm **478**
220.1 How to use the four layers . . . 478
220.2 Depth tracks . . . 478
220.3 Chapters . . . 479
220.4 Lab note . . . 479

221 Distribution shift **480**

222 Distribution shift **481**
222.1 Reference card . . . 481
222.2 Theory . . . 481
222.3 Derivation . . . 482
222.4 Code . . . 482
222.5 Experiment . . . 482
222.6 Project . . . 483
222.7 Depth tracks . . . 483
222.8 Failure modes . . . 483
222.9 See also . . . 483

223 Robustness and adversarial examples **484**

224 Robustness and adversarial examples **485**
224.1 Reference card . . . 485
224.2 Theory . . . 485
224.3 Derivation . . . 486
224.4 Code . . . 486
224.5 Experiment . . . 486
224.6 Project . . . 486
224.7 Depth tracks . . . 487
224.8 Failure modes . . . 487
224.9 See also . . . 487

225 Privacy **488**

226 Privacy **489**
226.1 Reference card . . . 489
226.2 Theory . . . 489
226.3 Derivation . . . 490
226.4 Code . . . 490
226.5 Experiment . . . 490
226.6 Project . . . 491
226.7 Depth tracks . . . 491
226.8 Failure modes . . . 491

226.9See also . . . 491

227 Fairness and measurement **492**

228 Fairness and measurement **493**
228.1 Reference card . . . 493
228.2 Theory . . . 493
228.3 Derivation . . . 494
228.4 Code . . . 494
228.5 Experiment . . . 495
228.6 Project . . . 495
228.7 Depth tracks . . . 495
228.8 Failure modes . . . 495
228.9 See also . . . 496

229 Safety cases **497**

230 Safety cases **498**
230.1 Reference card . . . 498
230.2 Theory . . . 498
230.3 Derivation . . . 499
230.4 Code . . . 499
230.5 Experiment . . . 499
230.6 Project . . . 500
230.7 Depth tracks . . . 500
230.8 Failure modes . . . 500
230.9 See also . . . 500

231 Project: a shift and harm audit **501**

232 Project: a shift and harm audit **502**
232.1 Reference card . . . 502
232.2 Theory . . . 502
232.3 Derivation . . . 503
232.4 Code . . . 503
232.5 Experiment . . . 503
232.6 Project . . . 503
232.7 Depth tracks . . . 504
232.8 Failure modes . . . 504
232.9 See also . . . 504

XVII Volume 17. Foundation models **505**

233 Volume 17. Foundation models **506**

234 Volume 17. Foundation models **507**
234.1 How to use the four layers . . . 507
234.2 Depth tracks . . . 507
234.3 Chapters . . . 508
234.4 Lab note . . . 508

235 Scale, data, and claimed emergence **509**

236 Scale, data, and claimed emergence **510**
236.1 Reference card . . . 510
236.2 Theory . . . 510
236.3 Derivation . . . 511
236.4 Code . . . 511
236.5 Experiment . . . 511
236.6 Project . . . 512
236.7 Depth tracks . . . 512
236.8 Failure modes . . . 512
236.9 See also . . . 512

237 Adaptation **513**

238 Adaptation **514**
238.1 Reference card . . . 514
238.2 Theory . . . 514
238.3 Derivation . . . 515
238.4 Code . . . 515
238.5 Experiment . . . 515
238.6 Project . . . 516
238.7 Depth tracks . . . 516
238.8 Failure modes . . . 516
238.9 See also . . . 516

239 Preferences and alignment **517**

240 Preferences and alignment **518**
240.1 Reference card . . . 518
240.2 Theory . . . 518
240.3 Derivation . . . 519
240.4 Code . . . 519
240.5 Experiment . . . 519
240.6 Project . . . 520
240.7 Depth tracks . . . 520
240.8 Failure modes . . . 520
240.9 See also . . . 520

241 Evaluating foundation models **521**

242 Evaluating foundation models **522**
242.1 Reference card . . . 522
242.2 Theory . . . 522
242.3 Derivation . . . 523
242.4 Code . . . 523
242.5 Experiment . . . 523
242.6 Project . . . 524
242.7 Depth tracks . . . 524
242.8 Failure modes . . . 524
242.9 See also . . . 524

243 Agents as systems **525**

244 Agents as systems **526**
244.1 Reference card . . . 526
244.2 Theory . . . 526
244.3 Derivation . . . 527
244.4 Code . . . 527
244.5 Experiment . . . 527
244.6 Project . . . 528
244.7 Depth tracks . . . 528
244.8 Failure modes . . . 528
244.9 See also . . . 528

245 Project: an evaluation harness **529**

246 Project: an evaluation harness **530**
246.1 Reference card . . . 530
246.2 Theory . . . 530
246.3 Derivation . . . 531
246.4 Code . . . 531
246.5 Experiment . . . 531
246.6 Project . . . 531
246.7 Depth tracks . . . 532
246.8 Failure modes . . . 532
246.9 See also . . . 532

XVII Volume 18. Research practice **533**

247 Volume 18. Research practice **534**

248 Volume 18. Research practice **535**
248.1 How to use the four layers . . . 535
248.2 Depth tracks . . . 535
248.3 Chapters . . . 536
248.4 Lab note . . . 536

249 Reading a paper into this reference **537**

250 Reading a paper into this reference **538**
250.1 Reference card . . . 538
250.2 Theory . . . 538
250.3 Derivation . . . 539
250.4 Code . . . 539
250.5 Experiment . . . 539
250.6 Project . . . 539
250.7 Depth tracks . . . 540
250.8 Failure modes . . . 540
250.9 See also . . . 540

251 Reproducing and revising a result **541**

252 Reproducing and revising a result **542**
252.1 Reference card . . . 542
252.2 Theory . . . 542
252.3 Derivation . . . 543
252.4 Code . . . 543
252.5 Experiment . . . 543
252.6 Project . . . 543
252.7 Depth tracks . . . 544
252.8 Failure modes . . . 544
252.9 See also . . . 544

253 Writing a research note **545**

254 Writing a research note **546**
254.1 Reference card . . . 546
254.2 Theory . . . 546
254.3 Derivation . . . 547
254.4 Code . . . 547
254.5 Experiment . . . 547
254.6 Project . . . 547
254.7 Depth tracks . . . 548
254.8 Failure modes . . . 548
254.9 See also . . . 548

255 A map of open problems **549**

256 A map of open problems **550**
256.1 Reference card . . . 550
256.2 Theory . . . 550
256.3 Derivation . . . 551
256.4 Code . . . 551
256.5 Experiment . . . 551

256.6Project . . . 551
256.7Depth tracks . . . 552
256.8Failure modes . . . 552
256.9See also . . . 552

257 Upstream books and how to use them 553

258 Upstream books and how to use them 554
258.1Reference card . . . 554
258.2Theory . . . 554
258.3Derivation . . . 555
258.4Code . . . 555
258.5Experiment . . . 555
258.6Project . . . 556
258.7Depth tracks . . . 556
258.8Failure modes . . . 556
258.9See also . . . 556

259 Capstone: a checkable study 557

260 Capstone: a checkable study 558
260.1Reference card . . . 558
260.2Theory . . . 558
260.3Derivation . . . 559
260.4Code . . . 559
260.5Experiment . . . 559
260.6Project . . . 559
260.7Depth tracks . . . 560
260.8Failure modes . . . 560
260.9See also . . . 560

1 Machine Learning Reference

2 Machine Learning Reference

A personal guide from first principles to a checkable study. The path runs from notation and least squares through probability, classical learning, neural nets, vision, language, generators, reinforcement learning, production systems, harm, foundation-model evaluation, and research practice.

This is a reference, not a short textbook. You look a result up, then you rederive it, implement it, and attach an experiment. Each chapter has a card, theory, a derivation, code, an experiment, and a project, with beginner, university, and research tracks.

2.1 The progression

Volume	Title	Stage	Question
01	Mathematical language	Beginner	What do the symbols mean, and which linear-algebra facts are load-bearing?
02	Calculus and optimization	Beginner to university	How does a training step move, and which guarantees need convexity?
03	Computation for experiments	Beginner	How does a formula become an experiment you can rerun?
04	Probability	Beginner to university	What is a model claiming about data?
05	Statistical inference	University	How wrong might an estimate be?
06	Supervised learning	University	Which predictor, which loss, which baseline?
07	Evaluation and model selection	University	Which number may choose a model, and which number may be published?

Volume	Title	Stage	Question
08	Unsupervised learning	University	What can you estimate without a target, and how do you test it?
09	Neural networks	University	What function does a network represent, and which part is bias rather than magic?
10	Training deep models	University to research	Which knobs are optimization, and which are generalization?
11	Vision	University	How is an image a measurement, and how is a vision claim evaluated?
12	Sequences and language	University to research	How do you model order without leaking the future?
13	Generative models	University to research	What distribution is a sampler claiming, and which column of the evaluation table applies?
14	Reinforcement learning	University to research	What is the agent optimizing, and when does a log lie about a new policy?
15	Systems and production	University to research	What artifacts make a model a system you can roll back?
16	Reliability, privacy, and harm	University to research	What do you measure when the average metric is the wrong harm?
17	Foundation models	Research	What can you conclude from a model whose training data you did not see?

18	Research practice	Research	How do you leave a claim someone else can check?

Start with How to study this reference. The glossary fixes words that later chapters refuse to blur. The upstream books page says what to read beside this series and what not to copy.

2.2 Four layers

Layer	You finish it when
Theory	You can state the result and the assumption that makes it false.
Code	A known case passes.
Experiment	You recorded a metric and a decision.
Project	Someone else can rerun the folder.

2.3 Formats

The sources are Markdown and Quarto (`.qmd`) in this repository. One source list renders three ways:

- an HTML book for the browser,
- a PDF,
- an EPUB.

The build script is `scripts/build.ps1`. It expects the Quarto CLI.

3 How to study this reference

4 How to study this reference

This book is a guide you return to, from a first definition through a checkable study. It is not a short textbook and it is not a tour of every algorithm. Each chapter is a reference entry: a card, the theory, a derivation, code, an experiment with a decision rule, and a project with a rubric. The same entry then splits into a beginner track, a university track, and a research track.

4.1 Who it is for

A complete progression means different chapters make different demands.

Track	What you do	What you skip
Beginner	The card, the code, and the smallest experiment. Learn the notation by using it.	Optional proofs and research extensions.
University	The derivation, the full experiment, and the project rubric.	Nothing in the main path of that chapter.
Research	The failure modes, a changed assumption, and a written limit on the claim.	The pretense that a benchmark name is a result.

You can be a beginner in Volume 14 and at research depth in Volume 04. The track is about the chapter.

4.2 How a session works

Ninety minutes is the default unit.

1. Read the reference card and restate the result in one sentence, including an assumption.
2. Read the theory and the derivation until you can point to the step that would fail if the assumption failed.
3. Run the code on the known case. If it fails, the experiment does not start.
4. Run the experiment and write the decision in `notes/`, with the metric and the command.
5. If the chapter's project is the day's work, leave a rerunnable folder, not only a plot.

A chapter is usable as a reference when you can look up the card and rerun the known case. It is university-complete when every layer is done. It is research-complete when you have written one way the assumption fails.

4.3 What "comprehensive" means here

Every serious entry has four layers.

Layer	Question it answers
Theory	What is true, under which assumptions, and what is not claimed?
Code	Can you implement the object and check a known case?
Experiment	What would change your mind, and did you measure it?
Project	Can someone else rerun the artifact and see the decision?

A paragraph you highlighted and did not measure is not yet part of your reference. The book is long so that the path exists. Your notes become the reference when they are attached to the experiments.

4.4 Suggested order

Read Volume 01 through Volume 07 in order the first time. Those volumes build notation, optimization, probability, inference, supervised baselines, and honest evaluation. After that, pick a path:

- **Representations.** Volumes 08, 09, 11, and 13.
- **Training and language.** Volumes 10, 12, and 17.
- **Decisions over time.** Volume 14, then Volume 17's agent chapter.
- **Operating a system.** Volume 15, then Volume 16.
- **Making a claim.** Volume 18, which you should open early as a way of taking notes and finish as a capstone.

Volume 18's upstream chapter says which external books to read beside this path. Those books stay in their own repositories.

4.5 What this book will not do

It will not paste the Harvard CS249r textbook, Dive into Deep Learning, the Think series, or the Hugging Face course into these chapters. Those works have their own licenses, described on the upstream page. It will not call a free PDF an open license. It will not treat a leaderboard number as a definition of understanding.

When a chapter says to refuse a claim, that refusal is part of the result.

5 Glossary

6 Glossary

These words are used in one sense throughout the series. If a chapter needs a second sense, it says so.

Term	Meaning in this book
Risk $R(f)$	Expected loss of a predictor under the data law. Not the training loss.
Empirical risk	Average loss on a named sample. Say which sample.
Estimator	The procedure, including preprocessing and selection.
Estimate	The number that procedure returned on one sample.
Bayes predictor	A minimizer of the risk over a rich class, for a named loss.
Approximation error	The gap from the Bayes predictor to the best function in a restricted family.
Estimation error	The gap from that best function to the one you fit.
Baseline	A simpler procedure on the same split and metric. A constant predictor counts.
Split unit	The entity a random split must not tear apart: row, patient, document, or time.
Leakage	Use of information the decision-time system will not have, including test labels and future events.
Validation	Data used to choose knobs. Not a license to also be the published test.
Test	A measurement of a procedure that was already frozen.
Proper score	A score minimized in expectation by stating the true probability. Log loss and Brier score are examples. Accuracy is not.
Calibration	Among cases given predicted probability p, about a fraction p are positive.
Inductive bias	What an architecture or penalty makes easy, stated as a restriction.

Term	Meaning in this book
Known case	An input with an answer you can compute without the experiment. The harness must pass it.
Shift	A named change in a law: of X, of Y given X, or of the mixture of slices.
Threat model	Who may change an input, under which budget, with what knowledge.
Safety case	Hazards, controls, evidence, assumptions, and a withdrawal condition.
Reproduction	The same claim and protocol, within a predeclared tolerance.
Upstream	A book you read under its own license. Not a chapter of this book.

7 Upstream books

8 Upstream books

This series is original. It does not include the text of the books below. Read them beside the volumes that point at them. Keep their licenses attached to their files if you keep your own copies.

Work	Read it for	License constraint
Harvard CS249r, *Machine Learning Systems*, Volumes I–IV, mlsysbook.ai	Systems from one machine through scale, agents, and physical AI. Deeper than Volume 15.	Textbook text is CC BY-NC-ND 4.0. Share it unmodified. Do not edit it into these chapters.
Dive into Deep Learning, d2l.ai	A full implemented deep-learning course.	Text is CC BY-SA 4.0. Adaptation carries attribution and share-alike. Not copied here.
Allen Downey, *Think Python*, *Think Stats*, *Think Bayes*	Computational introductions to Python, statistics, and Bayesian updating.	CC BY-NC-SA 4.0. Non-commercial adaptation is share-alike.
Hugging Face course, huggingface.co/course	A library-centered course on transformers.	Apache 2.0. Reuse requires attribution, the license notice, and a note of changes.

A free PDF is not the same thing as one of these licenses. Books you may read online and may not republish include, among others, *Understanding Deep Learning* (CC BY-NC-ND), and classic texts that are free to read but all rights reserved. Do not paste them into this repository.

Volume 18 has a chapter on how to write a reading note in your own words. Use that note instead of a close paraphrase.

8.1 Where they sit on the path

After you can do	Read
Volumes 01–03	Think Python and Think Stats, if the code and the statistics still feel thin.

After you can do	Read
Volumes 04–05	Think Bayes for a computational Bayesian companion.
Volumes 09–13	Dive into Deep Learning, as a broader implemented course than these notes.
Volume 12	The Hugging Face course, if you want the library path. Keep this volume's baselines.
Volume 15 and beyond	CS249r Volumes I–IV, unmodified, for hardware, scale, and the rest of the systems curriculum.

Part I

Volume 01. Mathematical language

9 Volume 01. Mathematical language

10 Volume 01. Mathematical language

Stage. Beginner, first weeks

Audience. A reader who can do high-school algebra and is willing to learn notation as a tool.

Question this volume answers. What do the symbols in the rest of this reference mean, and which linear-algebra facts are load-bearing?

This volume is a reference, not a short tour. Each chapter opens with a card you can look up later, then gives the theory, a reference implementation, an experiment with a decision rule, and a project with a rubric. Read in order the first time. After that, enter through the reference card of the chapter that matches the problem in front of you.

10.1 How to use the four layers

Layer	What you should be able to do when you finish it
Theory	State the definition, the assumptions, and the result, including what the result does not say.
Code	Implement the core object in a few dozen lines and check it against a known case.
Experiment	Change one factor, record a metric with uncertainty, and say what outcome would change your mind.
Project	Produce an artifact someone else can rerun: code, figure, table, and a one-page lab note.

10.2 Depth tracks

Every chapter ends with three tracks.

- **Beginner.** Do the reference card, the code, and the smallest experiment. Skip optional proofs.
- **University.** Do the derivation, the full experiment, and the project rubric.

- **Research.** Read the failure modes, change an assumption, and write what would have to be true for the method to fail on a new distribution.

10.3 Chapters

No.	Chapter	File
01	How to use this reference	`vol01/01_how_to_use.qmd`
02	Sets, functions, and notation	`vol01/02_notation.qmd`
03	Linear algebra for models	`vol01/03_linear_algebra.qmd`
04	Matrix calculus	`vol01/04_matrix_calculus.qmd`
05	Numerical linear algebra	`vol01/05_numerical_linear_algebra.qmd`
06	Project: least squares from the axioms	`vol01/06_least_squares_project.qmd`

10.4 Lab note

Keep one markdown file per experiment, `notes/YYYY-MM-DD-<slug>.md`, with the hypothesis, the command you ran, the metric, and the decision. The series is a reference only if the experiments you run are attached to it.

11 How to use this reference

12 How to use this reference

Reference question. How should a beginner-to-research reader move through a long reference without treating it as a short textbook?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

12.1 Reference card

Item	Statement
Object	A reference entry: card, theory, derivation, code, experiment, project.
Order	Volume order the first time. Chapter cards on every later visit.
Unit of work	One experiment folder with a hypothesis, a command, a metric, and a decision.
Done	You can restate the result, rerun the code, and say what would falsify it.
Not done	You highlighted a paragraph and did not change a number.

12.2 Theory

This series is a guide you return to, not a narrative you finish once. Each volume widens the object under study: a vector, a probability, a predictor, a training run, a deployed system, a research claim.

A result in this book is a triple: a statement, the assumptions that make it true, and a check you can run. If you drop the assumptions, you do not have the result. If you cannot check it, you do not yet own it. The four layers exist so that a lookup (the card), an argument (theory and derivation), an implementation (code), and a decision (experiment and project) stay attached to each other.

The progression has three speeds. Beginner work builds the objects. University work proves the standard lemmas and compares methods on a fixed split. Research work changes an assumption and writes the consequence. You may be a beginner in Volume 14 and at research

depth in Volume 04 at the same time. The track labels are about the chapter, not about your identity.

12.3 Derivation

Fix a study session of 90 minutes as the unit. Spend about 20 minutes on the card and theory, 25 on the derivation or a line-by-line reading of the proof sketch, 25 on the code until a known case passes, and 20 on writing the experiment note. A project spans several sessions and must leave a rerunnable artifact.

Let C be a chapter and $L(C)$ the set of layers you completed. The chapter is usable as a reference when $\{\text{card}, \text{code}\} \subseteq L(C)$. It is university-complete when all six layers are in $L(C)$. It is research-complete when you have added one written failure of an assumption, stored next to the lab note.

12.4 Code

```
from pathlib import Path

def new_note(slug: str, hypothesis: str, metric: str) -> Path:
    path = Path("notes") / f"{slug}.md"
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(
        "\n".join(
            [
                f"# {slug}",
                "",
                f"Hypothesis. {hypothesis}",
                "",
                f"Metric. {metric}",
                "",
                "Protocol.",
                "",
                "Result.",
                "",
                "Decision.",
                "",
            ]
        ),
        encoding="utf-8",
    )
    return path
```

12.5 Experiment

Hypothesis. A fixed note template increases the chance that a later session can reconstruct the decision.

Protocol. For the next four chapters, write the note before running code. On two of them, skip the template and write free prose. A week later, try to rerun both kinds from the note alone.

Metric. Fraction of notes from which you can recover the command, the metric, and the decision without opening old chat logs.

Decision rule. If templated notes are not easier to rerun, change the template. Do not keep a ritual that does not restore the result.

12.6 Project

Deliverable. A `notes/` directory, this function, and one completed note for the linear-algebra chapter.

Constraints. The note names the assumption you used and one place it can fail.

Rubric. Someone else can tell what you measured and what you would do differently. The code creates the file you claim it creates.

12.7 Depth tracks

Beginner. Read this chapter once, create the notes folder, and move to notation.

University. Write the session rule down and follow it for all of Volume 01.

Research. After Volume 06, reread this chapter and add a fifth field to the note: the assumption you are worried is false in your data.

12.8 Failure modes

- Treating the card as a substitute for the experiment.
- Collecting chapters without a question you are trying to answer.
- Skipping prerequisites and then blaming the later formula.

12.9 See also

Volume 03 for the experiment folder layout. Volume 18 for how a paper is filed into this reference. The upstream reading list names books you read beside this series. Their text is not copied here.

13 Sets, functions, and notation

14 Sets, functions, and notation

Reference question. Which symbols are fixed for the rest of the series, and what does a function claim?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

14.1 Reference card

Item	Statement
Data	A pair (x, y) with $x \in \mathcal{X}$ and $y \in \mathcal{Y}$. A sample is $\{(x_i, y_i)\}_{i=1}^n$.
Parameters	$\theta \in \Theta$. A predictor is $f_\theta : \mathcal{X} \to \mathcal{A}$, where $\mathcal{A}$ is the action or prediction space.
Loss	$\ell(f_\theta(x), y) \geq 0$. Empirical risk is $\widehat{R}_n(\theta) = n^{-1} \sum_{i=1}^n \ell(f_\theta(x_i), y_i)$.
Expectation	$R(\theta) = \mathbb{E}[\ell(f_\theta(X), Y)]$ is the risk under the unknown data law P.
Convention	Vectors are columns. $x^\top x$ is a scalar. $\|x\|_2 = \sqrt{x^\top x}$. Indices start at 1 in sums and at 0 in code.

14.2 Theory

A set is a collection we are willing to test membership in. A function $f : \mathcal{X} \to \mathcal{Y}$ assigns exactly one output to each input in its domain. In this series the domain is part of the claim. A neural network that expects a tensor of shape (n, d) is a different function from one that expects (n, c, h, w).

Random variables are functions on a sample space. Writing $X \sim P$ means we are about to use the law of X, not a single draw. A single dataset is one draw of the collection $(X_1, Y_1), \ldots, (X_n, Y_n)$. The risk $R(\theta)$ depends on P. The empirical risk $\widehat{R}_n(\theta)$ depends on the draw. Almost every later confusion between training loss and future error is a confusion between these two functions.

We write $\arg\min_{\theta \in \Theta} \widehat{R}_n(\theta)$ for a minimizer, not for a unique one. If the set of minimizers has more than one point, say so. Uniqueness is a theorem, not a notation.

14.3 Derivation

Let ℓ be squared error, $\ell(\hat{y}, y) = (\hat{y} - y)^2$, and let $f_\theta(x) = \theta^\top x$ with $x \in \mathbb{R}^d$ already containing a constant coordinate if you want an intercept. Then

$$\hat{R}_n(\theta) = n^{-1} \|X\theta - y\|_2^2,$$

where $X \in \mathbb{R}^{n \times d}$ has rows $x_i^\top$ and $y \in \mathbb{R}^n$. Expanding,

$$\|X\theta - y\|_2^2 = \theta^\top X^\top X \theta - 2 y^\top X \theta + y^\top y.$$

Differentiate in θ and set the gradient to zero:

$$X^\top X \theta = X^\top y.$$

This is the normal equation. It is a claim about critical points of squared error, under no claim yet that $X^\top X$ is invertible.

14.4 Code

```
import numpy as np

def empirical_risk(theta, X, y):
    residual = X @ theta - y
    return float(np.mean(residual**2))

def normal_equation(X, y):
    gram = X.T @ X
    rhs = X.T @ y
    # lstsq remains defined when gram is singular.
    theta, *_ = np.linalg.lstsq(gram, rhs, rcond=None)
    return theta
```

14.5 Experiment

Hypothesis. Adding a duplicate column to X leaves the minimal squared error unchanged and makes the parameter vector non-unique.

Protocol. Draw $X_0 \in \mathbb{R}^{40 \times 3}$ with independent standard normal entries and $y = X_0 \theta_\star + \varepsilon$. Build X by copying the first column. Compare `lstsq` on X_0 and on X, and compare $\hat{R}_n$.

Metric. Absolute difference of the two minimal risks, and the dimension of the numerical null space of $X^\top X$ (count singular values below 10^{-8}).

Decision rule. The risks should match to about 10^{-8} relative error. If they do not, the implementation, not the theorem, is wrong.

14.6 Project

Deliverable. A one-page notation sheet in your own words, plus the script above, plus the duplicate-column note.

Constraints. The sheet must define R, $\widehat{R}_n$, θ, X, and y with shapes.

Rubric. A reader can translate one formula from a later chapter back onto your sheet without asking you.

14.7 Depth tracks

Beginner. Copy the notation table by hand. Run the code on a 5 by 2 matrix you type yourself.

University. Prove the expansion of the squared norm and the gradient calculation without looking.

Research. Rewrite the normal equation for a weighted risk $\sum_i w_i (x_i^\top \theta - y_i)^2$ with $w_i > 0$. State when the weighted Gram matrix is singular.

14.8 Failure modes

- Using the same letter for a random variable and a single sample.
- Assuming $\arg\min$ returns one vector.
- Mixing row and column layouts between the formula and the array.

14.9 See also

The next chapter is the linear-algebra reference those shapes rely on. Volume 04 rereads R and $\widehat{R}_n$ as expectations.

15 Linear algebra for models

16 Linear algebra for models

Reference question. Which facts about matrices, ranks, and norms do models actually use?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

16.1 Reference card

Item	Statement
Space	$\mathbb{R}^d$ with the dot product $\langle u, v\rangle = u^\top v$ and the Euclidean norm.
Matrix	A linear map $A : \mathbb{R}^d \to \mathbb{R}^n$. Columns of A are the images of the standard basis.
Rank	$\operatorname{rank}(A) = \dim \operatorname{col}(A)$. $\operatorname{rank}(A) \le \min(n, d)$.
SVD	$A = U\Sigma V^\top$ with orthonormal U, V and nonnegative singular values on the diagonal of Σ.
PSD	$G = G^\top$ is positive semidefinite if $z^\top G z \ge 0$ for every z. Gram matrices $X^\top X$ are PSD.
Norms	$\|A\|_2$ is the largest singular value. $\|A\|_F^2 = \sum_{ij} A_{ij}^2 = \sum_k \sigma_k(A)^2$.

16.2 Theory

A model that predicts with $X\theta$ is using the column space of X. If y is not in that column space, squared error cannot be zero. The residual $y - X\hat{\theta}$ from least squares is orthogonal to the column space: $X^\top(y - X\hat{\theta}) = 0$. That geometric sentence is the normal equation.

Rank tells you how many independent directions the features supply. Two copies of the same feature do not give a new direction. A one-hot encoding of a categorical variable with a redundant level, together with an intercept, is the same phenomenon: the columns are linearly dependent, and the parameter is not identified even when predictions are.

Norms measure size. Parameter size and prediction size are different. $\|\theta\|_2$ can be large while $X\theta$ is moderate, which is what happens when columns of X are small or nearly collinear.

Later regularization chapters penalize $\|\theta\|_2$ or $\|\theta\|_1$. They are changing the parameter, not directly the residual, unless the design is well scaled.

16.3 Derivation

Suppose $X^\top X$ is invertible and $\hat{\theta} = (X^\top X)^{-1} X^\top y$. The fitted vector is $\hat{y} = Py$ with $P = X(X^\top X)^{-1} X^\top$. Then $P^2 = P$ and $P^\top = P$, so P is the orthogonal projector onto $\mathrm{col}(X)$. For any $v = Xa$,

$$(y - \hat{y})^\top v = y^\top (I - P) X a = 0,$$

because $(I - P)X = 0$. So the residual is orthogonal to every column. If $X = U\Sigma V^\top$ is a thin SVD with $\mathrm{rank}(X) = r$, the same projector is $U_r U_r^\top$, and the least-squares solution of minimal Euclidean norm is $\hat{\theta} = V_r \Sigma_r^{-1} U_r^\top y$.

16.4 Code

```
import numpy as np

def svd_least_squares(X, y, tol=1e-10):
    u, s, vt = np.linalg.svd(X, full_matrices=False)
    keep = s > tol
    coef = np.zeros(X.shape[1])
    if np.any(keep):
        coef = vt[keep].T @ ((u[:, keep].T @ y) / s[keep])
    fitted = X @ coef
    residual_norm = np.linalg.norm(y - fitted)
    return coef, fitted, float(residual_norm), int(np.sum(keep))
```

16.5 Experiment

Hypothesis. The residual from minimal-norm least squares is numerically orthogonal to the column space, including when two columns are copies.

Protocol. Build a rank-2 matrix in $\mathbb{R}^{30\times 4}$ by drawing two independent columns and repeating each once. Draw y at random. Compute `svd_least_squares`. Measure $\|X^\top (y - \hat{y})\|_\infty$ and the reported rank.

Metric. The infinity norm of $X^\top$ times the residual, and the gap between the kept and discarded singular values.

Decision rule. The orthogonality residual should be near 10^{-8} or smaller for this scale. A reported rank other than 2 means the tolerance does not match the singular-value gap.

16.6 Project

Deliverable. A script that, for a feature matrix you care about, prints rank, condition number of the kept singular values, residual norm, and whether an intercept column is redundant.

Constraints. Do not drop columns silently. Report them.

Rubric. The script distinguishes "predictions are fine" from "parameters are not unique."

16.7 Depth tracks

Beginner. Compute rank and a least-squares fit for a 4 by 2 example by hand and by code.

University. Prove that P is the orthogonal projector and that the SVD solution has minimal norm.

Research. State the condition number $\sigma_{\max}/\sigma_{\min}$ and how a small singular value amplifies noise in y into $\hat{\theta}$. Relate this to ridge regression in Volume 02.

16.8 Failure modes

- Inverting $X^\top X$ when it is singular or nearly singular.
- Comparing $\|\theta\|_2$ across features that have not been scaled.
- Interpreting a large coefficient on a duplicated column as a real effect.

16.9 See also

Volume 02 uses the same SVD to describe ridge shrinkage. Volume 08 uses projectors again for principal components.

17 Matrix calculus

18 Matrix calculus

Reference question. How do you differentiate a scalar loss with respect to a vector or a matrix without dropping a transpose?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

18.1 Reference card

Item	Statement
Layout	The gradient $\nabla_\theta L$ is a column vector, the same shape as θ.
Differential	$dL = \langle \nabla_\theta L, d\theta \rangle$. This identity defines the gradient.
Linear	$d(A\theta) = A\, d\theta$. $d(\theta^\top A\theta) = \theta^\top (A + A^\top) d\theta$ if you do not assume symmetry yet.
Quadratic	If $A = A^\top$, then $\nabla_\theta(\theta^\top A\theta) = 2A\theta$.
Least squares	$\nabla_\theta \lVert X\theta - y \rVert_2^2 = 2X^\top (X\theta - y)$.
Chain rule	If $z = Wx$ and L depends on z, then $\nabla_W L = (\nabla_z L) x^\top$.

18.2 Theory

Code autodiff and a hand derivative answer the same question: which direction of a small change in the parameter increases the scalar loss. The differential dL is the linear part of that change. Once you write dL as a dot product against $d\theta$, the vector on the other side is the gradient. This is safer than memorizing a table, because the shape checks itself.

Matrix parameters follow the same rule with the Frobenius inner product $\langle A, B \rangle = \sum_{ij} A_{ij} B_{ij} = \operatorname{tr}(A^\top B)$. If $dL = \langle G, dW \rangle_F$, then $\nabla_W L = G$. The outer product $(\nabla_z L) x^\top$ is the gradient of a loss that sees $z = Wx$, and it is the rank-one update used throughout backpropagation.

18.3 Derivation

Let $r = X\theta - y$ and $L = r^\top r$. Then $dr = X\,d\theta$ and

$$dL = 2r^\top dr = 2r^\top X\,d\theta = \langle 2X^\top r, d\theta \rangle.$$

Hence $\nabla_\theta L = 2X^\top(X\theta - y)$. Setting it to zero recovers the normal equation.

For $z = Wx$ with $L = \ell(z)$, the differential is $dz = dW\,x$, so

$$dL = \langle \nabla_z L, dW\,x \rangle = \langle (\nabla_z L)x^\top, dW \rangle_F.$$

Therefore $\nabla_W L = (\nabla_z L)x^\top$. If a bias b is added, dz also contains db, and $\nabla_b L = \nabla_z L$.

18.4 Code

```
import numpy as np

def mse_grad(theta, X, y):
    residual = X @ theta - y
    return 2.0 * X.T @ residual

def numerical_grad(loss, theta, eps=1e-6):
    grad = np.zeros_like(theta)
    for j in range(theta.size):
        step = np.zeros_like(theta)
        step[j] = eps
        grad[j] = (loss(theta + step) - loss(theta - step)) / (2 * eps)
    return grad
```

18.5 Experiment

Hypothesis. The analytic least-squares gradient matches the central-difference gradient on a random instance, to an accuracy limited by the step size.

Protocol. Draw $X \in \mathbb{R}^{25\times 4}$, θ, and y from a standard normal. Compare `mse_grad` with `numerical_grad` of $L(\theta) = \|X\theta - y\|_2^2$ at `eps` in $\{10^{-4}, 10^{-6}, 10^{-8}\}$.

Metric. $\|g_{\text{ana}} - g_{\text{num}}\|_\infty / (1 + \|g_{\text{ana}}\|_\infty)$.

Decision rule. The relative gap should be smallest near 10^{-6} and worse at 10^{-8} because subtraction cancels digits. If 10^{-6} is already wrong in the third digit, the analytic gradient is wrong.

18.6 Project

Deliverable. A gradient-check helper that takes any scalar loss and a parameter vector, plus a short note on a case you intentionally broke (for example, forgetting the factor 2).

Constraints. The helper must report the relative gap, not only a boolean.

Rubric. You can explain why a too-small `eps` fails even when the mathematics is right.

18.7 Depth tracks

Beginner. Check the gradient of $L(\theta) = \theta^\top \theta$ by hand and with the numerical helper.

University. Derive $\nabla_W L$ for $z = Wx + b$ and a squared loss on z.

Research. Write the differential of $\log\det G(\theta)$ for a symmetric positive-definite G, and name a model in Volume 04 that needs it.

18.8 Failure modes

- Treating the gradient of a vector-valued map as if it were a column. That object is a Jacobian.
- Dropping a transpose so that a $(d,)$ array is added to a $(1, d)$ array and broadcasts.
- Trusting autodiff on a loss that has detached a tensor or stopped the graph.

18.9 See also

Volume 09 uses the same outer-product gradient inside backpropagation. Volume 02 uses gradients for descent.

19 Numerical linear algebra

20 Numerical linear algebra

Reference question. When is a formula that is true in exact arithmetic the wrong thing to compute?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

20.1 Reference card

Item	Statement
Condition	For $A\theta = b$, relative errors can grow by about $\kappa_2(A) = \sigma_{\max}(A)/\sigma_{\min}(A)$.
Stability	An algorithm is stable when it returns the exact answer to a nearby problem.
Prefer	`solve` on a linear system, `lstsq` or a thin SVD on least squares, a Cholesky factor when the matrix is SPD.
Avoid	Forming an explicit inverse, or forming $X^\top X$ when X itself is ill-conditioned.
Float	float64 has about 16 decimal digits. Cancellation removes leading digits before you round.

20.2 Theory

Exact identities are not execution plans. The normal equation squares the condition number: $\kappa_2(X^\top X) = \kappa_2(X)^2$. A design that is merely somewhat collinear becomes a Gram matrix that is unusable in float64. QR or SVD on X avoids that squaring.

An ill-conditioned problem is hard even for a perfect algorithm, because the true answer moves a lot when y moves a little. An unstable algorithm is worse: it adds error that the problem did not require. Forming A^{-1} and multiplying is a common unstable habit. Solving $A\theta = b$ with a factorization answers the same mathematics and perturbs the data less.

Scaling features before forming a Gram matrix is not a cosmetic preprocessing step. It changes κ_2. Report the condition number you actually computed.

20.3 Derivation

Let $X = U\Sigma V^\top$ with singular values σ_j. The eigenvalues of $X^\top X$ are σ_j^2. If $\sigma_{\min}/\sigma_{\max} = 10^{-8}$, then the Gram matrix has eigenvalue ratio 10^{-16}, which is at the edge of float64. A perturbation of size 10^{-16} in the Gram matrix is then as large as the smallest eigenvalue, and the computed inverse is noise.

The minimal-norm least-squares map is $\theta = \sum_{\sigma_j > 0} \sigma_j^{-1}(u_j^\top y)v_j$. Truncating singular values below a tolerance τ replaces an explosive $1/\sigma_j$ with zero. That is a modeling choice: you refuse to fit directions the data do not identify. Ridge regression makes a related choice by replacing σ_j^{-1} with $\sigma_j/(\sigma_j^2 + \lambda)$.

20.4 Code

```
import numpy as np

def condition_report(X):
    s = np.linalg.svd(X, compute_uv=False)
    s = s[s > 0]
    kappa = float(s[0] / s[-1]) if s.size else np.inf
    return {"singular_values": s, "kappa": kappa, "gram_kappa_if_formed": kappa**2}

def ridge_from_svd(X, y, lam):
    u, s, vt = np.linalg.svd(X, full_matrices=False)
    shrink = s / (s**2 + lam)
    return vt.T @ (shrink * (u.T @ y))
```

20.5 Experiment

Hypothesis. Solving the normal equation in float64 loses accuracy before an SVD solver does, once columns are nearly collinear.

Protocol. Let the columns of X be v, $v + 10^{-8}z$, and an independent third column, with v, z standard normal in $\mathbb{R}^{200}$. Let $y = X\theta_\star$ with no noise. Compare $\hat{\theta}$ from `lstsq` on $X^\top X$ against `ridge_from_svd` with $\lambda = 0$ implemented as a tiny $\lambda = 10^{-12}$, and against `np.linalg.lstsq(X, y)`.

Metric. $\|\hat{\theta} - \theta_\star\|_2 / \|\theta_\star\|_2$ for each solver, and $\kappa_2(X)$.

Decision rule. The solver that forms the Gram matrix should be worse. If it is not, the columns are not collinear enough; shrink the 10^{-8}.

20.6 Project

Deliverable. A preprocessor that standardizes columns, refuses to drop them, prints κ_2 before and after, and fits least squares with `lstsq` on X rather than on the Gram matrix.

Constraints. Keep a column of ones unscaled, or handle the intercept separately, and say which you did.

Rubric. On the collinear construction above, your relative parameter error is in the same ballpark as `lstsq` on X, not worse by orders of magnitude.

20.7 Depth tracks

Beginner. Print κ_2 for a well-scaled matrix and for the collinear construction.

University. Show that the eigenvalues of $X^\top X$ are the squared singular values.

Research. Compare truncation of small singular values with the ridge shrink factor at matched effective degrees of freedom.

20.8 Failure modes

- Using `np.linalg.inv` as a default.
- Comparing algorithms on well-conditioned toy data and concluding they are equivalent on real data.
- Standardizing using statistics computed on the test split.

20.9 See also

Volume 02 for ridge as a numerical and a statistical method. Volume 06 for feature scaling inside model pipelines.

21 Project: least squares from the axioms

22 Project: least squares from the axioms

Reference question. Can you build an identified, numerically stable least-squares baseline and know what it does not estimate?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

22.1 Reference card

Item	Statement
Estimator	$\widehat{\theta} \in \arg\min_\theta \|X\theta - y\|_2^2$, choosing the minimal-norm solution if needed.
Identification	Unique θ if and only if $\operatorname{rank}(X) = d$.
Prediction	Unique fitted vector $\widehat{y} = P_{\operatorname{col}(X)} y$ even when θ is not unique.
Noise model	If $y = X\theta_\star + \varepsilon$ and $\mathbb{E}[\varepsilon \mid X] = 0$, then $\mathbb{E}[\widehat{\theta} \mid X] = \theta_\star$ when X has full column rank.
Out of scope	This chapter does not claim that squared error is the right loss for counts, ranks, or heavy tails.

22.2 Theory

Least squares is the first complete object in the series: a loss, a geometric estimator, a numerical method, and a statistical claim with an assumption. The statistical claim is about the conditional mean. If $\mathbb{E}[Y \mid X = x] = x^\top \theta_\star$, squared error is a coherent way to estimate that function, because the conditional mean minimizes expected squared error over all predictors. The linear restriction can still be wrong. Unbiasedness of $\widehat{\theta}$ given X does not imply that $x \mapsto x^\top \theta_\star$ is the conditional mean. It implies unbiasedness inside a model that may be false.

Use this chapter as the pattern for later projects. The artifact is not a plot alone. It is a statement of the target, the assumption, the algorithm, a check on a known case, a check on a broken case, and a decision about whether the estimator is usable for prediction, for parameter interpretation, or for neither.

22.3 Derivation

Among all functions g, $\mathbb{E}[(Y - g(X))^2]$ is minimized by $g_\star(x) = \mathbb{E}[Y \mid X = x]$, when the expectation exists. Restricting g to linear functions $x \mapsto x^\top \theta$ yields a different problem, whose population solution satisfies $\mathbb{E}[XX^\top]\theta = \mathbb{E}[XY]$ when the Gram matrix is invertible. The sample normal equation replaces those expectations by averages. Nothing in that derivation forces $g_\star$ itself to be linear. The gap $\mathbb{E}[(g_\star(X) - X^\top\theta)^2]$ is approximation error. Later volumes separate it from estimation error.

22.4 Code

```
import numpy as np

def fit_predict(X, y, X_new):
    theta, *_ = np.linalg.lstsq(X, y, rcond=None)
    return theta, X @ theta, X_new @ theta

def known_case():
    X = np.array([[1.0, 0.0], [1.0, 1.0], [1.0, 2.0], [1.0, 3.0]])
    theta_star = np.array([1.0, 2.0])
    y = X @ theta_star
    theta, fitted, _ = fit_predict(X, y, X)
    return np.allclose(theta, theta_star), np.allclose(fitted, y)
```

22.5 Experiment

Hypothesis. With a correct linear conditional mean and full rank, the parameter error shrinks as n grows. With a quadratic conditional mean, prediction error on a fresh sample stays bounded away from zero.

Protocol. Case A: $x \sim \text{Uniform}[-1, 1]$, $Y = 1 + 2x + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, 0.1^2)$. Case B: $Y = x^2 + \varepsilon$. Fit an intercept and a slope. Repeat 50 times at $n \in \{32, 128, 512\}$. Hold out 200 fresh points each time.

Metric. Median $\|\hat{\theta} - \theta_\star\|_2$ in case A. Median held-out mean squared error in both cases.

Decision rule. Case A parameter error should drop roughly like $1/\sqrt{n}$. Case B test error should stop improving once the linear approximation error dominates. If case B test error goes to zero, the data generator is wrong.

22.6 Project

Deliverable. A short report with the known-case check, the collinear non-uniqueness check from the linear-algebra chapter, and the two simulation cases. Include one table and a statement of what you would refuse to interpret.

Constraints. No library linear-model object. `lstsq` or SVD only, so the numerics stay visible.

Rubric. The report says whether you are estimating a parameter, a conditional mean, or a linear approximation to a conditional mean, and it matches the evidence.

22.7 Depth tracks

Beginner. Make `known_case` pass. Fit one line to ten points you choose.

University. Run both simulation cases and write the decision paragraph.

Research. Add a heavy-tailed noise law (Student-t with 3 degrees of freedom) and compare squared error with a Huber fit you implement by reweighted least squares. Say which functional of the conditional distribution each one estimates.

22.8 Failure modes

- Reporting coefficient signs when columns are collinear.
- Quoting in-sample R^2 as if it were risk.
- Centering or scaling using the full sample before a train-test split.

22.9 See also

Volume 06 returns to this estimator as a baseline. Volume 07 shows how to compare it with another model without fooling yourself.

Part II

Volume 02. Calculus and optimization

23 Volume 02. Calculus and optimization

24 Volume 02. Calculus and optimization

Stage. Beginner moving into university

Audience. A reader who can differentiate a polynomial and has finished Volume 01.

Question this volume answers. How does a training algorithm move in parameter space, and which guarantees require convexity?

This volume is a reference, not a short tour. Each chapter opens with a card you can look up later, then gives the theory, a reference implementation, an experiment with a decision rule, and a project with a rubric. Read in order the first time. After that, enter through the reference card of the chapter that matches the problem in front of you.

24.1 How to use the four layers

Layer	What you should be able to do when you finish it
Theory	State the definition, the assumptions, and the result, including what the result does not say.
Code	Implement the core object in a few dozen lines and check it against a known case.
Experiment	Change one factor, record a metric with uncertainty, and say what outcome would change your mind.
Project	Produce an artifact someone else can rerun: code, figure, table, and a one-page lab note.

24.2 Depth tracks

Every chapter ends with three tracks.

- **Beginner.** Do the reference card, the code, and the smallest experiment. Skip optional proofs.
- **University.** Do the derivation, the full experiment, and the project rubric.
- **Research.** Read the failure modes, change an assumption, and write what would have to be true for the method to fail on a new distribution.

24.3 Chapters

No.	Chapter	File
01	Derivatives, gradients, and Jacobians	`vol02/01_gradients.qmd`
02	Taylor expansions and local models	`vol02/02_taylor.qmd`
03	Convex sets and convex functions	`vol02/03_convexity.qmd`
04	Constraints and optimality	`vol02/04_constraints.qmd`
05	Stochastic gradients	`vol02/05_stochastic_gradients.qmd`
06	Project: a gradient-checking harness	`vol02/06_gradient_check_project.qmd`

24.4 Lab note

Keep one markdown file per experiment, `notes/YYYY-MM-DD-<slug>.md`, with the hypothesis, the command you ran, the metric, and the decision. The series is a reference only if the experiments you run are attached to it.

25 Derivatives, gradients, and Jacobians

26 Derivatives, gradients, and Jacobians

Reference question. What does a derivative say, and which object do you need for a vector parameter?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

26.1 Reference card

Item	Statement
Derivative	For $f : \mathbb{R} \to \mathbb{R}$, $f'(t)$ is the slope of the best linear approximation at t.
Gradient	For $L : \mathbb{R}^d \to \mathbb{R}$, $\nabla L(\theta)$ satisfies $L(\theta + h) = L(\theta) + \nabla L(\theta)^\top h + o(\|h\|)$.
Jacobian	For $F : \mathbb{R}^d \to \mathbb{R}^k$, $J_F(\theta) \in \mathbb{R}^{k \times d}$ is the matrix of partial derivatives.
Chain	$J_{F \circ G}(\theta) = J_F(G(\theta)) J_G(\theta)$.
Descent	For small $\eta > 0$, $\theta^+ = \theta - \eta \nabla L(\theta)$ decreases L if the gradient is nonzero and L is differentiable.

26.2 Theory

A derivative is a local linear model, not a slogan about change. The gradient is the direction in parameter space in which that linear model increases fastest per unit Euclidean length. Steepest descent walks the opposite way. The step length η is part of the method. The gradient does not choose it.

When the loss depends on parameters through a vector of predictions, the Jacobian is the object that carries derivatives through the chain rule. Backpropagation computes one row of interest, the gradient of a scalar, without forming the full Jacobian. You still need to know the Jacobian exists as the thing being multiplied.

Differentiability can fail at kinks, as with a ReLU at 0 or an absolute value. Subgradients are a later, convex repair. For now, check kinks explicitly or use a smooth surrogate when you need a classical gradient.

26.3 Derivation

Let $L(\theta + h) = L(\theta) + g^\top h + r(h)$ with $r(h)/\|h\| \to 0$. Then

$$L(\theta - \eta g) = L(\theta) - \eta \|g\|_2^2 + r(-\eta g).$$

Choose η small enough that $|r(-\eta g)| \le (\eta/2)\|g\|_2^2$. If $g \neq 0$, the right-hand side is at most $L(\theta) - (\eta/2)\|g\|_2^2$. So some positive step decreases the loss. This argument does not say that a constant step works for every η, and it does not say the iteration converges to a global minimizer.

26.4 Code

```
import numpy as np

def jacobian_rows(f, theta, eps=1e-6):
    base = f(theta)
    rows = []
    for j in range(theta.size):
        step = np.zeros_like(theta, dtype=float)
        step[j] = eps
        rows.append((f(theta + step) - base) / eps)
    return np.vstack(rows)
```

26.5 Experiment

Hypothesis. Forward differences recover the Jacobian of $F(\theta) = (\theta_1^2, \theta_1\theta_2)$ at $(2, -1)$, whose exact Jacobian is $\begin{bmatrix} 2\theta_1 & 0 \\ \theta_2 & \theta_1 \end{bmatrix}$.

Protocol. Compare `jacobian_rows` at three step sizes.

Metric. Max absolute entrywise error.

Decision rule. Error should fall as `eps` shrinks, until float noise dominates. A systematic factor-of-two error means an index bug, not a step-size issue.

26.6 Project

Deliverable. A function `descend(L, grad, theta0, eta, steps)` that records the loss every step, plus a plot or table for $L(\theta) = \theta^\top \theta$ and for $L(\theta) = \theta^4$ in one dimension, where a large constant step oscillates.

Constraints. Refuse a non-finite loss and stop.

Rubric. You can point to a step size that decreases L and one that does not, on the second function.

26.7 Depth tracks

Beginner. Differentiate three scalar functions by hand and match them with central differences.

University. Write the descent inequality with the little-o remainder in your own words.

Research. Give an example of a differentiable function, bounded below, whose gradient descent with any fixed step fails to approach the infimum. Start from a function with increasingly narrow valleys if you need a picture, and then look up a standard counterexample before claiming one.

26.8 Failure modes

- Calling a Jacobian a gradient.
- Using a step size copied from another model.
- Ignoring non-finite values after a too-large step.

26.9 See also

The stochastic-gradient chapter later in this volume. Volume 10 for practical step-size schedules.

27 Taylor expansions and local models

28 Taylor expansions and local models

Reference question. What local polynomial is a training step actually using?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

28.1 Reference card

Item	Statement
First order	$L(\theta + h) \approx L(\theta) + \nabla L(\theta)^\top h$.
Second order	Add $\frac{1}{2} h^\top H(\theta) h$, where $H_{ij} = \partial_i \partial_j L(\theta)$.
PSD Hessian	If $H(\theta) \succeq 0$, the quadratic model is convex in h.
Newton	If H is SPD, the quadratic model is minimized at $h = -H^{-1} \nabla L$.
Use	First order is cheap and global only by repetition. Second order is local and expensive.

28.2 Theory

Gradient descent trusts a linear model of the loss and then takes a short step so that the trust is not absurd. Newton's method trusts a quadratic model and jumps to that model's minimizer. When the Hessian is positive definite and the quadratic approximation is accurate, Newton can be dramatically faster. When the Hessian is indefinite, the quadratic model has no minimum, and the Newton step can climb.

In high dimension the Hessian is $d \times d$. For a network with millions of parameters you do not form it. You still use the Taylor picture to understand curvature: a learning rate that is safe in a flat direction can explode in a sharp one. Adaptive methods and normalization are attempts to rescale those directions without forming H.

28.3 Derivation

The second-order expansion with remainder says

$$L(\theta + h) = L(\theta) + g^\top h + \tfrac{1}{2} h^\top H(\theta) h + r_3(h),$$

where $r_3(h)/\|h\|^2 \to 0$ as $h \to 0$ if third derivatives exist and are continuous. Drop r_3 and differentiate in h: $g + Hh = 0$. If H is invertible, $h = -H^{-1} g$.

If H has an eigenvalue $\lambda < 0$ with eigenvector u, then along $h = tu$ the quadratic term is $(\lambda/2)t^2 \to -\infty$ as $|t|$ grows. The local model is then not a bowl.

28.4 Code

```
import numpy as np

def newton_step(grad, hess):
    return -np.linalg.solve(hess, grad)

def quadratic_bowl():
    # L(theta) = 0.5 * theta^T diag(1, 100) theta
    g = np.array([0.2, -4.0])
    H = np.diag([1.0, 100.0])
    return newton_step(g, H)
```

28.5 Experiment

Hypothesis. On a quadratic bowl, one Newton step from any start reaches the minimizer at the origin, while gradient descent with a constant step that is safe for the sharp axis is slow on the flat axis.

Protocol. Take $L(\theta) = \frac{1}{2}(\theta_1^2 + 100\theta_2^2)$. Start at $(1, 1)$. Run 50 gradient steps at $\eta = 1/100$ and one Newton step.

Metric. Distance to the origin after one Newton step, and after 50 gradient steps.

Decision rule. Newton should land within 10^{-8}. Gradient descent should still have a visible θ_1 component. If Newton does not land, `solve` was applied to the wrong Hessian.

28.6 Project

Deliverable. A contour or a table of the bowl, the Newton step, and a divergent gradient step at $\eta = 0.1$.

Constraints. Annotate the sharp and flat axes with their curvatures 100 and 1.

Rubric. A reader can see why one learning rate is not a property of the loss alone.

28.7 Depth tracks

Beginner. Expand e^t and $\log(1 + t)$ through second order and name the remainder.

University. Derive the Newton step and run the bowl experiment.

Research. Replace the Hessian by a diagonal approximation and measure the extra iterations on this bowl. This is a toy version of the compromise adaptive optimizers make.

28.8 Failure modes

- Using Newton when H is indefinite and treating the step as progress.
- Reading a local quadratic decrease as a global convergence proof.
- Forming a dense Hessian for a model whose d makes that matrix larger than memory.

28.9 See also

Volume 10 discusses curvature without forming H. Convex functions in the next chapter are where the local bowl is global.

29 Convex sets and convex functions

30 Convex sets and convex functions

Reference question. When is every local minimizer a global one?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

30.1 Reference card

Item	Statement
Set	C is convex if $x, y \in C$ and $t \in [0,1]$ imply $tx + (1-t)y \in C$.
Function	f is convex when $\operatorname{epi} f = \{(x, r) : r \geq f(x)\}$ is convex.
Inequality	$f(tx + (1-t)y) \leq tf(x) + (1-t)f(y)$.
Smooth	Differentiable f is convex iff $f(y) \geq f(x) + \nabla f(x)^\top (y - x)$.
Strong	$f(y) \geq f(x) + \nabla f(x)^\top (y-x) + (\mu/2)\|y-x\|_2^2$ for some $\mu > 0$.
Payoff	A local minimum of a convex function on a convex set is global. Strict convexity gives uniqueness.

30.2 Theory

Convexity is the assumption that makes optimization theory look like a guarantee rather than a heuristic. Linear least squares with a convex constraint, logistic regression, and ridge regression are convex in the parameters when the predictor is linear in those parameters. Deep networks are not convex in their weights. Carrying a convex guarantee into a nonconvex training run is a category error.

The first-order inequality says the graph sits above its tangents. That is why a point with $\nabla f(x) = 0$ is a global minimizer when f is convex and differentiable: every other point is at least $f(x)$. Without convexity, a zero gradient might be a saddle or a poor local minimum.

Strong convexity adds a quadratic bowl under the function and gives a unique minimizer and a linear rate for gradient descent with a suitable step. Squared error with a full-rank design and an L2 penalty is the prototype.

30.3 Derivation

Let f be convex and differentiable, and suppose $\nabla f(x_\star) = 0$. The tangent inequality at $x_\star$ says $f(y) \geq f(x_\star)$ for every y. So $x_\star$ is a global minimizer.

Conversely, if $x_\star$ is a local minimizer and f is convex, take any y and any small $t > 0$ so that $x_t = (1-t)x_\star + ty$ lies in the local neighborhood. Then $f(x_\star) \leq f(x_t) \leq (1-t)f(x_\star) + tf(y)$, which rearranges to $f(x_\star) \leq f(y)$. A local minimizer is global.

For $f(\theta) = \frac{1}{2}\|X\theta - y\|_2^2 + (\lambda/2)\|\theta\|_2^2$ with $\lambda > 0$, the Hessian is $X^\top X + \lambda I \succeq \lambda I$, so f is strongly convex with modulus λ.

30.4 Code

```
import numpy as np

def ridge_objective(theta, X, y, lam):
    resid = X @ theta - y
    return 0.5 * np.dot(resid, resid) + 0.5 * lam * np.dot(theta, theta)

def is_above_tangent(f, grad, x, y):
    return f(y) + 1e-8 >= f(x) + np.dot(grad(x), y - x)
```

30.5 Experiment

Hypothesis. The ridge objective lies above its tangents. The nonconvex function $f(t) = t^4 - t^2$ does not, at $t = 0$, where the derivative is zero and nearby values are negative.

Protocol. Check the tangent inequality for ridge at 20 random pairs (x, y) in $\mathbb{R}^5$. Check $f(0.1)$ against the tangent of f at 0.

Metric. The smallest slack $f(y) - f(x) - \nabla f(x)^\top (y - x)$ for ridge, and the slack for the scalar nonconvex example.

Decision rule. Ridge slacks should be nonnegative up to 10^{-8}. The scalar slack at 0.1 should be negative. If ridge fails, the gradient of the objective is inconsistent with the function.

30.6 Project

Deliverable. A note that classifies least squares, ridge, logistic regression with linear logits, and a one-hidden-layer network as convex or not convex in their parameters, with a one-line reason each.

Constraints. "Neural networks can have local minima" is not enough. Point at the composition of a nonlinear activation with a linear map.

Rubric. Each classification names the variable of convexity. A problem can be convex in θ and nonconvex in a feature transformation you also learn.

30.7 Depth tracks

Beginner. Draw a convex function and a function with two basins. Mark a point where the tangent inequality fails.

University. Prove the local-to-global argument in your own words.

Research. Show that strong convexity plus smoothness L implies that gradient descent with step $1/L$ contracts the distance to the minimizer by the factor $1 - \mu/L$. State where a deep network violates the hypothesis.

30.8 Failure modes

- Calling a loss convex because the log loss is convex, after composing it with a nonlinear network.
- Using uniqueness of the prediction to claim uniqueness of the parameter.
- Importing linear convergence rates into nonconvex training logs.

30.9 See also

Volume 06 for logistic regression, which is convex in the linear weights. Volume 09 for the point where convexity is given up on purpose.

31 Constraints and optimality

32 Constraints and optimality

Reference question. What changes when the parameter must lie in a set, such as a simplex or a box?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

32.1 Reference card

Item	Statement
Problem	Minimize $f(\theta)$ subject to $\theta \in C$, with C convex.
Projection	$\Pi_C(z) = \arg\min_{\theta \in C} \lVert \theta - z \rVert_2^2$ when the minimizer exists and is unique.
Projected step	$\theta^+ = \Pi_C(\theta - \eta \nabla f(\theta))$.
Simplex	Nonnegative vectors summing to 1. Projection is a standard algorithm, not a softmax.
Stationarity	For differentiable convex f, $\theta_\star$ is optimal iff $\nabla f(\theta_\star)^\top (\theta - \theta_\star) \geq 0$ for every $\theta \in C$.

32.2 Theory

Unconstrained stationarity is $\nabla f = 0$. On a constraint set, the negative gradient may point outside C. The right first-order test is that no feasible direction is a descent direction. Projected gradient descent follows the gradient and then returns to C by Euclidean projection.

Softmax outputs lie in the simplex, but softmax is a parameterization of an unconstrained vector, not a projection. Projection onto the simplex is what you want if the parameter itself must be a probability vector, as in some mixture weights or discrete optimal-transport problems. Box constraints $\theta_j \in [0, 1]$ project by clipping. Clipping is not a general substitute for projection onto other sets.

KKT conditions add multipliers for equality and inequality constraints. You need them when you want a certificate, dual variables, or a Lagrangian method. For many ML constraints, projection is the algorithm and the variational inequality above is the check.

32.3 Derivation

Let f be convex and differentiable. Suppose $\nabla f(\theta_\star)^\top(\theta - \theta_\star) \geq 0$ for all $\theta \in C$. Convexity gives $f(\theta) \geq f(\theta_\star) + \nabla f(\theta_\star)^\top(\theta - \theta_\star) \geq f(\theta_\star)$. So $\theta_\star$ is optimal.

If $\theta_\star$ is interior to C, every small direction is feasible, which forces $\nabla f(\theta_\star) = 0$. The unconstrained test is the special case $C = \mathbb{R}^d$.

Euclidean projection onto the unit box $[0, 1]^d$ is coordinatewise clipping, because the squared distance separates across coordinates. Projection onto the probability simplex does not separate in that naive way once the sum constraint binds.

32.4 Code

```
import numpy as np

def project_box(theta, low=0.0, high=1.0):
    return np.clip(theta, low, high)

def projected_step(theta, grad, eta):
    return project_box(theta - eta * grad)
```

32.5 Experiment

Hypothesis. Minimizing $(\theta - 2)^2$ on $[0, 1]$ by projected gradient descent ends at 1, and the variational inequality holds there even though the derivative is not zero.

Protocol. Start at 0.2, use $\eta = 0.1$, run 50 steps of `projected_step` with gradient $2(\theta - 2)$. Evaluate $\nabla f(1) \cdot (\theta - 1)$ for 20 random $\theta \in [0, 1]$.

Metric. Final θ, and the most negative directional derivative you find inside the interval.

Decision rule. The final value should be 1, and every feasible directional derivative should be nonnegative. A final value of 2 means you forgot to project.

32.6 Project

Deliverable. Projected gradient descent for nonnegative least squares: $\min_{\theta \geq 0} \|X\theta - y\|_2^2$, with clipping as the projection.

Constraints. Compare with unconstrained least squares on a problem whose true $\theta_\star$ has a negative coordinate.

Rubric. You report both the objective value and the constraint violation, which must be zero for the projected run.

32.7 Depth tracks

Beginner. Project ten random numbers onto $[0, 1]$ and onto the nonnegative reals.

University. Prove the box-constrained optimality argument for $(\theta - 2)^2$ on $[0, 1]$.

Research. Implement projection onto the probability simplex and test it on a 5-dimensional vector. Use the variational inequality as the unit test.

32.8 Failure modes

- Using softmax and calling it a Euclidean projection.
- Checking only $\nabla f = 0$ on a problem whose solution is on the boundary.
- Projecting after every epoch of an algorithm whose theory assumed an unconstrained step, and still quoting the unconstrained rate.

32.9 See also

Volume 08 for mixture weights. Volume 14 if you later parameterize a policy on a simplex.

33 Stochastic gradients

34 Stochastic gradients

Reference question. What remains true when each step uses a minibatch instead of the full gradient?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

34.1 Reference card

Item	Statement
Full gradient	$g(\theta) = \nabla \hat{R}_n(\theta) = n^{-1} \sum_{i=1}^{n} \nabla \ell_i(\theta)$.
Unbiased minibatch	If I is a random index, uniform on $1, \dots, n$, then $\mathbb{E}[\nabla \ell_I(\theta) \mid \theta] = g(\theta)$.
Noise	The step is $g(\theta) + \xi$ with $\mathbb{E}[\xi \mid \theta] = 0$. Variance need not be zero.
Step sizes	A constant step can hover. Steps with $\sum \eta_t = \infty$ and $\sum \eta_t^2 < \infty$ are the classical convergent schedule for noisy gradients.
Batch	Averaging B independent indices cuts the variance by about B, at B times the cost per step.

34.2 Theory

Empirical risk is an average. Its gradient is an average of per-example gradients. A minibatch gradient is a Monte Carlo estimate of that average. Unbiasedness says the estimate points the right way on average. It does not say a given step descends.

Variance is why training curves jitter and why a learning rate that worked on the full gradient can diverge on batch size 1. Increasing the batch reduces variance and raises the cost of each step. The useful comparison is loss decrease per unit of compute, not per iteration.

A constant step size on a strongly convex smooth objective converges to a neighborhood whose radius shrinks if you later decrease the step. People anneal learning rates for this reason, not only for folklore. In nonconvex deep learning the same update is used without the guarantee. You still diagnose it as noisy descent: if the batch gradient is systematically

biased, by a bug in shuffling or by dropout left in the wrong mode, the theory you think you are using is not the algorithm you are running.

34.3 Derivation

Let ℓ_i be the loss on example i and $g_B = B^{-1} \sum_{b=1}^{B} \nabla \ell_{I_b}(\theta)$ with I_b drawn uniformly with replacement. Linearity of expectation gives $\mathbb{E}[g_B \mid \theta] = g(\theta)$. If the per-example gradients have covariance bounded by $\sigma^2 I$ and the draws are independent,

$$\mathbb{E}[\|g_B - g(\theta)\|_2^2 \mid \theta] \le \frac{C\sigma^2}{B}$$

for a constant C that depends only on dimension in the crude bound. The $1/B$ scaling is the part to remember. It is a variance calculation, not a claim about generalization to new data. The batch is still drawn from the training set.

34.4 Code

```
import numpy as np

def full_grad(theta, X, y):
    return 2.0 * X.T @ (X @ theta - y) / X.shape[0]

def minibatch_grad(theta, X, y, batch, rng):
    idx = rng.integers(0, X.shape[0], size=batch)
    return full_grad(theta, X[idx], y[idx])
```

34.5 Experiment

Hypothesis. On a fixed θ, the mean of minibatch gradients approaches the full gradient, and the empirical variance of the gradient norm scales like $1/B$.

Protocol. Draw a least-squares problem with $n = 2000$, $d = 5$. At one θ, draw 200 minibatches for $B \in \{1, 8, 32\}$.

Metric. Distance between the average minibatch gradient and the full gradient. Sample variance of $\|g_B\|_2$.

Decision rule. The mean gap should shrink as you average more batches. Variance should drop by roughly 4 when B grows by 4. A bias that does not shrink means the index sampler is not uniform.

34.6 Project

Deliverable. Stochastic gradient descent for least squares with batch sizes 1, 16, and n, same number of data touches, and a plot of training objective against data touches.

Constraints. Use a decreasing step $\eta_t = \eta_0/\sqrt{t}$ and also a constant step. Report both.

Rubric. You can say which schedule reached a lower objective per data touch on this convex problem, and you do not claim the same ranking for a neural network.

34.7 Depth tracks

Beginner. At one parameter, average 100 single-example gradients and compare with the full gradient.

University. Run the variance experiment and the data-touch project.

Research. Derive a neighborhood bound for constant-step SGD on a μ-strongly convex, L-smooth objective in terms of η and σ^2. Identify which hypothesis a deep network drops.

34.8 Failure modes

- Confusing minibatch noise with test-set uncertainty.
- Forgetting to turn off dropout or data augmentation when you need the training gradient of a fixed loss.
- Comparing optimizers at equal iteration counts and unequal amounts of data read.

34.9 See also

Volume 10 for Adam and schedules. Volume 07 for the difference between training loss and risk.

35 Project: a gradient-checking harness

36 Project: a gradient-checking harness

Reference question. How do you certify a derivative before you trust a training run?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

36.1 Reference card

Item	Statement
Check	Compare an analytic gradient with a central difference on a tiny problem.
Central	$\partial_j L(\theta) \approx (L(\theta + \varepsilon e_j) - L(\theta - \varepsilon e_j))/(2\varepsilon)$.
Relative gap	$\|g_a - g_n\|/(1 + \|g_a\|)$, not a raw difference.
Where	On a one-example or tiny-batch problem in float64, with a deterministic loss.
Not a proof	Agreement at one point does not prove the code is right at every point.

36.2 Theory

A training run can decrease a loss for the wrong reason, or fail because the gradient is transposed. The cheap insurance is a finite-difference check on a problem small enough that d extra loss evaluations are irrelevant. Central differences cancel the even part of the Taylor error and are worth the extra evaluation.

The harness belongs in the repository next to the model, and it should fail loudly. Relative error protects you when the gradient is large. You also test a point where the gradient is supposed to be zero, and a point where a constraint or a kink makes the classical gradient the wrong object. ReLU at exactly zero is a bad place to demand agreement unless you define the subgradient you implemented.

This project is the template for every later model you write by hand. Library autodiff still deserves a check against a hand derivative on one layer, because the bug is often in the loss reduction, the mask, or the parameter you forgot to register.

36.3 Derivation

Taylor expansion gives

$$L(\theta + \varepsilon e_j) = L + \varepsilon g_j + \tfrac{1}{2}\varepsilon^2 H_{jj} + O(\varepsilon^3),$$

$$L(\theta - \varepsilon e_j) = L - \varepsilon g_j + \tfrac{1}{2}\varepsilon^2 H_{jj} + O(\varepsilon^3).$$

Subtract and divide by 2ε: the constant and the second-order term cancel, and the leading error is $O(\varepsilon^2)$. Forward differences leave an $O(\varepsilon)$ error. That is why the experiment should show central differences winning as ε decreases, until roundoff, which is about $\text{eps}_{\text{mach}}/\varepsilon$ in the subtraction, starts to grow.

36.4 Code

```
import numpy as np

def central_grad(loss, theta, eps=1e-6):
    grad = np.zeros_like(theta, dtype=float)
    for j in range(theta.size):
        step = np.zeros_like(theta, dtype=float)
        step[j] = eps
        grad[j] = (loss(theta + step) - loss(theta - step)) / (2.0 * eps)
    return grad

def relative_gap(a, b):
    return float(np.linalg.norm(a - b) / (1.0 + np.linalg.norm(a)))
```

36.5 Experiment

Hypothesis. For $L(\theta) = \|X\theta - y\|_2^2$, the relative gap between $2X^\top(X\theta - y)$ and the central difference is smallest at a moderate ε and worse for both very large and very small ε.

Protocol. Use $n = 8$, $d = 3$, float64, and $\varepsilon \in \{10^{-2}, 10^{-4}, 10^{-6}, 10^{-8}\}$.

Metric. Relative gap at each ε.

Decision rule. 10^{-5} to 10^{-6} should be clearly better than 10^{-2}. If every ε is bad, the analytic formula is wrong. If only the smallest ε is bad, you are seeing roundoff, and the formula may still be right.

36.6 Project

Deliverable. A harness that registers several losses: a quadratic, least squares, and logistic loss on a linear logit (you may implement the logistic gradient from $\sigma(x^\top \theta) - y$ times x). The harness prints a table of relative gaps and exits with a nonzero status if any gap at $\varepsilon = 10^{-6}$ exceeds 10^{-6}.

Constraints. Include one deliberately wrong gradient and show that the harness catches it.

Rubric. The command is one line, the table is saved, and the failure case is documented in the lab note.

36.7 Depth tracks

Beginner. Check $L(\theta) = \theta_1^2 + 3\theta_2$ only.

University. Complete the harness including the intentional bug.

Research. Add a complex-step or a two-sided check on a matrix-shaped parameter using the Frobenius pairing, and document the kink you refuse to test with a classical gradient.

36.8 Failure modes

- Testing in float32 and treating noise as a bug.
- Checking only shapes, not values.
- Disabling the harness in the training configuration and never turning it back on.

36.9 See also

Volume 09, where this harness is the acceptance test for a hand-written backward pass.

Part III

Volume 03. Computation for experiments

37 Volume 03. Computation for experiments

38 Volume 03. Computation for experiments

Stage. Beginner

Audience. A reader who can write a Python function and has the least-squares project from Volume 01.

Question this volume answers. How do you turn a formula into an experiment that still means the same thing next month?

This volume is a reference, not a short tour. Each chapter opens with a card you can look up later, then gives the theory, a reference implementation, an experiment with a decision rule, and a project with a rubric. Read in order the first time. After that, enter through the reference card of the chapter that matches the problem in front of you.

38.1 How to use the four layers

Layer	What you should be able to do when you finish it
Theory	State the definition, the assumptions, and the result, including what the result does not say.
Code	Implement the core object in a few dozen lines and check it against a known case.
Experiment	Change one factor, record a metric with uncertainty, and say what outcome would change your mind.
Project	Produce an artifact someone else can rerun: code, figure, table, and a one-page lab note.

38.2 Depth tracks

Every chapter ends with three tracks.

- **Beginner.** Do the reference card, the code, and the smallest experiment. Skip optional proofs.
- **University.** Do the derivation, the full experiment, and the project rubric.

- **Research.** Read the failure modes, change an assumption, and write what would have to be true for the method to fail on a new distribution.

38.3 Chapters

No.	Chapter	File
01	Python for reference implementations	`vol03/01_python_for_reference.qmd`
02	NumPy as a mathematical instrument	`vol03/02_numpy.qmd`
03	Data contracts	`vol03/03_data_contracts.qmd`
04	Figures, tables, and claims	`vol03/04_reporting.qmd`
05	Reproducible experiment folders	`vol03/05_experiment_folders.qmd`
06	Project: audit a dataset	`vol03/06_dataset_audit.qmd`

38.4 Lab note

Keep one markdown file per experiment, `notes/YYYY-MM-DD-<slug>.md`, with the hypothesis, the command you ran, the metric, and the decision. The series is a reference only if the experiments you run are attached to it.

39 Python for reference implementations

40 Python for reference implementations

Reference question. What coding habits keep a reference implementation checkable?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

40.1 Reference card

Item	Statement
Shape	Every public function documents the shape and dtype of each array.
Pure core	The mathematical step does not read files or global random state.
Generator	Randomness enters through an explicit `numpy.random.Generator`.
Check	A known case with an exact answer runs before any real data.
Failure	Non-finite values raise. They are not plotted as a quiet gap.

40.2 Theory

A reference implementation is an argument you can execute. If the mathematical step secretly reads a global seed, a previous cell's variable, or a file path, the argument cannot be replayed. Separate three layers: a pure function from arrays to arrays, a driver that builds those arrays and owns the generator, and a reporter that writes the table you will put in the lab note.

Shapes are part of the type. A function that accepts both row and column vectors by guessing will eventually align the wrong axis and still return a finite number. Require (n, d) for a design matrix and $(n,)$ for a target. Convert once at the boundary.

Determinism is a choice about the driver, not about the science. You fix a seed so a bug is reproducible. You vary the seed when the experiment asks about uncertainty. Those are different runs and should have different note names.

40.3 Derivation

Let f be the pure step and $D(s)$ the dataset drawn from seed s. A known-case test is a pair $(D, y^\star)$ with $f(D) = y^\star$ in exact arithmetic, and $\|f(D) - y^\star\| \le \tau$ in float64 for a tolerance τ you write down. If this fails, do not start the experiment. The experiment's metric is then a function $m(f(D(s)))$ of an explicit seed, so the map $s \mapsto m$ is a function you can average.

40.4 Code

```
import numpy as np

def as_design(X, y):
    X = np.asarray(X, dtype=np.float64)
    y = np.asarray(y, dtype=np.float64).reshape(-1)
    if X.ndim != 2 or X.shape[0] != y.shape[0]:
        raise ValueError(f"expected X (n, d) and y (n,), got {X.shape} and {y.shape}")
    if not np.isfinite(X).all() or not np.isfinite(y).all():
        raise ValueError("non-finite values")
    return X, y

def rng_from(seed: int) -> np.random.Generator:
    return np.random.default_rng(seed)
```

40.5 Experiment

Hypothesis. A shape check catches a row-vector target before a silent broadcast inflates the loss.

Protocol. Call `as_design` with a target of shape `(1, n)` and with a matching `(n,)` target. Then compute a mean squared error only on the path that passes the check, using a one-column design whose correct residual you know.

Metric. Whether the bad shape raises, and whether the good path matches the hand-computed error.

Decision rule. Both must hold. A warning instead of an exception is a failed check, because a warning does not stop the experiment.

40.6 Project

Deliverable. A one-file template `experiment.py` with `pure_step`, `known_case`, `run(seed)`, and `main` writing `notes/result.json`.

Constraints. No global `np.random.seed`. The seed is an argument.

Rubric. Running the file twice with the same seed writes the same JSON. A wrong shape fails before any metric is printed.

40.7 Depth tracks

Beginner. Wrap least squares from Volume 01 in `as_design` and `known_case`.

University. Make the JSON contain the seed, the metric, and the tolerance.

Research. Add a second backend for the same pure function and require both known cases to agree. This is the pattern you will want when a later chapter has a NumPy path and a library path.

40.8 Failure modes

- Sharing one Generator across unrelated experiments without recording the draw order.
- Catching exceptions and returning `NaN` so the table still fills.
- Testing only the happy shape.

40.9 See also

The dataset-audit project at the end of this volume. Volume 07 for splits that must happen inside `run(seed)`, not before it by hand.

41 NumPy as a mathematical instrument

42 NumPy as a mathematical instrument

Reference question. Which array operations match the linear algebra in Volume 01, and which ones secretly broadcast?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

42.1 Reference card

Item	Statement
Vector	Shape `(d,)`. `x @ y` is a dot product.
Design	Shape `(n, d)`. Row i is example i. `X @ theta` has shape `(n,)`.
Batch matmul	`A @ B` on shapes `(b, n, d)` and `(b, d, k)` maps each batch item.
Reduce	`mean(axis=0)` averages examples. `axis=1` averages features. The axis you reduce disappears.
Broadcast	A `(n, 1)` array and a `(d,)` array become `(n, d)`. That is often an accident.

42.2 Theory

NumPy is the computational form of the notation sheet. The dangerous feature is broadcasting: arrays with different shapes are stretched so an operation can continue. Stretching is the right tool for adding a bias of shape `(d,)` to predictions of shape `(n, d)`. It is the wrong tool when a residual of shape `(n, 1)` meets a residual of shape `(n,)` and you think you computed n squared errors rather than an $n \times n$ outer difference.

Write the shape beside the line in reference code until the operation is boring. Reductions must name the axis. A default `sum` over every entry hides whether you averaged over the batch. In later deep-learning code the same bug is a mean over pixels that was supposed to be a mean over the batch only.

42.3 Derivation

Let $X \in \mathbb{R}^{n \times d}$ and $\theta \in \mathbb{R}^d$. The ith coordinate of $X\theta$ is $\sum_{j=1}^{d} X_{ij}\theta_j$. In NumPy, `X @ theta` performs those n dots. The Gram matrix `X.T @ X` has entries $\sum_{i=1}^{n} X_{ij}X_{ik}$. If you instead write `X.T * X` you ask for an elementwise product and the shapes `(d, n)` and `(n, d)` do not match. The error is a gift. A silent wrong answer would be worse, and broadcasting is how silent wrong answers happen when one side is a vector.

42.4 Code

```
import numpy as np

def predictions(X, theta, bias):
    X = np.asarray(X, dtype=np.float64)
    theta = np.asarray(theta, dtype=np.float64).reshape(-1)
    if X.ndim != 2 or theta.shape != (X.shape[1],):
        raise ValueError("X (n, d), theta (d,)")
    return X @ theta + np.asarray(bias, dtype=np.float64)
```

42.5 Experiment

Hypothesis. Subtracting a column vector of shape `(n, 1)` from a prediction of shape `(n,)` builds an `(n, n)` matrix, while matching shapes `(n,)` builds a length-n residual whose mean square agrees with a hand calculation.

Protocol. Use $n = 4$, a one-column X, and $\theta = 1$. Compute both residuals inside `try` blocks that print shapes. Compare the flattened mean of squares with the true mean square only for the matching shapes.

Metric. Output shape, and absolute error of the mean square on the safe path.

Decision rule. The safe path must match to 10^{-12}. If the unsafe path does not produce shape `(4, 4)`, you have a NumPy version that refused the broadcast; record that, and still keep the shape assert.

42.6 Project

Deliverable. A shape-cheat sheet of ten operations you will use (`@`, `*`, `mean`, `sum`, `reshape`, `einsum` for a Gram matrix, indexing a minibatch, stacking, concatenating features, adding a bias), each with input shapes and output shape, checked by asserts.

Constraints. At least two asserts must fail if an axis is wrong.

Rubric. You can implement $X^\top X$ with `@` and with `einsum` and get the same matrix.

42.7 Depth tracks

Beginner. Do the cheat sheet for `@`, `mean(axis=0)`, and bias add.

University. Include the unsafe broadcast experiment in the lab note.

Research. Write the same three operations as Einstein summations and state the contraction that implements a batched Jacobian-vector product. You will want that language in Volume 09.

42.8 Failure modes

- Relying on broadcasting to fix a transpose.
- Reducing over all axes by default.
- Using `matrix` objects or `np.matrix`, which change the meaning of `*`.

42.9 See also

Volume 01 for the identities these operations must match. Volume 09 for batched tensor layouts.

43 Data contracts

44 Data contracts

Reference question. What must be true of a table before a model is allowed to see it?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

44.1 Reference card

Item	Statement
Row	One observational unit. Say what that unit is.
Target	The column you would not have at decision time. It is not a feature.
Missing	A missing value is a value of the measurement process, not a quiet zero, unless you justify the fill.
Split key	If several rows share an entity, the split is by entity, not by row.
Schema	Names, dtypes, units, and an allowed range, checked in code.

44.2 Theory

Most false results in applied work are data-contract failures wearing a model's clothes. A contract is a set of predicates the table must satisfy before fitting. Units are part of the predicate: a duration in milliseconds and a duration in days are different features that can share a column name across files.

The split is part of the contract when rows are dependent. Random row splits of a patient table, a time series, or a user's clicks put the same entity on both sides and make the test score a memory test. The contract names the split key. It also names leakage columns: identifiers that determine the label, timestamps from after the decision, and aggregates computed with the test rows included.

Imputation is a model. Filling with the training median is a simple one. Fitting that median on all rows uses the test sample to define the training features. The contract says which statistics are training-only.

44.3 Derivation

Let rows be pairs (e_i, x_i, y_i) with entity e_i. A row split draws a subset S of row indices. An entity split draws a subset E of entities and sets $S = \{i : e_i \in E\}$. The test risk of an entity split estimates performance on new entities. The test risk of a row split estimates performance on new rows from a mixture that still contains seen entities. Those are different functionals of the data law. No metric can repair a split that answers the wrong question. Write the functional down before choosing the number.

44.4 Code

```
import numpy as np

def check_table(columns, required, split_key):
    missing = [name for name in required if name not in columns]
    if missing:
        raise ValueError(f"missing columns: {missing}")
    if split_key not in columns:
        raise ValueError(f"split key {split_key} not in table")
    return True

def entity_split(entities, test_frac, rng):
    entities = np.asarray(entities)
    uniq = np.unique(entities)
    rng.shuffle(uniq)
    n_test = max(1, int(round(test_frac * len(uniq))))
    test_ents = set(uniq[:n_test])
    test_mask = np.array([e in test_ents for e in entities])
    return ~test_mask, test_mask
```

44.5 Experiment

Hypothesis. A row split of repeated entities yields a lower test error than an entity split when the label is constant inside each entity.

Protocol. Create 40 entities, 8 identical rows each, label equal to a standard normal draw per entity, and a single feature equal to the label plus tiny noise. Fit least squares under both splits, 20 seeds.

Metric. Median test mean squared error for row split and for entity split.

Decision rule. The row split should look optimistically good. If it does not, the feature is not actually a near copy of the label, or the splits coincide by chance and you should increase repeats.

44.6 Project

Deliverable. A contract checker for a dataset you already have or for the synthetic entity data: required columns, finite features, split key, and a written sentence naming the observational unit.

Constraints. The checker runs before fitting and fails the process on a violation.

Rubric. You can point to one column you refused to use as a feature, and say why.

44.7 Depth tracks

Beginner. Run the synthetic entity experiment once and write one sentence on the two test errors.

University. Add a training-only median imputer and show that fitting it on all rows changes a feature on the test set.

Research. State the dependence you would need to model if entities are independent but rows inside an entity are a time series. Volume 15 returns to this as leakage in production features.

44.8 Failure modes

- Filling missing targets with zero and training anyway.
- Splitting images from the same patient across train and test.
- Using a unique identifier as a numeric feature.

44.9 See also

Volume 07 for metrics after a valid split. Volume 15 for the same contract in a pipeline.

45 Figures, tables, and claims

46 Figures, tables, and claims

Reference question. What must a figure or table contain before it can support a sentence in the lab note?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

46.1 Reference card

Item	Statement
Comparison	A number without a baseline is not a result.
Uncertainty	If you report a mean over seeds, report the spread or a interval.
Axes	Labeled units, and a title that states the claim, not the chart type.
n	The sample size is visible.
Refuse	A bar chart of a single run with no baseline does not enter the note.

46.2 Theory

A reference book trains judgment about claims. The claim is a sentence with a verb: this method has lower held-out squared error than least squares under this split. The table is the evidence. Decoration is not evidence. A figure that omits the baseline invites the reader to treat a loss of 0.31 as good or bad by mood.

Uncertainty from a finite test set and uncertainty from the training procedure are different. A fixed fitted model still has a test-set standard error if the test points are random. A model refit on new seeds has additional spread. Say which one you computed. Averaging ten seeds and reporting three significant digits beyond the seed-to-seed scatter is a false precision.

Tables beat figures when the reader needs the number. Figures beat tables when the reader needs a shape: a residual plot, a calibration curve, a learning curve against sample size. One of each is enough for a project in this series.

46.3 Derivation

Suppose held-out losses $\ell_1, \dots, \ell_m$ are independent with mean μ and variance σ^2. The mean $\bar{\ell}$ has variance σ^2/m. The sample standard deviation s estimates σ, and $s/\sqrt{m}$ estimates the standard error of the mean. This interval says nothing about a new training seed. If you also have means $\bar{\ell}^{(1)}, \dots, \bar{\ell}^{(K)}$ from K training seeds, the sample standard deviation of those K numbers estimates training variability of the reported mean. Quote K and m separately.

46.4 Code

```
import numpy as np

def summarize(losses):
    losses = np.asarray(losses, dtype=np.float64).reshape(-1)
    m = losses.size
    mean = float(np.mean(losses))
    se = float(np.std(losses, ddof=1) / np.sqrt(m)) if m > 1 else np.nan
    return {"n": m, "mean": mean, "se": se}
```

46.5 Experiment

Hypothesis. Two methods with the same mean loss over seeds can have different seed-to-seed spread, and a report of means alone hides it.

Protocol. Draw method A as $\mathcal{N}(1.0, 0.01^2)$ and method B as $\mathcal{N}(1.0, 0.2^2)$, 30 seeds. Summarize each.

Metric. Mean and standard error, plus the standard deviation across seeds.

Decision rule. Means match. Spreads do not. A reporting template that prints only the mean fails this experiment and must be changed.

46.6 Project

Deliverable. A reporting snippet that writes a markdown table with columns `method`, `n_test`, `n_seeds`, `metric`, `mean`, `seed_sd`, and a baseline row.

Constraints. The function refuses to emit a row with a missing baseline flag on the table as a whole: the table must contain a row named `baseline`.

Rubric. You rebuild the Volume 01 case A versus case B comparison in this table, and the caption states the claim in one sentence.

46.7 Depth tracks

Beginner. Compute a standard error for ten coin flips of a 0-1 loss.

University. Produce the required table for the linear versus quadratic simulation.

Research. Distinguish a confidence interval for a fixed model's test risk from a prediction interval for a future seed. Write one sentence each into the note template.

46.8 Failure modes

- Truncating the y-axis of a bar chart to exaggerate a gap smaller than the standard error.
- Reporting the best seed as if it were the procedure.
- Comparing a tuned method against an untuned baseline and calling the gap architectural.

46.9 See also

Volume 07 for paired comparisons. Volume 18 for how a paper's main table is filed.

47 Reproducible experiment folders

48 Reproducible experiment folders

Reference question. What is the smallest folder layout that lets you rerun a result?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

48.1 Reference card

Item	Statement
Layout	`experiments/<id>/{config.json, run.py, metrics.json, notes.md}`.
Config	Every knob that changes the metric, including the seed and the data path.
Command	The note starts with the exact command.
Immutable	Do not edit `metrics.json` by hand.
Identity	The id is a slug plus a short hash of the config, not a date alone.

48.2 Theory

A result you cannot rerun is a rumor. The folder is the unit you will cite from later chapters. Config is the input. Metrics are the output. The note is the interpretation, and it is allowed to be wrong later; the metrics file is not edited to match a new opinion.

Hashing the config protects you from two runs that share a name and disagree. Dates are for humans. Hashes are for identity. Store the library versions next to the config when you use more than NumPy, because a metric can move when a solver changes.

This layout is intentionally smaller than a platform. You are building the habit a platform would enforce. Volume 15 can wrap the same folder in a pipeline. It should not have to invent the contents.

48.3 Derivation

Let c be the config and $h(c)$ a stable hash of its canonical JSON. Let $r(c)$ be the metric map. The folder name `<slug>-<h(c)>` is a key for $r(c)$. If you change a knob and reuse the folder, you have changed the key without changing the address, and later comparisons mix two functions. The rule "new config, new folder" is the statement that r is a function.

48.4 Code

```
import hashlib, json
from pathlib import Path

def canonical_hash(config: dict) -> str:
    raw = json.dumps(config, sort_keys=True, separators=(",", ":")).encode()
    return hashlib.sha256(raw).hexdigest()[:12]

def make_folder(root: str, slug: str, config: dict) -> Path:
    path = Path(root) / f"{slug}-{canonical_hash(config)}"
    path.mkdir(parents=True, exist_ok=True)
    (path / "config.json").write_text(json.dumps(config, indent=2, sort_keys=True), encodi
    return path
```

48.5 Experiment

Hypothesis. Two configs that differ only by seed produce different hashes and different folders. The same config produces the same hash.

Protocol. Hash `{n: 32, seed: 0}` twice and `{n: 32, seed: 1}` once.

Metric. Equality of the first two hashes, inequality with the third.

Decision rule. Both comparisons must hold. If key order in a hand-built dict changes the hash, the JSON was not canonical.

48.6 Project

Deliverable. Move the Volume 01 simulation into this folder layout. One folder per `(case, n, seed)` is acceptable; or one folder per case with a seed list stored in the config. Choose one and defend it.

Constraints. `notes.md` contains the command and the decision. `metrics.json` contains the numbers the decision uses.

Rubric. A new terminal, with only the folder, can tell what was run. Deleting `notes.md` does not destroy the numbers.

48.7 Depth tracks

Beginner. Create one folder for the known case of least squares.

University. Store the full simulation grid.

Research. Add `environment.json` with package versions and a check that fails if a rerun's versions differ. Record whether you consider that a failed reproduction or a new experiment.

48.8 Failure modes

- Overwriting `metrics.json` after peeking at the test set and changing the model.
- Putting the only copy of the result in a notebook cell you then edit.
- Hashing a config that omits the data path.

48.9 See also

Volume 18 for preregistering the metric before the run. Volume 15 for pipeline runs that should emit this same folder.

49 Project: audit a dataset

50 Project: audit a dataset

Reference question. What do you check before any model is allowed to look competent?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

50.1 Reference card

Item	Statement
Target	Type, units, missingness, and whether it is a proxy.
Split	Who the test score is about: new rows, new entities, or a later time.
Leakage	Columns that would be unavailable, or that encode the target.
Baseline	A constant predictor and a simple linear or majority predictor, on the same split.
Stop	If you cannot name the decision the prediction supports, do not tune a model yet.

50.2 Theory

An audit is a reference procedure you run every time the data source changes. It answers whether the prediction problem is well posed. A well-posed problem names the input available at decision time, the target, the population the test set represents, and a baseline a serious person already has. Models come after that.

The constant predictor is not an insult. For squared error it is the training mean, and it is the Bayes predictor when features carry no information. Any later model that cannot beat it has not used the features. For classification, the majority class is the analogous baseline. A rare-event problem can have 99 percent accuracy with a constant predictor. The audit records the base rate so that accuracy cannot launder it.

Proxies matter. Predicting a label that a human assigned from the same features you are using is not the same problem as predicting the event the human was judging. The audit says which problem the columns actually encode.

50.3 Derivation

For squared error, $\mathbb{E}[(Y - c)^2]$ is minimized at $c = \mathbb{E}[Y]$, with value $\mathrm{Var}(Y)$. On a training sample the minimizing constant is the training mean $\bar{y}$, and the training mean squared error is the sample variance with the $1/n$ normalization you chose in the loss, not necessarily the unbiased variance. On a test set you still predict $\bar{y}_{\text{train}}$, not the test mean. Using the test mean is leakage and makes the baseline look worse or better for a reason you will not have in deployment.

50.4 Code

```
import numpy as np

def constant_baseline(y_train, y_test):
    pred = float(np.mean(y_train))
    mse = float(np.mean((y_test - pred) ** 2))
    return {"prediction": pred, "test_mse": mse, "test_n": int(len(y_test))}
```

50.5 Experiment

Hypothesis. On data where the feature is independent of the target, least squares does not beat the constant baseline on a reasonably large test set by more than noise.

Protocol. Draw $n_{\text{train}} = n_{\text{test}} = 500$, feature standard normal, target standard normal, independent. Fit an intercept and a slope. Compare test MSE with `constant_baseline`. Repeat 30 seeds.

Metric. Fraction of seeds on which least squares wins, and the median difference in test MSE.

Decision rule. The fraction should be near one half and the median difference near zero. A large consistent win means the generator is coupled by accident.

50.6 Project

Deliverable. An audit note for either the synthetic entity table from the data-contract chapter or a dataset you have the rights to use. Required sections: observational unit, target definition, split, columns refused, missingness, constant baseline, one linear or majority baseline, and a go or no-go sentence.

Constraints. No model with more than a few dozen parameters. The point is the audit.

Rubric. A reader can tell what a "good" number would mean and whether your baseline already achieved it. The go or no-go sentence names the decision a prediction would support.

50.7 Depth tracks

Beginner. Audit the synthetic independent-feature data only.

University. Include the entity-split dataset and both baselines.

Research. Add a section on what would make the target a proxy, and write a second target definition that would change the decision. You do not need new data if you can argue from the measurement process.

50.8 Failure modes

- Tuning a flexible model to rescue a problem you cannot define.
- Reporting accuracy on a 95 percent base rate without the base rate.
- Computing the constant predictor from the test labels.

50.9 See also

Volume 06 will treat the linear baseline as a model class. Volume 07 will refuse metrics that ignore this audit.

Part IV

Volume 04. Probability

51 Volume 04. Probability

52 Volume 04. Probability

Stage. Beginner to university

Audience. A reader who has the notation sheet and can compute an average in NumPy.

Question this volume answers. What is a probability model claiming about data, and which distributions show up because of a theorem rather than a habit?

This volume is a reference, not a short tour. Each chapter opens with a card you can look up later, then gives the theory, a reference implementation, an experiment with a decision rule, and a project with a rubric. Read in order the first time. After that, enter through the reference card of the chapter that matches the problem in front of you.

52.1 How to use the four layers

Layer	What you should be able to do when you finish it
Theory	State the definition, the assumptions, and the result, including what the result does not say.
Code	Implement the core object in a few dozen lines and check it against a known case.
Experiment	Change one factor, record a metric with uncertainty, and say what outcome would change your mind.
Project	Produce an artifact someone else can rerun: code, figure, table, and a one-page lab note.

52.2 Depth tracks

Every chapter ends with three tracks.

- **Beginner.** Do the reference card, the code, and the smallest experiment. Skip optional proofs.
- **University.** Do the derivation, the full experiment, and the project rubric.
- **Research.** Read the failure modes, change an assumption, and write what would have to be true for the method to fail on a new distribution.

52.3 Chapters

No.	Chapter	File
01	Probability, events, and random variables	`vol04/01_probability_spaces.qmd`
02	Distributions you will actually use	`vol04/02_distributions.qmd`
03	Expectation, variance, and concentration	`vol04/03_expectation_concentration.qmd`
04	Multivariate Gaussians	`vol04/04_multivariate_gaussian.qmd`
05	Conditional independence	`vol04/05_conditional_independence.qmd`
06	Project: fit a law and show the misfit	`vol04/06_distribution_project.qmd`

52.4 Lab note

Keep one markdown file per experiment, `notes/YYYY-MM-DD-<slug>.md`, with the hypothesis, the command you ran, the metric, and the decision. The series is a reference only if the experiments you run are attached to it.

53 Probability, events, and random variables

54 Probability, events, and random variables

Reference question. What has to be specified before a probability calculation is about a dataset?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

54.1 Reference card

Item	Statement
Law	A probability P on events. A random variable X pushes P forward to a law on values.
Conditional	$P(A \mid B) = P(A \cap B)/P(B)$ when $P(B) > 0$.
Independence	$P(A \cap B) = P(A)P(B)$. Pairwise independence of many events is weaker than mutual independence.
iid	Independent and identically distributed. A strong assumption, not a default about the world.
Dataset	One realization of a collection of random variables, not the law itself.

54.2 Theory

Probability in this series is a model of ignorance and of variability, written so that expectations become the risk $R(\theta)$. You choose the sample space by saying what a draw is. If a draw is a patient, two visits are not two independent draws. If a draw is a visit, generalization to a new patient is a different question. The audit in Volume 03 was already this choice.

Conditioning is how features enter. $P(Y \in \cdot \mid X = x)$ is the object a predictor is trying to match, under a loss that says what "match" means. Squared error matches the conditional mean. Absolute error matches a conditional median. A 0-1 loss matches a conditional mode, which for two classes is a threshold on the conditional probability. Using squared error on a

problem whose decision needs a probability is a later chapter; the point here is that the loss picks a functional of the conditional law.

Independence is a knife. It lets expectations of sums become sums of expectations of products only under extra conditions, and it lets variances of sums add. It is false for time series, for entities with repeated measures, and for data drawn from a shifting policy. You may still use an independence model. You must then call the risk an approximation and test the approximation, not hide it in the word "sample."

54.3 Derivation

If X and Y are independent and the expectations exist, $\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$. Proof for simple random variables follows by writing both as finite sums of indicators of a partition and factoring the joint probabilities. The same identity fails if you only assume uncorrelatedness when you replace X and Y by nonlinear functions of them. Uncorrelated means $\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$ for that pair, not for X^2 and Y^2.

Linearity of expectation does not need independence: $\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$ whenever the expectations exist. Variance of a sum adds if and only if the covariance is zero, in the two-variable case: $\operatorname{Var}(X + Y) = \operatorname{Var}(X) + \operatorname{Var}(Y) + 2\operatorname{Cov}(X, Y)$.

54.4 Code

```
import numpy as np

def monte_carlo_mean(sampler, n, rng):
    draws = np.asarray([sampler(rng) for _ in range(n)], dtype=np.float64)
    return float(np.mean(draws)), float(np.std(draws, ddof=1) / np.sqrt(n))
```

54.5 Experiment

Hypothesis. The sample mean of a fair Bernoulli(1/2) concentrates, and the standard error $s/\sqrt{n}$ tracks the known standard deviation 1/2 divided by $\sqrt{n}$.

Protocol. Draw 2000 independent Bernoulli trials. At $n \in \{50, 200, 2000\}$, compute the mean of the prefix and the estimated standard error. Compare with 0.5 and with $0.5/\sqrt{n}$.

Metric. Absolute error of the mean, and the ratio of estimated standard error to the true one.

Decision rule. The mean error should be on the order of the true standard error, not systematically 0.1. A standard-error ratio far from 1 at $n = 2000$ means the `ddof` or the formula is wrong.

54.6 Project

Deliverable. A note that defines the sample space for a prediction problem you care about, states whether rows are iid, and computes one expectation by Monte Carlo with a reported standard error.

Constraints. If you claim independence, name one way the real data might violate it.

Rubric. The note distinguishes the law from the dataset.

54.7 Depth tracks

Beginner. Simulate the Bernoulli mean and write the sample space in one sentence.

University. Prove linearity without independence on a two-point example, and give a pair of uncorrelated random variables that are not independent.

Research. Write the conditional risk $\mathbb{E}[\ell(f(X), Y) \mid X = x]$ for squared error and show the minimizer is the conditional mean. This is the population version of the Volume 01 project.

54.8 Failure modes

- Describing a finite dataset as a distribution without saying whether you mean the empirical measure.
- Assuming variance adds for dependent time series.
- Conditioning on an event you selected after seeing the data and still using the original P.

54.9 See also

The distribution chapter next. Volume 05 for estimators of these expectations.

55 Distributions you will actually use

56 Distributions you will actually use

Reference question. Which families earn a place because of a closure property or a limit theorem?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

56.1 Reference card

Item	Statement
Bernoulli / binomial	A bit, or a count of independent bits. Classification labels and error counts.
Categorical	One of K classes. Parameterized by a probability vector.
Poisson	Counts with variance equal to the mean, as a limit of rare binomials.
Gaussian	A limit of normalized sums, and the maximum-entropy law on $\mathbb{R}$ with fixed mean and variance.
Exponential / Laplace	A positive waiting time with a memoryless property, and a symmetric heavy-shouldered location model for absolute error.
Student-t	A Gaussian scale mixture. Heavier tails. A robustness probe, not a default noise model you skip checking.

56.2 Theory

A named distribution is a constraint. $\mathcal{N}(\mu, \sigma^2)$ says the law is determined by a mean and a variance and has light tails. If your residuals have a point mass, a skew, or outliers that dominate the variance, the name is false even if the sample mean and variance match. Matching two moments is not matching a law. A QQ plot or a tail count is the check.

The Gaussian earns its place from the central limit theorem: a standardized sum of independent, not-too-heavy random variables approaches a standard normal. Means of many roughly independent effects look Gaussian. Individual observations often do not. The Laplace

distribution earns a place because the median, not the mean, is its location estimator under absolute error, and because it is a simple heavier-tailed alternative. The Bernoulli earns a place because labels are bits. Use a family when a theorem or a measurement process points to it. Do not use it because a library constructor was nearby.

56.3 Derivation

If $Y_1, \dots, Y_n$ are independent Bernoulli(p), then $S = \sum_i Y_i$ is Binomial(n, p), because each sequence with s ones has probability $p^s(1-p)^{n-s}$ and there are $\binom{n}{s}$ of them. The mean is np and the variance is $np(1-p)$, by independence. As $n \to \infty$ and $p = \lambda/n$, $P(S = k) \to e^{-\lambda}\lambda^k/k!$. That Poisson limit is why rare counts sometimes look Poisson. It fails when the trials are dependent or the rate is not small.

Standardize a sample mean: $\sqrt{n}(\bar{Y} - \mu)/\sigma \Rightarrow \mathcal{N}(0, 1)$ under finite variance and iid sampling. This is a theorem about the mean, and it is the reason standard errors in Volume 03 look Gaussian at large n even when Y itself is Bernoulli.

56.4 Code

```
import numpy as np

def sample_laws(n, rng):
    return {
        "bernoulli": rng.binomial(1, 0.3, size=n),
        "poisson": rng.poisson(2.0, size=n),
        "gaussian": rng.normal(0.0, 1.0, size=n),
        "laplace": rng.laplace(0.0, 1.0 / np.sqrt(2.0), size=n),
        "student_t": rng.standard_t(3, size=n),
    }
```

56.5 Experiment

Hypothesis. Laplace and Student-t draws with matched variance still produce more points beyond 4 standard deviations than a Gaussian does.

Protocol. Draw $n = 20000$ from a standard normal, a Laplace scaled to variance 1, and a Student-t with 3 degrees of freedom scaled to variance 1 (the raw t_3 has variance 3, so divide by $\sqrt{3}$).

Metric. Fraction of absolute values larger than 4.

Decision rule. The Gaussian fraction should be near 6×10^{-5}. The others should be visibly larger. If they are not, the scaling is wrong and the comparison is unfair.

56.6 Project

Deliverable. A one-page card for each of the five families: story, parameters, mean, variance, and one ML loss or model that implies it.

Constraints. Include a sixth law you refuse, with the reason (for example a Gaussian for raw counts).

Rubric. Each card names a check you would run on residuals.

56.7 Depth tracks

Beginner. Simulate the tail experiment for Gaussian versus Laplace.

University. Derive the binomial variance and the Poisson limit probabilities for $k = 0, 1$.

Research. State the Lindeberg condition in words and give a case with infinite variance where the central limit theorem fails. A stable law is enough as a pointer if you do not want the full proof.

56.8 Failure modes

- Using a Gaussian likelihood for nonnegative counts and predicting negative means.
- Matching a histogram in the center and ignoring the tails that dominate squared error.
- Treating the Poisson mean-variance link as optional after you have assumed the Poisson.

56.9 See also

Volume 05 for maximum likelihood under these laws. Volume 06 for the losses they induce.

57 Expectation, variance, and concentration

58 Expectation, variance, and concentration

Reference question. How fast does an average approach its expectation, and what breaks the bound?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

58.1 Reference card

Item	Statement
Expectation	The integral of the variable against its law. Linear even without independence.
Variance	$\mathbb{E}[(X - \mathbb{E}X)^2]$. Scales as a^2 when you multiply X by a.
Chebyshev	$P(\|X - \mu\| \geq t) \leq \mathrm{Var}(X)/t^2$.
Hoeffding	For independent bounded $[0, 1]$ variables, $P(\|\bar{X} - \mu\| \geq t) \leq 2e^{-2nt^2}$.
Breaks	Dependence, unbounded heavy tails, or a mean that does not exist.

58.2 Theory

Concentration inequalities turn a sample size into a promise about an average. Chebyshev needs only a variance and is loose. Hoeffding needs bounded independent summands and gives a sub-Gaussian tail. Empirical risk is an average of losses. If losses are bounded and rows are independent, Hoeffding applies to $\hat{R}_n - R$ at a fixed θ. It does not apply uniformly over an infinite set of θ without more work. That gap is learning theory in Volume 06's neighborhood: a uniform law, not a single-average law.

If the loss is unbounded, a few outliers dominate $\hat{R}_n$ under squared error. The mean may converge, but the constant in the bound is the variance, which those outliers inflate. This is why a reported training loss on unclipped squared error can move when one row is corrected.

Dependence breaks the $1/n$ variance scaling. In a time series with strong correlation, the effective sample size can stay small as n grows. The formula $s/\sqrt{n}$ then understates uncertainty.

58.3 Derivation

Chebyshev: let $Z = (X - \mu)^2 \geq 0$. Then $\mathbb{E}[Z] \geq t^2 P(Z \geq t^2)$, which is the claim. For an iid average, $\mathrm{Var}(\bar{X}) = \sigma^2/n$, so $P(|\bar{X} - \mu| \geq t) \leq \sigma^2/(nt^2)$.

Hoeffding's inequality for independent $X_i \in [0, 1]$ can be taken as a tool here: the exponent $-2nt^2$ is the fact to remember. Solving $2e^{-2nt^2} = \delta$ for t gives $t = \sqrt{\log(2/\delta)/(2n)}$. A deviation of a few percent at $\delta = 0.05$ needs n in the hundreds or thousands, not the dozens, when the variables are order-1.

58.4 Code

```
import numpy as np

def chebyshev_bound(variance, t):
    return float(variance / t**2)

def hoeffding_bound(n, t):
    return float(2 * np.exp(-2 * n * t**2))
```

58.5 Experiment

Hypothesis. The fraction of Bernoulli averages that deviate from p by more than $t = 0.1$ is below the Hoeffding bound and well below a loose reading of the data, and it is much larger if you replace independent draws by a constant within each block of length 50.

Protocol. $p = 0.5$, $n = 200$, $t = 0.1$, 500 independent experiments. Then a dependent version: draw 4 independent Bernoulli values and repeat each 50 times.

Metric. Empirical deviation frequency versus `hoeffding_bound(200, 0.1)`.

Decision rule. The independent frequency must sit under the bound. The dependent frequency should be higher. If the independent run violates the bound often, the sampler is wrong; the bound is not the suspect first.

58.6 Project

Deliverable. A calculator that, given a bounded loss in $[0, 1]$, a tolerance t, and a failure probability δ, returns the Hoeffding sample size, plus a paragraph saying why your actual loss may not be bounded.

Constraints. Also compute the Chebyshev sample size using a variance you specify, and compare.

Rubric. You apply it to 0-1 loss and explain why squared error on an unbounded target does not get the same promise.

58.7 Depth tracks

Beginner. Compute both bounds for $n = 100$, $t = 0.1$, and simulate the independent Bernoulli case.

University. Include the block-dependence comparison.

Research. Explain, without a full proof, why a uniform bound over a finite class of size M costs an extra $\log M$ inside the Hoeffding square root. Name the failure mode when M is infinite.

58.8 Failure modes

- Quoting $s/\sqrt{n}$ for a strongly dependent series.
- Applying Hoeffding to unbounded squared error.
- Treating a fixed-θ concentration result as a guarantee for the θ you picked by looking at the data.

58.9 See also

Volume 05 for central-limit intervals, which are tighter and more assumption-hungry. Volume 07 for test-set intervals.

59 Multivariate Gaussians

60 Multivariate Gaussians

Reference question. What does a covariance matrix say, and what does it refuse to say?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

60.1 Reference card

Item	Statement
Law	$X \sim \mathcal{N}(\mu, \Sigma)$ in $\mathbb{R}^d$ has density proportional to $\exp(-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu))$ when Σ is SPD.
Affine	$AX + b$ is Gaussian with mean $A\mu + b$ and covariance $A\Sigma A^\top$.
Independence	For a Gaussian vector, uncorrelated coordinates are independent. This equivalence is special to Gaussians.
Conditional	The conditional law of some coordinates given others is Gaussian. The precision matrix encodes conditional independence.
Mahalanobis	$(x-\mu)^\top \Sigma^{-1}(x-\mu)$ is squared distance after whitening.

60.2 Theory

The multivariate Gaussian is the linear-algebra distribution. Its geometry is ellipses of constant Mahalanobis distance. PCA, linear discriminant analysis, and Gaussian graphical models are this geometry plus a decision. The covariance Σ controls spread and linear dependence. It does not control nonlinear dependence. A Gaussian cannot represent a ring, a cluster with different tail weights, or a dependence that appears only in extremes.

Whitening by $\Sigma^{-1/2}$ turns the law into spherical standard normal, when Σ is SPD. In code, a Cholesky factor is the stable square root: $\Sigma = LL^\top$ and $L^{-1}(X - \mu)$ is standard normal. Forming Σ^{-1} explicitly is the same habit Volume 01 discouraged.

If coordinates are jointly Gaussian, a zero in the precision matrix Σ^{-1} means the corresponding pair is conditionally independent given the rest. A zero in Σ itself means ordinary uncorrelatedness, hence independence, of that pair, ignoring the others. Those are different graphs. Do not read a covariance heatmap as a conditional-independence map.

60.3 Derivation

Let $Z \sim \mathcal{N}(0, I_d)$ and $X = \mu + LZ$ with $\Sigma = LL^\top$. Then $\mathbb{E}[X] = \mu$ and

$$\operatorname{Cov}(X) = \mathbb{E}[LZZ^\top L^\top] = LL^\top = \Sigma,$$

using $\mathbb{E}[ZZ^\top] = I$. This is the sampling recipe. The quadratic form in the density is $Z^\top Z = (x-\mu)^\top (LL^\top)^{-1}(x-\mu)$.

For a partition of a two-dimensional Gaussian, the conditional mean of X_1 given $X_2 = x_2$ is linear in x_2: $\mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_2)$. That linear conditional mean is why Gaussian linear regression and the multivariate Gaussian are the same calculation in different clothes.

60.4 Code

```
import numpy as np

def sample_gaussian(mu, cov, n, rng):
    mu = np.asarray(mu, dtype=np.float64)
    cov = np.asarray(cov, dtype=np.float64)
    left = np.linalg.cholesky(cov)
    z = rng.normal(size=(n, mu.shape[0]))
    return mu + z @ left.T
```

60.5 Experiment

Hypothesis. Samples from `sample_gaussian` recover the requested covariance as n grows, and a coordinate pair with covariance 0 is empirically uncorrelated.

Protocol. $\mu = 0$, $\Sigma = \begin{bmatrix} 2 & 0.5 \\ 0.5 & 1 \end{bmatrix}$, $n \in \{200, 5000\}$, one seed is enough if you also report a second seed as a check.

Metric. Frobenius norm of the difference between the sample covariance (with $1/n$) and Σ.

Decision rule. The error should drop roughly like $1/\sqrt{n}$. If the Cholesky fails, Σ was not SPD in the code, whatever the formula said.

60.6 Project

Deliverable. A whitening function using Cholesky, a round-trip test (`whiten` then `unwhiten`), and a short note on a dataset of two coordinates that are uncorrelated but dependent (sample $Z \sim \mathcal{N}(0,1)$ and set $X = Z$, $Y = Z^2 - 1$). Show the sample correlation is near 0 while a scatter plot is not a blob.

Constraints. Do not call the second pair Gaussian.

Rubric. The note says where the Gaussian implication "uncorrelated means independent" would have lied.

60.7 Depth tracks

Beginner. Sample a diagonal Gaussian and check the sample variances.

University. Derive the conditional mean formula in two dimensions and check it by binning samples.

Research. Connect a zero in the precision matrix to a conditional regression coefficient of zero. Implement the precision matrix via `lstsq` or a solver, not an explicit inverse, if you form normal equations.

60.8 Failure modes

- Estimating a $d \times d$ covariance from $n \le d$ samples and inverting it.
- Reading correlation as causation or as conditional independence.
- Using a Gaussian bottleneck layer and claiming it can represent any dependence.

60.9 See also

Volume 08 for PCA as the eigendecomposition of a covariance. Volume 05 for the Gaussian likelihood.

61 Conditional independence

62 Conditional independence

Reference question. When does a feature become irrelevant once you know some other features?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

62.1 Reference card

Item	Statement
Definition	$A \perp B \mid C$ means $P(A, B \mid C) = P(A \mid C)P(B \mid C)$ whenever the conditioning event has positive probability.
Chain	A Markov chain has $X_{t+1} \perp (X_s)_{s<t} \mid X_t$.
Classifier	If $Y \perp X_j \mid X_{-j}$, feature j adds nothing once the others are known.
Not closed	Independence of pairs does not give you independence of functions of those pairs without more structure.
Graph	A missing edge is a claim. It is not a default.

62.2 Theory

Conditional independence is the grammar of "this feature is redundant given those." It is stronger than a coefficient being small in one particular model. A coefficient can be small because of scaling, collinearity, or a penalty. Conditional independence says the conditional law of the target does not change when the feature moves, once the conditioning set is fixed.

Markov models in sequences and states in reinforcement learning are conditional-independence claims. S_{t+1} depends on the past only through S_t and the action. If the real process remembers a hidden variable, the state you wrote down is not a state. Later, a "Markov decision process" chapter will assume this. The assumption belongs here, where it is just probability.

You cannot test conditional independence by looking at a single correlation. You can test a consequence: after fitting a sufficiently rich model on X_{-j}, adding X_j should not improve held-out risk if the independence is true and the first model already captured $P(Y \mid X_{-j})$. With a weak first model, a gain from X_j may only mean the first model was incomplete.

62.3 Derivation

Let Y, X_1, X_2 be discrete. $Y \perp X_2 \mid X_1$ iff $P(Y = y \mid X_1 = x_1, X_2 = x_2) = P(Y = y \mid X_1 = x_1)$ for all values in the support. The conditional risk of any predictor that sees (X_1, X_2) then equals the conditional risk of a predictor that sees only X_1, for a loss that is a function of the conditional law, because the conditional law did not change. In particular, the Bayes risk given (X_1, X_2) equals the Bayes risk given X_1. Extra features that are conditionally independent of the label given the rest do not improve the best possible risk. They can still make a finite-sample estimate worse.

62.4 Code

```
import numpy as np

def sample_chain(n, rng):
    x1 = rng.normal(size=n)
    x2 = x1 + rng.normal(scale=0.1, size=n)
    y = x1 + rng.normal(scale=0.5, size=n)
    return x1, x2, y
```

62.5 Experiment

Hypothesis. In `sample_chain`, Y is independent of X_2 given X_1 by construction. A linear model that already has X_1 gains nothing from X_2 beyond noise. A model that sees only X_2 still predicts Y, because X_2 carries X_1.

Protocol. $n_{\text{train}} = n_{\text{test}} = 400$, 30 seeds. Compare test MSE of least squares on $(1, X_1)$, on $(1, X_1, X_2)$, and on $(1, X_2)$.

Metric. Median test MSE of the three models.

Decision rule. The first two medians should be close. The third can be worse but should beat a constant. A large gain from adding X_2 after X_1 means the code coupled Y to the noise in X_2.

62.6 Project

Deliverable. A note that states one conditional independence you believe in a problem you care about, one you do not, and the chain experiment as a positive control.

Constraints. The belief must name the conditioning set.

Rubric. You distinguish "the feature is useless alone" from "the feature is useless given the others," using the third model as the illustration.

62.7 Depth tracks

Beginner. Run the chain experiment at one seed and write the three test errors.

University. Argue from the sampling code that the conditional law of Y given (X_1, X_2) ignores X_2.

Research. Give a collider example: X_1 and X_2 independent, Y depends on both, and conditioning on Y induces dependence between X_1 and X_2. Simulate it. This is the trap in reading conditional correlations as mechanisms.

62.8 Failure modes

- Dropping a feature because its marginal correlation with Y is zero, when it matters given other features (a suppressor or a component of an interaction).
- Conditioning on a consequence and calling the induced dependence a direct effect.
- Asserting the Markov property because it makes the code easier.

62.9 See also

Volume 12 for sequences. Volume 14 for state in reinforcement learning. Volume 08 for features that add nothing once a representation is fixed.

63 Project: fit a law and show the misfit

64 Project: fit a law and show the misfit

Reference question. How do you attach a named distribution to data without pretending the name is true?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

64.1 Reference card

Item	Statement
Fit	Match the parameters the family uses, on the training sample only.
Check	A distance or a plot the family could fail: tails, zeros, skew, or a holdout log likelihood.
Baseline family	Always include a simpler or neighboring family, not only the one you like.
Decision	Use the family for a stated purpose, or refuse it.
Purpose	A Gaussian approximation to a mean can be fine when a Gaussian model of individuals is absurd.

64.2 Theory

Fitting a distribution is estimation plus a test of the constraint the family imposes. Maximum likelihood for a Gaussian is the sample mean and the sample variance (with the $1/n$ normalization). The fit can be perfect as a moment match and still a bad model of tails. The project forces a misfit display so the card does not become an identity.

Holdout log likelihood compares families on new draws from the same process. It does not prove the winner is well specified. It says which constraint predicted the new points better. If both lose to a histogram you trust, the parametric families are the wrong summary.

Separate the purpose. Using a Gaussian approximation for the sampling distribution of a mean is the central limit theorem. Using a Gaussian to simulate individual incomes or latencies is a claim about tails and support. Write the purpose in the first line of the note so a later reader does not reuse the fitted law for a purpose you did not check.

64.3 Derivation

For Y_i iid $\mathcal{N}(\mu, \sigma^2)$, the log likelihood is

$$-\tfrac{n}{2}\log(2\pi\sigma^2) - \tfrac{1}{2\sigma^2}\sum_i (Y_i - \mu)^2.$$

Maximizing in μ gives $\hat{\mu} = \bar{Y}$. Plugging in and maximizing in σ^2 gives $\hat{\sigma}^2 = n^{-1}\sum_i (Y_i - \bar{Y})^2$. The unbiased variance with $n-1$ is a different functional. Do not mix them inside a likelihood comparison. For a Laplace location family with known scale, the maximizer of the likelihood in the location is a median, because the loss being minimized is absolute error. The project should show that these two location estimates diverge when the sample is skewed.

64.4 Code

```
import numpy as np

def gaussian_mle(y):
    y = np.asarray(y, dtype=np.float64)
    mu = float(np.mean(y))
    var = float(np.mean((y - mu) ** 2))
    return mu, var

def gaussian_loglik(y, mu, var):
    y = np.asarray(y, dtype=np.float64)
    return float(-0.5 * np.sum(np.log(2 * np.pi * var) + (y - mu) ** 2 / var))
```

64.5 Experiment

Hypothesis. A Gaussian MLE fit on Laplace data assigns lower holdout log likelihood than a Laplace fit, and the reverse is true on Gaussian data.

Protocol. Fit on 500 points, evaluate on 2000 fresh points, 20 seeds, both generators, variance matched. For the Laplace fit, use the median and the mean absolute deviation from the median as the scale (a standard Laplace MLE when both parameters are free is acceptable if you implement it).

Metric. Mean holdout log likelihood by family and generator.

Decision rule. The matching family should win. If the Gaussian wins on Laplace data, the Laplace scale is wrong or the holdout is too small to see the tails.

64.6 Project

Deliverable. A note with two generators, both fitted families, a tail-fraction table, holdout log likelihoods, and a decision sentence of the form "I would use family A to simulate new individual observations" or "I would use family A only to describe the mean."

Constraints. Training points and holdout points are disjoint. Parameters are not refit on the holdout.

Rubric. The decision matches the table. A win on likelihood with a failing tail check is called a partial win, not a specification.

64.7 Depth tracks

Beginner. Fit a Gaussian to Gaussian data and report $\hat{\mu}$, $\hat{\sigma}^2$, and a holdout log likelihood.

University. Run the crossed families experiment.

Research. Add a two-component Gaussian mixture fit by a simple EM or by a library, and show a dataset where both single Gaussians lose. Keep the EM initialization in the config hash from Volume 03.

64.8 Failure modes

- Selecting the family on the same sample you use to quote the likelihood, with no holdout and no complexity penalty.
- Using the $n - 1$ variance inside a Gaussian likelihood comparison.
- Exporting a fitted individual-level Gaussian for a quantity whose support is nonnegative.

64.9 See also

Volume 05 for a fuller account of maximum likelihood. Volume 08 for mixtures.

Part V

Volume 05. Statistical inference

65 Volume 05. Statistical inference

66 Volume 05. Statistical inference

Stage. University

Audience. A reader who can compute a mean, a variance, and a holdout log likelihood.

Question this volume answers. How do you estimate a functional of an unknown law, and how do you say how wrong the estimate might be?

This volume is a reference, not a short tour. Each chapter opens with a card you can look up later, then gives the theory, a reference implementation, an experiment with a decision rule, and a project with a rubric. Read in order the first time. After that, enter through the reference card of the chapter that matches the problem in front of you.

66.1 How to use the four layers

Layer	What you should be able to do when you finish it
Theory	State the definition, the assumptions, and the result, including what the result does not say.
Code	Implement the core object in a few dozen lines and check it against a known case.
Experiment	Change one factor, record a metric with uncertainty, and say what outcome would change your mind.
Project	Produce an artifact someone else can rerun: code, figure, table, and a one-page lab note.

66.2 Depth tracks

Every chapter ends with three tracks.

- **Beginner.** Do the reference card, the code, and the smallest experiment. Skip optional proofs.
- **University.** Do the derivation, the full experiment, and the project rubric.
- **Research.** Read the failure modes, change an assumption, and write what would have to be true for the method to fail on a new distribution.

66.3 Chapters

No.	Chapter	File
01	Estimators and sampling distributions	`vol05/01_estimators.qmd`
02	Maximum likelihood	`vol05/02_maximum_likelihood.qmd`
03	Tests and intervals	`vol05/03_intervals.qmd`
04	Bayesian updating	`vol05/04_bayes.qmd`
05	Experimental design and power	`vol05/05_power.qmd`
06	Project: analyze a comparison without fooling yourself	`vol05/06_ab_project.qmd`

66.4 Lab note

Keep one markdown file per experiment, `notes/YYYY-MM-DD-<slug>.md`, with the hypothesis, the command you ran, the metric, and the decision. The series is a reference only if the experiments you run are attached to it.

67 Estimators and sampling distributions

68 Estimators and sampling distributions

Reference question. What is an estimator, as distinct from the estimate sitting in one notebook?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

68.1 Reference card

Item	Statement
Functional	A map $T(P)$ from a law to a number or a vector, such as a mean or a coefficient.
Estimator	A function of the sample, $\hat{T}_n$, intended to track $T(P)$.
Estimate	The number $\hat{T}_n$ returns on the sample you have.
Sampling law	The law of $\hat{T}_n$ when the sample is redrawn.
Bias and variance	$\mathbb{E}[\hat{T}_n] - T(P)$, and the variance of $\hat{T}_n$ under redraws.

68.2 Theory

The estimate is one draw. The estimator is the procedure. Judgments about reliability are judgments about the procedure's sampling law, not about the dignity of a single number. Unbiasedness, $\mathbb{E}[\hat{T}_n] = T(P)$, is optional. Mean squared error $\mathbb{E}[(\hat{T}_n - T(P))^2]$ equals variance plus squared bias, and a little bias can buy a lot of variance reduction. Ridge regression is that trade on purpose.

The sampling law depends on assumptions: iid draws, the form of P, and which parts of the procedure were fixed before seeing the data. A procedure that chooses a model by looking at the test set does not have the sampling law you would compute for a fixed model. Volume 07 is about freezing that procedure. Here you learn to name $T(P)$ before you name a method.

Plug-in estimators replace P by the empirical measure $P_n = n^{-1} \sum_i \delta_{Z_i}$. The sample mean is the plug-in estimator of the mean. The $1/n$ Gaussian MLE variance is the plug-in estimator

of variance. They inherit simplicity and they inherit every mistake in the empirical measure, including outliers.

68.3 Derivation

Let $\hat{\mu} = \bar{Y}$ and $\mu = \mathbb{E}[Y]$, with iid draws and finite variance σ^2. Then $\mathbb{E}[\bar{Y}] = \mu$ and $\mathrm{Var}(\bar{Y}) = \sigma^2/n$, so MSE is σ^2/n. For the plug-in variance $\hat{\sigma}^2 = n^{-1}\sum_i (Y_i - \bar{Y})^2$, a standard calculation gives $\mathbb{E}[\hat{\sigma}^2] = (1 - 1/n)\sigma^2$ under iid sampling. It is biased low. The $n-1$ version is unbiased for σ^2 and is not the Gaussian MLE. Report which functional you meant.

MSE decomposition: $\mathbb{E}[(\hat{T} - T)^2] = \mathrm{Var}(\hat{T}) + (\mathbb{E}[\hat{T}] - T)^2$ when the expectations exist. The cross term vanishes because $\hat{T} - \mathbb{E}[\hat{T}]$ has mean zero.

68.4 Code

```
import numpy as np

def plug_in_mean_var(y):
    y = np.asarray(y, dtype=np.float64)
    mu = float(np.mean(y))
    var = float(np.mean((y - mu) ** 2))
    return mu, var
```

68.5 Experiment

Hypothesis. Over repeated samples, the plug-in variance averages about $(1 - 1/n)\sigma^2$, and the $n - 1$ variance averages about σ^2.

Protocol. Gaussian data, $\sigma^2 = 4$, $n = 10$, 2000 replicates.

Metric. Mean of each variance estimator across replicates.

Decision rule. The plug-in mean should sit near $4 \times 0.9 = 3.6$. If it sits near 4, you used `ddof=1` by accident.

68.6 Project

Deliverable. A note that names $T(P)$ for three procedures you have already run: the training mean baseline, the least-squares coefficient, and a Gaussian variance. For one of them, simulate the sampling distribution and report bias and variance.

Constraints. The procedure includes any preprocessing. If you standardize, the estimator is "standardize then fit," not "fit."

Rubric. You never describe a single estimate as "the estimator's bias." Bias is a property of the procedure under a stated P.

68.7 Depth tracks

Beginner. Run the variance simulation.

University. Write the MSE decomposition and check it numerically on a deliberately biased estimator, such as $\bar{Y}/2$.

Research. Define the sampling distribution of a two-step estimator that selects a subset of features on the same sample it fits. Explain why an analytic standard error that ignores the selection step is for a different procedure.

68.8 Failure modes

- Quoting a standard error from a formula whose assumptions the selection procedure violated.
- Calling an estimator unbiased because the residual mean on the training sample is zero.
- Comparing MSEs of procedures aimed at different functionals.

68.9 See also

Maximum likelihood next. Volume 07 for selection that changes the procedure.

69 Maximum likelihood

70 Maximum likelihood

Reference question. What problem does maximum likelihood solve, and when is the solution a bad match to the decision you care about?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

70.1 Reference card

Item	Statement
Model	A family $\{P_\theta : \theta \in \Theta\}$.
Likelihood	$L(\theta) = p_\theta(z_{1:n})$ on the observed sample. Log likelihood $\ell(\theta) = \log L(\theta)$.
MLE	$\hat{\theta} \in \arg\max_\theta \ell(\theta)$.
iid shortcut	If the model says the rows are iid, $\ell(\theta) = \sum_i \log p_\theta(z_i)$.
Misspecification	If no P_θ equals the true law, the MLE targets the P_θ closest in KL divergence, not every functional you might have wanted.

70.2 Theory

Maximum likelihood is minimum KL divergence inside a family. You pick the member of the family that makes the observed sample least surprising, in the sense of that family's own density. If the family contains the truth and regularity conditions hold, the MLE is asymptotically efficient for θ: you cannot do better, asymptotically, with an unbiased estimator. That sentence is about θ in a well-specified parametric model. It is not a general blessing.

Under misspecification the maximizer converges to the projection of the true law onto the family in KL. For a Gaussian family that projection matches the true mean and variance, and ignores every other feature of the truth. If your decision depends on a tail probability, the MLE of a Gaussian can be a poor plug-in even though it is a fine estimator of the mean. Match the estimator to $T(P)$. Likelihood is the right tool when the parameter of the family is the functional you want, or when you will use the whole fitted law and have checked it.

The iid log-likelihood sum is invalid when the model says the rows are dependent. A Markov model has a likelihood, but it is not $\sum_i \log p_\theta(z_i)$ with a marginal p_θ. Using the wrong likelihood estimates the wrong projection.

70.3 Derivation

KL divergence from the true density p to a model density q_θ is $\mathbb{E}_p[\log p(Z)] - \mathbb{E}_p[\log q_\theta(Z)]$. The first term does not depend on θ. Minimizing KL is maximizing $\mathbb{E}_p[\log q_\theta(Z)]$. The average $n^{-1} \sum_i \log q_\theta(z_i)$ is the plug-in version of that expectation under an iid empirical measure. Its maximizer is the MLE.

For a Bernoulli(p) sample the log likelihood is $s \log p + (n - s) \log(1 - p)$ with s the number of ones. The derivative $s/p - (n - s)/(1 - p)$ vanishes at $\hat{p} = s/n$. Second derivative negative on $(0, 1)$ makes it a maximum. At $s \in \{0, n\}$ the maximum is on the boundary, 0 or 1, where a Gaussian approximation to the sampling law of $\hat{p}$ is a poor description at small n.

70.4 Code

```
import numpy as np

def bernoulli_mle(y):
    y = np.asarray(y, dtype=np.float64).reshape(-1)
    if np.any((y != 0) & (y != 1)):
        raise ValueError("Bernoulli MLE expects 0/1")
    return float(np.mean(y))

def bernoulli_loglik(y, p, eps=1e-12):
    p = float(np.clip(p, eps, 1 - eps))
    y = np.asarray(y, dtype=np.float64)
    return float(np.sum(y * np.log(p) + (1 - y) * np.log(1 - p)))
```

70.5 Experiment

Hypothesis. On Bernoulli data the MLE matches s/n, and on data generated from a two-component mixture of Bernoullis with a latent group, a single Bernoulli MLE matches the marginal mean and misses the group structure.

Protocol. Case A: $p = 0.2$, $n = 500$. Case B: with probability $1/2$ a row is Bernoulli(0.05), else Bernoulli(0.55). Fit a single $\hat{p}$. Compare $\hat{p}$ with the average of the row's true success probability.

Metric. $|\hat{p} - p|$ in case A. In case B, the difference between $\hat{p}$ and the marginal mean, which should be near 0, and a written statement that a single p assigns the wrong probability to a known group.

Decision rule. Case A should be within sampling error of 0.2. Case B's $\hat{p}$ near 0.3 is a success at estimating the marginal and a failure at describing a group. The note must say which functional was requested.

70.6 Project

Deliverable. MLE fits for Bernoulli, Gaussian, and Poisson on synthetic data from the matching family and from one misspecified family, with holdout log likelihood.

Constraints. State the target functional in the first line.

Rubric. You refuse at least one MLE as an answer to a decision it does not target, even if the optimization succeeded.

70.7 Depth tracks

Beginner. Derive and code the Bernoulli MLE.

University. Include the mixture misspecification case.

Research. Write the Gaussian score equation and the observed information $-\partial^2 \ell / \partial \theta^2$ for a scalar parameter. Say what the inverse information estimates when the model is correct, and what sandwich estimation changes when it is not. You may state the sandwich formula without proving it.

70.8 Failure modes

- Maximizing likelihood under an independence model for a time series and calling the result efficient.
- Using a boundary MLE of 0 or 1 as a probability in a product that then becomes zero forever.
- Treating a likelihood comparison on the training sample as an unbiased model selection rule.

70.9 See also

Volume 04 for the families. Volume 06 for the losses that are negative log likelihoods.

71 Tests and intervals

72 Tests and intervals

Reference question. What claim does a confidence interval make, and what claim does it refuse?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

72.1 Reference card

Item	Statement
Interval	A random set $C(Z_{1:n})$ computed from the sample.
Coverage	$P(T(P) \in C(Z_{1:n}))$, the probability over samples that the set covers the true functional.
Not	A probability that the true value lies in the numerical interval you already computed, unless you did a Bayesian calculation on purpose.
Test	A rule that rejects a hypothesis H_0 when a statistic falls in a critical region. Size is the rejection probability when H_0 is true.
p-value	The probability, under H_0, of a statistic at least as extreme as the one observed. It is not $P(H_0 \mid \text{data})$.

72.2 Theory

A 95 percent confidence interval is a procedure that covers the true value on 95 percent of repeated samples, under the assumptions used to build it. After you compute $[1.2, 1.8]$, the true value either is in it or is not. The 95 percent has been spent on the procedure. This frequentist reading is the one the formulas in this chapter use. Bayesian credible intervals answer a different question, in the next chapter, and they should not share a label.

A test of $H_0 : \mu = 0$ that fails to reject is not evidence that μ is zero. It is an absence of a particular kind of evidence against zero, at the sample size you have. Report the interval.

The interval shows the effects you have not ruled out. A significant result on a huge sample can be an effect too small to matter. Put the scientific threshold in the note before you look at the p-value.

Assumptions enter coverage. A t interval for a mean assumes iid draws with a finite variance, and the t critical value is exact when the draws are Gaussian. Under mild skew and large n it is often a decent approximation for the mean, by the central limit theorem. It is not a check on the median, and it is not valid under the dependence you already know how to construct.

72.3 Derivation

If $\bar{Y} \approx \mathcal{N}(\mu, \sigma^2/n)$ and s estimates σ, then $(\bar{Y} - \mu)/(s/\sqrt{n})$ is approximately standard normal, or Student-t with $n - 1$ degrees of freedom under an exact Gaussian model. A level $1 - \alpha$ interval is $\bar{Y} \pm t_{n-1,1-\alpha/2}\, s/\sqrt{n}$. Coverage is the probability this random interval contains μ.

For a test of $\mu = \mu_0$, the statistic is the same ratio at μ_0. Rejecting when the absolute value exceeds the critical value is the same event as μ_0 falling outside the interval. The duality is useful: if you would not trust the interval, do not trust the test.

72.4 Code

```
import numpy as np

def mean_interval(y, alpha=0.05):
    y = np.asarray(y, dtype=np.float64).reshape(-1)
    n = y.size
    mean = float(np.mean(y))
    se = float(np.std(y, ddof=1) / np.sqrt(n))
    # Normal critical value is a large-n stand-in. Small n should use a t table.
    z = 1.959963984540054
    if alpha != 0.05:
        raise ValueError("this helper is written for alpha=0.05")
    return mean - z * se, mean + z * se
```

72.5 Experiment

Hypothesis. The normal-approximation interval for a Gaussian mean covers at about 95 percent. The same numerical procedure under block dependence undercover.

Protocol. Independent case: $n = 40$ Gaussian draws, 1000 replicates, record the fraction of intervals containing the true mean. Dependent case: 8 independent Gaussian draws repeated 5 times each, and apply `mean_interval` as if $n = 40$.

Metric. Empirical coverage.

Decision rule. Independent coverage should be near 95 percent, a bit low or high is plausible at 1000 replicates (the standard error of a coverage estimate is about $\sqrt{0.95 \cdot 0.05/1000} \approx 0.7$ percentage points, so several points of error can happen; look for a gross miss). Dependent coverage should fall. Using a z critical value instead of t at $n = 40$ is a small effect; dependence is the large one.

72.6 Project

Deliverable. Intervals for a mean and for a difference of means on synthetic data, plus the dependence counterexample, plus a sentence that translates a non-rejection into "effects larger than this are unlikely," using the interval endpoints rather than the p-value.

Constraints. Do not write "the probability that the true mean is in $[a, b]$ is 95 percent."

Rubric. The wording matches the coverage definition. The dependent case is not swept into a footnote without a number.

72.7 Depth tracks

Beginner. Compute one interval for a sample of size 30 and state the coverage claim correctly.

University. Run both coverage experiments.

Research. Construct a bootstrap percentile interval for the median and compare its coverage with a mean interval on skewed data (exponential draws). Say which functional each interval is for.

72.8 Failure modes

- Peeking at several outcomes and reporting the smallest p-value as if it were the only test.
- Interpreting a confidence interval from a misspecified model as if the assumptions were decorative.
- Stopping data collection when the interval first excludes zero and then quoting a 95 percent coverage that the stopping rule destroyed.

72.9 See also

The Bayesian chapter for a different interval. Volume 07 for paired model comparisons.

73 Bayesian updating

74 Bayesian updating

Reference question. What extra ingredient does a Bayesian calculation use, and what does the posterior mean?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

74.1 Reference card

Item	Statement
Prior	A law $p(\theta)$ on parameters, chosen before the sample.
Likelihood	$p(z \mid \theta)$ from the sampling model.
Posterior	$p(\theta \mid z) \propto p(z \mid \theta)p(\theta)$.
Predictive	$p(z_{\text{new}} \mid z) = \int p(z_{\text{new}} \mid \theta)p(\theta \mid z)d\theta$.
Credible interval	A set with posterior probability $1 - \alpha$. It is not a confidence interval.

74.2 Theory

Bayesian updating takes the prior times the likelihood and renormalizes. The prior is part of the estimator. Two priors are two procedures. A flat prior is still a prior: it says, in a chosen parameterization, that wide regions start equally weighted, and it is not invariant to reparameterization unless you did the extra work to make it so.

The posterior is a distribution on θ given the sample, inside the joint model. A 95 percent credible interval contains 95 percent of that posterior. It answers "where does the updated belief sit?" It does not, by itself, answer "what fraction of repeated samples produce intervals that cover the truth?" Sometimes a Bayesian procedure also has good coverage. That is a separate theorem. Label the interval by the question it answers.

The posterior predictive distribution is what you want for a new observation. A plug-in prediction at the posterior mode ignores leftover uncertainty about θ. For a Bernoulli with a Beta prior, the predictive probability of the next one is a simple ratio you should be able to derive in your sleep. It shrinks the raw frequency toward the prior mean, which is useful exactly when n is small and dangerous when the prior was a wish.

74.3 Derivation

Take $Y \mid p \sim \text{Binomial}(n, p)$ and $p \sim \text{Beta}(\alpha, \beta)$, with density proportional to $p^{\alpha-1}(1-p)^{\beta-1}$. The posterior is $\text{Beta}(\alpha+s, \beta+n-s)$, because the p powers add. The prior mean is $\alpha/(\alpha+\beta)$. The posterior mean is $(\alpha+s)/(\alpha+\beta+n)$, a weighted average of the prior mean and s/n.

The probability the next draw is 1, given the past, is the posterior mean of p. That is the predictive probability. As $n \to \infty$ with $s/n \to p_\star$, the posterior mean goes to $p_\star$ for fixed α, β. The prior washes out when the likelihood concentrates and the model is correct. It does not wash out of a misspecified likelihood.

74.4 Code

```
import numpy as np

def beta_binomial_update(successes, n, alpha=1.0, beta=1.0):
    a = alpha + successes
    b = beta + (n - successes)
    mean = a / (a + b)
    return {"alpha": a, "beta": b, "mean": float(mean), "predictive_next": float(mean)}
```

74.5 Experiment

Hypothesis. With a $\text{Beta}(1, 1)$ prior and $s = 0$ successes in $n = 3$ trials, the posterior mean is $1/5$, not 0. With $n = 300$ and $s = 0$, the posterior mean is near 0 but not exactly 0.

Protocol. Compare `beta_binomial_update` with the MLE on both samples. Draw posterior samples by `rng.beta` if available and check that their mean matches the formula.

Metric. Posterior mean versus MLE.

Decision rule. The small-sample gap must be visible. If the $n = 3$ posterior mean is 0, the prior was not added.

74.6 Project

Deliverable. A note with one Beta-Binomial update, one sentence labeling a credible summary as Bayesian, and one sentence saying what coverage experiment you would need before calling it a confidence interval. Include a prior you consider foolish ($\text{Beta}(50, 1)$ on a coin you think is fair) and show that $n = 10$ does not wash it out.

Constraints. Do not grid-search the prior on the same data you report as confirmation.

Rubric. The foolish prior is visible in the posterior mean, and you say what additional data would be required to move it.

74.7 Depth tracks

Beginner. Compute the Beta$(1, 1)$ update for 7 successes in 10 trials by hand and in code.

University. Include the foolish-prior calculation.

Research. State the posterior for a Gaussian mean with known variance and a Gaussian prior. Identify the posterior mean as a precision-weighted average. Compare its shrinkage with ridge regression in one sentence, carefully: related linear algebra, different problem.

74.8 Failure modes

- Reporting a credible interval and a p-value as if they were the same object.
- Tuning the prior until the posterior matches a hoped-for story, on the analysis sample.
- Forgetting that a wrong likelihood is not repaired by a careful prior.

74.9 See also

Volume 02 for ridge as penalized least squares. Volume 14 if a posterior over a policy appears later; do not import this chapter's coin math without a new model.

75 Experimental design and power

76 Experimental design and power

Reference question. How do you choose a sample size for an effect you would be unwilling to miss?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

76.1 Reference card

Item	Statement
Effect	The smallest difference that would change a decision, in the units of the target.
Power	The probability that the test rejects H_0 when a stated alternative is true.
Inputs	Effect, noise scale, test size α, desired power, and the design (paired or not, split unit).
Output	A sample size, plus the assumption list.
Not	A sample size copied from a similar paper with a different noise scale.

76.2 Theory

Power is the sensitivity of a procedure. A test with size 5 percent and power 10 percent at the effect you care about is a procedure that almost never finds the thing you are looking for, and occasionally raises a false alarm. Collecting that sample and "seeing what happens" is a way to spend the sample without answering the question.

The effect size is a decision input, not a statistical output. "A 1 percent drop in squared error" might matter in a huge system and not matter in a class project. Write it down first. Then estimate the noise from a pilot or from the constant-baseline residual variance. Then solve for n. If the required n is impossible, change the question, the design, or the measurement. Do not pretend the affordable n answers the original question.

Paired designs reduce variance when the two measurements share a shock. Entity-level randomization increases the effective noise when you must randomize at the entity even though you measure rows. The sample-size formula has to use the variance of the unit you randomize, not the variance of the noisiest subplot.

76.3 Derivation

For a one-sample z test of a mean, $H_0 : \mu = 0$ against $\mu = \delta > 0$, the rejection threshold at level α is about $z_{1-\alpha}\sigma/\sqrt{n}$. Power is the probability that $\bar{Y}$ exceeds that threshold when the true mean is δ. Under a normal approximation,

$$1 - \beta \approx 1 - \Phi\left(z_{1-\alpha} - \frac{\delta\sqrt{n}}{\sigma}\right).$$

Set this equal to a desired power and solve

$$n \approx \left(\frac{(z_{1-\alpha} + z_{1-\beta})\sigma}{\delta}\right)^2.$$

For $\alpha = 0.05$ and power 0.8, the z values are about 1.64 (one-sided) and 0.84, so $n \approx (2.48\sigma/\delta)^2$. A two-sided test uses 1.96 instead of 1.64. Halving δ quadruples n.

76.4 Code

```
import numpy as np

def n_one_sample(delta, sigma, z_alpha=1.64485, z_power=0.84162):
    if delta <= 0 or sigma <= 0:
        raise ValueError("delta and sigma must be positive")
    return int(np.ceil(((z_alpha + z_power) * sigma / delta) ** 2))
```

76.5 Experiment

Hypothesis. At the computed n, a one-sided z test of a normal mean has power near 80 percent against the δ you designed for, and substantially lower power at $n/4$.

Protocol. $\sigma = 1$, $\delta = 0.2$, compute n. Simulate 500 experiments at that n and at $n/4$, testing $\bar{Y} > 1.64485\sigma/\sqrt{n}$.

Metric. Rejection frequency.

Decision rule. The designed n should land near 80 percent, with Monte Carlo error of a couple of points. $n/4$ should be obviously worse. A designed-sample power near 50 percent means the z quantiles or the threshold used the wrong σ.

76.6 Project

Deliverable. A design note for an A/B comparison of two prediction models on a metric you specify. Include the smallest effect you care about, a noise estimate, α, desired power, the resulting n, and what you will do if that n is unaffordable.

Constraints. The noise estimate must say whether it is per row or per entity.

Rubric. A reader can recompute n from the note. The effect is in metric units, not in "Cohen's d" alone, unless you also translate it.

76.7 Depth tracks

Beginner. Compute n for $\sigma = 1$, $\delta = 0.5$, and explain what happens to n if δ is halved.

University. Run the power simulation.

Research. Modify the formula for a two-sample test with variance σ^2 in each arm. Then say what changes if the true variances differ by a factor of four and you used a pooled estimate.

76.8 Failure modes

- Powering a study for a huge effect and then claiming a precise estimate of a small one.
- Using a row-level variance when randomization is by entity.
- Computing power after seeing a p-value, with an effect size taken from the same data, and calling it the design.

76.9 See also

Volume 07 for the metric the design must name. Volume 03 for entity splits.

77 Project: analyze a comparison without fooling yourself

78 Project: analyze a comparison without fooling yourself

Reference question. How do you report a two-policy comparison so the interval, the unit, and the decision stay attached?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

78.1 Reference card

Item	Statement
Policies	Two procedures frozen before the test sample is scored.
Unit	The independent randomization or sampling unit.
Metric	One primary metric, chosen before looking.
Estimate	Difference in means on that unit, with an interval.
Decision	Ship, keep, or redesign, compared with the effect the study was powered for.

78.2 Theory

This project is the inference volume in one folder. You will generate two policies with a known true difference so you can see whether your analysis recovers it. The discipline is identical when the truth is unknown: the analysis does not get to know it.

A comparison of already-fitted models on a test set estimates the difference in risk for those two frozen predictors. It does not include training-seed variability. Say so. If you refit both models inside each replicate, the interval is about the procedures, which is usually what a paper wants and not what a one-shot deployment check wants.

Peeking at five metrics and highlighting the significant one destroys the size of the test. Predeclare the primary metric in `config.json` before the script prints it. Secondary metrics can be exploratory and must be labeled as such.

78.3 Derivation

Let the unit-level outcomes be A_i and B_i under the two policies, and let $D_i = B_i - A_i$ when the comparison is paired on the same unit. The target is $\mathbb{E}[D]$. The paired interval is the one-sample interval for the D_i. Pairing is valid when each unit yields both measurements without interference. If policy B's presence changes policy A's outcome, you do not have a paired difference of stable potential outcomes. For synthetic data you can enforce no interference by simulation. In a letter to your future self, note that production traffic often interferes.

78.4 Code

```
import numpy as np

def paired_difference(a, b):
    d = np.asarray(b, dtype=np.float64) - np.asarray(a, dtype=np.float64)
    mean = float(np.mean(d))
    se = float(np.std(d, ddof=1) / np.sqrt(d.size))
    z = 1.959963984540054
    return {"diff": mean, "low": mean - z * se, "high": mean + z * se, "n": int(d.size)}
```

78.5 Experiment

Hypothesis. If $B = A + 0.3 +$ noise on paired units, the interval for the mean difference covers 0.3 at roughly 95 percent and excludes 0 in a study powered for 0.3.

Protocol. Gaussian noise scale 1, n from the power formula for $\delta = 0.3$, two-sided, so use $z_\alpha \approx 1.96$ in a modified calculator. Run 400 replicates. Also run one underpowered replicate at $n = 20$ and write how you would talk about it.

Metric. Coverage of 0.3, and the fraction of intervals that exclude 0.

Decision rule. Coverage near 95 percent. Power should be high by construction. The $n = 20$ write-up must not convert "interval contains 0" into "the policies are equal."

78.6 Project

Deliverable. A folder, in the Volume 03 layout, with the predeclared metric, the power calculation, the paired analysis, the coverage simulation, and a decision paragraph a teammate could act on.

Constraints. One primary metric. Any other number is labeled exploratory.

Rubric. The decision uses the interval and the predeclared effect, not a p-value alone. The unit of analysis is named in the first five lines of the note.

78.7 Depth tracks

Beginner. Analyze one paired sample of size 40 with a known difference and interpret the interval in the approved wording.

University. Complete the coverage simulation and the underpowered write-up.

Research. Add a second, entity-clustered version: 10 independent pairs repeated 20 times, analyzed once correctly at the entity level and once incorrectly at the repeated-row level. Compare interval widths.

78.8 Failure modes

- Scoring the test set, tuning, and scoring it again as if the first look did not happen.
- Reporting a relative improvement with no base rate.
- Letting the randomization unit and the analysis unit differ without a reason written down.

78.9 See also

Volume 07, which uses this comparison style for model selection. Volume 16 for when the metric itself is the wrong decision.

Part VI

Volume 06. Supervised learning

79 Volume 06. Supervised learning

80 Volume 06. Supervised learning

Stage. University

Audience. A reader who can fit least squares, audit a table, and state a sampling unit.

Question this volume answers. Which predictor should you fit, under which loss, and what baseline must it beat?

This volume is a reference, not a short tour. Each chapter opens with a card you can look up later, then gives the theory, a reference implementation, an experiment with a decision rule, and a project with a rubric. Read in order the first time. After that, enter through the reference card of the chapter that matches the problem in front of you.

80.1 How to use the four layers

Layer	What you should be able to do when you finish it
Theory	State the definition, the assumptions, and the result, including what the result does not say.
Code	Implement the core object in a few dozen lines and check it against a known case.
Experiment	Change one factor, record a metric with uncertainty, and say what outcome would change your mind.
Project	Produce an artifact someone else can rerun: code, figure, table, and a one-page lab note.

80.2 Depth tracks

Every chapter ends with three tracks.

- **Beginner.** Do the reference card, the code, and the smallest experiment. Skip optional proofs.
- **University.** Do the derivation, the full experiment, and the project rubric.
- **Research.** Read the failure modes, change an assumption, and write what would have to be true for the method to fail on a new distribution.

80.3 Chapters

No.	Chapter	File
01	The supervised problem	`vol06/01_supervised_problem.qmd`
02	Linear and logistic regression	`vol06/02_linear_logistic.qmd`
03	Trees and ensembles	`vol06/03_trees_ensembles.qmd`
04	Kernel methods	`vol06/04_kernels.qmd`
05	Calibration and proper scoring	`vol06/05_calibration.qmd`
06	Project: a baseline bake-off	`vol06/06_bakeoff_project.qmd`

80.4 Lab note

Keep one markdown file per experiment, `notes/YYYY-MM-DD-<slug>.md`, with the hypothesis, the command you ran, the metric, and the decision. The series is a reference only if the experiments you run are attached to it.

81 The supervised problem

82 The supervised problem

Reference question. What is the object a supervised method is trying to estimate?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

82.1 Reference card

Item	Statement
Task	Learn f from samples of (X, Y) so that $\ell(f(X), Y)$ is small on a new draw.
Bayes predictor	A minimizer $f_\star$ of the risk $R(f) = \mathbb{E}[\ell(f(X), Y)]$ over a rich class of functions.
Model class	A restricted family $\mathcal{F}$. Approximation error is the gap between $f_\star$ and the best element of $\mathcal{F}$.
Algorithm	A map from a sample to $\hat{f} \in \mathcal{F}$. Estimation error is how far $\hat{f}$ is from that best element.
Generalization gap	$R(\hat{f}) - \hat{R}_n(\hat{f})$, a random quantity, not a moral failing.

82.2 Theory

Supervised learning splits error into two debts. Approximation error is the price of a small family: linear functions cannot represent a quadratic conditional mean. Estimation error is the price of a finite sample: even the best linear function is not exactly the one you fit. A larger family can cut approximation error and raise estimation error. Regularization, more data, and a better representation are different ways of paying these debts. They are not interchangeable slogans.

The Bayes predictor depends on the loss. Under squared error it is the conditional mean. Under 0-1 loss, for two classes, it is the thresholded conditional probability. A model that estimates $P(Y = 1 \mid X)$ well can still be a poor 0-1 classifier if you choose a bad threshold, and a hard classifier can have good 0-1 loss and be useless when you needed a probability. Say which Bayes predictor you are chasing.

The empirical risk minimizer picks $\hat{f}$ by looking at $\hat{R}_n$ on the training sample. Its training loss is optimistic for $R(\hat{f})$ because the same points were used to choose $\hat{f}$. The next volume is entirely about not lying with that optimism. This volume builds families that are worth evaluating.

82.3 Derivation

For squared error, expand $\mathbb{E}[(Y - f(X))^2 \mid X = x] = \mathrm{Var}(Y \mid X = x) + (\mathbb{E}[Y \mid X = x] - f(x))^2$. The variance term does not depend on f. The second term is zero when $f(x) = \mathbb{E}[Y \mid X = x]$. So the conditional mean is Bayes for squared error. Restricting to linear f leaves a nonnegative approximation term $\mathbb{E}[(\mathbb{E}[Y \mid X] - X^\top \theta)^2]$.

For 0-1 loss, $\mathbb{E}[\mathbf{1}\{f(X) \neq Y\} \mid X = x] = 1 - P(Y = f(x) \mid X = x)$ in classification. Maximizing the conditional probability of the chosen label minimizes the loss. The decision is $\arg\max_y P(Y = y \mid X = x)$.

82.4 Code

```
import numpy as np

def bayes_squared(conditional_mean, x):
    return conditional_mean(x)

def hard_threshold(prob_one, threshold=0.5):
    return (np.asarray(prob_one) >= threshold).astype(np.float64)
```

82.5 Experiment

Hypothesis. When $Y = \sin(2\pi X) + \varepsilon$ on $[0, 1]$, a linear family has a visible approximation floor, and a piecewise-constant family with enough bins lowers that floor until estimation error in the bins rises.

Protocol. $n_{\text{train}} = 80$, $n_{\text{test}} = 1000$, noise standard deviation 0.2, 30 seeds. Linear regression on $(1, x)$. Piecewise means on $b \in \{2, 8, 20\}$ equal-width bins, with empty bins falling back to the global training mean.

Metric. Median test MSE by method, compared with an oracle that predicts $\sin(2\pi x)$.

Decision rule. Linear should lose to a moderate binning. Very fine binning should worsen relative to moderate binning at this n. If fine binning wins cleanly, n is too large to see estimation error; reduce n.

82.6 Project

Deliverable. A diagram or table that splits test MSE into an oracle Bayes term (noise plus irreducible error), an approximation gap, and the remaining estimation gap, for the linear model and one binning.

Constraints. The oracle uses the true sin function, which you may use only for evaluation in this synthetic study.

Rubric. You can point to which knob cuts which gap. "Use a flexible model" is not an adequate sentence.

82.7 Depth tracks

Beginner. Fit the linear model and the oracle and report the two test MSEs.

University. Complete the binning sweep.

Research. Write the bias-variance decomposition for squared error at a fixed x for the piecewise-constant estimator, assuming bins were fixed before seeing Y. Note where data-dependent bins would change the argument.

82.8 Failure modes

- Calling training error the approximation error.
- Changing the loss between training and the decision without saying the Bayes predictor changed.
- Using the test set to pick the number of bins and then quoting that same test number as final risk.

82.9 See also

Volume 07 for honest selection of the bin count. Volume 09 for families that are flexible and still structured.

83 Linear and logistic regression

84 Linear and logistic regression

Reference question. What do linear regression and logistic regression estimate, and how do you fit the second one?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

84.1 Reference card

Item	Statement
Linear	$f(x) = \theta^\top x$. Squared loss estimates a linear approximation to the conditional mean.
Logistic	$P(Y = 1 \mid X = x) = \sigma(\theta^\top x)$ with $\sigma(t) = 1/(1 + e^{-t})$.
Log loss	$-(y \log p + (1 - y) \log(1 - p))$, the negative Bernoulli log likelihood.
Gradient	For one row, $(\sigma(\theta^\top x) - y)x$.
Fit	No closed form in general. Use gradient descent or a Newton method on the convex loss.

84.2 Theory

Linear regression is the Volume 01 estimator. Logistic regression keeps the linear score and changes the loss so the fitted number is a probability. The mean of a Bernoulli is its probability, and the logistic function maps the real line to $(0, 1)$. The model class is therefore a family of conditional Bernoulli laws, not a hard classifier. You can threshold the probability afterward. The threshold is a decision rule and can be chosen for asymmetric costs. It is not part of the likelihood.

The log loss is convex in θ because it is a convex decreasing function of the margin $y'\theta^\top x$ in the ± 1 coding, or by direct Hessian calculation: the Hessian is $X^\top W X$ with W a diagonal of $\sigma(1 - \sigma) > 0$ except at degenerate probabilities. Full column rank makes it strongly convex on compact sets where probabilities stay away from 0 and 1. Separable data send $\|\theta\|$ to infinity and probabilities to 0 and 1. A penalty or early stopping is then part of the estimator, not a numerical detail.

Coefficients in a logistic model are changes in log odds per unit of the feature, holding other features fixed, inside the model. They are not causal effects. Collinearity makes them unidentified in the same way as in least squares.

84.3 Derivation

Let $p_i = \sigma(\theta^\top x_i)$ and $\ell(\theta) = -\sum_i \left(y_i \log p_i + (1 - y_i) \log(1 - p_i)\right)$. The derivative of $\log \sigma(t)$ and $\log(1 - \sigma(t))$ yields $\partial \ell / \partial \theta = \sum_i (p_i - y_i) x_i$. Set the step

$$\theta \leftarrow \theta - \eta X^\top (p - y)/n$$

if you use the mean loss. The Hessian contribution of one row is $p_i(1 - p_i) x_i x_i^\top$, hence $H = X^\top W X \succeq 0$. Newton's method applies when H is invertible. A standard L2 penalty $\frac{\lambda}{2}\|\theta\|_2^2$ adds λI to H and $\lambda\theta$ to the gradient, and keeps separable data from diverging.

84.4 Code

```
import numpy as np

def sigmoid(t):
    t = np.clip(t, -40.0, 40.0)
    return 1.0 / (1.0 + np.exp(-t))

def fit_logistic(X, y, lam=1.0, steps=200, eta=0.5):
    theta = np.zeros(X.shape[1])
    n = X.shape[0]
    for _ in range(steps):
        p = sigmoid(X @ theta)
        grad = X.T @ (p - y) / n + lam * theta
        theta = theta - eta * grad
    return theta
```

84.5 Experiment

Hypothesis. On linearly separable 2D data, unpenalized gradient descent grows $\|\theta\|$, while $\lambda = 1$ keeps it bounded, and both can reach low training error if the step is stable.

Protocol. Two Gaussian clouds centered at $(\pm 1.5, 0)$, 40 points each. Fit $\lambda = 0$ and $\lambda = 1$ for several step counts. Track $\|\theta\|$ and training log loss.

Metric. Norm and mean log loss at the final step.

Decision rule. The unpenalized norm should be larger and still growing if you double the step count. If $\lambda = 0$ converges in norm, the clouds overlap and are not separable; move the centers.

84.6 Project

Deliverable. A comparison, on one synthetic classification set and one regression set, of least squares on a 0/1 target versus logistic regression. Report log loss and 0-1 error. Threshold logistic probabilities at 0.5 and, if you move the threshold, justify it by a cost.

Constraints. Standardize features using training statistics only. Include an intercept.

Rubric. You say which loss each method targets. You do not rank them by a metric neither one optimized without saying so.

84.7 Depth tracks

Beginner. Fit logistic regression on the two clouds and print probabilities, not only labels.

University. Show the Hessian is PSD and run the penalty comparison.

Research. Implement one Newton step with an L2 penalty and compare iterations against gradient descent at equal training log loss. Discuss the cost of forming H when d is large.

84.8 Failure modes

- Interpreting odds ratios from unstandardized, collinear features as importance.
- Using 0-1 loss as a training objective and expecting a useful gradient.
- Predicting hard labels inside a pipeline that the next stage treats as probabilities.

84.9 See also

Volume 07 for calibration. Volume 10 for penalties as regularization.

85 Trees and ensembles

86 Trees and ensembles

Reference question. What function class does a tree represent, and why does averaging trees change the error?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

86.1 Reference card

Item	Statement
Tree	A partition of feature space into boxes, with a constant prediction in each box.
Split	Chosen to reduce a loss: squared error, or a classification impurity such as Gini or entropy.
Depth	A capacity knob. A deep tree can memorize. A stump is a single split.
Bagging	Average trees fit on bootstrap samples. Variance reduction when the trees are unstable and not perfectly dependent.
Boosting	Add trees sequentially to fit the residual or the negative gradient of the loss.

86.2 Theory

A regression tree is a piecewise-constant model. It can approximate jumps and interactions that a linear model misses, which is an approximation-error win. The partition is chosen using the training labels, so the family is data-dependent and the "number of leaves" understates the search. Deep trees have low bias and high variance: a small change in the sample can move a split.

Bagging attacks variance. A bootstrap sample draws n rows with replacement. Trees fit on different bootstraps make different errors. Their average has variance $\rho\sigma^2 + (1-\rho)\sigma^2/M$ in the equicorrelated caricature, where M is the number of trees and ρ is the correlation of their predictions. If the trees are identical, averaging does nothing. Random feature subsets at each split (a random forest) try to lower ρ.

Boosting attacks bias. Each new tree fits the gradient of the current loss, a functional gradient step in function space. Shrinkage, subsampling, and early stopping are the regularizers. A boosted ensemble can overfit. The staged addition sometimes overfits slowly, which is convenient and not a guarantee.

Trees do not require monotone scaling of a single feature, because splits are threshold rules. They are awkward at linear trends unless depth is spent to staircase the line. A linear baseline remains mandatory.

86.3 Derivation

For squared error, the best constant on a fixed region R is the mean of Y on that region, because a constant predictor's optimal value is the mean. A split of R into R_L and R_R is scored by the drop in total squared error

$$\sum_{i \in R} (y_i - \bar{y}_R)^2 - \sum_{i \in R_L} (y_i - \bar{y}_L)^2 - \sum_{i \in R_R} (y_i - \bar{y}_R)^2.$$

Searching over feature indices and thresholds picks the split with the largest drop. This search is part of the estimator. The Bayes predictor is still the conditional mean; the tree is a restricted, data-dependent approximation.

86.4 Code

```
import numpy as np

def best_stump(x, y):
    x = np.asarray(x, dtype=np.float64)
    y = np.asarray(y, dtype=np.float64)
    order = np.argsort(x)
    x, y = x[order], y[order]
    best = None
    for i in range(1, len(y)):
        if x[i] == x[i - 1]:
            continue
        left, right = y[:i], y[i:]
        sse = np.sum((left - left.mean()) ** 2) + np.sum((right - right.mean()) ** 2)
        if best is None or sse < best[0]:
            best = (float(sse), float(0.5 * (x[i - 1] + x[i])), float(left.mean()), float(
    return {"sse": best[0], "threshold": best[1], "left_mean": best[2], "right_mean": best
```

86.5 Experiment

Hypothesis. A stump recovers a step function in one dimension better than a linear fit, and loses to a linear fit on a straight line with modest noise.

Protocol. Case step: X uniform on $[-1, 1]$, $Y = 1\{X > 0\} + \varepsilon$. Case line: $Y = 2X + \varepsilon$. Noise scale 0.2, $n = 200$, test size 1000, 20 seeds. Compare stump and least squares.

Metric. Median test MSE.

Decision rule. The ranking should flip between cases. If the stump wins on the line, the line is too noisy or the stump implementation peeks at the test set.

86.6 Project

Deliverable. The stump, a depth-2 tree you build by splitting each side once, and a linear baseline, on both synthetic cases. If you use a library forest or booster, add it as a fourth row and record its depth and tree count in the config.

Constraints. The split search may use training data only.

Rubric. You explain the ranking using approximation versus estimation, not using a leaderboard adjective.

86.7 Depth tracks

Beginner. Implement the stump and the step-function case.

University. Add the line case and the depth-2 tree.

Research. Estimate the correlation ρ of predictions from two bootstrap stumps on the step data, and say whether averaging two of them should help. Check by computing the average's test MSE.

86.8 Failure modes

- Letting a tree split on an identifier column.
- Comparing a tuned forest with an untuned linear model.
- Reading a split as a causal threshold.

86.9 See also

Volume 07 for choosing depth on a validation set. Volume 10 for gradient boosting as optimization.

87 Kernel methods

88 Kernel methods

Reference question. How can a linear method operate in a large feature space without writing the features down?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

88.1 Reference card

Item	Statement
Feature map	$\phi(x)$, possibly high or infinite dimensional.
Kernel	$k(x, x') = \langle \phi(x), \phi(x') \rangle$.
Gram	The $n \times n$ matrix $K_{ij} = k(x_i, x_j)$.
Representer	A penalized linear fit in ϕ-space has a solution $\sum_{i=1}^{n} \alpha_i \phi(x_i)$.
Prediction	$f(x) = \sum_{i=1}^{n} \alpha_i k(x_i, x)$, after the penalty identifies α.

88.2 Theory

Kernels are inner products of features you may never form. A polynomial kernel corresponds to polynomial features. A Gaussian (RBF) kernel corresponds to a smooth infinite feature map. The computational object is the Gram matrix, which is $n \times n$. That is attractive when n is moderate and the feature dimension would have been huge, and unattractive when n is millions.

The representer theorem is the reason the prediction depends on the training points only through n coefficients. It applies to objectives that depend on the data through inner products plus an increasing penalty on the norm in the feature space. It does not say every learning method is a kernel method.

The kernel chooses a notion of similarity. If that notion is wrong, the method is a linear method in the wrong space. Scaling of x changes distances and therefore changes an RBF kernel. Standardize with a purpose, and put the scale in the config. Kernel ridge regression is ridge regression after the feature map, written with K so you solve an $n \times n$ system instead of a $d \times d$ system.

88.3 Derivation

Ridge regression in feature space minimizes $\frac{1}{2}\|\Phi\theta - y\|_2^2 + \frac{\lambda}{2}\|\theta\|_2^2$, where Φ has rows $\phi(x_i)^\top$. The gradient condition is $\Phi^\top(\Phi\theta - y) + \lambda\theta = 0$, so $\theta = \Phi^\top\alpha$ with $\alpha = (\Phi\theta - y)/(-\lambda)$ waiting to be identified. Then $\Phi\theta = K\alpha$, and substitution yields $(K + \lambda I)\alpha = y$ when the loss is unnormalized squared error with the factor arranged so the λ term matches; if your loss has a $1/n$ or a $1/2$, the λ placement changes. Be consistent. Predictions at new x are $k(x)^\top\alpha$ with $k(x)_i = k(x_i, x)$.

For the linear kernel $k(x, x') = x^\top x'$, this solution matches ordinary ridge when algebra is aligned. That match is the unit test.

88.4 Code

```
import numpy as np

def rbf(a, b, length_scale):
    a = np.asarray(a, dtype=np.float64)
    b = np.asarray(b, dtype=np.float64)
    d2 = np.sum((a[:, None, :] - b[None, :, :]) ** 2, axis=-1)
    return np.exp(-0.5 * d2 / length_scale**2)

def kernel_ridge(K, y, lam):
    alpha = np.linalg.solve(K + lam * np.eye(K.shape[0]), y)
    return alpha
```

88.5 Experiment

Hypothesis. Kernel ridge with a linear kernel and a matching penalty agrees with ridge regression from Volume 02's SVD formula on a small full-rank problem. An RBF kernel beats the linear kernel on the sine wave from the previous volume's approximation example, at a length scale you declare in advance.

Protocol. $n = 60$ for the agreement test. For the sine wave, fix λ and the length scale before looking at test error, using a validation slice of the training set if you must choose them.

Metric. Parameter or prediction gap on the linear-kernel test. Test MSE on the sine wave.

Decision rule. Linear-kernel predictions should match to about 10^{-6} relative error once penalties match. If they do not, do not interpret the RBF result; fix the algebra.

88.6 Project

Deliverable. Linear kernel versus RBF versus the stump, on the step data and the sine data, with λ and length scale in the config hash.

Constraints. The Gram matrix is built from training points only. Test points enter only $k(x_{\text{test}}, x_{\text{train}})$.

Rubric. You state the $n \times n$ cost and a case where you would refuse a kernel method because n is too large.

88.7 Depth tracks

Beginner. Implement the linear kernel and check it against a dot product.

University. Complete the agreement test with ridge and the sine comparison.

Research. Show that the RBF Gram matrix is PSD by construction in exact arithmetic, and watch its condition number as the length scale shrinks. Connect a tiny length scale to near-interpolation and a large κ.

88.8 Failure modes

- Building the Gram matrix on train and test together and then slicing.
- Choosing the length scale by the final test number.
- Forgetting that a new categorical level has no training row and no kernel coordinate until you define one.

88.9 See also

Volume 01 for the ridge SVD shrink factor, which is the feature-space version of this penalty. Volume 10 for penalties on neural nets, a different family.

89 Calibration and proper scoring

90 Calibration and proper scoring

Reference question. When is a predicted probability a probability, and which scores punish a false one?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

90.1 Reference card

Item	Statement
Calibration	Among examples with predicted probability p, the fraction of positive labels is about p.
Refinement	How much the predictions vary. A constant $\bar{y}$ can be calibrated and useless.
Proper score	A score minimized in expectation by reporting the true conditional probability. Log loss and squared error on the probability (Brier) are proper.
Improper	Accuracy at a threshold can prefer a miscalibrated score that ranks well.
Fix	Fit the probability model, then recalibrate on data that did not choose the model class.

90.2 Theory

Classification pipelines often emit scores that are not probabilities: margins from a linear model, leaf fractions from a tree grown for Gini, or logits from a network trained long past the point where the probabilities sharpened into 0 and 1. Decisions that need expected cost need probabilities. Ranking the positive class above the negative class needs only an order. The metric has to match the need.

A proper scoring rule makes the truthful conditional probability the optimal report. Log loss is harsh in the tails: predicting 10^{-6} when the event happens costs about 13.8 nats, while predicting 0.5 costs $\log 2$. That harshness is correct when a confident mistake is expensive, and it is unstable when your model is wrongly confident. The Brier score $(p - y)^2$ is proper and milder.

Calibration diagrams bin the predictions. Inside a bin, the average prediction should match the average label. A model can be calibrated and have no discrimination, or discriminate well and be miscalibrated. Report both: a calibration plot and a proper score, plus a ranking metric if ranking is the decision.

90.3 Derivation

For binary Y and a reported probability q, expected log loss given $X = x$ is

$$-p \log q - (1 - p) \log(1 - q), \quad p = P(Y = 1 \mid X = x).$$

The derivative in q is $-p/q + (1 - p)/(1 - q)$, zero at $q = p$, and the second derivative is positive. So log loss is proper. The Brier score $p(1 - q)^2 + (1 - p)q^2$ has derivative zero at $q = p$ as well.

If you pass a score s through a fitted map $q = \sigma(a + bs)$ using a validation set, you are fitting a one-dimensional logistic regression. That map can repair monotone miscalibration. It cannot create information that was not in s.

90.4 Code

```
import numpy as np

def brier(p, y):
    p = np.asarray(p, dtype=np.float64)
    y = np.asarray(y, dtype=np.float64)
    return float(np.mean((p - y) ** 2))

def reliability_bins(p, y, n_bins=5):
    p = np.asarray(p, dtype=np.float64)
    y = np.asarray(y, dtype=np.float64)
    edges = np.linspace(0.0, 1.0, n_bins + 1)
    rows = []
    for i in range(n_bins):
        mask = (p >= edges[i]) & (p < edges[i + 1] if i < n_bins - 1 else p <= edges[i + 1])
        if np.any(mask):
            rows.append((float(p[mask].mean()), float(y[mask].mean()), int(mask.sum())))
    return rows
```

90.5 Experiment

Hypothesis. If the true probability is $p(x) = \sigma(2x)$ and you report $q = \sigma(8x)$, the reports are overconfident: reliability bins pull toward the corners relative to the diagonal, and log loss is worse than for $q = p$.

Protocol. Draw $x \sim \mathcal{N}(0, 1)$, $y \sim \text{Bernoulli}(p(x))$, $n = 4000$. Compare $q = p$ and $q = \sigma(8x)$ by log loss, Brier score, and a 5-bin reliability table. Also compare a constant $q = \bar{y}$.

Metric. The two scores and the bin gaps $\bar{q} - \bar{y}$.

Decision rule. The true p must win on log loss. The overconfident q should show bin average predictions more extreme than bin frequencies. The constant should be better calibrated than the overconfident model and worse as a ranking of risk. If you only report accuracy at 0.5, you may miss this; that is part of the lesson.

90.6 Project

Deliverable. Take the logistic regression from earlier in the volume, emit probabilities, and build a reliability table on a validation set. Fit a temperature or an affine logistic map on that validation set only if the table is bent, and quote a final test score once.

Constraints. The test set does not choose the map.

Rubric. You state whether the downstream decision needs probabilities or only a ranking, and you include the matching metric.

90.7 Depth tracks

Beginner. Compute Brier score for a constant 0.5 and for a constant equal to the base rate, on a sample with base rate 0.1.

University. Run the overconfidence experiment.

Research. Show on paper that accuracy is not a proper scoring rule: give two reported probabilities where the wrong one wins on expected accuracy at threshold 0.5. A simple discrete p is enough.

90.8 Failure modes

- Temperature-scaling on the test set.
- Treating a high accuracy on an imbalanced problem as calibration.
- Clipping probabilities to exactly 0 or 1 before log loss, which makes the score infinite at the first mistake.

90.9 See also

Volume 07 for threshold choice under costs. Volume 12 if a token probability is later treated as a calibrated chance; it usually is not.

91 Project: a baseline bake-off

92 Project: a baseline bake-off

Reference question. How do you compare a small set of supervised methods without a leaderboard fantasy?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

92.1 Reference card

Item	Statement
Frozen split	Train, validation, and test, or a cross-validation plan, written before fitting.
Baselines	Constant predictor, linear or logistic model, and one nonlinear method.
Knobs	Each knob is in the config. The validation set chooses knobs. The test set is used once.
Table	Primary metric with uncertainty across seeds or folds, plus the sample size.
Claim	One sentence: which method won by enough to matter, or that none did.

92.2 Theory

A bake-off is an estimation procedure for the identity of the best method in a finite list. The list is part of the procedure. Searching a thousand hidden variants and publishing the winner is a different, more optimistic procedure. This project keeps the list short and written down: a constant, a linear model, a stump or tree, and optionally kernel ridge or logistic regression.

The validation set is for knobs such as tree depth, λ, and length scale. The test set answers one question about the chosen procedures. If you need a second test look, you no longer have that test set. You may say so and call the number exploratory.

Practical significance is the effect size from Volume 05, not the smallest difference a large test set can detect. If the linear model is within that margin of the flexible model, prefer the linear model for interpretation and speed unless you have another reason. Prefer is a decision. The table does not make it for you.

92.3 Derivation

Let methods $m = 1, \dots, M$ be chosen from a fixed list by validation risk, and let $m^\star$ be the one you then evaluate on the test set. The test risk of $m^\star$ is an unbiased estimate of the risk of that frozen method, not of the selection rule's future performance on a new draw of the whole pipeline, unless the test set was never used in any choice. If the validation choice is unstable across seeds, the test number is about whichever method won this time. Report the selection frequency. A method that wins validation on 51 percent of seeds is not a stable champion.

92.4 Code

```
import numpy as np

def pick_by_validation(val_scores):
    names = list(val_scores)
    winner = min(names, key=lambda name: val_scores[name])
    return winner, {name: float(val_scores[name]) for name in names}
```

92.5 Experiment

Hypothesis. On the sine-wave data, a flexible method wins validation more often than linear regression. On the straight-line data, linear regression wins or ties within a margin you set to the noise level.

Protocol. Two datasets from earlier chapters, three methods, 20 seeds, a validation slice of 30 percent of the training data, a fixed test set per seed drawn fresh.

Metric. Selection frequency and median test MSE of the selected method versus always-linear.

Decision rule. If selection frequency is split and test gaps are smaller than your practical margin, the claim is "no meaningful winner," not the name of the method that won eleven times.

92.6 Project

Deliverable. One experiment folder with the method list, the practical margin, the validation rule, the test table, and a decision. Include the constant baseline in every table.

Constraints. No method enters the list after you see the test table.

Rubric. The decision would still look reasonable if the seed-level winner flipped. You name what you would need (more data, a different feature, a different loss) if it does not.

92.7 Depth tracks

Beginner. Compare constant versus linear on the line data only, with the test set used once.

University. Run the full three-method, two-dataset protocol.

Research. Add a paired comparison of test losses between the two best methods, using the Volume 05 interval on per-point loss differences. State whether the mean gap's interval excludes your practical margin.

92.8 Failure modes

- A bake-off whose only nonlinear method was tuned on the test set.
- Dropping the constant baseline because it "is not a real model."
- Declaring a winner on a gap smaller than the seed-to-seed scatter.

92.9 See also

Volume 07 is this protocol made systematic. Volume 10 adds neural nets to the list only after the baselines exist.

Part VII

Volume 07. Evaluation and model selection

93 Volume 07. Evaluation and model selection

94 Volume 07. Evaluation and model selection

Stage. University

Audience. A reader who can run the bake-off protocol and compute an interval.

Question this volume answers. Which number is allowed to choose a model, and which number is allowed to be published?

This volume is a reference, not a short tour. Each chapter opens with a card you can look up later, then gives the theory, a reference implementation, an experiment with a decision rule, and a project with a rubric. Read in order the first time. After that, enter through the reference card of the chapter that matches the problem in front of you.

94.1 How to use the four layers

Layer	What you should be able to do when you finish it
Theory	State the definition, the assumptions, and the result, including what the result does not say.
Code	Implement the core object in a few dozen lines and check it against a known case.
Experiment	Change one factor, record a metric with uncertainty, and say what outcome would change your mind.
Project	Produce an artifact someone else can rerun: code, figure, table, and a one-page lab note.

94.2 Depth tracks

Every chapter ends with three tracks.

- **Beginner.** Do the reference card, the code, and the smallest experiment. Skip optional proofs.
- **University.** Do the derivation, the full experiment, and the project rubric.

- **Research.** Read the failure modes, change an assumption, and write what would have to be true for the method to fail on a new distribution.

94.3 Chapters

No.	Chapter	File
01	Train, validation, test, and leakage	`vol07/01_splits.qmd`
02	Metrics by decision	`vol07/02_metrics.qmd`
03	Error analysis	`vol07/03_error_analysis.qmd`
04	Cross-validation	`vol07/04_cross_validation.qmd`
05	Comparing models under uncertainty	`vol07/05_comparisons.qmd`
06	Project: a model card and an error taxonomy	`vol07/06_model_card_project.qmd`

94.4 Lab note

Keep one markdown file per experiment, `notes/YYYY-MM-DD-<slug>.md`, with the hypothesis, the command you ran, the metric, and the decision. The series is a reference only if the experiments you run are attached to it.

95 Train, validation, test, and leakage

96 Train, validation, test, and leakage

Reference question. What is each split for, and which operations leak across it?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

96.1 Reference card

Item	Statement
Train	Fits parameters of a frozen procedure.
Validation	Chooses knobs that are not fit inside the training objective: depth, λ, step count, thresholds.
Test	Estimates risk of the procedure the validation set already chose. One look.
Leakage	Any use of a split's labels, or of its inputs when the statistic will be applied outside that split, that the deployment distribution will not provide in the same way.
Time	If the world moves forward, the test set is the future, not a random subset of the past.

96.2 Theory

A split is a firewall between fitting and bragging. Parameters see the training labels. Knobs see the validation labels. The test labels are a measurement instrument. Using an instrument to build the object changes what the instrument measures.

Leakage is not only a column named `target`. It is a scaler fit on all rows, a feature-selection pass on all rows, an entity that appears on both sides, a moving-average feature that used tomorrow, or a test-set threshold chosen to maximize test accuracy. The deployment system will not have tomorrow, and it will not have the test labels.

When data are a time series, a random split trains on the future to predict the past. Use a cutoff. When rows share an entity, split on the entity. These rules came from Volume 03. Here they are preconditions for every metric in the volume.

96.3 Derivation

Let $\hat{f}_S$ be the predictor obtained when the procedure is allowed to see the set of examples S. If the test indices T intersect the information used to build $\hat{f}$, the quantity $m^{-1}\sum_{i\in T}\ell(\hat{f}(x_i),y_i)$ is not an unbiased estimate of the risk of a predictor built without those examples. In particular, choosing $k=\arg\min_k \widehat{\mathrm{err}}_T(k)$ and then reporting $\widehat{\mathrm{err}}_T(k)$ estimates the minimum of several dependent test errors, which is biased low for the risk of the selected k. The bias shrinks only if you pay for selection with a fresh sample or a nested procedure.

96.4 Code

```
import numpy as np

def random_split(n, fractions, rng):
    if abs(sum(fractions) - 1) > 1e-8:
        raise ValueError("fractions must sum to 1")
    idx = rng.permutation(n)
    cuts = (np.cumsum(fractions) * n).astype(int)
    a, b = cuts[0], cuts[1]
    return idx[:a], idx[a:b], idx[b:]
```

96.5 Experiment

Hypothesis. Choosing the polynomial degree on the test set and reporting that same test error is optimistic relative to choosing the degree on a validation set and reporting a fresh test error.

Protocol. $Y = X + 0.1X^3 + \varepsilon$, X uniform on $[-2, 2]$, noise scale 0.5, $n = 80$. Degrees $1, 3, 5, 9, 15$. Protocol A: pick the degree by test MSE. Protocol B: a 50/25/25 train-validation-test split, pick by validation, report test. 50 seeds.

Metric. Median reported test MSE of the selected degree, and the median degree chosen.

Decision rule. Protocol A's reported error should be lower than Protocol B's fresh test error. If it is not, the test sets are too small to show selection bias; increase the degree list or the number of seeds before doubting the principle.

96.6 Project

Deliverable. A split helper that supports random, entity, and time-cutoff splits, plus the polynomial selection experiment.

Constraints. The helper returns indices only. Fitting happens outside it, so the firewall is visible.

Rubric. You can explain which protocol's number you would put in a report, and you refuse Protocol A's number as a final claim.

96.7 Depth tracks

Beginner. Implement the three-way split and print sizes.

University. Run the selection-bias experiment.

Research. Describe nested cross-validation in a paragraph and say what computational cost it pays to estimate the risk of the selection rule rather than the risk of one chosen model.

96.8 Failure modes

- Fitting a vectorizer or a scaler on the full corpus before splitting text data.
- Using a time series' shuffled rows.
- "Just one peek" at the test set to confirm a knob.

96.9 See also

Volume 03 for entity contracts. Volume 15 for the same firewall in a pipeline.

97 Metrics by decision

98 Metrics by decision

Reference question. Which metric matches the decision, the base rate, and the loss you trained?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

98.1 Reference card

Item	Statement
Regression	MSE for the mean, MAE for the median, and a custom weighted loss if costs are asymmetric.
Classification	Log loss or Brier for probabilities. Accuracy only when classes are balanced and errors are symmetric.
Imbalance	Precision, recall, and false-positive rate at a stated threshold. A curve if the threshold is not yet chosen.
Ranking	A pairwise ranking metric if the decision is an order, not a label.
Cost	If missing a positive costs c times a false alarm, the threshold solves that cost, and accuracy does not.

98.2 Theory

A metric is a functional of predictions and labels. It is the right functional when it ranks procedures the way the decision would rank them. Accuracy on a 99 percent negative problem ranks the constant predictor above a detector that spends a few false alarms to catch rare positives. If the decision needs those positives, accuracy is the wrong functional even though the arithmetic is correct.

Training loss and reporting metric can differ, but the gap must be intentional. Training with log loss and reporting a thresholded cost is coherent if you keep probabilities and then choose the threshold for the cost. Training with accuracy's non-differentiable step and

hoping is not a method. Reporting five metrics and letting the best one be the story is a multiple-comparison problem from Volume 05.

Curves are for unfixed thresholds. Once a product has a capacity constraint, such as "at most 10 false alarms per hour," the operating point is fixed and a single point on the curve is the metric. Quote the constraint.

98.3 Derivation

Let a positive prediction cost c_{FP} if the label is negative and a missed positive cost c_{FN}. The expected cost of predicting positive when $P(Y=1 \mid x)=p$ is $(1-p)c_{FP}$. The expected cost of predicting negative is $p\,c_{FN}$. Predict positive when $p/(1-p) > c_{FP}/c_{FN}$, i.e. when $p > c_{FP}/(c_{FP}+c_{FN})$. The Bayes threshold is not 0.5 unless the costs are equal. Accuracy assumes they are equal and that you care about the fraction of mistakes, not their weighted cost.

98.4 Code

```
import numpy as np

def binary_counts(pred, y):
    pred = np.asarray(pred).astype(bool)
    y = np.asarray(y).astype(bool)
    tp = int(np.sum(pred & y))
    fp = int(np.sum(pred & ~y))
    fn = int(np.sum(~pred & y))
    tn = int(np.sum(~pred & ~y))
    return {"tp": tp, "fp": fp, "fn": fn, "tn": tn}

def precision_recall(counts):
    tp, fp, fn = counts["tp"], counts["fp"], counts["fn"]
    prec = tp / (tp + fp) if tp + fp else float("nan")
    rec = tp / (tp + fn) if tp + fn else float("nan")
    return {"precision": prec, "recall": rec}
```

98.5 Experiment

Hypothesis. On data with 5 percent positives, a constant negative predictor has accuracy 0.95 and recall 0. A predictor that flags the top 5 percent of a slightly informative score trades accuracy for recall.

Protocol. $n = 4000$, score equal to the latent log-odds plus noise, threshold the score so that 5 percent of rows are flagged. Compare with predicting all negative.

Metric. Accuracy, precision, recall.

Decision rule. The constant must win on accuracy and lose on recall. A write-up that ranks by accuracy alone fails the project even if the code is right.

98.6 Project

Deliverable. A metric card for a problem you define: the decision, the cost or constraint, the primary metric, two metrics you will not use as primary, and the synthetic imbalance experiment as a worked warning.

Constraints. The primary metric is one number, or one number at a predeclared operating point.

Rubric. A teammate can tell how the constant predictor scores on your primary metric.

98.7 Depth tracks

Beginner. Compute accuracy and recall for the all-negative predictor on a 5 percent base rate.

University. Derive the cost threshold and apply it to a logistic model's probabilities.

Research. Define a precision-recall curve as a set of operating points and say why the area under it is not the quantity you ship when the operating point is constrained by a review budget.

98.8 Failure modes

- Macro-averaging away a rare class you actually care about.
- Comparing models at different recall levels and calling the higher precision a better model.
- Using MSE on raw prices when the decision cares about relative error, without a transformation you can justify.

98.9 See also

Volume 06 for proper scores. Volume 16 for metrics that encode a harm you did not intend.

99 Error analysis

100 Error analysis

Reference question. How do you turn a residual into a next action?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

100.1 Reference card

Item	Statement
Slice	A subset defined by a feature available at decision time, or by a tag you can apply without the prediction.
Residual	For regression, $y - \hat{y}$. For classification, a false positive or false negative.
Mass	The slice's share of total loss, not only its average loss.
Action	More data, a new feature, a new family, a bug fix, or a decision to stop.
Not an action	"The model struggles on hard examples" with no slice and no mass.

100.2 Theory

Average error hides the slice that dominates the loss. A slice with terrible conditional error and tiny mass may not be the best place to spend the next week. A slice with moderate error and half the mass is. Compute both the conditional metric and the contribution n_{slice}/n times that metric, in the same units as the global metric.

Slices must be things you could act on. Slicing by the residual itself ("the points we got wrong") does not tell you a feature to add. Slicing by a region of X, by a data source, or by a time period does. Looking at individual losses is still useful as a bug detector: a single row with a corrupted label can dominate MSE. That is a data-contract failure, and the action is to fix the row or clip the loss, then say which you did.

Error analysis is how approximation error becomes a plan. If residuals against a linear fit are shaped like a parabola, the next family should be able to represent a parabola. If residuals are a few huge spikes, the next change is robustness or data cleaning, not a bigger network.

100.3 Derivation

Let the test loss be the average of pointwise losses ℓ_i. For a partition of the test set into slices S,

$$\frac{1}{n}\sum_{i}\ell_i = \sum_{S}\frac{n_S}{n}\left(\frac{1}{n_S}\sum_{i\in S}\ell_i\right).$$

The term in parentheses is the slice metric. The weight n_S/n turns it into mass. Ranking slices by the product is ranking them by contribution to the number you report. Ranking by the slice metric alone ranks by conditional difficulty and can send you to an empty corner of the space.

100.4 Code

```
import numpy as np

def slice_mass(losses, groups):
    losses = np.asarray(losses, dtype=np.float64)
    groups = np.asarray(groups)
    rows = []
    for g in np.unique(groups):
        mask = groups == g
        rows.append(
            {
                "slice": g.item() if hasattr(g, "item") else g,
                "n": int(mask.sum()),
                "mean_loss": float(losses[mask].mean()),
                "mass": float(losses[mask].sum() / losses.size),
            }
        )
    return sorted(rows, key=lambda r: -r["mass"])
```

100.5 Experiment

Hypothesis. On data with two groups, a model that is slightly worse on a large group contributes more total MSE than a model that is terrible on a tiny group.

Protocol. Group A, 90 percent of rows, residual variance 1. Group B, 10 percent, residual variance 9. Compute slice mass for a predictor that predicts the mean everywhere, so the loss is the variance in each group.

Metric. Mass of A versus mass of B.

Decision rule. Group A should carry more total mass despite the lower variance. An analysis that sends you to "fix B only" because its mean loss is higher needs the mass column before it is allowed to choose the work.

100.6 Project

Deliverable. An error table for the linear fit on the sine-wave data, sliced by the sign of x and by $|x|$ bins, plus one concrete next action tied to the largest mass.

Constraints. At least one slice must be checked for a single corrupted point: multiply one training label by 50, refit, and show the change in MSE.

Rubric. The action is something you can implement in a later chapter, not a mood.

100.7 Depth tracks

Beginner. Compute the two-group mass table.

University. Add the corruption check and the sine-wave slices.

Research. Discuss Simpson's paradox in one paragraph: a model can be worse on every slice of a training attribute and better overall if the slice mix differs. Say how a shifted test mix would change which mass matters.

100.8 Failure modes

- Slicing only by the prediction and calling it a population subgroup.
- Chasing the noisiest slice regardless of mass.
- Fixing a label in the test set and reporting the improved test number as a model improvement.

100.9 See also

Volume 16 for slices that correspond to people rather than to regions of x. Volume 06 for choosing the next family.

101 Cross-validation

102 Cross-validation

Reference question. How do you reuse a small sample for selection without pretending the test set grew?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

102.1 Reference card

Item	Statement
K-fold	Partition the training data into K blocks. Fit on $K-1$, score on the held block, rotate.
Mean	The average of the K scores estimates the risk of the training procedure on a sample of size about $n(1-1/K)$.
Selection	Choose the knob with the best average validation score. Refit on all training data if you will deploy.
Nested	An outer loop estimates the risk of the whole selection procedure. The inner loop selects.
Dependence	Folds that share rows' entities or times are not K independent tests.

102.2 Theory

A single train-validation split spends a chunk of data on knobs and never trains on it. K-fold cross-validation lets every training row serve as a validation row once. The price is K fits, and the score is not the risk of the model fit on all n rows. It is closer to the risk of a fit on $n(K-1)/K$ rows. When K is 5 or 10 that gap is often small relative to other errors. When $K = n$, leave-one-out trains on almost the full sample and can be expensive and high-variance as an estimate of the risk.

Cross-validation inside the training set does not create a test set. If you select on cross-validated scores and also report those same scores as the final number, you have used the data twice. The optimistic bias is usually smaller than test-set shopping, and it is still bias. A small locked test set, or a nested loop, is the honest final number.

Folds must respect the split contract. A grouped K-fold keeps an entity inside one fold. A time series uses a forward chain: train on the past, validate on the next block, and never the reverse.

102.3 Derivation

Let $\widehat{\mathrm{CV}}(\lambda) = K^{-1} \sum_{k=1}^{K} \widehat{R}^{(k)}(\hat{f}_{\lambda}^{(-k)})$, where $\hat{f}_{\lambda}^{(-k)}$ is trained without fold k. For a fixed λ, each $\widehat{R}^{(k)}$ is an average of losses on data not used to fit that function, so it is unbiased for the risk of the procedure "train on a sample of that size," up to the dependence among folds that share most of their training rows. The minimizer $\hat{\lambda} = \arg\min_{\lambda} \widehat{\mathrm{CV}}(\lambda)$ is a random knob. $\widehat{\mathrm{CV}}(\hat{\lambda})$ is a minimum and is biased low as an estimate of $R(\hat{f}_{\hat{\lambda}})$. Nested CV estimates that selected risk by putting the whole minimization inside an outer fold.

102.4 Code

```
import numpy as np

def kfold_indices(n, k, rng):
    if k < 2 or k > n:
        raise ValueError("k must be between 2 and n")
    idx = rng.permutation(n)
    folds = np.array_split(idx, k)
    splits = []
    for i in range(k):
        val = folds[i]
        train = np.concatenate([folds[j] for j in range(k) if j != i])
        splits.append((train, val))
    return splits
```

102.5 Experiment

Hypothesis. For ridge λ on a small linear problem, 5-fold CV selects a λ whose test MSE is close to the best λ on a locked test set, and the CV score of the winner is slightly optimistic relative to that test MSE.

Protocol. $n = 100$, $d = 10$, sparse linear signal, noise 1, λ on a log grid. Locked test size 500. 30 seeds.

Metric. Test MSE of the CV-chosen λ minus test MSE of the oracle λ chosen on the test set (the oracle is an evaluation-only ceiling). Also CV score minus true test MSE of the chosen model.

Decision rule. The oracle gap should be small but nonnegative on average. A large negative gap means information leaked into the folds.

102.6 Project

Deliverable. Grouped or ordinary K-fold selection of one knob, a refit on all training data using the chosen knob, and a locked test evaluation. Document whether you report the CV score, the locked test score, or both, and which one is final.

Constraints. The locked test set is not a fold.

Rubric. You can say what sample size the CV score refers to, and you do not call the CV score of the winner an unbiased test error.

102.7 Depth tracks

Beginner. Print the five pairs of index arrays and check they cover each index once as validation.

University. Run the ridge selection experiment.

Research. Implement one outer fold of nested CV and compare its estimate with the locked test estimate. Discuss variance: one outer fold is honest and noisy.

102.8 Failure modes

- Tuning on CV and shipping the CV score of the best fold rather than the average.
- Random folds on a time series.
- Refitting on train+test after selection because "the test set should not be wasted."

102.9 See also

Volume 05 for the bias of a minimum. Volume 10 for early stopping, which is this chapter applied to a step count.

103 Comparing models under uncertainty

104 Comparing models under uncertainty

Reference question. When is a measured gap between two models large enough to believe?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

104.1 Reference card

Item	Statement
Paired	Compare losses on the same test points. The unit of the interval is the per-point loss difference.
Seeds	If both models are refit, the relevant scatter includes seeds, not only the test sample.
Margin	A practical margin, declared first, is the smallest gap that would change the decision.
Interval	If the interval for the mean gap contains the margin and zero, you have not settled the question.
Ranking noise	A leaderboard sort of single runs is not an estimator with a known error bar.

104.2 Theory

Two test errors differ by some amount on every run. The question is whether the procedure difference is distinguishable from noise and from your practical margin. Pairing on test points removes a shared component: easy points are easy for both models. The variance of the difference is what matters, and it can be much smaller than the variance of either error.

A significant difference can be smaller than any effect you would ship. A non-significant difference can hide an effect you are not powered to see. Both sentences are Volume 05. Here they apply to model gaps. Report the interval for the mean paired difference and compare it with the margin.

Multiple models inflate the chance that the luckiest gap looks real. If you compared a fixed pair, say so. If you picked the pair after seeing five models, the interval for that post-selected gap does not have its nominal coverage. Predeclare the comparison or correct for the search. The bake-off project already asked you to keep the list short for this reason.

104.3 Derivation

Let $D_i = \ell(f(x_i), y_i) - \ell(g(x_i), y_i)$ on a test set that neither model was chosen with. The mean $\bar{D}$ estimates $R(f) - R(g)$ for those two frozen predictors. A normal interval is $\bar{D} \pm z s_D / \sqrt{m}$. If f and g themselves depend on the test set, this derivation does not apply. If f and g are random through the training seed and you only draw one seed, the interval omits that variance. A conservative engineering report repeats the paired test across seeds and shows the seed-wise intervals, or treats the seed-level mean gap as the observation.

104.4 Code

```
import numpy as np

def paired_gap(loss_a, loss_b):
    d = np.asarray(loss_a, dtype=np.float64) - np.asarray(loss_b, dtype=np.float64)
    mean = float(np.mean(d))
    se = float(np.std(d, ddof=1) / np.sqrt(d.size))
    z = 1.959963984540054
    return {"gap": mean, "low": mean - z * se, "high": mean + z * se}
```

104.5 Experiment

Hypothesis. Two identical predictors produce a paired interval that covers zero. A predictor with an added bias of 0.3 in its residual produces an interval that eventually excludes zero as the test size grows, and may not at a small test size.

Protocol. Shared noise, $m \in \{30, 300\}$, 200 replicates of coverage or exclusion. Practical margin 0.1 in MSE units if you compare squared residuals of a constant shift; set the simulation so you can compute the true gap.

Metric. Coverage of the true gap, and fraction of intervals excluding 0.

Decision rule. Identical predictors must cover 0 near 95 percent. The shifted predictor at $m = 30$ should often include 0 if the per-point variance is order 1. Do not call that a proof of equality.

104.6 Project

Deliverable. A paired comparison between the linear model and one flexible model on a locked test set from Volume 06, with a practical margin and a written decision: switch, stay, or collect more test data.

Constraints. The models are frozen before the test losses are computed.

Rubric. The decision quotes the interval. "The flexible model won" is not sufficient if the interval contains the margin.

104.7 Depth tracks

Beginner. Run `paired_gap` on two identical loss vectors and on a shifted pair.

University. Complete the project comparison from a real bake-off folder.

Research. Explain how a Wilcoxon signed-rank test on the D_i changes the assumptions relative to the normal interval. Say when the mean difference remains the quantity you care about even if you use a rank test as a robustness check.

104.8 Failure modes

- Comparing training losses as if they were paired test losses.
- Declaring a winner from one seed when seed-to-seed gaps change sign.
- Using an unpaired test when the models were scored on the same points and a paired test was available.

104.9 See also

Volume 05 for power. Volume 18 for how a paper should display this interval.

105 Project: a model card and an error taxonomy

106 Project: a model card and an error taxonomy

Reference question. What does a stranger need to know before reusing your fitted procedure?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

106.1 Reference card

Item	Statement
Procedure	Data contract, split, family, knobs, seed policy, and code version.
Metric	Primary metric, baseline, interval or seed spread, and test size.
Slices	Where the error mass sits.
Limits	What the procedure is not for, including populations you did not sample.
Taxonomy	A short list of error types with counts, not a gallery of anecdotes alone.

106.2 Theory

A model card is the reference page for a procedure you might reuse. It is not a press release. The card should let someone decide not to use the model. That requires the limits and the baseline, not only the best number.

An error taxonomy groups failures by cause you can distinguish: wrong label, missing feature, out-of-range input, threshold cost, or a family that cannot represent the slice. Counts turn the taxonomy into data. A card with twenty cherry-picked mistakes and no denominators is a demo.

This project closes the evaluation volume. Later volumes that train larger models still owe a card. The card gets longer. It does not become optional because the model became harder to train.

106.3 Derivation

Treat the card as a specification of the estimator $\hat{f}$ and of the measurement of $R(\hat{f})$. If a field is missing, a later user will fill it with an assumption. The dangerous assumptions are a random split when you used entities, a balanced accuracy when the base rate is 1 percent, and an in-domain test set when they will deploy out of domain. Writing “not evaluated under shift” is a result. It blocks a false claim. Volume 16 is where shift is measured rather than only confessed.

106.4 Code

```
def model_card(fields):
    required = [
        "name",
        "target",
        "split",
        "baseline",
        "primary_metric",
        "test_n",
        "limits",
    ]
    missing = [key for key in required if key not in fields or fields[key] in ("", None)]
    if missing:
        raise ValueError(f"model card missing {missing}")
    lines = [f"# {fields['name']}", ""]
    for key in required:
        lines.append(f"- {key}: {fields[key]}")
    return "\n".join(lines)
```

106.5 Experiment

Hypothesis. A card that omits `limits` is rejected by the helper, and a completed card for the constant predictor on imbalanced data states a limit that accuracy would have hidden.

Protocol. Pass a dictionary without `limits` and confirm the exception. Pass a complete card whose primary metric is recall at a fixed false-positive budget, with the limit “not evaluated on a new entity.”

Metric. Whether the incomplete card raises, and whether the complete card contains the baseline accuracy or the reason you refused it.

Decision rule. Both checks pass before you write a card for a flexible model. The constant predictor is the control card.

106.6 Project

Deliverable. A one-page card for the winner of your bake-off, an error taxonomy with at least three labeled types and counts on the test set, and the slice-mass table from earlier in this volume.

Constraints. The card names a population you did not test.

Rubric. A reader can reimplement the measurement and can also refuse the model for a stated reason.

106.7 Depth tracks

Beginner. Write the constant-predictor card only.

University. Write the bake-off card and the taxonomy.

Research. Add a known-failure-under-shift line with a concrete shift you can simulate now, such as a change in the noise scale or the base rate, and report the metric under that shift without retuning.

106.8 Failure modes

- A card that reports the training loss as performance.
- A taxonomy whose categories overlap so thoroughly that counts cannot be added.
- Limits written as “use responsibly” with no population and no metric.

106.9 See also

Volume 16 for harm and shift sections that this card should grow. Volume 15 for the owner of the deployed procedure.

Part VIII

Volume 08. Unsupervised learning

107 Volume 08. Unsupervised learning

108 Volume 08. Unsupervised learning

Stage. University

Audience. A reader who can fit a supervised baseline and state a data contract.

Question this volume answers. What can you estimate when there is no target, and how do you know you estimated it?

This volume is a reference, not a short tour. Each chapter opens with a card you can look up later, then gives the theory, a reference implementation, an experiment with a decision rule, and a project with a rubric. Read in order the first time. After that, enter through the reference card of the chapter that matches the problem in front of you.

108.1 How to use the four layers

Layer	What you should be able to do when you finish it
Theory	State the definition, the assumptions, and the result, including what the result does not say.
Code	Implement the core object in a few dozen lines and check it against a known case.
Experiment	Change one factor, record a metric with uncertainty, and say what outcome would change your mind.
Project	Produce an artifact someone else can rerun: code, figure, table, and a one-page lab note.

108.2 Depth tracks

Every chapter ends with three tracks.

- **Beginner.** Do the reference card, the code, and the smallest experiment. Skip optional proofs.
- **University.** Do the derivation, the full experiment, and the project rubric.
- **Research.** Read the failure modes, change an assumption, and write what would have to be true for the method to fail on a new distribution.

108.3 Chapters

No.	Chapter	File
01	Clustering	`vol08/01_clustering.qmd`
02	Dimensionality reduction	`vol08/02_dimensionality_reduction.qmd`
03	Density estimation and anomalies	`vol08/03_density_anomaly.qmd`
04	Mixture models	`vol08/04_mixtures.qmd`
05	Self-supervised representations	`vol08/05_self_supervised.qmd`
06	Project: evaluate a representation	`vol08/06_embedding_project.qmd`

108.4 Lab note

Keep one markdown file per experiment, `notes/YYYY-MM-DD-<slug>.md`, with the hypothesis, the command you ran, the metric, and the decision. The series is a reference only if the experiments you run are attached to it.

109 Clustering

110 Clustering

Reference question. What does a clustering method minimize, and what does a cluster not mean?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

110.1 Reference card

Item	Statement
K-means	Partition points into K sets and place a center at each set's mean, minimizing total squared distance to the assigned center.
Alternation	Assign each point to the nearest center, then recompute means. Each step drops the objective. The result can be a poor local minimum.
Shape	Squared Euclidean K-means prefers blobs of similar variance. It splits long ellipses and chains.
K	A knob. Likelihood-style scores on the same points used to fit are optimistic. Stability across seeds is a partial check.
Not	A cluster is not a type of person, a market segment, or a cause, unless a later measurement says so.

110.2 Theory

Clustering assigns labels the algorithm invented. Those labels can be useful features, compression codes, or hypotheses. They are not supervised targets. Evaluating them with a supervised accuracy requires a human labeling you claim to match, and then you are in a supervised problem with a fuzzy label.

K-means is a vector-quantization method with squared error. Standardize features or the coordinate with the largest variance owns the distance. A random initialization can land

in a bad partition; several restarts and the best objective are part of the estimator. The objective falling is not evidence that K is correct. A larger K always can achieve a lower or equal training objective.

Stability is a sanity check: if two runs with different seeds disagree on almost every point, the partition is not a feature of the data under this method. Agreement does not make the clusters real in a scientific sense. It makes the algorithm repeatable.

110.3 Derivation

Given centers, the optimal assignment of a point is the nearest center, because the objective is a sum of independent squared distances. Given assignments, the optimal center of a cluster is the mean, because squared error around a constant is minimized by the mean. The alternation therefore cannot increase the objective. It reaches a fixed point in finitely many assignments when ties are broken deterministically, but not necessarily a global minimizer. A standard bad case is two centers that fall into one blob and leave another blob empty or split.

110.4 Code

```
import numpy as np

def kmeans(X, k, rng, n_init=5, steps=30):
    X = np.asarray(X, dtype=np.float64)
    best = None
    for _ in range(n_init):
        centers = X[rng.choice(X.shape[0], size=k, replace=False)].copy()
        labels = np.zeros(X.shape[0], dtype=int)
        for _step in range(steps):
            d2 = ((X[:, None, :] - centers[None, :, :]) ** 2).sum(axis=-1)
            labels = np.argmin(d2, axis=1)
            for j in range(k):
                mask = labels == j
                if np.any(mask):
                    centers[j] = X[mask].mean(axis=0)
        inertia = float(((X - centers[labels]) ** 2).sum())
        if best is None or inertia < best[0]:
            best = (inertia, centers, labels)
    return {"inertia": best[0], "centers": best[1], "labels": best[2]}
```

110.5 Experiment

Hypothesis. On two round Gaussian blobs, K-means with $K = 2$ and several restarts recovers the generating groups up to label permutation. On one elongated Gaussian, $K = 2$ cuts the ellipse rather than refusing to split it.

Protocol. Blobs centered at $(\pm 3, 0)$ with identity covariance, 200 points each. Ellipse: covariance $\mathrm{diag}(8, 0.2)$, 400 points. Standardize only within a run you declare. Compare a few seeds.

Metric. For the blobs, accuracy after the best label flip. For the ellipse, the fraction of points on each side of the cut, and a sentence.

Decision rule. Blob recovery should be near perfect. If the ellipse run returns two groups of very different radius in the long direction, you have illustrated the squared-error bias. Do not "fix" it by switching K after seeing a desired story.

110.6 Project

Deliverable. The two simulations, a note on scaling, and a clustering of a dataset only if you also state what decision the cluster label would feed. If you have no decision, stop at the simulations.

Constraints. Report inertia and a stability check across three seeds.

Rubric. You explicitly say "these clusters are a quantization of squared distance," not "we discovered two kinds of users," unless you brought an external measurement.

110.7 Depth tracks

Beginner. Run K-means on the two blobs.

University. Add the ellipse counterexample and the scaling note.

Research. Compare K-means with a Gaussian mixture at $K = 2$ on the ellipse. Say which likelihood each method is closer to, and why equal spherical covariance is the K-means-like mixture.

110.8 Failure modes

- Choosing K by the inertia on the fitting data.
- Clustering on the target column.
- Interpreting a cluster of missingness codes as a behavior.

110.9 See also

The mixture chapter later in this volume. Volume 07 if a human label arrives and the problem becomes classification.

111 Dimensionality reduction

112 Dimensionality reduction

Reference question. What information does a low-dimensional projection keep, and what does it destroy?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

112.1 Reference card

Item	Statement
PCA	Orthogonal directions of maximal variance. The first r principal components minimize squared reconstruction error among rank-r linear maps of that form.
SVD	After centering, the principal directions are the right singular vectors of the data matrix.
Variance explained	The sum of the kept eigenvalues divided by the sum of all eigenvalues. A reconstruction metric, not a prediction metric.
Not invariant	Scaling a feature changes the components. Centering is part of the method.
Supervision	PCA ignores Y. A direction of small variance can be the only predictive one.

112.2 Theory

PCA compresses x into scores $z = V_r^\top (x - \bar{x})$ and reconstructs $\tilde{x} = \bar{x} + V_r z$. The discarded coordinates are the ones with least variance, not the ones with least predictive value. Use PCA to denoise a measurement you believe is approximately low rank, to visualize, or to build a baseline representation. Check a supervised model on the original features before you trust the compressed ones.

The number r is a knob. Reconstruction error on the fitting set falls as r grows. A validation reconstruction error, or a downstream validation risk, is the selection criterion. A scree plot is a picture, not a test.

Nonlinear embeddings can preserve local neighborhoods that PCA tears. They also hide the map. If you cannot transform a new point, you have a picture of the training set, not a feature function. A reference implementation should define the map on new x.

112.3 Derivation

Let the centered data matrix be X. Among rank-r matrices, the minimizer of $\|X - M\|_F$ is the truncated SVD $U_r \Sigma_r V_r^\top$. The columns of V_r are principal axes. The Frobenius identity $\|X\|_F^2 = \sum_j \sigma_j^2$ says the discarded energy is the sum of the squared singular values you drop. Because centering was done first, this energy is about variance, not about the mean. A new point uses the same $\bar{x}$ and V_r learned on the training matrix. Recomputing PCA on train plus test leaks.

112.4 Code

```
import numpy as np

def pca_fit(X, r):
    X = np.asarray(X, dtype=np.float64)
    mean = X.mean(axis=0)
    xc = X - mean
    _u, s, vt = np.linalg.svd(xc, full_matrices=False)
    r = min(r, vt.shape[0])
    return {"mean": mean, "components": vt[:r], "singular_values": s}

def pca_transform(model, X):
    return (np.asarray(X, dtype=np.float64) - model["mean"]) @ model["components"].T
```

112.5 Experiment

Hypothesis. On data whose third coordinate is small noise, $r = 2$ reconstruction catches the signal plane. A supervised label that depends only on the third coordinate is lost by that PCA, and a linear model on the scores fails while a linear model on the raw third coordinate works.

Protocol. X_1, X_2 standard normal, $X_3 \sim \mathcal{N}(0, 0.01^2)$, $Y = 5X_3 + \varepsilon$. Fit PCA on training X only.

Metric. Reconstruction MSE of X, and test MSE of Y from PCA scores versus from X_3 alone.

Decision rule. Reconstruction can look excellent while prediction from scores is near the constant baseline. If prediction from scores works, X_3 was not actually low variance.

112.6 Project

Deliverable. The simulation, plus PCA on a dataset with a declared scaling policy, a plot or table of cumulative energy, and a downstream linear model on the scores compared with the same model on the original features.

Constraints. The mean and components come from training rows only.

Rubric. You state whether you care about reconstruction or about predicting Y, and the conclusion matches that purpose.

112.7 Depth tracks

Beginner. Fit PCA with $r = 1$ on a 2D line of points and reconstruct.

University. Run the low-variance predictive coordinate experiment.

Research. Relate the PCA truncation to the ridge shrink factor from Volume 01. Both shrink directions associated with small singular values, under different objectives. Write the two formulas side by side.

112.8 Failure modes

- Fitting PCA on all rows, then splitting.
- Interpreting a component loading as a causal factor.
- Choosing r because it explains 95 percent of variance when the predictive signal is in the last 5 percent.

112.9 See also

Volume 01 for the SVD. Volume 09 for learned nonlinear representations that are trained with a target or with a self-supervised loss.

113 Density estimation and anomalies

114 Density estimation and anomalies

Reference question. How do you score whether a new point looks like the training distribution?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

114.1 Reference card

Item	Statement
Density	An estimate $\hat{p}(x)$ of the law of inputs, trained without labels, or of residuals, trained on normal operation.
Anomaly score	Often $-\log \hat{p}(x)$ or a distance to a fitted center. A threshold turns the score into a decision and needs a false-alarm budget.
Fit	Use normal data only, or you will teach the detector that failures are typical.
Dimension	A kernel density estimate needs a bandwidth and suffers as d grows. A simple Gaussian model is biased and usable as a baseline.
Evaluation	If you have labeled anomalies, this becomes a supervised detection metric at a fixed false-alarm rate. If you do not, you only have a score.

114.2 Theory

Anomaly detection is a one-class or low-label problem. The training contract is: these rows are the normal class, or they are unlabeled and you are willing to assume anomalies are rare enough not to dominate the fit. A Gaussian density on standardized features is the baseline. Points with large Mahalanobis distance are the candidates. The threshold is a Volume 07 operating point. Without labeled failures you can still set it to flag a chosen fraction of a validation stream, and you cannot quote a recall.

High dimension makes fixed-bandwidth kernel density estimates concentrate in misleading ways: distances become similar and the likelihood values become extreme. A low-dimensional projection or a simple model with a checked residual is often more honest than a flexible density you cannot evaluate.

If the process drifts, yesterday's normal is today's false alarm. A density is a model of a time window. Put the window in the card.

114.3 Derivation

For $X \sim \mathcal{N}(\mu, \Sigma)$, $-2 \log p(x)$ equals the Mahalanobis distance plus $\log \det \Sigma$ plus a constant. Ranking points by Mahalanobis distance is ranking them by Gaussian likelihood. The squared distance is approximately χ^2_d when the Gaussian assumption holds, which suggests a threshold, and the suggestion is wrong when the tails are heavier. A validation histogram of the score on held-out normal data is the empirical threshold source. Labeled anomalies, if they exist, are a test set and do not choose the threshold.

114.4 Code

```
import numpy as np

def gaussian_anomaly_score(X_train, X_new, ridge=1e-6):
    mu = X_train.mean(axis=0)
    xc = X_train - mu
    cov = (xc.T @ xc) / X_train.shape[0] + ridge * np.eye(X_train.shape[1])
    left = np.linalg.cholesky(cov)
    delta = np.asarray(X_new, dtype=np.float64) - mu
    # Solve L w = delta^T for whitened coordinates.
    w = np.linalg.solve(left, delta.T)
    return np.sum(w**2, axis=0)
```

114.5 Experiment

Hypothesis. Inliers drawn from the training Gaussian have smaller scores than points shifted by several standard deviations, and a heavier-tailed inlier distribution produces more high scores than the χ^2 story expects.

Protocol. Train on 500 standard normal points in $\mathbb{R}^4$. Score 500 fresh inliers and 200 points with mean $\mathbf{1}$. Then score Student-t points with 3 degrees of freedom as if they were anomalies under a Gaussian fit.

Metric. Fraction of shifted points above the 99th percentile of training scores, and the same fraction for t inliers.

Decision rule. The shifted cloud should mostly flag. The t inliers should flag more often than 1 percent. If they do not, the ridge or the percentile is masking tails; look at the score histogram before adding model complexity.

114.6 Project

Deliverable. A Gaussian anomaly baseline with a predeclared false-alarm rate on a validation slice of normal data, plus one injected shift. If you add a second detector, compare them at the same false-alarm rate.

Constraints. The threshold is not chosen on the injected anomalies.

Rubric. You report false-alarm rate and detection rate separately. A single accuracy is not accepted.

114.7 Depth tracks

Beginner. Score a 1D normal sample and set a threshold at mean plus three standard deviations.

University. Run the shift and heavy-tail experiment in $\mathbb{R}^4$.

Research. Explain why a deep density model can assign higher likelihood to a constant input than to training images, as a caution that likelihood is not a perceptual typicality score. You do not need to reproduce a paper; state what evaluation you would require before trusting such a score.

114.8 Failure modes

- Fitting the "normal" model on a mixture of failures and normal operation.
- Choosing the threshold to maximize detection on the only labeled anomalies you have, and then quoting that detection rate.
- Ignoring drift and paging a human for a new typical regime.

114.9 See also

Volume 04 for the Gaussian tail. Volume 16 for adversaries who try to sit inside the normal score.

115 Mixture models

116 Mixture models

Reference question. What does a mixture claim that K-means does not, and how do you fit it?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

116.1 Reference card

Item	Statement
Model	$p(x) = \sum_{k=1}^{K} \pi_k p(x \mid k)$, with π on the simplex.
Latent	A component indicator Z. The responsibilities $p(Z = k \mid x)$ are soft assignments.
EM	Alternate between computing responsibilities under current parameters and maximizing the expected complete log likelihood.
K-means link	Hard assignments and equal spherical Gaussians, with variances shrunk toward zero, approach a K-means-like objective.
Likelihood	The observed-data likelihood is not convex in the parameters. Restarts matter. A larger K fits the training density better and can overfit.

116.2 Theory

A mixture is a density with a discrete latent cause. It gives you a likelihood, a way to score new points, and soft memberships. K-means gives you a partition and a squared-error objective. Use a mixture when you want probabilities or ellipses with covariance. Use K-means when you want a simple quantization and will not pretend you fit a density.

EM is coordinate ascent on a lower bound of the log likelihood. The bound is tight at the current responsibilities, so the observed log likelihood should not decrease. It can converge to a bad local maximum, and a component can collapse onto one point, sending a variance

toward zero and the likelihood toward infinity in a Gaussian mixture without a variance floor. A floor or a prior on the covariance is part of a practical estimator.

Selecting K by training likelihood is invalid in the naive way: more components win. Hold out log likelihood, or a penalized criterion you can define, on a validation set. Even then, a good density estimate is not a discovery of K true types.

116.3 Derivation

For a Gaussian mixture, the E step sets

$$r_{ik} \propto \pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k), \quad \sum_k r_{ik} = 1.$$

The M step for the weights is $\pi_k = n^{-1} \sum_i r_{ik}$. The mean μ_k is the responsibility-weighted average of x_i. These updates maximize the expected complete-data log likelihood given the responsibilities. The observed log likelihood after the M step is at least the previous one when the maximization is exact. In code, compute responsibilities in the log domain or the first term underflows to zero and a component dies by rounding.

116.4 Code

```
import numpy as np

def log_gaussian_diag(X, mu, var):
    # var is a vector of per-coordinate variances.
    X = np.asarray(X, dtype=np.float64)
    mu = np.asarray(mu, dtype=np.float64)
    var = np.asarray(var, dtype=np.float64)
    return -0.5 * np.sum(np.log(2 * np.pi * var) + (X - mu) ** 2 / var, axis=-1)
```

116.5 Experiment

Hypothesis. On two well-separated 1D Gaussians, a two-component EM recovers means near the truth and a higher validation log likelihood than a one-component Gaussian. On a single Gaussian, the two-component model does not win by a meaningful validation margin if you keep a variance floor.

Protocol. Implement EM for diagonal or 1D Gaussians with a variance floor of 10^{-3}, several restarts, $K \in \{1, 2\}$. Validation size at least 1000.

Metric. Parameter error when K matches the generator, and validation log likelihood.

Decision rule. The separated case should favor $K = 2$. The single-Gaussian case should not produce a dramatic likelihood win for $K = 2$. A huge win with a near-zero variance means the floor failed and a component collapsed.

116.6 Project

Deliverable. A 1D two-component EM with a log-likelihood trace that is monotone, a variance floor, and the model-selection comparison above.

Constraints. Responsibilities are computed from log densities with a log-sum-exp, not from raw densities that underflow.

Rubric. You can show the monotone trace and a restart that found a worse likelihood.

116.7 Depth tracks

Beginner. Fit a single Gaussian by the MLE formulas and score a validation set with `log_gaussian_diag`.

University. Implement EM and the two simulations.

Research. Derive the EM auxiliary function inequality $ ( ) Q(') $ in words and identify the step that becomes a hard assignment in the small-variance limit.

116.8 Failure modes

- Selecting K on training likelihood.
- Letting a component variance go to zero.
- Describing mixture weights as the prior probability of a real latent type without a measurement model.

116.9 See also

Volume 04 for the Gaussian MLE. Volume 13 for mixture-of-experts and latent-variable generative models, which inherit this nonconvexity.

117 Self-supervised representations

118 Self-supervised representations

Reference question. What target are you manufacturing when you train without labels?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

118.1 Reference card

Item	Statement
Pretext	A supervised task built from the input alone, such as predicting a masked coordinate or identifying a transformed copy.
Representation	Intermediate features $h(x)$ that you will reuse for a later task.
Leakage	The pretext can be solved by a shortcut that ignores the structure you wanted, such as a border artifact or a timestamp.
Evaluation	A downstream task with a frozen or fine-tuned h, compared with a baseline that does not use h.
Not free labels	A pretext loss going down does not mean h helps the decision you care about.

118.2 Theory

Self-supervision turns unsupervised data into a supervised problem by hiding part of x and asking the model to recover it, or by declaring two augmentations of the same row to be a match. The hope is that a representation sufficient for the pretext is useful for a later target Y. That hope is an empirical claim about the relationship between the pretext and Y. It can fail when the pretext is solved by a feature that is independent of Y.

Evaluate h by a simple downstream head, often linear, trained on a modest labeled set, with the same split discipline as Volume 07. Compare with the raw features and with PCA scores. If a linear head on h cannot beat a linear head on x, you have not yet earned a complicated encoder.

Augmentations are part of the model. If your augmentation destroys the signal, for example by shuffling the only predictive coordinate, the representation will learn what remains, which may be nothing you wanted. Write the augmentation next to the loss.

118.3 Derivation

A denoising objective $\mathbb{E}[\|x - g(h(\tilde{x}))\|^2]$, where $\tilde{x}$ is a corrupted copy, is minimized by a $g \circ h$ that approximates the conditional mean of x given $\tilde{x}$, under squared error. The representation h is whatever bottleneck you force that conditional mean through. If the corruption leaves a coordinate untouched and that coordinate already determines x, h can ignore everything else. The conditional-independence chapter is the test: if $Y \perp h(X) \mid s(X)$ for a shortcut s, the downstream task will not see new information.

118.4 Code

```
import numpy as np

def mask_and_target(X, rng, mask_prob=0.3):
    X = np.asarray(X, dtype=np.float64)
    mask = rng.random(X.shape) < mask_prob
    corrupted = X.copy()
    corrupted[mask] = 0.0
    return corrupted, X, mask
```

118.5 Experiment

Hypothesis. Predicting a masked coordinate of a two-dimensional Gaussian with strong correlation produces a representation that helps impute the missing coordinate, and does not automatically help predict an independent label.

Protocol. $X_2 = X_1 +$ small noise, Y independent of X. Train a least-squares map from the unmasked coordinate to the masked one on corrupted rows. Evaluate imputation MSE against predicting the mean. Evaluate predicting Y from the imputed features versus from X.

Metric. Imputation MSE and test MSE for Y.

Decision rule. Imputation should beat the mean. Prediction of Y should not beat the constant baseline by a meaningful margin. A win on Y means the generator coupled Y to X.

118.6 Project

Deliverable. The masking experiment, plus a one-paragraph pretext design for a data type you care about, including the shortcut you are afraid of and the downstream check that would catch it.

Constraints. The downstream check uses labels that did not enter the pretext fit.

Rubric. You do not claim a representation is useful because the pretext loss decreased.

118.7 Depth tracks

Beginner. Implement masking and a mean-imputation baseline.

University. Complete the correlated-coordinate versus independent-label experiment.

Research. Describe a contrastive pretext in one formula: pull together two views of the same row and push apart views of different rows. State a shortcut (a shared border, a camera id) that would satisfy the contrastive loss without helping Y.

118.8 Failure modes

- Tuning the pretext on the downstream test metric without a validation firewall.
- Augmentations that remove the class-relevant signal and a celebration of invariance to that signal.
- Using labels to build the pretext and still calling the method unsupervised.

118.9 See also

Volume 11 for representation learning with images. Volume 17 for large pretrained models, which are this idea at a scale that does not change the evaluation logic.

119 Project: evaluate a representation

120 Project: evaluate a representation

Reference question. What evaluation would convince you to replace raw features by a learned or reduced representation?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

120.1 Reference card

Item	Statement
Candidates	Raw features, PCA scores, and one learned or clustered representation.
Probe	A linear model or logistic regression on each representation, with the same split.
Budget	The same number of labeled training rows for each probe.
Map	A function that applies to new rows. A layout of the training points is not enough.
Decision	Adopt, reject, or keep as a visualization only.

120.2 Theory

This project stops unsupervised methods from grading their own homework. The representation may be built without Y, but the decision to use it is supervised if you have Y. If you do not have Y, the only honest products are reconstruction, density, or a declared quantization error, and the card should say "no downstream task."

Give every candidate the same labeled budget. A representation that needs 10 times more labels to win has not won at the budget you have. Freeze the representation before the probe is selected, or include its knobs inside the validation loop and pay for that selection.

Visualization is a separate deliverable. A 2D picture can be pedagogically useful and predictively harmful. The project requires you to say which of the two you are claiming.

120.3 Derivation

Let $R(h)$ be the risk of the best linear probe on representation h inside a fixed probe family, and let $\hat{R}(h)$ be a validation estimate. You adopt h over raw x when $\hat{R}(h)$ beats $\hat{R}(\mathrm{id})$ by the practical margin after the probe's own regularization is chosen fairly. The gap $R(\mathrm{id}) - R(h)$ can be negative even when h is a sufficient statistic for a nonlinear family, because the probe is linear. A loss with a linear probe is a test of linear readability, not of whether any head could use h. If you allow a flexible head, say so, and give the raw features the same flexible head or you have confounded the representation with the head.

120.4 Code

```
import numpy as np

def linear_probe_mse(Z_train, y_train, Z_test, y_test, lam=1e-3):
    Z_train = np.asarray(Z_train, dtype=np.float64)
    Z_test = np.asarray(Z_test, dtype=np.float64)
    y_train = np.asarray(y_train, dtype=np.float64).reshape(-1)
    y_test = np.asarray(y_test, dtype=np.float64).reshape(-1)
    ones_tr = np.concatenate([np.ones((Z_train.shape[0], 1)), Z_train], axis=1)
    ones_te = np.concatenate([np.ones((Z_test.shape[0], 1)), Z_test], axis=1)
    penalty = lam * np.eye(ones_tr.shape[1])
    penalty[0, 0] = 0.0
    theta = np.linalg.solve(ones_tr.T @ ones_tr + penalty, ones_tr.T @ y_train)
    pred = ones_te @ theta
    return float(np.mean((pred - y_test) ** 2))
```

120.5 Experiment

Hypothesis. PCA with r large enough to keep a predictive coordinate matches the raw-feature probe. PCA that drops that coordinate loses. A cluster id represented as a single integer code is a poor linear feature unless you one-hot it, and the one-hot probe still loses to the raw coordinates on a linear truth.

Protocol. Use the low-variance predictive coordinate construction from the PCA chapter, and a separate two-blob dataset where the blob id is a one-hot feature. Same labeled budget, λ fixed in advance or chosen on validation.

Metric. Probe test MSE or log loss, with a paired comparison if you built per-point losses.

Decision rule. The representation that discarded the predictive coordinate must lose. If the one-hot cluster id wins on the blob data, that is allowed: the cluster recovered the Bayes partition. Say so as a special case, not as a general endorsement of clustering.

120.6 Project

Deliverable. A table of probes, the practical margin, the paired gap if you have per-point losses, and a decision: ship the representation, use it only for a picture, or discard it. Include the map's definition on a new row.

Constraints. The same labels and the same split for every row of the table.

Rubric. The decision would be unchanged if you replaced adjectives with the table. A picture without a probe is labeled as a picture.

120.7 Depth tracks

Beginner. Probe raw features versus the first principal component on a linear problem.

University. Complete the discard-the-signal and the cluster one-hot comparisons.

Research. Add a self-supervised imputation feature from the previous chapter to the table and interpret a negative result without changing the pretext after seeing the probe.

120.8 Failure modes

- A flexible head on the learned features and a linear head on the raw features.
- Fitting the representation on the test rows.
- Declaring success from a 2D scatter plot of the training set.

120.9 See also

Volume 11 and Volume 17, where the same probe protocol applies to features you did not train yourself.

Part IX

Volume 09. Neural networks

121 Volume 09. Neural networks

122 Volume 09. Neural networks

Stage. University

Audience. A reader who can differentiate a vector loss and can run the gradient-check harness.

Question this volume answers. What function does a network represent, how do gradients flow, and which parts are inductive bias rather than magic?

This volume is a reference, not a short tour. Each chapter opens with a card you can look up later, then gives the theory, a reference implementation, an experiment with a decision rule, and a project with a rubric. Read in order the first time. After that, enter through the reference card of the chapter that matches the problem in front of you.

122.1 How to use the four layers

Layer	What you should be able to do when you finish it
Theory	State the definition, the assumptions, and the result, including what the result does not say.
Code	Implement the core object in a few dozen lines and check it against a known case.
Experiment	Change one factor, record a metric with uncertainty, and say what outcome would change your mind.
Project	Produce an artifact someone else can rerun: code, figure, table, and a one-page lab note.

122.2 Depth tracks

Every chapter ends with three tracks.

- **Beginner.** Do the reference card, the code, and the smallest experiment. Skip optional proofs.
- **University.** Do the derivation, the full experiment, and the project rubric.

- **Research.** Read the failure modes, change an assumption, and write what would have to be true for the method to fail on a new distribution.

122.3 Chapters

No.	Chapter	File
01	Computation graphs	`vol09/01_computation_graph.qmd`
02	Backpropagation	`vol09/02_backprop.qmd`
03	Architectures and inductive bias	`vol09/03_inductive_bias.qmd`
04	Initialization and activations	`vol09/04_initialization.qmd`
05	What universal approximation does not give you	`vol09/05_approximation.qmd`
06	Project: a network in NumPy	`vol09/06_numpy_network_project.qmd`

122.4 Lab note

Keep one markdown file per experiment, `notes/YYYY-MM-DD-<slug>.md`, with the hypothesis, the command you ran, the metric, and the decision. The series is a reference only if the experiments you run are attached to it.

123 Computation graphs

124 Computation graphs

Reference question. How should you represent a network so that a forward value and a backward gradient are the same object?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

124.1 Reference card

Item	Statement
Graph	A directed acyclic graph of tensors. Each node is a function of its parents.
Forward	Evaluate nodes in topological order. Cache the inputs each node needs for its derivative.
Backward	Apply the chain rule from the scalar loss toward the parameters, using cached values.
Parameter	A leaf the loss depends on and that you intend to update. A constant is a leaf you do not update.
Shape	Every node has a shape contract. A broadcast inside a node is part of the contract, not an accident.

124.2 Theory

A network is a function built by composition. The computation graph is the bookkeeping that makes the composition differentiable and implementable. Autodiff libraries hide the graph. A reference implementation should show one graph you can draw: an affine map, a pointwise nonlinearity, another affine map, and a loss. If you cannot name the intermediate tensors, you cannot check them.

Caching is required because the local derivative often depends on the forward value. ReLU's derivative depends on the sign of the pre-activation. A sigmoid's derivative can be written from the sigmoid output. Throwing away the forward pass and hoping to differentiate the formula from parameters alone is algebraically possible and operationally the wrong interface.

The loss is a node that reduces to a scalar. Gradients are defined for that scalar. If you forget the reduction and backpropagate a vector, you have invented an implicit sum or you have a bug. Say which reduction you used, mean or sum, because it rescales every gradient and therefore the learning rate.

124.3 Derivation

Let $z = Wx + b$, $h = \phi(z)$, and L a scalar depending on h. The chain rule in differentials is $dL = \langle \nabla_h L, dh \rangle$ and $dh = \phi'(z) \odot dz$, so $\nabla_z L = \nabla_h L \odot \phi'(z)$. From Volume 01, $\nabla_W L = (\nabla_z L) x^\top$ and $\nabla_b L = \nabla_z L$. A graph engine stores $\nabla_z L$ as the message sent to the affine node. Nothing in this calculation requires W to be small. It requires the shapes of $\nabla_z L$ and z to match.

124.4 Code

```
import numpy as np

def affine_forward(x, W, b):
    return x @ W + b, (x, W, b)

def affine_backward(grad_z, cache):
    x, W, _b = cache
    grad_W = x.T @ grad_z
    grad_b = grad_z.sum(axis=0)
    grad_x = grad_z @ W.T
    return grad_x, grad_W, grad_b
```

124.5 Experiment

Hypothesis. The affine backward pass matches central differences on a scalar loss $L = \sum z_{ij}^2$ through $z = xW + b$, at a small random shape.

Protocol. $n = 4$, input dimension 3, output dimension 2. Use the Volume 02 harness on the flattened parameters of W and b.

Metric. Relative gradient gap at $\varepsilon = 10^{-6}$.

Decision rule. The gap should be below 10^{-6}. A gap near 1 often means a missing transpose or a mean/sum mismatch. Fix that before any training chapter.

124.6 Project

Deliverable. A drawn graph for a two-layer network, the affine check, and a note that states whether your training loss is a mean or a sum.

Constraints. The check uses float64 and a deterministic loss.

Rubric. The relative gap is in the lab note next to the picture of the graph.

124.7 Depth tracks

Beginner. Implement affine forward and check one matrix product by hand.

University. Pass the gradient check for W and b.

Research. Add a batch-norm or layer-norm node only after you can write its forward and backward contracts. If you use a library instead, check one parameter gradient against a numerical difference with the library's own forward.

124.8 Failure modes

- Broadcasting a bias of the wrong rank and accepting the forward value because it is finite.
- Reducing with `mean` in the loss and `sum` in the hand-derived gradient.
- Detaching a tensor so the graph looks connected and the gradient is zero.

124.9 See also

The next chapter for the nonlinearity. Volume 02 for the harness you must reuse.

125 Backpropagation

126 Backpropagation

Reference question. What does backpropagation compute, and which bugs does a gradient check miss if you only test one layer?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

126.1 Reference card

Item	Statement
Output	The gradient of a scalar loss with respect to each registered parameter.
Local	Each node contributes a Jacobian-vector product, not a materialised Jacobian, when the output dimension is large.
ReLU	The local factor is $1\{z > 0\}$, with a declared value at 0.
Sigmoid	The local factor is $s(1 - s)$ if $s = \sigma(z)$.
Init	Gradients can vanish or explode with depth even when the code is correct. That is a modeling issue, checked separately from the chain rule.

126.2 Theory

Backpropagation is the chain rule evaluated from the loss backward, reusing the forward cache. It is not an approximation. If the numerical check fails, the code is wrong or the point sits on a kink you did not define. If the check passes and training still diverges, the learning rate, the scaling, or the loss surface is the suspect, not the chain rule.

Deep stacks multiply local factors. A product of many numbers smaller than 1 vanishes. A product of numbers larger than 1 explodes. ReLU's factor is 0 or 1, so it avoids shrinking, and it kills gradients on the inactive side. Sigmoid factors are at most 1/4. These facts are why activation choice and initialization share a chapter later. Do not "fix" a vanished gradient by multiplying a fudge factor you cannot explain.

A two-layer check should cover the composition, not only the affine piece. Bugs hide in the interaction: a ReLU backward that uses the wrong pre-activation, or a loss backward that forgets a factor of 2.

126.3 Derivation

Let $h = \mathrm{ReLU}(z) = \max(z, 0)$ elementwise, and define the derivative at 0 to be 0. Then $dh = 1\{z > 0\} \odot dz$, so the incoming gradient is multiplied by that mask. For $s = \sigma(z)$, $ds = s(1-s) \odot dz$. For squared error $L = \|h-y\|_2^2$, $\nabla_h L = 2(h-y)$. Compose these messages to get $\nabla_z L$. The parameter gradients then use the affine rules. Write this composition for one hidden layer before you write a loop over layers.

126.4 Code

```
import numpy as np

def relu(z):
    return np.maximum(z, 0.0), (z > 0.0)

def relu_backward(grad_h, mask):
    return grad_h * mask

def mse_backward(pred, y):
    return 2.0 * (pred - y)
```

126.5 Experiment

Hypothesis. A one-hidden-layer map $f(x) = W_2 \mathrm{ReLU}(W_1 x + b_1) + b_2$ has analytic gradients matching central differences at a random parameter, on a single example, for the squared loss.

Protocol. Input dimension 3, hidden 4, output 2, float64, $\varepsilon = 10^{-6}$. Include a point where some hidden units are off, so the mask is not all ones.

Metric. Relative gap for each parameter tensor.

Decision rule. Every gap below 10^{-5}. If only b_1 fails, the mask or the sum over the batch is wrong. Do not start the training volume on a failed check.

126.6 Project

Deliverable. A minimal network class with `forward`, `backward`, and `grad_check`, plus a lab note of the gaps. No library autodiff in the checked path.

Constraints. One hidden layer is enough. The loss reduction is written in the note.

Rubric. The check covers a case with inactive ReLU units. A check where every unit is active is marked incomplete.

126.7 Depth tracks

Beginner. Check the ReLU mask on a vector with positive and negative entries.

University. Pass the full one-hidden-layer gradient check.

Research. Replace the scalar loss by a softmax cross-entropy and derive the gradient in the logits, $p - y_{\text{one hot}}$. Check it. This is the classification head you will actually train.

126.8 Failure modes

- Checking gradients only at initialization when every ReLU happens to be active, then failing later.
- Using an in-place operation that corrupts the cached pre-activation.
- Comparing a mean-reduced library gradient with a sum-reduced hand gradient and "fixing" it by changing the mathematics at random.

126.9 See also

Volume 02's harness. Volume 10, which assumes this check already passed.

127 Architectures and inductive bias

128 Architectures and inductive bias

Reference question. What does an architecture assume about the data, before any weight is fit?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

128.1 Reference card

Item	Statement
Bias	A restriction on the functions the model can represent easily, or at all.
MLP	A stack of affine maps and pointwise nonlinearities. It assumes little spatial structure and pays for that with data.
Weight sharing	The same parameter applied at many positions, as in a convolution or a recurrent step. It assumes a repeated local rule.
Equivariance	Transforming the input transforms the output in a matching way. A claim about the architecture, true only if the implementation honors it.
Choice	Pick the bias that matches a symmetry you are willing to defend. A flexible model is a bias toward needing more data.

128.2 Theory

An architecture is a family $\mathcal{F}$. The inductive bias is what that family prefers. Without a preference, finite data do not select a unique continuation of a finite sample. Convolutions prefer local, translation-structured rules. Attention, in a later volume, prefers content-based routing and pays quadratic cost in sequence length unless you change it. A fully connected layer on pixels prefers nothing about neighborhoods and must learn locality from scratch.

Bias is not a vibe. It is a statement you can falsify. If a convolution cannot represent a function that depends on absolute position, and your labels depend on absolute position,

the architecture is wrong or you must add a position feature. If it can represent the function only by using many filters to simulate a lookup table, it can represent it in principle and still be the wrong bias in practice.

Depth changes which functions are compact. A deep composition can represent some hierarchical functions with fewer parameters than a shallow network of the same family. That is a theorem in restricted settings and a tendency in practice. It is not a reason to add layers when a linear model has not been beaten.

128.3 Derivation

A 1D convolution with kernel $w \in \mathbb{R}^k$ and stride 1, ignoring boundaries, produces $(w * x)_t = \sum_{u=0}^{k-1} w_u x_{t+u}$. Shifting the input by s shifts the output by s. That is equivariance to translation. A fully connected layer Wx does not have this property unless W itself is constrained to be constant along diagonals, which is the convolution written as a matrix. Global average pooling then turns translation equivariance of the feature map into translation invariance of the pooled vector. If your label flips when the pattern moves, invariance is the wrong last step.

128.4 Code

```
import numpy as np

def conv1d_same_width(x, w):
    x = np.asarray(x, dtype=np.float64)
    w = np.asarray(w, dtype=np.float64)
    pad = len(w) // 2
    xp = np.pad(x, pad)
    out = np.array([np.dot(w, xp[i : i + len(w)]) for i in range(len(x))])
    return out
```

128.5 Experiment

Hypothesis. `conv1d_same_width` commutes with a shift: shifting a long input and then convolving matches convolving and then shifting, away from the borders. A dense random matrix does not.

Protocol. A random kernel of length 3, an input of length 30, shift by 4. Compare the overlapping region. Apply a random 30×30 matrix to the input and to the shifted input and show the outputs are not shifts of each other.

Metric. Max absolute error on the overlap for the convolution. A qualitative counterexample for the dense map is enough if you also report one numeric gap.

Decision rule. Convolution error near 10^{-12} on the overlap. If it fails, the padding or the kernel alignment is wrong. Do not claim equivariance you did not test.

128.6 Project

Deliverable. A short architecture card for MLP versus 1D convolution on a synthetic problem where the label is a local pattern that can appear at any position, and a second problem where the label depends on the absolute index. Say which family is appropriate for each.

Constraints. You may demonstrate with a hand-built convolutional feature rather than a trained network if training is not ready. The claim is about the family.

Rubric. Each choice names the symmetry it assumes and the experiment that would embarrass it.

128.7 Depth tracks

Beginner. Implement the convolution and test a kernel `[1, 0, -1]` on a short array.

University. Complete the shift experiment and the two-problem card.

Research. State, for images, what a 3×3 convolution assumes that a vision transformer with a position embedding does not. You will return to this in Volume 11. Do not treat either as universally more powerful.

128.8 Failure modes

- Using a fully connected stack on raw pixels because it is "more general," with a small sample.
- Padding in a way that breaks the shift test and still claiming translation equivariance.
- Adding capacity until the training loss vanishes and calling the result an inductive bias.

128.9 See also

Volume 11 for images. Volume 12 for sequence models and attention.

129 Initialization and activations

130 Initialization and activations

Reference question. How should weights start so that a forward pass and a backward pass have a sane scale?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

130.1 Reference card

Item	Statement
Goal	Pre-activations with order-1 variance at the start of training, not 0 and not 10^6.
Scale	For a layer with fan-in d, a common choice is variance $1/d$ or $2/d$ for ReLU, so a sum of d terms does not grow like d.
Symmetry	Identical rows of W stay identical under many updates. Initialize randomly.
Activation	Match the init to the activation. A sigmoid wants smaller inputs than a ReLU.
Check	Histogram or variance of each layer's pre-activation on a minibatch, before any claim about learning rate.

130.2 Theory

Initialization is a scale model of the untrained network. If the first forward pass saturates every sigmoid, the gradient check may still pass and the training step will not move. If the first forward pass overflows, you will chase a learning rate that was never the bug.

The fan-in rule is a variance calculation under an independence assumption that is only roughly true. It is good enough to start, and it is something you verify by printing variances. Residual connections and normalization layers change the right init. Copying an init from a paper whose architecture you did not copy is a superstition.

Biases often start at zero. A ReLU network with zero bias and symmetric data can start with half the units off. That can be acceptable. A dead unit that never turns on for any training point stays dead if its gradient is zero. Watch the fraction of active units.

130.3 Derivation

Let $z = \sum_{j=1}^{d} W_j x_j$, with x_j independent, mean 0, variance 1, and W_j independent, mean 0, variance v. Then $\mathrm{Var}(z) = dv$. Choosing $v = 1/d$ keeps $\mathrm{Var}(z) = 1$. For ReLU, $h = \max(z, 0)$ roughly halves the variance if z is symmetric, because half the values are zero and the positive half carries the second moment. The He heuristic uses $v = 2/d$ to put that factor back. It is a heuristic: ReLU of a Gaussian is not a unit-variance Gaussian. Measure the variance anyway.

130.4 Code

```
import numpy as np

def init_linear(fan_in, fan_out, rng, relu=True):
    scale = np.sqrt(2.0 / fan_in) if relu else np.sqrt(1.0 / fan_in)
    W = rng.normal(scale=scale, size=(fan_in, fan_out))
    b = np.zeros(fan_out)
    return W, b
```

130.5 Experiment

Hypothesis. With $v = 2/d$ and ReLU, a stack of several layers keeps the empirical variance of activations within a small factor of 1 on standard normal inputs. With $v = 1$, the variance collapses as depth grows. With $v = 1$ replaced by a constant 0.5 independent of d, the variance explodes.

Protocol. $d = 64$, depth in $\{1, 4, 8\}$, 256 random inputs, three init rules.

Metric. Mean over units of the empirical variance of each layer's output.

Decision rule. The $2/d$ stack should stay order-1 longer than the others. If all three explode, the input variance is not 1 or the ReLU is not applied. This experiment is about initialization, so do not tune it until the story appears; report the curves you get.

130.6 Project

Deliverable. The depth sweep, plus the fraction of dead ReLU units at initialization on your synthetic input. Choose one init for the later training project and write the variance rule in the config.

Constraints. No data-dependent init that uses the test set.

Rubric. You can say what would change if the inputs were not standardized.

130.7 Depth tracks

Beginner. Initialize one layer both ways and print $\mathrm{Var}(z)$.

University. Run the depth sweep.

Research. Derive the backward-variance heuristic: gradients can require a fan-out scaling that disagrees with the forward fan-in scaling. Name a normalization layer as the practical ceasefire, and say what it does to the scale.

130.8 Failure modes

- Zero initialization of a whole weight matrix in a hidden layer.
- A learning-rate search on a saturated network.
- Importing a pretrained model's init into a different depth without measuring activations.

130.9 See also

Volume 10 for learning rates that assume this scale. Volume 11 for convolutions, where fan-in includes the kernel area.

131 What universal approximation does not give you

132 What universal approximation does not give you

Reference question. A wide enough network can approximate many functions. Why is that not a training plan?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

132.1 Reference card

Item	Statement
Theorem shape	For a non-polynomial activation, one hidden layer can approximate continuous functions on a compact set, arbitrarily closely, if the width may grow.
Not included	A bound on the width you need, the sample size, the optimization, or the test risk.
Compact set	The guarantee is on a bounded domain. Extrapolation is a different claim.
Existence	The theorem says some weights exist. It does not say gradient descent finds them.
Use	A sanity check that the family is not the obstacle. The obstacle may still be data, optimization, or the wrong loss.

132.2 Theory

Universal approximation is an existence result about families. It tells you not to blame "neural nets cannot represent that" for every failure, once the domain is compact and the function is continuous, given enough width. It does not tell you that your width, your finite sample, or your optimizer are enough. Polynomials are also dense on a compact interval. You do not ship a degree-10,000 polynomial because of that sentence.

The rate at which width must grow can be ugly. A function with a sharp localized bump needs the bump to be covered by some unit. In high dimension, covering a domain with

localized bumps is expensive. Architecture and features are how you avoid paying that price. A convolution that matches the symmetry can represent the function you want with far fewer samples than a dense net that could also represent it in the limit.

Optimization makes the gap practical. A representable function is irrelevant if the loss landscape and the initialization do not reach it. Cite the theorem when someone claims a network family is incapable. Do not cite it when the training loss has stalled.

132.3 Derivation

A standard constructive picture uses localized bumps. Suppose ϕ is a sigmoid. A unit $\phi(w(x - c))$ with large w approximates a step at c. A difference of two steps approximates a spike. Summing spikes of chosen heights approximates a continuous function uniformly on an interval, given enough spikes. The number of spikes is the width. The construction uses weights that grow with the desired sharpness. Gradient descent from a small init is not this construction. The proof shows membership in the closure of the family, not a training algorithm.

132.4 Code

```
import numpy as np

def spike_basis(x, centers, width):
    x = np.asarray(x, dtype=np.float64)
    c = np.asarray(centers, dtype=np.float64)
    return np.exp(-0.5 * ((x[:, None] - c[None, :]) / width) ** 2)
```

132.5 Experiment

Hypothesis. On $[0, 1]$, a bump basis with enough centers approximates $\sin(2\pi x)$ in least squares better than a linear function, and a bump basis with centers only on $[0, 0.2]$ fails on $[0.8, 1]$.

Protocol. Fit ridge regression on the bump features. Evaluate on a grid. Compare widths corresponding to 4 and 20 centers, and the localized-center failure.

Metric. Max absolute error on the grid, and on the right-hand interval separately.

Decision rule. More centers covering the interval should reduce error. Centers that do not cover the interval must leave a large error there, which is the compact-set and support caveat in miniature.

132.6 Project

Deliverable. The bump approximation, plus a paragraph that rewrites one real failure you have seen as representation, estimation, or optimization, with the evidence that distinguishes them. Training loss near zero and test loss high points at estimation or leakage, not at universal approximation. Training loss stuck high can be optimization or representation.

Constraints. Do not invoke the theorem without naming which hypothesis of the theorem you checked.

Rubric. The paragraph would change a teammate's next experiment.

132.7 Depth tracks

Beginner. Fit 8 bumps to a sine wave and plot or tabulate the error.

University. Add the uncovered-interval failure.

Research. State a curse-of-dimensionality caveat: a grid of bumps with 10 centers per axis in d dimensions has 10^d centers. Connect this to why image models do not one-hot the pixel cube.

132.8 Failure modes

- "The network can approximate any function, so more epochs will fix the test error."
- Quoting a theorem that assumes a compact domain while predicting far outside the training box.
- Confusing a failed optimizer with a limited family, or the reverse, without looking at the training loss.

132.9 See also

Volume 06 for approximation versus estimation. Volume 10 for the optimization half.

133 Project: a network in NumPy

134 Project: a network in NumPy

Reference question. Can you train a small network whose gradients you checked, and beat a linear baseline for a reason you can name?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

134.1 Reference card

Item	Statement
Model	One hidden layer, ReLU, squared loss or softmax cross-entropy.
Check	The gradient check passes before training.
Baseline	Linear or logistic regression on the same features and split.
Data	A synthetic problem with a known nonlinearity, plus the linear problem where the network should not win by much.
Report	Training loss, validation loss, test metric, and the baseline, across a few seeds.

134.2 Theory

This is the first trained network in the series. Its job is to be understood. A library model that beats a baseline without a gradient check is a later convenience, not a substitute. The synthetic nonlinear problem should be one where Volume 06 already showed a linear floor: the sine wave, or a product interaction $Y = X_1 X_2 + \varepsilon$. The network should be able to lower that floor. On a purely linear problem it should tie the linear model within noise after you give the linear model the same features.

Overfitting a tiny sample is part of the lesson. If the hidden width is large and n is small, training loss will drop and validation loss will rise. Record that curve. It is the picture Volume 10 will try to control. Do not hide it by reporting only the best epoch on the test set.

134.3 Derivation

The update is $\theta \leftarrow \theta - \eta \widehat{\nabla} L(\theta)$ with the analytic gradient from backpropagation and a minibatch or the full sample. For a full-batch step on a smooth loss, a sufficiently small η decreases L. ReLU is not smooth at 0. In practice a small step that does not sit exactly on the kink still behaves. If the loss increases on the first step, η is too large or the gradient is scaled for a different reduction. Divide η by 10 before rewriting the network.

134.4 Code

```
import numpy as np

def sgd_step(params, grads, eta):
    return [p - eta * g for p, g in zip(params, grads)]
```

134.5 Experiment

Hypothesis. On $Y = \sin(2\pi X_1) + 0.5 X_2 + \varepsilon$, the checked network achieves lower test MSE than linear regression in (X_1, X_2). On $Y = X_1 + X_2 + \varepsilon$, the gap disappears within seed noise.

Protocol. $n_{\text{train}} = 200$, validation 200, test 1000, hidden width 32, a few seeds, η chosen on the validation loss from a short log grid. Stop using the validation set for early stopping and do not retune on the test set.

Metric. Median test MSE versus the linear baseline.

Decision rule. The nonlinear problem should show a gap larger than the seed scatter. The linear problem should not crown the network if the gap is inside the paired interval from Volume 07. If the network loses on the sine wave, look at training loss. High training loss means optimization or init. Low training loss and high test loss means capacity and a small sample.

134.6 Project

Deliverable. The gradient-checked network, both datasets, the baseline table, learning curves of training and validation loss, and a model card.

Constraints. NumPy only on the checked path. A library reimplementation is optional and must match the NumPy loss on one batch within a tight tolerance.

Rubric. You can explain a failure mode you actually hit: dead units, a too-large step, or a mean/sum bug. A smooth success with no note of the check is incomplete.

134.7 Depth tracks

Beginner. Train on the sine wave with full-batch gradient descent for a fixed small η and plot the loss.

University. Complete both datasets, the baseline, and the card.

Research. Add a second hidden layer only after the first layer's check passes. Compare parameter counts at equal validation loss. Do not add depth because the theorem said depth is expressive.

134.8 Failure modes

- Early-stopping on the test curve.
- A width so large that you memorize 200 points and call it a solved sine wave.
- Skipping the linear problem where the network should not look brilliant.

134.9 See also

Volume 10 for a disciplined training loop. Volume 11 if you replace the MLP with a convolution on images.

Part X

Volume 10. Training deep models

135 Volume 10. Training deep models

136 Volume 10. Training deep models

Stage. University to research

Audience. A reader who has a gradient-checked network and a validation split.

Question this volume answers. Which knobs control optimization, which control generalization, and how do you tell them apart?

This volume is a reference, not a short tour. Each chapter opens with a card you can look up later, then gives the theory, a reference implementation, an experiment with a decision rule, and a project with a rubric. Read in order the first time. After that, enter through the reference card of the chapter that matches the problem in front of you.

136.1 How to use the four layers

Layer	What you should be able to do when you finish it
Theory	State the definition, the assumptions, and the result, including what the result does not say.
Code	Implement the core object in a few dozen lines and check it against a known case.
Experiment	Change one factor, record a metric with uncertainty, and say what outcome would change your mind.
Project	Produce an artifact someone else can rerun: code, figure, table, and a one-page lab note.

136.2 Depth tracks

Every chapter ends with three tracks.

- **Beginner.** Do the reference card, the code, and the smallest experiment. Skip optional proofs.
- **University.** Do the derivation, the full experiment, and the project rubric.
- **Research.** Read the failure modes, change an assumption, and write what would have to be true for the method to fail on a new distribution.

136.3 Chapters

No.	Chapter	File
01	Losses for modern models	`vol10/01_losses.qmd`
02	Optimizers	`vol10/02_optimizers.qmd`
03	Schedules and normalization	`vol10/03_schedules_normalization.qmd`
04	Regularization and augmentation	`vol10/04_regularization.qmd`
05	Debugging a training run	`vol10/05_debugging_training.qmd`
06	Project: a controlled training ablation	`vol10/06_ablation_project.qmd`

136.4 Lab note

Keep one markdown file per experiment, `notes/YYYY-MM-DD-<slug>.md`, with the hypothesis, the command you ran, the metric, and the decision. The series is a reference only if the experiments you run are attached to it.

137 Losses for modern models

138 Losses for modern models

Reference question. What does each common training loss estimate, and when is it the wrong reduction?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

138.1 Reference card

Item	Statement
Squared	A conditional mean. Sensitive to outliers.
Logistic / cross-entropy	A conditional categorical distribution. Proper for probabilities.
Margin	A ranking or a separation constraint. Not a probability unless you add a calibration step.
Class weights	A change of the empirical measure toward rare classes. They change the functional you estimate.
Reduction	Sum and mean differ by n or by the batch size. The optimizer must know which you used.

138.2 Theory

The loss is the definition of the task inside the optimizer. Changing it changes the Bayes predictor. Class-weighted cross-entropy does not estimate $P(Y \mid X)$ under the original data law. It estimates the conditional probability under a reweighted law. That can be what you want for a rare class, and it will miscalibrate probabilities for the original law unless you correct them. Write the reweighting in the card.

Label smoothing replaces one-hot targets by a mixture with a uniform distribution. It is a regularizer and a change of target. The model's best possible log loss is no longer zero. A training loss that stops falling at a positive number may be correct.

For sequence models the loss is usually a sum over tokens. A longer document then contributes more gradient. Mean-over-tokens and mean-over-documents are different estimators. Pick the one that matches the decision, and do not compare two runs that reduced differently.

138.3 Derivation

The gradient of mean squared error on a batch of size B is B^{-1} times the gradient of the sum. If you tune η for the sum and then switch the code to the mean, you have multiplied the effective step by $1/B$. A mysterious need to increase the learning rate by about B after a “cleanup” refactor is this factor. Write η next to the reduction in the config hash.

For softmax cross-entropy, if $p = \mathrm{softmax}(z)$ and y is one-hot, $\nabla_z L = p - y$ for the sum of per-example losses, or that quantity over B for the mean. The $p - y$ form is the one to gradient-check. A sign error trains the model to be confidently wrong, and the loss rises.

138.4 Code

```
import numpy as np

def softmax(z):
    z = np.asarray(z, dtype=np.float64)
    z = z - z.max(axis=-1, keepdims=True)
    e = np.exp(z)
    return e / e.sum(axis=-1, keepdims=True)

def cross_entropy_mean(z, y_index):
    p = softmax(z)
    picked = p[np.arange(len(y_index)), y_index]
    return float(-np.mean(np.log(np.clip(picked, 1e-12, 1.0))))
```

138.5 Experiment

Hypothesis. Doubling the batch size while switching from a summed loss to a mean loss, without changing η, changes the parameter step unless η is adjusted by the same factor.

Protocol. One affine logistic model, one batch, compute the gradient under sum and under mean. Compare the proposed steps ηg.

Metric. The ratio of step norms.

Decision rule. The ratio should equal the batch size when η is fixed. If your training code hides the ratio inside a library reduction, print it. Do not tune around an unknown factor.

138.6 Project

Deliverable. A loss card for the network project: the formula, the reduction, the Bayes target, and a gradient check of the classification head if you use one. Include a reweighted variant only as a labeled alternative, with a sentence on calibration.

Constraints. The config stores `reduction` and `class_weights`.

Rubric. A change in reduction forces a new config hash, by Volume 03's rules.

138.7 Depth tracks

Beginner. Implement softmax cross-entropy and check that a one-hot prediction of the true class has loss near 0.

University. Complete the sum-versus-mean step comparison.

Research. Derive the gradient of label-smoothed cross-entropy and say what probability vector minimizes it. Check the gradient at one point.

138.8 Failure modes

- Comparing losses across runs that used different reductions.
- Reweighting classes and still reporting unweighted accuracy as if the training objective matched it.
- Clipping probabilities at 0 inside the log.

138.9 See also

Volume 06 for proper scoring. Volume 12 for token-level reductions.

139 Optimizers

140 Optimizers

Reference question. What state does an optimizer keep besides the parameters, and what does that state assume?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

140.1 Reference card

Item	Statement
SGD	Step along the minibatch gradient. Optional momentum keeps a velocity.
Momentum	A leaky average of past gradients. It accelerates in consistent directions and adds a state that must be checkpointed.
Adam	Per-coordinate gradient scale from first and second moments. A step size that is plausible in one coordinate can still be wrong globally.
Batch size	A larger batch reduces gradient noise and changes the η you should use. It is not a free improvement in generalization.
Fair comparison	Equal data touches or equal wall time, with each method's step size tuned on validation.

140.2 Theory

An optimizer is a dynamical system with its own state. Momentum's velocity and Adam's moment estimates are parameters of the training trajectory. A checkpoint that stores weights and not optimizer state does not resume the same trajectory. That matters for reproducibility and for a crash in the middle of a long run.

Adam rescales coordinates that have historically small gradients so they can move, and it shrinks coordinates with huge gradients. That is helpful when features or layers have mismatched scales. It is not a proof of convergence to a good minimum. The epsilon in the

denominator, the moment decay rates, and whether you debias the moments are part of the estimator. Defaults are a choice a paper made for a different problem.

Tune the step size. A fixed default that works on a tutorial is not a tuned method. The comparison that matters in this series is against SGD with momentum at its own best validation step size, at matched data budget. If Adam wins only at an untuned SGD baseline, you learned less than you think.

140.3 Derivation

Classical momentum replaces the gradient step by $v \leftarrow \beta v + g$ and $\theta \leftarrow \theta - \eta v$, or the heavy-ball variant with the gradient evaluated at a shifted point. On a quadratic with a poor condition number, a well chosen β damps the zig-zag along the sharp axis that Volume 02 illustrated. The optimal β depends on that condition number, which you do not know. Treat β as a knob with a default near 0.9 and a small validation check, not as a constant of nature.

Adam's second moment $\hat{v}$ estimates a coordinatewise gradient energy. The step is proportional to $\hat{m}/(\sqrt{\hat{v}} + \epsilon)$. If $\hat{v}$ is tiny and ϵ is tinier, early steps can be enormous. If ϵ is large, Adam becomes a sluggish sign method. Print the step norm on the first iteration.

140.4 Code

```
import numpy as np

def sgd_momentum(theta, grad, velocity, eta, beta):
    velocity = beta * velocity + grad
    theta = theta - eta * velocity
    return theta, velocity
```

140.5 Experiment

Hypothesis. On the poorly conditioned quadratic bowl from Volume 02, momentum with a tuned β reaches a small radius in fewer steps than plain SGD at its best stable constant step. On a well-conditioned bowl, the gap shrinks.

Protocol. $L = \frac{1}{2}(\theta_1^2 + \kappa\theta_2^2)$ for $\kappa \in \{1, 100\}$. Grid η and, for momentum, $\beta \in \{0, 0.9\}$.

Metric. Steps to reach $\|\theta\| < 10^{-3}$, or the radius after a fixed budget.

Decision rule. Momentum should help more at $\kappa = 100$. If it diverges, the step is outside the stable region; that run does not count as a win for SGD, it counts as an untuned momentum run. Retune.

140.6 Project

Deliverable. SGD, momentum, and a from-scratch Adam or a carefully wrapped library Adam, on the NumPy network's nonlinear synthetic problem. Each gets a validation-chosen learning rate. Report validation loss against data touches.

Constraints. Optimizer state is part of any checkpoint you save.

Rubric. You do not crown a method that saw more data or a method whose only competitor used $\eta = 0.1$ by tradition.

140.7 Depth tracks

Beginner. Implement momentum on the quadratic bowl.

University. Complete the three-way comparison on the network.

Research. Show a small example where Adam's coordinatewise scaling helps when one input feature is multiplied by 100 and you forgot to standardize. Then say why fixing the scale is still the first move.

140.8 Failure modes

- Comparing optimizers at a single default learning rate.
- Restoring weights without optimizer state and expecting the loss curve to continue.
- Increasing batch size, keeping η, and attributing the change to "more stable gradients" without the reduction factor from the loss chapter.

140.9 See also

Volume 02 for condition numbers. Volume 15 for training runs as jobs with checkpoints.

141 Schedules and normalization

142 Schedules and normalization

Reference question. How do learning-rate schedules and normalization layers change the scale of training?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

142.1 Reference card

Item	Statement
Schedule	A rule η_t. Common patterns are constant, step drops, and cosine decay. The rule is a knob.
Warmup	A short increase from a tiny step, used when a large initial step is unstable, especially with adaptive methods or large batches.
Normalization	A layer that recenters and rescales activations using batch or feature statistics, plus learned gain and bias.
Train versus eval	Batch normalization uses batch statistics in training and a running estimate at evaluation. Forgetting the switch changes the function.
Interaction	Normalization changes the effective step. Retune η after you add it. Do not stack it on an old learning rate and interpret the crash as the data.

142.2 Theory

A schedule encodes the belief that the useful step size shrinks as you approach a minimum or as the gradient noise starts to dominate. Volume 02's stochastic-gradient chapter already said a constant step hovers in a neighborhood. Decay is how you leave the neighborhood. Decay too early and you freeze a bad solution. The validation loss, not the folklore, decides.

Normalization makes the scale of activations less dependent on the scale of incoming weights. That often lets you use a larger step and deeper stacks. It also introduces a dependence across the batch, in batch normalization, so the function of one example depends on the other examples in the minibatch. At evaluation you must freeze the statistics or the prediction of a single deployed example is undefined or depends on its accidental neighbors.

Layer normalization computes statistics per example and avoids that particular train-eval trap. It is a different function. Copying a recipe that used one of them into an architecture that used the other is a change of model, not a cleanup.

142.3 Derivation

Batch normalization of a pre-activation z in a batch replaces it by $\hat{z} = (z - \mu_B)/\sqrt{\sigma_B^2 + \epsilon}$ and then $\gamma \hat{z} + \beta$. The Jacobian with respect to z includes the dependence of μ_B and σ_B on z. A gradient that treats μ_B as constant is wrong. Library implementations include those terms. A hand implementation that forgets them can still train and will fail a strict gradient check.

A step decay multiplies η by a factor at fixed epochs. A cosine schedule sets $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t / T))$ for a horizon T. Both are deterministic functions of the step index. Put T in the config. Changing the number of epochs without changing T changes the schedule.

142.4 Code

```
import numpy as np

def cosine_eta(t, t_max, eta_max, eta_min=0.0):
    if t_max <= 0:
        raise ValueError("t_max must be positive")
    frac = min(max(t, 0), t_max) / t_max
    return float(eta_min + 0.5 * (eta_max - eta_min) * (1 + np.cos(np.pi * frac)))
```

142.5 Experiment

Hypothesis. On a convex quadratic with added gradient noise, a cosine decay ends at a smaller radius than a constant step equal to $\eta_{\max}$, at matched step count, when $\eta_{\max}$ is large enough to hover.

Protocol. The bowl $\kappa = 20$, plus isotropic Gaussian gradient noise of scale 0.05. Compare constant $\eta_{\max}$ and cosine from $\eta_{\max}$ to 0.

Metric. Radius at the final step, median over 20 seeds.

Decision rule. Cosine should finish closer to the origin when the constant step visibly jitters. If both diverge, reduce $\eta_{\max}$. This is a sanity check of the schedule, not a claim about networks.

142.6 Project

Deliverable. Two training curves for the small network: constant η and cosine decay, with $\eta_{\max}$ chosen separately on validation for each. If you implement layer normalization, add a third curve and a new learning-rate sweep rather than reusing η.

Constraints. Evaluation mode is irrelevant until you use batch statistics. If you do, the card must say how a single example is normalized at test time.

Rubric. The curves share axes of data touches and validation loss. The schedule parameters sit in the config hash.

142.7 Depth tracks

Beginner. Plot `cosine_eta` over 100 steps.

University. Run the noisy quadratic comparison and the two network curves.

Research. Explain the train-eval discrepancy of batch normalization in one formula, and name a failure mode in which a batch of size 1 at deployment has undefined variance.

142.8 Failure modes

- A cosine period that assumes 100 epochs after you shortened the run to 20.
- Leaving batch normalization in training mode at evaluation so the score depends on batch composition.
- Adding normalization and three other changes in one run, then attributing the gain to all of them.

142.9 See also

Volume 07 for early stopping, which can be viewed as a data-dependent schedule. Volume 11 for normalization inside convolutional nets.

143 Regularization and augmentation

144 Regularization and augmentation

Reference question. Which knobs reduce the gap between training and test loss, and which ones just blur the data?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

144.1 Reference card

Item	Statement
Penalty	An added term such as $(\lambda/2)\|\theta\|_2^2$. It changes the estimator and the Bayes target inside the penalized family.
Early stopping	Choosing the step count on validation. A regularizer with a cheap path along the training trajectory.
Dropout	Randomly masking units in training and rescaling at evaluation. Another train-eval switch.
Augmentation	Label-preserving transformations. They enlarge the training distribution toward variations you claim should not change Y.
Evidence	A smaller train-test gap together with a better validation loss. A smaller gap with a worse validation loss is not a win.

144.2 Theory

Regularization is any restriction that trades training loss for expected test loss. The trade can fail. A huge λ makes a network a constant and the gap vanishes because both losses are bad. Always read the gap next to the validation number.

Augmentation is a modeling statement. Flipping an image preserves a species label and destroys a "which side is the tumor on" label. The augmentation distribution should match variations you want invariance to, and it should not use information from the test image. Augmenting after the split is fine. Augmenting by mixing train and test images is leakage.

Dropout's evaluation multiplier, or the equivalent of averaging the mask, must be on when you report a number. Forgetting it makes the test-time function a randomly scaled cousin of the training function. As with batch normalization, the switch is part of the model definition.

144.3 Derivation

The L2-penalized objective $L(\theta)+(\lambda/2)\|\theta\|_2^2$ has gradient $\nabla L+\lambda\theta$. Weight decay in plain SGD, $\theta \leftarrow (1-\eta\lambda)\theta - \eta\nabla L$, matches that gradient step. In adaptive methods, decoupling weight decay from the adaptive scale is a different update from adding $\lambda\theta$ before the coordinatewise rescaling. If you compare "weight decay" across optimizers, say which formula you used.

Early stopping selects $t_\star = \arg\min_{t \le T} \widehat{R}_{\mathrm{val}}(\theta_t)$. It is cross-validation along one path. The selected $\theta_{t_\star}$ is then a validation-chosen model. The test evaluation happens after $t_\star$ is frozen.

144.4 Code

```
import numpy as np

def l2_penalty_grad(theta, lam):
    return lam * np.asarray(theta, dtype=np.float64)

def dropout_mask(shape, p, rng):
    if not 0 <= p < 1:
        raise ValueError("p is the drop probability")
    return rng.random(shape) >= p
```

144.5 Experiment

Hypothesis. On a small training set for the sine network, a moderate L2 penalty or early stopping lowers validation MSE relative to a long unpenalized run, and a penalty that is far too large raises both training and validation loss.

Protocol. Width 128, $n_{\mathrm{train}} = 60$, a grid of λ including 0, plus one long run with epoch chosen on validation.

Metric. Validation MSE and the train-validation gap.

Decision rule. You should be able to show one setting that generalizes better than the longest unregularized run, and one setting that underfits. If the unregularized run does not overfit, the width is too small or n is too large for the demonstration; adjust the capacity until the lesson is visible, and record that you designed the demonstration.

144.6 Project

Deliverable. A regularization sweep on the synthetic network with the selection rule predeclared. If you add an augmentation, use a synthetic one you can defend, such as small input noise, and include a harmful augmentation, such as permuting the predictive coordinate, as a negative control.

Constraints. The test set is not used to pick λ or the stopping epoch.

Rubric. The note says which regularizer you would keep and what invariance it assumes.

144.7 Depth tracks

Beginner. Add an L2 term to the linear model and watch coefficients shrink as λ grows.

University. Complete the overfitting demonstration and the sweep.

Research. Implement dropout at train and eval on one layer and show the gradient check fails if the eval rescale is left on during the analytic training gradient, or succeeds only when the mask is fixed. State the correct check: fix the mask, differentiate, then average.

144.8 Failure modes

- Augmentations chosen because they improved the test number, with no validation split.
- Dropout left on at test time so every evaluation is random.
- Five regularizers added at once with a single before-after comparison.

144.9 See also

Volume 07 for the selection firewall. Volume 16 for augmentations that hide a sensitive attribute and may or may not be ethical or effective.

145 Debugging a training run

146 Debugging a training run

Reference question. In what order do you investigate a run that does not learn?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

146.1 Reference card

Item	Statement
Order	Data contract, loss at initialization, gradient check, one-batch overfit, then scale up.
One batch	A correct flexible model should drive the loss on a tiny batch near the minimum of that loss. If it cannot, the bug is not the dataset.
Baseline	A linear model or a constant. If the network cannot beat it on one batch, stop.
Scale	Activation variances, gradient norms per layer, fraction of dead units, fraction of non-finite values.
Change one thing	A debug log that changes the learning rate, the init, and the loss together does not isolate a cause.

146.2 Theory

Training bugs masquerade as research questions. The discipline is to restore a regime where success is mandatory. A tiny batch with a large width and a correct loss has a minimum you should reach. Failure there is a bug in the graph, the reduction, the optimizer state, or the input pipeline. Success there, and failure on the full problem, is capacity, regularization, distribution shift, or an optimization issue at scale. Those are different projects.

Look at per-layer gradient norms. A layer with a zero gradient is disconnected. A layer whose gradient grows by orders of magnitude each step will overflow. Non-finite losses should abort the run and dump the last finite checkpoint's statistics. A plot that drops NaNs and continues will show a comforting curve that is no longer the loss you defined.

Compare to a baseline on the same batch. If linear regression fits the batch better than the network, the network is not yet a superset in practice, even if it is a superset in theory. Usually the step size or the initialization has pinned it.

146.3 Derivation

Let L_B be the loss on a fixed batch of size B. If the model can represent the targets and the loss is bounded below by zero, the infimum is zero for squared error on a realizable batch, or the entropy of the labels for cross-entropy when the labels conflict. Gradient descent on a smooth approximation can drive L_B near that floor given a small enough step and enough steps. ReLU networks can fail this test when units die on every point in the batch. The diagnostic action is to print the active fraction, not to collect more data.

146.4 Code

```
import numpy as np

def assert_finite(name, value):
    arr = np.asarray(value)
    if not np.isfinite(arr).all():
        raise FloatingPointError(f"{name} has non-finite values")
```

146.5 Experiment

Hypothesis. A network with a deliberately transposed gradient cannot drive a single-batch squared loss near zero, while the corrected gradient can. A learning rate 100 times too large also fails, with non-finite or oscillating loss, even when the gradient is correct.

Protocol. One batch of 16 points from the sine generator, width 64, two bug conditions and one correct run.

Metric. Loss at step 0 and step 200, and whether the run raised on a non-finite value.

Decision rule. The correct run should reduce loss by a large factor. Each bug should fail differently. If the transposed gradient still trains, the loss is invariant to that parameter and the test is vacuous; choose a parameter the loss actually uses.

146.6 Project

Deliverable. A debug checklist you actually run, stored as `DEBUG.md` next to the training script, with the one-batch result, the linear baseline on that batch, gradient norms, and the two intentional bugs.

Constraints. The checklist is ordered. Later items are not attempted while earlier items fail.

Rubric. A new failure you introduce in the loss reduction is caught by an item already on the list, or you add that item because it was not.

146.7 Depth tracks

Beginner. Overfit 8 points with a linear model, then with the network.

University. Complete the two intentional bugs and the checklist.

Research. Add a per-layer gradient-norm log to a deeper stack and identify a vanishing layer you create by using sigmoid and a bad init. Fix it by the init chapter's rule, not by an unexplained learning-rate change.

146.8 Failure modes

- Collecting more data while one batch does not fit.
- Averaging away NaNs.
- Changing five hyperparameters after each failed run.

146.9 See also

Volume 03 for the experiment folder that should store the failed runs too. Volume 09 for the gradient check that belongs at the top of the checklist.

147 Project: a controlled training ablation

148 Project: a controlled training ablation

Reference question. How do you attribute a gain to one training choice?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

148.1 Reference card

Item	Statement
Factor	One change: optimizer, schedule, penalty, width, or augmentation.
Control	The previous best configuration, otherwise untouched.
Selection	Each factor's internal knob, such as η, is tuned on validation under that factor.
Budget	Matched data touches or matched wall-clock time, declared first.
Report	A table of differences with uncertainty across seeds, and a decision about what enters the default.

148.2 Theory

An ablation is an experiment, not a tour of features. The scientific claim is that factor A, not the company it kept, moved the validation metric. If you turn on normalization, Adam, and augmentation together, you have a bundle. Bundles are how products are built and how papers get refuted. This project allows one bundle only as a final row, after the single-factor rows.

Each factor deserves its own step-size or penalty sweep when the factor changes scale. Holding η fixed "to be fair" is unfair to the method whose optimum moved. The fairness is in the validation budget for the sweep, which you should cap so a method with more knobs does not get a larger search.

Seeds matter more here than in convex problems. Report the median and the spread. A gain smaller than the spread is not a default-changing result. It is a request for more seeds or a simpler claim.

148.3 Derivation

Let $\Delta_s = M_s(A) - M_s(C)$ be the validation metric gap on seed s, with the sign oriented so positive is better. The mean of Δ_s over seeds estimates the procedure gap under your seed distribution. It does not estimate the gap on a new test set. After you choose the default, one locked test evaluation measures that chosen procedure. Choosing the factor with the best test Δ is the selection bias of Volume 07. The ablation table is validation. The test number is a single subsequent line, if you spend it.

148.4 Code

```
import numpy as np

def summarize_deltas(deltas):
    d = np.asarray(deltas, dtype=np.float64)
    return {
        "n": int(d.size),
        "median": float(np.median(d)),
        "mean": float(np.mean(d)),
        "sd": float(np.std(d, ddof=1) if d.size > 1 else 0.0),
    }
```

148.5 Experiment

Hypothesis. On the small-sample sine network, early stopping or L2 regularization improves validation MSE relative to a fixed long run, and adding both does not necessarily beat the better single factor by more than seed noise.

Protocol. At least 5 seeds. Factors: longer unregularized run, L2 at its best λ from a small grid, early stopping, and the combination. Matched maximum step budget.

Metric. Validation MSE deltas versus the long unregularized run.

Decision rule. Adopt a factor only if the median delta is in the direction you wanted and larger than a practical margin you set from the residual noise. Otherwise keep the simpler training loop.

148.6 Project

Deliverable. An ablation table, the configs, the seed spread, and a default training section in the model card. One locked test evaluation of the chosen default against the linear baseline.

Constraints. No test-based ablation. The checklist from the debugging chapter has been run on the default.

Rubric. A reader can see which single change mattered. A bundle-only table is rejected.

148.7 Depth tracks

Beginner. Compare two learning rates at one seed and write why that comparison is incomplete.

University. Complete the ablation and the locked test line.

Research. Add a compute column. If a factor wins only by using twice the data touches, repeat the control at that budget before you attribute the gain to the idea.

148.8 Failure modes

- Ablating on the test set.
- A table of single runs sorted to two decimal places.
- Declaring five factors “all necessary” because the bundle worked.

148.9 See also

Volume 07 for paired gaps. Volume 18 for how an ablation becomes a paper table.

Part XI

Volume 11. Vision

149 Volume 11. Vision

150 Volume 11. Vision

Stage. University

Audience. A reader who can train a small network and can explain a convolution's shift behavior.

Question this volume answers. How should images be represented, transformed, and evaluated so a vision model is a measurement rather than a demo?

This volume is a reference, not a short tour. Each chapter opens with a card you can look up later, then gives the theory, a reference implementation, an experiment with a decision rule, and a project with a rubric. Read in order the first time. After that, enter through the reference card of the chapter that matches the problem in front of you.

150.1 How to use the four layers

Layer	What you should be able to do when you finish it
Theory	State the definition, the assumptions, and the result, including what the result does not say.
Code	Implement the core object in a few dozen lines and check it against a known case.
Experiment	Change one factor, record a metric with uncertainty, and say what outcome would change your mind.
Project	Produce an artifact someone else can rerun: code, figure, table, and a one-page lab note.

150.2 Depth tracks

Every chapter ends with three tracks.

- **Beginner.** Do the reference card, the code, and the smallest experiment. Skip optional proofs.
- **University.** Do the derivation, the full experiment, and the project rubric.

- **Research.** Read the failure modes, change an assumption, and write what would have to be true for the method to fail on a new distribution.

150.3 Chapters

No.	Chapter	File
01	Images as tensors	`vol11/01_images.qmd`
02	Convolution and equivariance	`vol11/02_convolution.qmd`
03	Detection and segmentation	`vol11/03_detection.qmd`
04	Modern vision architectures	`vol11/04_modern_vision.qmd`
05	Evaluation for vision	`vol11/05_vision_eval.qmd`
06	Project: a small vision study	`vol11/06_vision_project.qmd`

150.4 Lab note

Keep one markdown file per experiment, `notes/YYYY-MM-DD-<slug>.md`, with the hypothesis, the command you ran, the metric, and the decision. The series is a reference only if the experiments you run are attached to it.

151 Images as tensors

152 Images as tensors

Reference question. What is an image in this reference, and which transforms change the label?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

152.1 Reference card

Item	Statement
Tensor	A batch of images as an array of shape (n, c, h, w) in this series. Channels are the color or sensor axis.
Values	A declared numeric range. Bytes in $[0, 255]$ and floats in $[0, 1]$ are different inputs.
Geometry	Resize, crop, and flip are functions on the tensor and, sometimes, on the label.
Split	Augment after the split. Fit normalization statistics on the training images only.
Baseline	A linear model on pooled color, or a tiny convolution, before a large architecture.

152.2 Theory

An image is a measurement on a grid, not a bag of pixels and not a photograph's caption. The grid spacing is part of the contract. A model trained at 224×224 is a different function from the same code at 32×32. The label contract says whether a horizontal flip preserves Y. For species classification it might. For left-versus-right lesion location it does not.

Channel order and layout bugs are silent. A model can learn to read a permuted channel as if it were color, then fail on a camera that uses the declared order. Assert the shape and the range at the boundary. A mean of 120 and a mean of 0.47 should not both be called "normalized" without the denominator.

The constant and linear baselines still apply. Average brightness predicts some labels and is the thing a careless split will leak through JPEG artifacts or borders. Error analysis by

slice, from Volume 07, should include a slice for image source or site, because cameras shift the input law.

152.3 Derivation

A per-channel standardization $(x_c - \mu_c)/\sigma_c$ uses μ_c, σ_c from training pixels. Applying it with test-set moments uses the test distribution to define the features. The same formula with two different moment vectors is two different maps. Put the moments in the config or the checkpoint. At evaluation, reuse them. A resize from (h, w) to (h', w') is a linear interpolation or a pooling rule you must name. It is not invertible, so two different original images can become one tensor. That aliasing is a property of the measurement, and fine labels can become impossible after it.

152.4 Code

```
import numpy as np

def standardize_images(batch, mean, std):
    x = np.asarray(batch, dtype=np.float64)
    if x.ndim != 4:
        raise ValueError("expected (n, c, h, w)")
    mean = np.asarray(mean, dtype=np.float64).reshape(1, -1, 1, 1)
    std = np.asarray(std, dtype=np.float64).reshape(1, -1, 1, 1)
    if np.any(std <= 0):
        raise ValueError("std must be positive")
    return (x - mean) / std
```

152.5 Experiment

Hypothesis. Standardizing with training moments and with test moments produces different tensors when the test images are brighter, and a linear probe on the wrong moments is a different procedure.

Protocol. Synthetic 8×8 single-channel images, training brightness $\mathcal{N}(0, 1)$, test brightness $\mathcal{N}(2, 1)$. Compare the two standardized test batches' means.

Metric. Difference of spatial means after standardization.

Decision rule. The gap should be near 2 if someone wrongly recenters the test set by its own mean and you compare to a training-moment standardization that leaves the shift in place… Design the comparison explicitly: training-moment standardization of the test

set should leave a mean near $(2 - \mu_{\text{train}})/\sigma$, while cheating recentering removes it. If those match, the test brightness was not shifted.

152.6 Project

Deliverable. A loader contract: layout, range, resize rule, which flips preserve your label, and where moments come from. Demonstrate on synthetic images if you do not have rights to a real set.

Constraints. No silent channel swaps. An assert must fail if a batch is (n, h, w, c) under this series' layout, or you document a single layout and convert at the edge.

Rubric. A new image can be turned into the tensor the model expects without reading the training loop.

152.7 Depth tracks

Beginner. Standardize a batch and state the shape after the call.

University. Run the brightness shift experiment.

Research. Describe one label that a random crop can destroy, such as a count of objects that the crop cuts in half, and how the target would have to change to a crop-consistent label.

152.8 Failure modes

- Dividing uint8 values by 255 in one script and forgetting in another.
- Flipping images whose labels encode orientation.
- Computing dataset mean on a folder that mixes train and test.

152.9 See also

Volume 03 for contracts. Volume 09 for convolutions as the layer that uses the grid.

153 Convolution and equivariance

154 Convolution and equivariance

Reference question. What parameter sharing does a convolutional layer implement, and when does padding break the story?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

154.1 Reference card

Item	Statement
Kernel	A local weight tensor applied at every spatial position.
Fan-in	For the initialization rule, fan-in includes input channels times kernel height times kernel width.
Stride and padding	They change the output grid and the exact equivariance. They are part of the architecture, not cosmetics.
Pooling	A local reduction. Global pooling yields a vector whose size does not depend on the input size, and it discards position.
Check	The shift test from Volume 09, extended to two dimensions, away from the border.

154.2 Theory

A convolutional layer is a linear map with tied weights and a local support. The tying is the inductive bias: a detector learned in one corner is available in another. That is appropriate when the same visual pattern can appear at many positions. It is inappropriate when position itself is the label, unless position is reintroduced by a coordinate channel or by avoiding global pooling.

Padding changes the border. “Same” padding tries to preserve spatial size and uses values that were not in the image, often zeros. Those zeros are a cue. A model can detect the border by the padding pattern. The shift test will fail near the border even when the interior is equivariant. Report the region where you claim the property.

Detection and segmentation need positions. Classification of a centered object sometimes does not. Choosing global average pooling is a decision that translation of the object should not change the logit. If the dataset always centers the object, the decision is invisible until a shifted test set.

154.3 Derivation

In two dimensions, $(K * X)_{c',r,col} = \sum_{c,u,v} K_{c',c,u,v} X_{c,r+u,col+v}$, with a stride skipping positions and a padding rule defining X outside the grid. Shifting X by one pixel shifts the unpadded valid output by one pixel. With zero padding, the output near the new border also depends on the zeros that entered the window, so the equality fails for windows that overhang. A gradient check still applies: the convolution is linear in K and in X, and a small random kernel should match a numerical gradient of a scalar loss on the output.

154.4 Code

```
import numpy as np

def conv2d_single_channel(image, kernel):
    image = np.asarray(image, dtype=np.float64)
    kernel = np.asarray(kernel, dtype=np.float64)
    kh, kw = kernel.shape
    out_h = image.shape[0] - kh + 1
    out_w = image.shape[1] - kw + 1
    if out_h <= 0 or out_w <= 0:
        raise ValueError("kernel larger than image")
    out = np.zeros((out_h, out_w))
    for r in range(out_h):
        for c in range(out_w):
            out[r, c] = np.sum(kernel * image[r : r + kh, c : c + kw])
    return out
```

154.5 Experiment

Hypothesis. Valid convolution commutes with a one-pixel shift in the interior: shifting the image and then convolving matches convolving and then shifting, on the overlap. A kernel of all ones computes a local sum you can check by hand on a constant image.

Protocol. A 12×12 random image and a 3×3 kernel. Constant image 7, kernel of ones, expected output 63 everywhere.

Metric. Max absolute error of the shift test and of the constant test.

Decision rule. Both errors near 10^{-12}. A shift error that is large only on the border means you included an invalid region; restrict the comparison.

154.6 Project

Deliverable. The 2D convolution, the two checks, and a one-layer convolutional featurizer for a synthetic shape task: a bright 3×3 block on a dark background, label 1 if present. Compare a linear model on raw pixels with a linear model on the convolution response.

Constraints. The block's position varies. The split is by image, not by pixel.

Rubric. You predict that the convolutional feature needs fewer labeled images to beat chance, and you measure that learning curve at two sample sizes.

154.7 Depth tracks

Beginner. Hand-check the constant-image case.

University. Add the shift test and the synthetic block experiment.

Research. State how the parameter count of a 3×3 convolution depends on channels and not on $h \times w$, and how the compute does depend on $h \times w$. This is the systems fact Volume 15 will price.

154.8 Failure modes

- Claiming translation invariance for a feature map that is only equivariant.
- Using padding as a location cue and failing on a differently padded deployment image.
- Flattening $224 \times 224 \times 3$ into a vector as the first trained model, with 200 images.

154.9 See also

Volume 09 for the 1D shift test. Volume 15 for the cost of the spatial reduction.

155 Detection and segmentation

156 Detection and segmentation

Reference question. What is the target when an image can contain many objects, or a label per pixel?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

156.1 Reference card

Item	Statement
Classification	One label for the image.
Detection	A set of boxes and class labels. The set can be empty. Order does not matter.
Segmentation	A label per pixel, or a mask per object.
Matching	A predicted box must be matched to a true box before it counts as a true positive. The matching threshold is part of the metric.
Empty	Predicting nothing on a negative image is a true negative region, not a failure to find something.

156.2 Theory

Detection is not classification of a resized crop you already know how to find. The model must propose locations. The evaluation must not reward a duplicate box that overlaps the same object twice, and it must not punish an empty prediction on an image with no objects. Those rules are the matching protocol. Changing the overlap threshold changes the metric's meaning. Report the threshold.

Class imbalance is structural: most windows are background. A loss that treats every window equally will be happy predicting background. The foreground weight or the sampling of windows is part of the loss, with the calibration consequences from Volume 10.

Segmentation metrics that score pixel accuracy are dominated by the majority class, often background. Mean overlap on the rare class is closer to the decision if the rare class is the point of the system. State the decision.

156.3 Derivation

Intersection over union of two boxes is the area of their intersection divided by the area of their union. A true positive at threshold τ is a predicted box whose best unmatched true box of the same class has overlap at least τ. Unmatched predictions are false positives. Unmatched truths are false negatives. Precision and recall then follow Volume 07's definitions. Average precision integrates the precision-recall behavior of a ranked list of detections. It is not the accuracy of an image classifier, and it is not comparable across datasets with different size distributions without a new reading of the protocol.

156.4 Code

```
import numpy as np

def iou(box_a, box_b):
    ay0, ax0, ay1, ax1 = box_a
    by0, bx0, by1, bx1 = box_b
    ih = max(0.0, min(ay1, by1) - max(ay0, by0))
    iw = max(0.0, min(ax1, bx1) - max(ax0, bx0))
    inter = ih * iw
    area_a = max(0.0, ay1 - ay0) * max(0.0, ax1 - ax0)
    area_b = max(0.0, by1 - by0) * max(0.0, bx1 - bx0)
    union = area_a + area_b - inter
    return float(inter / union) if union > 0 else 0.0
```

156.5 Experiment

Hypothesis. A box that matches a truth with a one-pixel shrink has high IoU. A box shifted completely off the truth has IoU 0. A second identical prediction matched greedily should not count as a second true positive.

Protocol. Hand-built boxes. Implement a greedy one-to-one match at $\tau = 0.5$ for a single class and apply it to a duplicated prediction.

Metric. IoU values and the matched counts.

Decision rule. The duplicate must become a false positive. If both copies count as true positives, the matcher is wrong even if `iou` is right.

156.6 Project

Deliverable. The IoU function, the matcher, and a synthetic detection problem: one bright block per some images, none on others. A baseline that always predicts a centered box should fail on the empty images and on the off-center blocks in a way your metric shows.

Constraints. Report precision and recall at one predeclared τ, plus the false-positive count on empty images.

Rubric. You can explain a failure that image-level accuracy would have missed.

156.7 Depth tracks

Beginner. Compute IoU for identical boxes and for disjoint boxes.

University. Implement the matcher and the synthetic baseline.

Research. Define mean intersection-over-union for a three-class segmentation and show a case where pixel accuracy is above 95 percent while the rare class has overlap near 0.

156.8 Failure modes

- Counting two overlapping predictions of one object as two detections.
- Reporting classification accuracy on a detection task.
- Tuning the overlap threshold on the test set.

156.9 See also

Volume 07 for precision, recall, and operating points. Volume 16 for detectors that fail on a subpopulation the aggregate AP hides.

157 Modern vision architectures

158 Modern vision architectures

Reference question. What changed between a small convnet and a current vision backbone, in terms you can test?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

158.1 Reference card

Item	Statement
Residual path	A skip connection $x + F(x)$ so a layer can learn a perturbation of the identity.
Stem and stage	Early downsampling, then stages that trade spatial size for channels.
Attention block	A content-based mix of spatial tokens. It does not by itself know the grid unless position is encoded.
Pretraining	Weights fit on a previous dataset. They are a prior, and they can carry that dataset's labels and biases.
Test	A shifted-object set, a different camera, and a linear probe on frozen features versus full fine-tuning.

158.2 Theory

Modern backbones are still functions from tensors to tensors. Residual branches make depth easier to optimize because the identity path is present at initialization if the branch starts small. Downsampling stages are a computational bet that high resolution is needed only early. Attention mixes information globally within a window or an image and needs a position signal to break permutation symmetry of the tokens.

These are testable designs, not brand names. A residual network that cannot fit the synthetic block task from the convolution chapter has a bug, regardless of its depth. A pretrained backbone that fails when the object moves has learned the centering of its training set. Freeze the backbone and train a linear head before you fine-tune everything. The probe tells

you whether the features are readable. Fine-tuning confounds the features with a new head and a new optimization.

Pretraining data are part of the model card. If you cannot name the data, you cannot name the population the features saw. License and consent of that data are part of whether you may use the weights, not a footnote.

158.3 Derivation

If $F(x) = 0$ at initialization, $x + F(x) = x$, so a stack of residual blocks starts as an identity in those units. Gradients along the skip do not pass through F. A product of many small derivatives is no longer the only path. This does not prevent vanishing gradients inside F, and it does not make the function class "more universal" in the sense of the approximation chapter. It changes optimization. A shift test still applies to a convolutional residual block in the interior of the grid, because both the skip and a translation-equivariant F commute with the shift.

158.4 Code

```
import numpy as np

def residual_add(x, branch):
    x = np.asarray(x, dtype=np.float64)
    branch = np.asarray(branch, dtype=np.float64)
    if x.shape != branch.shape:
        raise ValueError(f"residual shape mismatch: {x.shape} vs {branch.shape}")
    return x + branch
```

158.5 Experiment

Hypothesis. A residual block whose branch is zero returns the input unchanged, and a shifted input remains a shifted output. A position-free average of tokens is invariant to permutation and therefore cannot encode "the block is in the left half" unless a coordinate feature is added.

Protocol. Implement the zero-branch check. For the position check, compare a global mean of pixel values on a left-bright image and a right-bright image. They match, so the mean cannot classify side. Adding the column index as a feature makes a linear rule possible; demonstrate that with a hand-built score.

Metric. Max error of the identity check, and accuracy of the side rule with and without the coordinate.

Decision rule. The identity check is exact in float64 up to 10^{-12}. The mean-only side classifier must fail. If it succeeds, the images differ in total brightness, not only in position.

158.6 Project

Deliverable. An architecture card for a small convnet you train and, if you use a public pretrained model, a second card for that model naming its training data as well as you can, the probe result, and a shifted-object test.

Constraints. The shifted test is not used to pick the architecture and then quoted as a fresh result. If you tune on it, label the number exploratory.

Rubric. You can say which inductive bias you relied on, in a sentence a Volume 09 reader would recognize.

158.7 Depth tracks

Beginner. Check residual addition and the global-mean position failure.

University. Train the small convnet on the synthetic block task and run a shifted test.

Research. Compare a frozen pretrained representation's linear probe with full fine-tuning on a tiny labeled set you are allowed to use. Discuss what a win for fine-tuning confounds.

158.8 Failure modes

- Treating a leaderboard architecture as a prior that replaces baselines.
- Fine-tuning on a test set because pretrained models "only need a few epochs."
- Ignoring the license of pretrained weights.

158.9 See also

Volume 17 for foundation models that include vision. Volume 08 for probes.

159 Evaluation for vision

160 Evaluation for vision

Reference question. Which vision numbers survive a change of camera, crop, or class frequency?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

160.1 Reference card

Item	Statement
In-domain test	New images from the same collection protocol as the training images.
Shift test	A change you can name: brightness, shift, crop, site, or year.
Class mix	Accuracy is not comparable across test sets with different base rates.
Localization	If the decision needs a location, an image-level score is not sufficient.
Leakage	Near-duplicate frames of the same scene in train and test, or the same patient, inflate the number.

160.2 Theory

Vision test sets are often easier than the world the model will see, because the collection protocol is stable. A high in-domain accuracy is a real result about that protocol and a weak result about a new hospital, a new phone, or a new year. The model card should separate these.

Near-duplicates are the entity problem. Frames from one video are not independent images. Splitting them at random lets the model memorize the scene. Group by video, patient, or location. A shift in brightness is the standardization experiment from the first chapter, promoted to a model test: do not recompute moments on the shifted test set if deployment will not do that either. Be consistent about which moments the deployed system uses, and test that procedure.

Calibration of a vision classifier still matters when a human will use the probability. Accuracy does not certify it. A reliability table on an in-domain validation set is the minimum. A reliability table under shift is the honest one, and it usually looks worse.

160.3 Derivation

Suppose accuracy on a balanced test set is a, and you deploy at base rate $p \ll 1/2$ with the same class-conditional detection behavior. Accuracy becomes a different function of true and false positive rates and p. Two models with the same balanced accuracy can rank differently at the deployment base rate. Report the class-conditional rates, or the metric at the deployment mix, using the threshold rule from Volume 07. Do not transport a balanced accuracy number into an imbalanced decision.

160.4 Code

```
import numpy as np

def accuracy_at_base_rate(tpr, fpr, p):
    # Predict positive with the given class-conditional rates.
    return float(tpr * p + (1 - fpr) * (1 - p))
```

160.5 Experiment

Hypothesis. Two operating points with the same balanced accuracy, $0.5(\mathrm{TPR} + 1 - \mathrm{FPR})$, can have different accuracy at $p = 0.01$.

Protocol. Point A: TPR 0.9, FPR 0.1. Point B: TPR 0.99, FPR 0.19. Check balanced accuracy matches. Evaluate `accuracy_at_base_rate` at $p = 0.5$ and $p = 0.01$.

Metric. The two accuracies.

Decision rule. They should agree at $p = 0.5$ and disagree at $p = 0.01$. If they do not, the formula was applied to the wrong rate. This numerical fact is the reason a vision paper's headline accuracy is not your operating metric.

160.6 Project

Deliverable. An evaluation note for the synthetic block detector: in-domain metric, a shifted-block metric, a brightness-shifted metric, and a grouped split if you created near-duplicate images. Include the base-rate calculation if you threshold a score.

Constraints. Moments and thresholds follow the deployment rule you wrote down.

Rubric. You refuse to quote a single accuracy as the performance of the system.

160.7 Depth tracks

Beginner. Compute the two base-rate accuracies by hand and in code.

University. Write the full synthetic evaluation note.

Research. Design a grouped split for a video dataset and explain a leakage path through background identity rather than through the object.

160.8 Failure modes

- A random frame split.
- Re-tuning on each shifted test set and calling it robustness.
- Comparing two papers' top-1 accuracy across different class mixes.

160.9 See also

Volume 16 for a fuller shift and harm audit. Volume 07 for the threshold.

161 Project: a small vision study

162 Project: a small vision study

Reference question. Can you carry a vision claim from a contract through a baseline, a convnet, and a shift test?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

162.1 Reference card

Item	Statement
Task	Synthetic or a dataset you have the right to use. The label definition fits in five lines.
Baselines	Constant, linear on pooled features, and one convolutional model.
Ablation	One factor: depth, augmentation, or input size.
Shift	One predeclared shift.
Card	Limits include the collection protocol you did not vary.

162.2 Theory

This project is the volume's integration test. A reader of Volumes 07, 09, and 10 should recognize every piece: a split, a baseline bake-off, a checked or library model with a stated reduction, one ablation, and a card. The new piece is the grid. If the convolutional model cannot beat pooled linear features on a task you designed to be local and shift-friendly, believe that measurement. Either the task is easier than you thought or the convnet is buggy. Both outcomes are publishable in your lab note.

Keep the images small enough that a CPU run finishes. The point is the protocol. A large backbone you cannot train carefully is a worse reference than a small kernel you can ablate.

162.3 Derivation

The claim structure is the same as the paired gap in Volume 07. Let D_i be the per-image loss difference between the convnet and the pooled linear model. The interval for the mean difference is the in-domain result. Repeat the losses under the shifted distribution without retuning, and report a second interval. A model that wins in-domain and ties under the shift you care about has not won the deployment claim. The card's primary metric must say which distribution it refers to.

162.4 Code

```
import numpy as np

def pooled_brightness(images):
    x = np.asarray(images, dtype=np.float64)
    return x.mean(axis=(2, 3))
```

162.5 Experiment

Hypothesis. On the moving bright-block task, a convolutional feature beats pooled brightness, and pooled brightness fails once positive and negative images are matched for total brightness.

Protocol. Construct negatives with the same total intensity and no compact block, and positives with a compact block. Fit the pooled baseline and a one-filter threshold or a small trained convnet.

Metric. Accuracy or average precision at a predeclared score threshold, in-domain and with the block shifted.

Decision rule. If pooled brightness matches the convnet, the negatives are darker and the task is not spatial. Rebuild the negatives. Do not celebrate a model for measuring brightness.

162.6 Project

Deliverable. The dataset generator, the three baselines or models, one ablation, the shift evaluation, and a model card.

Constraints. Training moments and the decision threshold are frozen before the shift test.

Rubric. The note's first paragraph states the label, the split unit, and the shift. A screenshot of a detection is not a substitute.

162.7 Depth tracks

Beginner. Generate the blocks and show the pooled baseline failing the equal-brightness control.

University. Complete the project as specified.

Research. Add a grouped near-duplicate split: several jitters of the same block location. Show the optimistic gap of a random split versus a group split.

162.8 Failure modes

- Tuning on the shifted images.
- A task that pooled color already solves.
- Weights from a dataset whose license you have not read.

162.9 See also

Volume 15 if this model is served. Volume 16 if the images are of people.

Part XII

Volume 12. Sequences and language

163 Volume 12. Sequences and language

164 Volume 12. Sequences and language

Stage. University to research

Audience. A reader who can train a classifier and can state a split unit.

Question this volume answers. How do you model ordered symbols without leaking the future or trusting a probability you did not calibrate?

This volume is a reference, not a short tour. Each chapter opens with a card you can look up later, then gives the theory, a reference implementation, an experiment with a decision rule, and a project with a rubric. Read in order the first time. After that, enter through the reference card of the chapter that matches the problem in front of you.

164.1 How to use the four layers

Layer	What you should be able to do when you finish it
Theory	State the definition, the assumptions, and the result, including what the result does not say.
Code	Implement the core object in a few dozen lines and check it against a known case.
Experiment	Change one factor, record a metric with uncertainty, and say what outcome would change your mind.
Project	Produce an artifact someone else can rerun: code, figure, table, and a one-page lab note.

164.2 Depth tracks

Every chapter ends with three tracks.

- **Beginner.** Do the reference card, the code, and the smallest experiment. Skip optional proofs.
- **University.** Do the derivation, the full experiment, and the project rubric.
- **Research.** Read the failure modes, change an assumption, and write what would have to be true for the method to fail on a new distribution.

164.3 Chapters

No.	Chapter	File
01	Tokens and sequences	`vol12/01_tokens.qmd`
02	Recurrent models	`vol12/02_recurrent.qmd`
03	Attention and transformers	`vol12/03_attention.qmd`
04	Training and adapting language models	`vol12/04_finetune.qmd`
05	Evaluation of generation	`vol12/05_generation_eval.qmd`
06	Project: a small language pipeline	`vol12/06_language_project.qmd`

164.4 Lab note

Keep one markdown file per experiment, `notes/YYYY-MM-DD-<slug>.md`, with the hypothesis, the command you ran, the metric, and the decision. The series is a reference only if the experiments you run are attached to it.

165 Tokens and sequences

166 Tokens and sequences

Reference question. What is the unit of a language or sequence model, and what does a split have to protect?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

166.1 Reference card

Item	Statement
Token	An element of a finite vocabulary, or a bin of a continuous stream. The tokenizer is part of the model.
Document	The leakage boundary. Sentences from one document are not independent draws.
Vocabulary	Built on the training text only. A test token that is absent has a defined unknown id.
Length	Padding is not content. The loss mask must ignore pad positions.
Order	A bag-of-tokens baseline is mandatory. If it wins, the model is not yet using order.

166.2 Theory

A tokenizer is a function from raw text or signals to ids. Changing it changes the input space. Two runs with different vocabularies are not the same architecture on the same data. Fit the vocabulary, including frequency cutoffs, on the training split. A vocabulary that saw the test documents has leaked rare names and labels.

The split unit is the document, the user, or the time period. Splitting sentences independently lets a model memorize a document from its other sentences. Time-ordered text should be tested on the future. A random document split answers a different question: new documents from the same period.

Padding exists so a batch can be rectangular. A loss that predicts the pad id will look strangely accurate and will waste capacity. Mask it. A metric that does not mask it is a different metric.

166.3 Derivation

Let a document be a token sequence $w_{1:T}$. A next-token model factorizes $p(w_{1:T}) = \prod_t p(w_t \mid w_{<t})$ under the model. The average negative log likelihood per non-pad token is the usual training loss. Dividing by the number of documents instead of tokens gives a different weight to long documents. The mask $m_t \in \{0, 1\}$ enters as $\sum_t m_t \ell_t / \sum_t m_t$. If m_t is zero on pads and also on the token you are not allowed to predict, the denominator is the count of real predictions. A bug that sets $m_t = 1$ on the position being predicted from itself is label leakage inside the sequence.

166.4 Code

```
import numpy as np

def masked_mean(values, mask):
    values = np.asarray(values, dtype=np.float64)
    mask = np.asarray(mask, dtype=np.float64)
    denom = mask.sum()
    if denom <= 0:
        raise ValueError("empty mask")
    return float((values * mask).sum() / denom)
```

166.5 Experiment

Hypothesis. Including pad positions in the loss, when pads are easy to predict, lowers the reported mean loss without improving token predictions on real positions.

Protocol. A batch with half the positions masked as pad, real-token loss 2.0, pad loss 0.0. Compare `masked_mean` with a plain mean.

Metric. The two means.

Decision rule. The plain mean should be optimistically smaller. If your training log uses the plain mean, the number is not the masked language-model loss. Fix the log before comparing runs.

166.6 Project

Deliverable. A tokenizer or a simple whitespace vocabulary fit on training documents only, an unknown-token rule, a pad mask, and a bag-of-tokens logistic or least-squares baseline on a synthetic labeling task.

Constraints. The task must have a document id so the split can be grouped. Synthetic documents are acceptable: a keyword determines the label, and an order-sensitive variant uses a two-token phrase.

Rubric. You show one case where the bag of tokens suffices and one where order matters, analogous to the stump-versus-linear flip in Volume 06.

166.7 Depth tracks

Beginner. Implement `masked_mean` and the pad experiment.

University. Build the vocabulary and the two synthetic tasks.

Research. Describe byte-pair encoding in a paragraph and the leakage path if merge rules are learned on the full corpus, including the test set.

166.8 Failure modes

- A shared vocabulary built before the split.
- Padding tokens in the accuracy.
- Treating two sentences from one article as independent test points.

166.9 See also

Volume 03 for grouped splits. Volume 10 for the reduction.

167 Recurrent models

168 Recurrent models

Reference question. What state does a recurrent net carry, and what does truncation do to the dependency it can learn?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

168.1 Reference card

Item	Statement
Recurrence	$h_t = f(h_{t-1}, x_t)$. The same f at every time, which is weight sharing along the sequence.
State	h_t is a function of $x_{1:t}$ and h_0. It is a bottleneck.
BPTT	Backpropagation through the unrolled graph. Gradients accumulate along the path.
Truncation	Stopping the backward path after K steps makes the update blind to dependencies longer than K, even if the forward state could carry them.
Vanish	Products of Jacobians along time vanish or explode. Gates are a mitigation, not a proof.

168.2 Theory

A recurrent net is a deep net in time, with tied weights. The inductive bias is that the same update is useful at every position. That fits a stationary process and misfires when the beginning of a document should be treated by a different rule than the end, unless the inputs mark position.

The hidden state is a finite summary. If the label depends on a detail the state was too small to store, the model cannot represent the function, whatever the optimizer does. This is the approximation question again, with a bottleneck you chose.

Truncated backpropagation is an optimization approximation. You can still, in the forward pass, carry state across the whole sequence, and only backpropagate through the last K

steps. The early inputs then affect the forward state but not that update. Long-range learning can fail even though a long-range path exists. If the experiment needs a dependency of length 20, truncation at 5 is a reason.

168.3 Derivation

Unroll $h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$ for three steps and differentiate a loss on h_3 with respect to h_0. The chain contains the product $J_3 J_2 J_1$ of Jacobians of the tanh map. Each Jacobian includes $\mathrm{diag}(1 - \tanh^2)$ times W_h. If the singular values of that product are much less than 1, the gradient with respect to early inputs is tiny. Clipping the gradient norm bounds the step but does not restore information that the product destroyed. It only stops a divergent step.

168.4 Code

```
import numpy as np

def rnn_step(h, x, Wh, Wx, b):
    z = h @ Wh + x @ Wx + b
    return np.tanh(z)
```

168.5 Experiment

Hypothesis. A hand-built state that copies a bit can remember an input from K steps ago, and a truncated analytic story that ignores the path cannot update the copy weight from a loss that depends only on that memory. In code, demonstrate memory with an identity recurrence, and demonstrate a vanished product by powering a scalar 0.5 twenty times.

Protocol. Identity: $h_t = x_t$ if you overwrite, or $h_t = h_{t-1}$ if x_t is a blank. Report h_{20} given a bit at step 0. Separately compute 0.5^{20}.

Metric. Whether the bit survived, and the value of the product.

Decision rule. The bit should survive the identity recurrence. The product should be numerically tiny. If you claim a gated RNN fixes this, the experiment must show a gate that is open on the remembered path, not a citation.

168.6 Project

Deliverable. A tiny recurrent classifier on the order-sensitive synthetic phrase task, compared with the bag-of-tokens baseline, plus a truncation or depth note: what is the longest dependency in the synthetic language, and can the unroll see it?

Constraints. Pad masks apply. Documents are the split unit.

Rubric. You can say whether a failure is the bag-of-tokens baseline winning, a state that is too small, or a dependency longer than the truncation.

168.7 Depth tracks

Beginner. Step a one-dimensional recurrence by hand for three steps.

University. Complete the memory demo and the synthetic comparison.

Research. Derive the gradient clip's effect: it rescales g when $\|g\| > c$ and leaves it otherwise. Explain why clipping can stabilize training without lengthening the dependency the optimizer can credit.

168.8 Failure modes

- Feeding the target token as an input at the same time step.
- Evaluating a language model with teacher forcing and calling that a free-running generation error.
- Splitting a sequence in the middle so the state at the boundary has no past, then scoring the prefix.

168.9 See also

Volume 09 for products of Jacobians. The attention chapter for a path that does not multiply through every intermediate time.

169 Attention and transformers

170 Attention and transformers

Reference question. What does an attention layer compute, and what cost and bias come with it?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

170.1 Reference card

Item	Statement
Scores	A compatibility $q_i^\top k_j/\sqrt{d}$ between a query at position i and a key at position j.
Weights	A softmax over allowed positions. The mask sets disallowed scores to a large negative number before the softmax.
Output	A weighted sum of values. The layer is a routed average, not a lookup of a single token, unless the softmax is peaked.
Causal mask	Position i may not read $j > i$ in a next-token model. A bug here is label leakage.
Cost	Full attention is quadratic in the sequence length, in memory and in the matmul.

170.2 Theory

Self-attention lets a position read any other allowed position in one step. The path length between distant tokens is constant, which is the contrast with the recurrent product of Jacobians. The price is quadratic compute and a weak inductive bias about order. Without a position encoding, the layer is permutation-equivariant in a precise sense: it does not know which token came first. Sequences are ordered, so you add positions, and those features are part of the model.

The causal mask is the difference between a language model and a model that peeks at the answer. Bidirectional encoders are appropriate when the whole sequence is an input, as in classification of a complete document. They are inappropriate when you claim to predict

the future. The mask test is the unit test. It is not optional because a library "usually" masks.

Multi-head attention is several of these maps concatenated. It raises capacity. It does not by itself create linguistic understanding. Evaluate it as a function.

170.3 Derivation

Let Q, K, V be $T \times d$ matrices for one head and one example. Scores $S = QK^\top / \sqrt{d}$. For a causal mask, set $S_{ij} = -\infty$ when $j > i$, then $A = \mathrm{softmax}(S)$ row-wise, and the output is AV. Row i of A sums to 1 and is zero after i. The gradient of a loss on the output with respect to V is $A^\top \nabla_{\text{out}}$. The gradient with respect to S uses the Jacobian of the softmax. A numerical check on a short T catches a mask that was applied after the softmax, which cannot zero the future once the weights are positive.

170.4 Code

```
import numpy as np

def causal_attention(Q, K, V):
    Q = np.asarray(Q, dtype=np.float64)
    K = np.asarray(K, dtype=np.float64)
    V = np.asarray(V, dtype=np.float64)
    scale = np.sqrt(Q.shape[-1])
    scores = (Q @ K.T) / scale
    t = scores.shape[0]
    future = np.triu(np.ones((t, t), dtype=bool), k=1)
    scores = np.where(future, -1e9, scores)
    scores = scores - scores.max(axis=-1, keepdims=True)
    weights = np.exp(scores)
    weights = weights / weights.sum(axis=-1, keepdims=True)
    return weights @ V, weights
```

170.5 Experiment

Hypothesis. With a causal mask, attention weights on the future are numerically zero and the output at position i does not change when a future value vector changes. Without the mask, it does.

Protocol. $T = 4$, random Q, K, V. Perturb V at the last position and compare outputs at position 0.

Metric. Max future weight, and the change in the position-0 output.

Decision rule. Future weights below 10^{-8} and a zero change at position 0 for the masked layer. If the unmasked layer also shows a zero change, the query-key pair assigned that position no weight by chance; resample.

170.6 Project

Deliverable. The masked attention function, the peek test, and a tiny trained or closed-form sequence task where a bag of tokens fails and a one-layer causal or bidirectional model matches the task's information constraint. Classification of a finished string may be bidirectional. Next-token prediction must be causal.

Constraints. The peek test is in the debug checklist.

Rubric. You can explain the quadratic cost at $T = 1024$ versus $T = 128$ in arithmetic operations for the score matrix.

170.7 Depth tracks

Beginner. Implement causal attention and print the weight matrix.

University. Run the peek test and the bag-of-tokens comparison.

Research. Describe rotary or absolute position features in one paragraph and the failure mode if two sequences are concatenated without a segment boundary the model can see.

170.8 Failure modes

- Applying the mask after the softmax.
- A bidirectional encoder advertised as a next-token model.
- Ignoring the pad mask so padding positions become keys the model attends to.

170.9 See also

Volume 15 for the quadratic cost as a systems constraint. Volume 17 for models that use this block at large scale.

171 Training and adapting language models

172 Training and adapting language models

Reference question. What is the difference between training a small model from scratch and adapting a model you did not train?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

172.1 Reference card

Item	Statement
From scratch	You own the data, the tokenizer, and the loss. The baselines of this volume apply.
Fine-tune	Start from someone else's parameters and continue training. You inherit their data and their failures.
Freeze	Train a head on frozen features. A probe, in the sense of Volume 08.
Prompt	No gradient. The model is a fixed function and the prompt is an input. It is not a guarantee of the same behavior next month if the model is hosted.
Split	The downstream split is still yours. Their pretraining data may already contain your test documents.

172.2 Theory

Adapting a pretrained model can beat a small model on a tiny labeled set because the pretraining was a large self-supervised task. That is a hypothesis about feature reuse, tested by a probe and by a fine-tune, not a law. The pretrained model may have seen text that overlaps your test set. Deduplication and a truly new domain are the checks. If you cannot check, the card says the overlap is unknown.

Fine-tuning all weights on a small set can erase useful features and can memorize the small set. A frozen head, a small learning rate, and early stopping are regularizers from Volume

10. The bag-of-tokens baseline remains mandatory. On some classification tasks it is close enough that a transformer is an expensive tie.

A hosted model you call through an API is not a checkpoint you can hash. The config must store the model name and the date. A result that cannot name the model version is not reproducible. This series prefers a local checkpoint when the claim is about your training.

172.3 Derivation

Let θ_0 be the pretrained parameter and $\theta_0 + \Delta$ the fine-tuned one. A penalty $\|\Delta\|$ or a low-rank constraint on Δ keeps the new function near the old one. Full fine-tuning is the unconstrained case. The validation loss decides the constraint. The test set does not. If pretraining already optimized a loss that includes your test documents, then even $\Delta = 0$ is not an out-of-sample model. No penalty on Δ repairs contamination that happened before you started.

172.4 Code

```
import numpy as np

def lora_delta(A, B, x):
    # x (n, d), A (d, r), B (r, d_out) or match your layout and document it.
    A = np.asarray(A, dtype=np.float64)
    B = np.asarray(B, dtype=np.float64)
    x = np.asarray(x, dtype=np.float64)
    return (x @ A) @ B
```

172.5 Experiment

Hypothesis. A low-rank update can represent a small correction to a linear map and cannot, at rank 1, represent an arbitrary full-rank correction of a $d \times d$ map when $d > 1$.

Protocol. $d = 4$. Fit a rank-1 $\Delta = ab^\top$ by least squares to a random full-rank target matrix on many x, and to a rank-1 target.

Metric. Frobenius residual of the two fits.

Decision rule. The rank-1 target should be fit closely. The full-rank target should leave a visible residual. If both residuals vanish, the parameterization is not rank 1.

172.6 Project

Deliverable. A downstream classification note: bag-of-tokens baseline, a small from-scratch model or a frozen probe, and a fine-tune only if you have a checkpoint you may use. Include a contamination caveat.

Constraints. The tokenizer and the split follow this volume's rules.

Rubric. You do not claim a general language ability. You claim a metric on a named split.

172.7 Depth tracks

Beginner. Show the rank-1 residual gap with `lora_delta` or an outer product.

University. Complete the baseline and one adapted or from-scratch model on the synthetic document task.

Research. Write a deduplication check: exact overlap between train and test strings, and a limit of that check for paraphrases it will not catch.

172.8 Failure modes

- Fine-tuning on the test split because the set is "small."
- An API result with no date and no model id.
- Dropping the bag-of-tokens baseline because a transformer "should" win.

172.9 See also

Volume 08 for probes. Volume 17 for scale and contamination as a research topic.

173 Evaluation of generation

174 Evaluation of generation

Reference question. What may you conclude from a generation metric, and what requires a human or a task-specific check?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

174.1 Reference card

Item	Statement
Likelihood	A held-out negative log likelihood or perplexity, under a stated tokenization, measures the predictive model. It is not a judgment of truth.
Reference overlap	A score against one reference string punishes valid paraphrases and rewards overlap.
Task metric	If generation is a means to an extracted answer, score the answer, not the prose.
Degeneration	Repetition, empty answers, and copied inputs need their own counters.
Calibration	A high token probability is not a calibrated probability that a claim is true.

174.2 Theory

Generation has several different questions. “Does this model predict the next token of this corpus?” is a likelihood question. “Does the string match a reference translation?” is a string-overlap question with known failure modes. “Is the answer correct?” is a task question and needs an answer key or a checker. “Should a person trust the prose?” is not settled by any of those.

Perplexity is exp of the mean token negative log likelihood. It depends on the tokenizer. You cannot compare perplexities across different tokenizations without a conversion you justify. A model that memorizes the test set has excellent perplexity and has not generalized. The contamination check sits next to the number.

Automatic overlap metrics are baselines. They are useful when they correlate with a decision you have validated on a sample of human or programmatic checks. They are not the decision. For a synthetic task, prefer a programmatic checker: did the output contain the required phrase, did the arithmetic hold, did the format parse.

174.3 Derivation

If the model factorizes token probabilities, the log probability of a completed string under teacher forcing is the sum of conditional log probabilities. That sum uses the true prefix at every step. Free-running generation feeds the model's own samples back in, so errors compound. A high teacher-forced likelihood does not imply a high probability for the string the model actually emits when it is allowed to condition on its own mistakes. Report which protocol you used. A metric computed on teacher-forced completions of the reference is not a metric of free-running text.

174.4 Code

```
import numpy as np

def perplexity_from_nll(mean_nll):
    if mean_nll < 0:
        raise ValueError("mean negative log likelihood must be nonnegative")
    return float(np.exp(mean_nll))
```

174.5 Experiment

Hypothesis. Two tokenizations of the same characters can yield different per-token negative log likelihoods for a model that is uniform over its vocabulary, and the character-level numbers can be reconciled only if you account for tokens per character.

Protocol. Vocabulary sizes 30 and 100, uniform model, sequences of 10 tokens. Compute per-token NLL and perplexity. Note that the uniform NLL is $\log |V|$.

Metric. The two perplexities.

Decision rule. They must differ. A comparison of two models that hides a tokenizer change is the same kind of mismatch. Record both vocabulary sizes whenever you quote perplexity.

174.6 Project

Deliverable. An evaluation note for a small generator or a prompted fixed model on a synthetic task with a programmatic checker, plus likelihood on held-out text if you trained a model. Include repetition rate or exact-copy rate.

Constraints. The checker is written before you generate the test outputs, as far as that is operationally possible. If you adjust the checker after seeing outputs, say so.

Rubric. You separate "predicts the corpus," "matches a reference," and "satisfies the checker." You do not use one of those phrases to mean the others.

174.7 Depth tracks

Beginner. Compute perplexity of a uniform model over a vocabulary of size 10.

University. Complete the synthetic checker evaluation.

Research. Discuss why a model can assign high probability to a false statement that is common in the training text. Tie this to proper scoring: the likelihood scores the string distribution, not the truth of a proposition under an external world.

174.8 Failure modes

- Comparing perplexity across tokenizers.
- Selecting checkpoints by a free-running demo you reran until it looked good.
- Treating a fluent paragraph as a cited fact.

174.9 See also

Volume 16 for claims that affect people. Volume 17 for evaluation of large models.

175 Project: a small language pipeline

176 Project: a small language pipeline

Reference question. Can you ship a language classification or tagging pipeline whose baseline, mask, and errors are visible?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

176.1 Reference card

Item	Statement
Task	Document label or a synthetic tag. Not an open-ended claim about understanding.
Pipeline	Raw text, tokenizer, features or model, masked metric, error slices.
Baselines	Majority class and bag of tokens.
Model	A linear model, a small recurrent net, or a small attention layer.
Errors	A taxonomy: unknown tokens, order failures, leaked duplicates, pad-mask bugs.

176.2 Theory

The pipeline is the product. A good attention block inside a leaky split is a bad system. This project scores the pipeline. The write-up should let someone rerun it from the experiment folder and disagree with your decision using your table.

Prefer a task with a known mechanism so error analysis can name the mechanism. A synthetic phrase task, a copied bag-of-tokens task, and a duplicate-document leak task are enough. If you use a public dataset, the card still needs the split unit and the license.

The decision is whether the order-sensitive model earns its cost over the bag of tokens on the primary metric, by the paired-gap rule. If it does not, ship the linear model in the note's recommendation and keep the neural result as a negative. Negative results are part of a reference.

176.3 Derivation

The pipeline risk is $R(\text{decode} \circ f \circ \text{tokenize})$. A comparison that swaps the tokenizer between the baseline and the neural model confounds the representation with the family. Use one tokenizer for both, or treat the tokenizer as a factor in an ablation with its own row. The primary metric uses the pad mask and the document-level split. The optimistic sentence-level split can appear only as a labeled anti-pattern, the way Protocol A did in Volume 07.

176.4 Code

```
import numpy as np

def majority_label(y_train, n_test):
    y_train = np.asarray(y_train)
    values, counts = np.unique(y_train, return_counts=True)
    label = values[np.argmax(counts)]
    return np.full(n_test, label)
```

176.5 Experiment

Hypothesis. On documents whose label is the majority keyword, the bag-of-tokens model matches a sequence model. On documents whose label is whether token A appears before token B, the bag model falls to chance if the counts are balanced, and a causal or bidirectional sequence model can exceed it.

Protocol. Generate both datasets. Same vocabulary and document-level split. Majority baseline on both.

Metric. Accuracy or log loss, with a paired comparison on the order-sensitive set.

Decision rule. A sequence model that does not win on the order-sensitive set is not finished. A bag model that wins on the order-sensitive set means the generator leaks order into counts; fix the generator.

176.6 Project

Deliverable. The generator, the majority and bag baselines, one sequence model, the mask, the document split, an error taxonomy, and a model card. Include the leaky sentence split only as a warning row.

Constraints. Unknown tokens and pads are defined. The primary metric ignores pads.

Rubric. The recommendation names the simpler model if the gap does not clear the practical margin.

176.7 Depth tracks

Beginner. Implement the majority baseline and the keyword dataset.

University. Complete both datasets and the card.

Research. Add a contamination row: copy 10 percent of test documents into training and measure the optimistic gap. Leave the row in the note as a warning, not as a system to ship.

176.8 Failure modes

- A fluent demo in place of the table.
- Different tokenizers for the baseline and the model.
- Pads counted as correct predictions.

176.9 See also

Volume 17 if you replace the small model with a pretrained one. The same baselines stay.

Part XIII

Volume 13. Generative models

177 Volume 13. Generative models

178 Volume 13. Generative models

Stage. University to research

Audience. A reader who can fit a density and a classifier and can read a likelihood.

Question this volume answers. What distribution is a generator claiming to sample, and which evaluation tells you it failed?

This volume is a reference, not a short tour. Each chapter opens with a card you can look up later, then gives the theory, a reference implementation, an experiment with a decision rule, and a project with a rubric. Read in order the first time. After that, enter through the reference card of the chapter that matches the problem in front of you.

178.1 How to use the four layers

Layer	What you should be able to do when you finish it
Theory	State the definition, the assumptions, and the result, including what the result does not say.
Code	Implement the core object in a few dozen lines and check it against a known case.
Experiment	Change one factor, record a metric with uncertainty, and say what outcome would change your mind.
Project	Produce an artifact someone else can rerun: code, figure, table, and a one-page lab note.

178.2 Depth tracks

Every chapter ends with three tracks.

- **Beginner.** Do the reference card, the code, and the smallest experiment. Skip optional proofs.
- **University.** Do the derivation, the full experiment, and the project rubric.
- **Research.** Read the failure modes, change an assumption, and write what would have to be true for the method to fail on a new distribution.

178.3 Chapters

No.	Chapter	File
01	Autoregressive models	`vol13/01_autoregressive.qmd`
02	Variational autoencoders	`vol13/02_vae.qmd`
03	Adversarial generators	`vol13/03_gans.qmd`
04	Diffusion models	`vol13/04_diffusion.qmd`
05	Evaluating generators	`vol13/05_gen_eval.qmd`
06	Project: a small generator with an evaluation	`vol13/06_gen_project.qmd`

178.4 Lab note

Keep one markdown file per experiment, `notes/YYYY-MM-DD-<slug>.md`, with the hypothesis, the command you ran, the metric, and the decision. The series is a reference only if the experiments you run are attached to it.

179 Autoregressive models

180 Autoregressive models

Reference question. What does an autoregressive factorization claim, and how do you sample from it?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

180.1 Reference card

Item	Statement
Factorization	$p(x_{1:T}) = \prod_t p(x_t \mid x_{<t})$, with a declared start token.
Training	Teacher-forced conditional likelihood of the observed next symbol.
Sampling	Draw $\hat{x}_t$ from the conditional and feed it back. This is not the teacher-forced path.
Exposure	The state distribution at training, where prefixes are real, differs from the state distribution at sampling, where prefixes are the model's own draws.
Tractable likelihood	The factorization gives you a log likelihood. A model without one needs a different evaluation.

180.2 Theory

Autoregressive models are the sequence models of Volume 12 trained as generative models of the whole string, not only as classifiers. The likelihood is a proper score for the string distribution under the factorization. It does not score whether a free-running sample looks right to a person, and it does not score whether a conditional sample matches a prompt you added at deployment unless that prompt was part of the model.

Exposure bias is the train-test mismatch of the prefix distribution. A model can assign high likelihood to a corpus and still degenerate when sampled, because it never had to continue from its own errors during training. Report both teacher-forced likelihood and a free-running checker.

Temperature and top-k truncation change the distribution you sample from. They are not the distribution you evaluated with likelihood. If you deploy a truncated sampler, the card names it. The likelihood of the original model is then not the likelihood of the deployed sampler.

180.3 Derivation

The log likelihood of a sequence is $\sum_t \log p_\theta(x_t \mid x_{<t})$. A gradient step on this sum, with a causal mask, is ordinary supervised learning of the next token. Ancestral sampling draws x_1, then x_2 given x_1, and so on. The probability of a sampled trajectory under the model is the product of the conditionals that were sampled, not the product along the ground-truth string. Truncating the support of each conditional defines a new model $q \neq p_\theta$. $\mathrm{KL}(q \| p_\theta)$ need not be zero. Do not quote p_θ's perplexity as a property of q.

180.4 Code

```
import numpy as np

def ancestral_sample(next_probs, start, steps, rng):
    seq = [start]
    for _ in range(steps):
        p = np.asarray(next_probs(seq), dtype=np.float64)
        p = p / p.sum()
        seq.append(int(rng.choice(len(p), p=p)))
    return seq
```

180.5 Experiment

Hypothesis. A bigram model fit on a language with a strict alternation pattern has high teacher-forced likelihood on held-out strings from that language, and a sampler that is not masked to the support can still emit illegal pairs if you deliberately use a uniform conditional.

Protocol. Alphabet `{0, 1}`, legal strings alternate. Fit a bigram conditional by counts with a small smoothing parameter. Sample from the fitted conditionals and from a uniform conditional.

Metric. Fraction of adjacent pairs that violate alternation, and held-out mean NLL of the fitted model.

Decision rule. The fitted sampler should violate rarely, at a rate explained by smoothing. The uniform sampler should violate often. If the fitted sampler violates often, the conditional tables were built with a leakage from the future or a bug in the counts.

180.6 Project

Deliverable. A bigram or tiny neural autoregressive model on the synthetic alternating language, with teacher-forced NLL, free-running violation rate, and a temperature or truncation variant labeled as a different sampler.

Constraints. The causal constraint is tested. Pads are masked.

Rubric. You never describe the truncated sampler's behavior using the untruncated perplexity alone.

180.7 Depth tracks

Beginner. Fit the bigram counts and compute one string's log probability by hand.

University. Complete the sample-versus-likelihood comparison.

Research. Explain scheduled sampling in one paragraph and why it optimizes a different objective than maximum likelihood. Do not claim it fixes exposure bias without a measurement.

180.8 Failure modes

- Reporting teacher-forced loss as sample quality.
- Changing temperature and comparing likelihoods as if the model were fixed.
- A non-causal factorization presented as autoregressive.

180.9 See also

Volume 12 for masks and perplexity. The diffusion chapter for a generator without this factorization.

181 Variational autoencoders

182 Variational autoencoders

Reference question. What does a VAE optimize, and why is the objective not the exact log likelihood?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

182.1 Reference card

Item	Statement
Model	A latent $z \sim p(z)$, an observation model $p_\theta(x \mid z)$, and an encoder $q_\phi(z \mid x)$.
ELBO	A lower bound on $\log p_\theta(x)$. Maximizing the bound is not the same as computing the bound's gap.
Terms	A reconstruction term minus a KL penalty that pulls $q_\phi(z \mid x)$ toward $p(z)$.
Gap	The ELBO equals the true log likelihood only when q matches the posterior. Usually it does not.
Samples	Draw $z \sim p(z)$ and then $x \sim p(x \mid z)$ to sample the prior predictive. Encoding a training point and decoding it is a reconstruction, not a sample from the model.

182.2 Theory

A VAE is a likelihood model with an intractable posterior, plus a variational approximation you also train. The evidence lower bound is a surrogate. A high ELBO is encouraging. It does not identify $\log p(x)$, and two models with the same ELBO can have different gaps. If you need a number that is a log likelihood, say "ELBO" or compute a tighter estimate you can defend.

The KL term can collapse. If $q(z \mid x)$ ignores x and matches the prior, the reconstruction term must carry the data with an uninformative latent. The model has then learned a

decoder from noise, and the encoder is decorative. Watch the KL per dimension. A dimension with KL near zero is unused.

Reconstructions look sharper than prior samples when the decoder is deterministic and the latent is a copy of x through a narrow code. Judge the prior samples if you claimed a generator. Judge reconstructions if you claimed a compressor. They are different products.

182.3 Derivation

For a single x,

$$\log p_\theta(x) = \mathbb{E}_q[\log p_\theta(x \mid z)] - \mathrm{KL}(q_\phi(z \mid x) \| p(z)) + \mathrm{KL}(q_\phi(z \mid x) \| p_\theta(z \mid x)).$$

The last KL is nonnegative, so dropping it yields the ELBO, a lower bound. Equality holds when q is the true posterior. The reparameterization $z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$, lets you differentiate a Gaussian q through a sample. A discrete latent needs a different estimator. Do not backpropagate through a `stop_gradient` you forgot about and then wonder why the encoder never moves.

182.4 Code

```
import numpy as np

def gaussian_kl_standard(mu, log_var):
    # KL(N(mu, diag(exp(log_var))) || N(0, I)), summed over latent dimensions.
    mu = np.asarray(mu, dtype=np.float64)
    log_var = np.asarray(log_var, dtype=np.float64)
    return float(0.5 * np.sum(np.exp(log_var) + mu**2 - 1.0 - log_var))
```

182.5 Experiment

Hypothesis. The KL helper is zero at $\mu = 0, \log \sigma^2 = 0$ and positive if μ moves. A collapsed encoder with $\mu = 0$ and $\log \sigma^2 = 0$ for every x has reconstruction loss equal to a decoder that does not see x.

Protocol. Check the KL values. Then, on 1D data with two modes, compare a decoder that receives $z \sim \mathcal{N}(\mu(x), \sigma(x))$ with one that receives $\epsilon \sim \mathcal{N}(0, I)$ only. You may fit both with least squares if you keep the lesson about information flow.

Metric. KL at the standard normal parameters, and reconstruction error with and without x-dependent latents.

Decision rule. KL at the prior parameters should be about 0. The x-dependent code should win on two well-separated modes if the decoder is linear in z and z can encode the mode. If it does not, the code has no path into the decoder.

182.6 Project

Deliverable. A small VAE or a diagonal-Gaussian latent model on a 2D mixture, with ELBO terms logged separately, prior samples, and reconstructions. State which picture the card claims.

Constraints. Do not call the ELBO a log likelihood in the table.

Rubric. A collapsed KL is visible if you cause one, and the note says how you noticed.

182.7 Depth tracks

Beginner. Check `gaussian_kl_standard(0, 0)` and a nonzero mean.

University. Fit the 2D mixture model and separate the plots.

Research. Describe the variational gap in a sentence a reviewer would accept, and one diagnostic that the gap might be large even when the ELBO looks stable.

182.8 Failure modes

- Reporting reconstructions as unconditional samples.
- Adding the KL with the wrong sign and calling the result an ELBO.
- A latent that copies a label you are not allowed to use at generation time.

182.9 See also

Volume 08 for mixtures. Volume 04 for the Gaussian KL's ingredients.

183 Adversarial generators

184 Adversarial generators

Reference question. What game does a GAN play, and why is a lower loss not a better generator?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

184.1 Reference card

Item	Statement
Players	A generator G maps noise z to a sample. A discriminator D scores samples versus data.
Equilibrium	In the idealized game, G matches the data law and D is uninformative.
Practice	The finite-sample game can oscillate, collapse modes, or let D become perfect and uninformative as a gradient.
No likelihood	You do not get $\log p(x)$ from the usual GAN objective.
Eval	A task metric or a distance you predeclare. The discriminator loss is not that metric.

184.2 Theory

A GAN trains a sampler by a critic. When the critic is strong and the generator matches the data, the critic cannot do better than chance. A generator "winning" by driving the discriminator loss to a floor can mean the discriminator collapsed, not that the samples are good. Log both losses and look at samples or at a held-out task metric.

Mode collapse is the failure in which G covers one mode of a mixture and ignores the others. The discriminator may still be fooled on that mode. A coverage check, such as the fraction of true modes that have a nearby sample, belongs in the experiment. A single pretty sample does not answer coverage.

GANs do not give a density value. Do not compare a GAN to a VAE by likelihood unless you add a density estimator you trust. Compare them by a sample metric you would actually use, and give the autoregressive model the likelihood column it earned.

184.3 Derivation

The idealized discriminator, given a fixed generator, scores a point by the odds ratio of the data density and the generator density. The generator is then trained to move samples toward regions the discriminator calls "data." At equilibrium the densities match and the odds ratio is 1. Finite models replace both densities by neural nets and take alternating steps. Alternating gradient steps on a nonconvex game are not guaranteed to reach that equilibrium. A diverging discriminator accuracy with a generator that emits one point is compatible with the algebra of a failed optimization, not a counterexample to the equilibrium statement.

184.4 Code

```
import numpy as np

def mode_coverage(samples, mode_centers, radius):
    samples = np.asarray(samples, dtype=np.float64)
    centers = np.asarray(mode_centers, dtype=np.float64)
    covered = 0
    for c in centers:
        if np.any(np.linalg.norm(samples - c, axis=-1) <= radius):
            covered += 1
    return covered / len(centers)
```

184.5 Experiment

Hypothesis. Samples concentrated at one mode of a two-mode mixture have coverage 1/2 under a moderate radius, while samples drawn from both modes have coverage 1. A discriminator accuracy near 1 can occur in the collapsed case if the discriminator only learned that one region.

Protocol. You do not need a full GAN for the metric check. Draw collapsed samples and good samples. Optionally train a tiny linear discriminator and show it can be confident while coverage is poor, or skip the training and treat the metric as the lesson if time is short. Prefer to train if you can keep it small.

Metric. `mode_coverage` and, if you train, discriminator accuracy.

Decision rule. Coverage must distinguish the two samplers. If the radius is so large that one mode covers both centers, shrink it. The metric is only meaningful when a collapsed sampler does not accidentally cover everything.

184.6 Project

Deliverable. A small generator on a 2D mixture, or a clear negative result, with mode coverage, a baseline that samples the training points themselves (the memorization ceiling), and a note that there is no likelihood column.

Constraints. The coverage radius is chosen from the known mode separation before you look at the generator, or chosen on a validation draw from the true mixture, not to maximize the generator's score after the fact.

Rubric. Memorization is reported as a ceiling, not as a competitor you forgot to beat honestly on new draws.

184.7 Depth tracks

Beginner. Compute coverage for a one-mode and a two-mode cloud.

University. Train or simulate the collapse comparison and write the discriminator caveat.

Research. State one distance between distributions, such as a critic-based estimate, and the way it can be gamed if the critic class is too weak or too strong. Keep the formula short and the caveat explicit.

184.8 Failure modes

- Picking the checkpoint with the prettiest sample.
- Reporting discriminator loss as a quality metric.
- Comparing likelihoods you do not have.

184.9 See also

Volume 04 for mixtures as the data law. Volume 18 for checkpoint selection rules.

185 Diffusion models

186 Diffusion models

Reference question. What is a diffusion model trained to predict, and how is a sample produced?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

186.1 Reference card

Item	Statement
Forward	A fixed noising process that turns data into nearly standard Gaussian noise over many steps.
Reverse	A learned denoise that walks from noise back toward the data.
Training target	Often the added noise, or a clean image, at a randomly chosen noise level. Not a single-step autoencoder unless the schedule says so.
Schedule	The noise levels are part of the model. Training and sampling must use the same schedule.
Likelihood	Some formulations give a bound. A sample-only comparison does not require you to pretend you computed $\log p(x)$.

186.2 Theory

Diffusion models separate a simple forward destruction of structure from a learned reversal. The network usually predicts the noise that was added at a given time, which is a regression target with a known answer. That regression is the training loss. Sampling applies the learned reverse step many times. A one-step reconstruction at a low noise level can look good while the full reverse chain from pure noise fails. Evaluate the procedure you ship.

The schedule determines how hard each regression is. If you train only at tiny noise, you have not trained the steps that start from pure noise. If you sample with a different number of steps than you are willing to pay at deployment, you are evaluating a different sampler.

As with other generators, mode coverage and a task metric beat a gallery. A diffusion model can still memorize. A nearest-neighbor check from generated samples to the training set is a useful embarrassment test on small data.

186.3 Derivation

A common forward step is $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$. The network $\epsilon_\theta(x_t, t)$ is trained by squared error against ϵ. When $\bar{\alpha}_t \approx 1$, $x_t \approx x_0$ and the problem is easy. When $\bar{\alpha}_t \approx 0$, x_t is almost noise and the network must recover structure from the time index and the residual signal. The training distribution over t is therefore a knob. Sampling plugs ϵ_θ into a reverse update whose coefficients come from the same $\bar{\alpha}$. Mixing two papers' coefficient conventions is a silent bug.

186.4 Code

```
import numpy as np

def forward_noise(x0, alpha_bar, rng):
    eps = rng.normal(size=np.shape(x0))
    x_t = np.sqrt(alpha_bar) * x0 + np.sqrt(1.0 - alpha_bar) * eps
    return x_t, eps
```

186.5 Experiment

Hypothesis. At $\bar{\alpha}$ near 1, a predictor that returns 0 has small error if the true noise is small… more sharply: the squared error of predicting the true ϵ is zero by construction in a synthetic test of the target, and the error of predicting 0 is about the dimension.

Protocol. Draw x_0 and ϵ, form x_t at $\bar{\alpha} \in \{0.99, 0.1\}$, and compute MSE of the true ϵ versus a zero predictor.

Metric. Mean squared error per coordinate.

Decision rule. The true ϵ scores 0. The zero predictor scores near 1. If your training code does not approach the true-ϵ score on a batch where you pass ϵ as the prediction, the loss is wired to the wrong tensor.

186.6 Project

Deliverable. A one-dimensional diffusion-style denoiser on a two-mode dataset, trained at several noise levels, with samples from a short reverse chain and a mode-coverage number. A nearest-neighbor distance from samples to training points.

Constraints. The schedule is in the config and is shared by training and sampling.

Rubric. You can show a failure mode of sampling at a noise level you did not train, or of a reverse chain that is too short.

186.7 Depth tracks

Beginner. Check `forward_noise` recovers x_0 when $\bar{\alpha} = 1$.

University. Train the 1D denoiser and report coverage.

Research. State the difference between predicting ϵ, predicting x_0, and predicting a score $\nabla \log p_t$. You only need the algebraic relationship at one noise level, plus the warning that the parameterization changes the loss scale.

186.8 Failure modes

- Evaluating only low-noise reconstructions.
- A sampling schedule copied from a paper that used a different $\bar{\alpha}$ definition.
- A gallery of five samples as the only metric.

186.9 See also

Volume 11 if the data are images. The evaluation chapter next.

187 Evaluating generators

188 Evaluating generators

Reference question. Which generator comparisons are even defined?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

188.1 Reference card

Item	Statement
Likelihood models	Autoregressive models and others with a density or a bound can report held-out likelihood, labeled correctly as a bound when it is one.
Sample models	GANs and many samplers need a sample metric. Do not invent a likelihood.
Coverage	A check that modes or subclasses appear, not only that some sample is sharp.
Memorization	Distance to the nearest training example. Small distance means copying.
Downstream	If the generator feeds another system, evaluate that system with and without the synthetic data.

188.2 Theory

Generative evaluation fails when numbers from different questions are placed in one column. Likelihood, sample fidelity, coverage, and downstream utility can rank models differently. The reference rule is to put them in different columns and to leave a cell blank rather than fill it with a proxy you do not believe.

Human preference is a measurement with a protocol: who judged, what they saw, what the question was. Five friends and a screenshot are not a protocol. A programmatic checker on synthetic data is a protocol. Use it when you can, and do not generalize it to “the model understands images.”

Synthetic data that trains a downstream classifier must be tested against real held-out data. A classifier that is perfect on synthetic test images and useless on real ones learned the generator's artifacts. That result belongs in the note.

188.3 Derivation

Let p be the data law and q the model. $\mathrm{KL}(p\|q)$ is an expectation under p and punishes a q that misses regions where p has mass. It is related to likelihood. A critic that looks at samples from q can miss the same failure if those samples never land in the missing region and the critic is not shown the uncovered data. Coverage metrics are attempts to see the miss. No single divergence is the decision. The decision is the column that matches the use: compression and prediction favor likelihood; a simulator for a downstream task favors that task's risk.

188.4 Code

```
import numpy as np

def nearest_train_distance(samples, train):
    samples = np.asarray(samples, dtype=np.float64)
    train = np.asarray(train, dtype=np.float64)
    d2 = ((samples[:, None, :] - train[None, :, :]) ** 2).sum(axis=-1)
    return np.sqrt(d2.min(axis=1))
```

188.5 Experiment

Hypothesis. Samples copied from the training set have nearest-neighbor distance 0. Samples from a shifted Gaussian have distances on the order of the shift. A coverage metric and a nearest-neighbor metric disagree on a collapsed memorizing sampler versus a slightly noisy full-support sampler.

Protocol. Two-mode data in 2D. Sampler A copies five training points from one mode. Sampler B draws both modes with small noise.

Metric. Mode coverage and median nearest-neighbor distance.

Decision rule. Sampler A looks "close to the data" and fails coverage. Sampler B covers and is not an exact copy. A note that reports only one of the metrics must say which failure it cannot see.

188.6 Project

Deliverable. An evaluation table for at least two generators you built in this volume, with blank cells where a number is undefined, plus the memorization and coverage checks.

Constraints. Do not fill a likelihood cell for a GAN.

Rubric. The recommendation names the use case. A different use case may rank the models differently, and you say so.

188.7 Depth tracks

Beginner. Compute nearest-neighbor distances for copies and for shifted points.

University. Build the two-sampler table.

Research. Write the downstream-utility protocol: train a classifier on real data only, and on real plus synthetic, test on real only, and state the leakage path if synthetic images were made from the test set.

188.8 Failure modes

- One column called “score.”
- Human preference without a question text.
- Synthetic data generated from test examples.

188.9 See also

Volume 07 for not shopping metrics. Volume 18 for reporting negative rankings.

189 Project: a small generator with an evaluation

190 Project: a small generator with an evaluation

Reference question. Can you train one generator and defend the number you publish?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

190.1 Reference card

Item	Statement
Data	A low-dimensional mixture or a tiny image set you have rights to.
Model	Autoregressive, VAE, GAN, or diffusion. One primary choice.
Baseline	The empirical training distribution as a memorizing sampler, and a single Gaussian.
Metrics	The columns from the evaluation chapter that apply.
Failure	One induced failure: collapse, a wrong schedule, or a peeked future token.

190.2 Theory

The project freezes the habit of matching the claim to the metric. A Gaussian baseline that wins on likelihood and loses on a sample-coverage task you care about is a successful experiment. It tells you the task is not likelihood. A generator that only ties the memorizing sampler has not learned a density. It has learned the training set.

Keep the dimension small enough to plot. A 2D mixture with three modes will teach more than a blurry face generator you cannot evaluate. If you move to images, reuse the vision contract and the shift test: a generator trained on centered digits and sampled for a downstream task should be checked off-center.

190.3 Derivation

The single Gaussian MLE baseline has a closed form from Volume 05. Its held-out log likelihood is defined. An autoregressive model on discretized 2D bins also has a likelihood, after you state the binning. A GAN does not. The table's likelihood column is filled for the first two and blank for the GAN. The coverage column is filled for all three. This is the whole methodological point, and the project is incomplete if the blank cell was replaced by a dash that a reader might read as zero.

190.4 Code

```
import numpy as np

def gaussian_mle_nll(train, test):
    mu = train.mean(axis=0)
    var = np.mean((train - mu) ** 2, axis=0) + 1e-6
    test = np.asarray(test, dtype=np.float64)
    return float(0.5 * np.mean(np.sum(np.log(2 * np.pi * var) + (test - mu) ** 2 / var, ax
```

190.5 Experiment

Hypothesis. On three well-separated modes, the Gaussian MLE has worse mode coverage than a sampler that draws the empirical modes, and it may still assign a finite likelihood everywhere the mixture model with a missing mode assigns almost none.

Protocol. Fit the Gaussian baseline. Compare with a categorical draw over the three training means plus small noise. Score coverage and, for the Gaussian, held-out NLL.

Metric. Coverage and NLL where defined.

Decision rule. The table must show a tradeoff or a clear win, not a single champion invented by dropping a column. If the modes are so close that the Gaussian covers them, separate the means.

190.6 Project

Deliverable. One learned generator plus the two baselines, the metric table, the induced failure, and a card that says what the generator is for.

Constraints. Hyperparameters are chosen on validation draws or validation likelihood, not on the final test numbers.

Rubric. A blank cell is present if a metric does not apply. The failure mode is something you ran, not a paragraph you copied from a survey.

190.7 Depth tracks

Beginner. Fit the Gaussian baseline and report NLL and a scatter or a table of means.

University. Complete the generator and the table.

Research. Add a downstream classifier that uses synthetic samples and test it on real held-out draws. State whether synthetic data helped at a matched labeled budget.

190.8 Failure modes

- A generator with no baseline.
- Test-set tuning of the noise schedule.
- Calling a reconstruction a sample.

190.9 See also

Volume 14 if a generator becomes a world model in reinforcement learning. Keep the evaluation columns.

Part XIV

Volume 14. Reinforcement learning

191 Volume 14. Reinforcement learning

192 Volume 14. Reinforcement learning

Stage. University to research

Audience. A reader who can talk about conditional independence, expectations, and off-policy data.

Question this volume answers. What is an agent optimizing, under which state assumption, and how can a logged dataset lie about a new policy?

This volume is a reference, not a short tour. Each chapter opens with a card you can look up later, then gives the theory, a reference implementation, an experiment with a decision rule, and a project with a rubric. Read in order the first time. After that, enter through the reference card of the chapter that matches the problem in front of you.

192.1 How to use the four layers

Layer	What you should be able to do when you finish it
Theory	State the definition, the assumptions, and the result, including what the result does not say.
Code	Implement the core object in a few dozen lines and check it against a known case.
Experiment	Change one factor, record a metric with uncertainty, and say what outcome would change your mind.
Project	Produce an artifact someone else can rerun: code, figure, table, and a one-page lab note.

192.2 Depth tracks

Every chapter ends with three tracks.

- **Beginner.** Do the reference card, the code, and the smallest experiment. Skip optional proofs.
- **University.** Do the derivation, the full experiment, and the project rubric.

- **Research.** Read the failure modes, change an assumption, and write what would have to be true for the method to fail on a new distribution.

192.3 Chapters

No.	Chapter	File
01	Markov decision processes	`vol14/01_mdp.qmd`
02	Dynamic programming and Monte Carlo	`vol14/02_dynamic_programming.qmd`
03	Temporal difference and Q-learning	`vol14/03_td_q.qmd`
04	Policy gradients and actor-critic	`vol14/04_policy_gradients.qmd`
05	Offline data and partial observations	`vol14/05_offline_partial.qmd`
06	Project: a tabular agent with ablations	`vol14/06_agent_project.qmd`

192.4 Lab note

Keep one markdown file per experiment, `notes/YYYY-MM-DD-<slug>.md`, with the hypothesis, the command you ran, the metric, and the decision. The series is a reference only if the experiments you run are attached to it.

193 Markov decision processes

194 Markov decision processes

Reference question. What has to be in the state before the Markov decision process is the process you have?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

194.1 Reference card

Item	Statement
Process	States, actions, a transition kernel, a reward, and a discount $\gamma \in [0, 1)$.
Markov	S_{t+1} and R_{t+1} depend on the past only through S_t and A_t.
Policy	A map $\pi(a \mid s)$, or a deterministic $a = \pi(s)$.
Return	$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+1+k}$.
Value	$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$. $Q^\pi(s, a)$ conditions on the action too.

194.2 Theory

An MDP is a conditional-independence claim plus an objective. If two different histories lead to the same recorded state and different futures, the recorded state is not a state. The algorithm will treat those histories as exchangeable and will be wrong. Add the missing memory or stop calling the problem Markov.

The discount γ makes the return finite and says how much you care about later rewards. It is part of the objective, not a numerical trick only. A policy that is optimal at $\gamma = 0.9$ need not be optimal at $\gamma = 0.99$. Report γ.

Reward is the objective you will get. If the reward is a proxy, the value function is the value of the proxy. Volume 16's metric warnings apply here with more force, because the agent will search for a high proxy. Specify the reward as a contract.

194.3 Derivation

Condition on $S_t = s$ and $A_t = a$, then follow π. The Bellman equation for the action value is

$$Q^\pi(s,a) = \mathbb{E}[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s, A_t = a],$$

and $V^\pi(s) = \sum_a \pi(a \mid s) Q^\pi(s,a)$. The derivation uses the Markov property when the future after S_{t+1} depends on the past only through S_{t+1}, and it uses the law of total expectation. If the state is missing a bit that the reward depends on, the conditional expectation given the recorded s is an average over that bit, and the equation is about a coarser process than the one you think you control.

194.4 Code

```
import numpy as np

def discounted_return(rewards, gamma):
    total = 0.0
    scale = 1.0
    for r in rewards:
        total += scale * float(r)
        scale *= gamma
    return total
```

194.5 Experiment

Hypothesis. A two-step reward sequence $(0, 1)$ has discounted return γ when the first reward is 0, and a state alias that merges two positions with different next rewards makes a value estimate that is the average, not either truth.

Protocol. Compute `discounted_return([0, 1], 0.9)`. Then tabulate a two-state MDP you can solve by hand: from state A, action “go” yields reward 1; from state B, reward 0; an aliased observation O merges A and B with equal probability.

Metric. The return number, and the conditional expected next reward given O versus given the true state.

Decision rule. The aliased expectation should sit between the two true rewards. A controller that conditions only on O cannot choose differently in A and B. That is the Markov failure, in miniature.

194.6 Project

Deliverable. A small MDP written as tables: states, actions, transitions, rewards, γ. A sentence that says what was left out of the state on purpose, and what decision that omission forbids.

Constraints. The tables are small enough to solve by enumeration in the next chapter.

Rubric. A reader can compute one return and one aliased expectation from the note.

194.7 Depth tracks

Beginner. Compute discounted returns at $\gamma \in \{0, 0.9, 1\}$ for a finite reward list, and see the $\gamma = 1$ sum grow with the horizon.

University. Write the aliased-state table.

Research. Give an example of a belief state that restores the Markov property when the observation is partial. You will need it again in the partial-observability chapter.

194.8 Failure modes

- Treating a frame of video as a state when velocity is missing and the reward depends on motion.
- Changing γ between training and the reported evaluation.
- A reward that includes a term unavailable at decision time.

194.9 See also

Volume 04 for conditional independence. Volume 16 for proxy rewards.

195 Dynamic programming and Monte Carlo

196 Dynamic programming and Monte Carlo

Reference question. How do you compute values when the model is known, and what changes when you only have episodes?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

196.1 Reference card

Item	Statement
Policy evaluation	Solve or iterate the Bellman equation for a fixed π.
Improvement	A greedy policy with respect to Q is at least as good, under the MDP assumptions.
Monte Carlo	Average returns from complete episodes. Unbiased for the return under the policy that generated the episodes, given enough coverage.
Model	Dynamic programming needs the transition and reward tables. Monte Carlo needs samples.
Coverage	A state you never visit has no Monte Carlo estimate, whatever the equation says.

196.2 Theory

Dynamic programming is the exact arithmetic of a known finite MDP. It is the unit test for later algorithms: on a table you can solve, a new method should agree up to optimization error. If it does not, the bug is not "instability of deep RL."

Monte Carlo replaces expectations by episode averages. The return is a high-variance estimate of the value. It does not bootstrap, so an error in one state's value does not immediately poison another, and you must wait until the episode ends. A state that is missing from the data is not estimated. A policy that never takes a risky action will not see the reward behind it. That is an exploration problem, and it is already present in a four-state grid.

Policy improvement is a theorem about the true Q^π. Improvement with respect to a noisy $\hat{Q}$ can make the policy worse. The theorem is the target, not a description of every update you code.

196.3 Derivation

For a finite state-action space, policy evaluation iterates $Q_{k+1} = T^\pi Q_k$, where $(T^\pi Q)(s,a) = R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \sum_{a'} \pi(a' \mid s') Q(s',a')$. The map is a contraction in the max norm with modulus γ, so the iteration converges to Q^π from any finite start. Monte Carlo instead averages G_t for visits to (s,a). Under the policy that generated the data, and with every pair visited infinitely often, the averages converge to Q^π by the law of large numbers. Finite data leave variance and holes.

196.4 Code

```
import numpy as np

def value_iteration_step(Q, reward, transitions, gamma):
    # transitions[s, a, s_next] = probability. reward[s, a] expected immediate reward.
    n_s, n_a, _ = transitions.shape
    q_next = np.max(Q, axis=1)
    updated = reward + gamma * np.einsum("saj,j->sa", transitions, q_next)
    return updated
```

196.5 Experiment

Hypothesis. Value iteration on a two-state MDP you solve by hand reaches the hand solution. A Monte Carlo estimate from a policy that never visits state 1 leaves state 1 undefined or at its initialization.

Protocol. Build a deterministic MDP, run the iteration to a tolerance of 10^{-8}, and compare. Then generate episodes that avoid one state.

Metric. Max absolute error against the hand solution, and the visit count per state.

Decision rule. The dynamic-programming error should be near the tolerance. The Monte Carlo table must show the hole. Filling the hole with zero and calling it a value is a bug in the report.

196.6 Project

Deliverable. A solved grid or chain, the iteration code, a Monte Carlo comparison with visit counts, and a note on one state the behavior policy did not see.

Constraints. The MDP tables are the ground truth for this project. Do not "improve" them to match a broken algorithm.

Rubric. The hand solution and the iteration agree, and the missing-state issue is visible.

196.7 Depth tracks

Beginner. Solve a one-state, one-action discounted reward by summing the geometric series and by iteration.

University. Complete the grid or chain and the Monte Carlo hole.

Research. State the contraction argument in two lines and the step that fails if $\gamma = 1$ and the horizon is infinite without a termination condition.

196.8 Failure modes

- A Monte Carlo value of 0 for an unvisited high-reward state.
- Comparing a partial-episode return with a discounted infinite-horizon value without a terminal rule.
- Using the optimal policy's samples to claim you evaluated a worse policy.

196.9 See also

The next chapter for bootstrapping. Volume 05 for the variance of an average.

197 Temporal difference and Q-learning

198 Temporal difference and Q-learning

Reference question. What does a temporal-difference update bootstrap, and when is Q-learning's target the wrong policy?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

198.1 Reference card

Item	Statement
TD(0)	Update $V(s)$ toward $r + \gamma V(s')$, a bootstrap target.
Bias	The target is biased if V is wrong, and lower variance than a full return because it cuts the future short.
Q-learning	Target $r + \gamma \max_{a'} Q(s', a')$. This learns about the greedy policy while following a different behavior policy, under coverage conditions.
SARSA	Target $r + \gamma Q(s', a')$ with a' actually taken. It evaluates the behavior policy, including its exploration.
Deadly triad	Function approximation, bootstrapping, and off-policy updates can diverge. A table is the safe unit test.

198.2 Theory

Temporal-difference learning replaces the rest of the return by the current value estimate. That bootstrapping is why it can learn before an episode ends, and why a bad estimate propagates. In a table with a proper step size and enough visits, TD(0) converges to V^π for the policy being followed. The conditions matter. A neural network is a different theorem, and sometimes there is no theorem.

Q-learning's max operator targets the greedy action. If the behavior policy does not explore, the max is a max over fiction. If it explores but you deploy the greedy policy, SARSA's

value, which includes exploratory actions, is not the value of the deployed policy. Say which policy's value you learned.

With function approximation, off-policy TD can diverge even on simple linear approximators. The practical warning is to unit-test new code on a table where dynamic programming is available, then add function approximation on a problem you can still visualize.

198.3 Derivation

The TD error $\delta = r + \gamma V(s') - V(s)$ has expectation zero when $V = V^\pi$ and s' is drawn from the policy's transition, because of the Bellman equation. The update $V(s) \leftarrow V(s) + \alpha\delta$ is stochastic approximation of that fixed point. Q-learning replaces $V(s')$ by $\max_{a'} Q(s', a')$, which is the Bellman optimality backup, not the backup of the behavior policy. The behavior policy must still visit the pairs that the optimal policy uses, or the fixed point of the sampled updates is about a restricted state-action set.

198.4 Code

```
import numpy as np

def q_learning_update(Q, s, a, r, s2, alpha, gamma):
    target = r + gamma * np.max(Q[s2])
    Q = Q.copy()
    Q[s, a] = (1 - alpha) * Q[s, a] + alpha * target
    return Q
```

198.5 Experiment

Hypothesis. On the tabular MDP you solved by dynamic programming, Q-learning with an exploring behavior policy approaches the optimal action values. With a behavior policy that never takes the optimal action in a key state, the corresponding entry does not move from its initialization.

Protocol. ϵ-greedy behavior with $\epsilon = 0.2$, and a second run with $\epsilon = 0$ and a bad start. Compare with `value_iteration_step` run to convergence for the optimal values, or with the hand solution.

Metric. Max absolute error on visited pairs, and the visit count.

Decision rule. The exploring run should approach the solution on visited pairs. The non-exploring run must show a hole. If both fail, the reward or transition table disagrees with the DP solver; do not tune α to hide that.

198.6 Project

Deliverable. Tabular Q-learning and SARSA on one MDP, compared with dynamic programming. A plot or table of the greedy policy's return, not only of the TD loss.

Constraints. The evaluation episodes use the greedy policy without exploration, and they are separate from the training episodes.

Rubric. You can say whether SARSA and Q-learning should differ at high ϵ, and your table agrees.

198.7 Depth tracks

Beginner. Perform one Q-learning update by hand on a two-state table.

University. Match dynamic programming under exploration, and show the no-exploration hole.

Research. Construct or cite a small divergent off-policy linear TD example and say which leg of the deadly triad you removed to restore stability in your tabular code.

198.8 Failure modes

- Evaluating with ϵ-greedy and calling it the greedy policy's return.
- A target network or a max operator coded with the wrong index so you bootstrap from the current action only.
- Function approximation added before the tabular test passes.

198.9 See also

Volume 02 for stochastic approximation. Volume 10 for the optimizer you will be tempted to drop in too early.

199 Policy gradients and actor-critic

200 Policy gradients and actor-critic

Reference question. What does a policy-gradient estimator actually estimate, and how does a baseline change it?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

200.1 Reference card

Item	Statement
Policy gradient	The gradient of $J(\pi_\theta)$ can be written as an expectation of $\nabla \log \pi_\theta(a \mid s)$ times a return factor.
Score	$\nabla \log \pi$ does not depend on the unknown transition model. That is the point of the likelihood-ratio trick.
Baseline	Subtracting a state-dependent baseline that does not depend on a leaves the expectation unchanged and can cut variance.
Actor-critic	A learned V or Q replaces the Monte Carlo return. It adds bias when the critic is wrong and cuts variance.
Credit	A noisy advantage assigns credit to actions that did not cause the return. Variance is the failure mode, not a rounding error.

200.2 Theory

Policy gradient methods optimize a parameterized policy directly. They can represent stochastic policies, which matter when the optimal action is mixed or when the MDP is partially observed and a deterministic function of a non-Markov observation is not enough. The estimator is a score-function estimator. It is unbiased for the gradient of the expected return when the return is the true return and the expectation is over the policy you are differentiating. A replay buffer of old policies is a different distribution. You then need an off-policy correction or you are estimating a different gradient.

A baseline that depends only on the state does not change the expected gradient and often reduces variance. An actor-critic baseline that depends on a learned value is only a baseline in that unbiased sense when it is treated as constant with respect to the action in the score term, and the critic's own error is a second source of bias if you bootstrap.

Entropy bonuses and clipping, as in common practical algorithms, change the objective. They are not the theorem you just proved. Put them in the config as objective changes.

200.3 Derivation

For a one-step bandit, $J(\theta) = \mathbb{E}_{a\sim\pi_\theta}[R(a)]$. Then $\nabla J = \mathbb{E}[\nabla \log \pi_\theta(a) R(a)]$, because $\nabla \pi = \pi \nabla \log \pi$ and the score of a normalized policy has mean zero. Subtracting b inside gives $\mathbb{E}[\nabla \log \pi(a)(R(a) - b)]$, and $\mathbb{E}[\nabla \log \pi(a) b = 0$. The variance of the estimator depends on b. The choice $b \approx \mathbb{E}[R]$ is the simple baseline. In an MDP the same cancellation holds for a baseline that depends on the state at the decision, not on the sampled action.

200.4 Code

```
import numpy as np

def bernoulli_score(action, prob):
    p = np.clip(prob, 1e-6, 1 - 1e-6)
    # action in {0, 1}, derivative of log probability w.r.t. the logit is action - p
    # if prob = sigmoid(logit). Return the logit score.
    return float(action - p)
```

200.5 Experiment

Hypothesis. On a one-step bandit with a higher mean for action 1, a score-function update with a baseline moves the logit toward action 1 on average, and the baseline reduces the variance of the update relative to the raw return.

Protocol. Bernoulli rewards, means 0.2 and 0.8, analytic $\nabla \log \pi$ from a sigmoid policy. Compare update variance with $b = 0$ and $b = \mathbb{E}[R]$ estimated from a large sample, over many draws of the action and reward.

Metric. Mean update direction and variance of the scalar update.

Decision rule. The mean direction should agree with and without the baseline, up to Monte Carlo error. The variance should drop when the baseline is the mean reward. If the means disagree, the baseline depended on the action or the score is wrong.

200.6 Project

Deliverable. The bandit experiment, plus a short-horizon tabular policy gradient on the MDP from earlier chapters, compared with Q-learning's greedy policy. Report evaluation return of the learned policy, not the training surrogate.

Constraints. The behavior during evaluation is the learned policy without an extra exploration bonus you forgot to turn off.

Rubric. You can point to the baseline's variance reduction in the bandit and to one bias you accepted if you used a critic.

200.7 Depth tracks

Beginner. Compute the score of a nearly deterministic wrong action and see that it can be large.

University. Complete the bandit variance comparison and the short MDP run.

Research. Write the importance weight $\pi_\theta(a \mid s)/\pi_{\text{old}}(a \mid s)$ and the failure mode when the old policy never took the action the new policy wants. That is the off-policy policy-gradient hole.

200.8 Failure modes

- Optimizing a surrogate and never measuring episodic return.
- A baseline that uses the sampled action and breaks the unbiasedness argument.
- Reporting the best training episode as the policy's value.

200.9 See also

Volume 05 for variance of estimators. Volume 10 for the optimizer wrapped around this noisy gradient.

201 Offline data and partial observations

202 Offline data and partial observations

Reference question. When does a logged dataset fail to answer what a new policy will do, and when is the observation not a state?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

202.1 Reference card

Item	Statement
Behavior	The policy that produced the log. A new policy's actions may be rare or absent in the log.
Support	You cannot estimate the value of an action that was never taken, without a model that extrapolates.
Confounding	If the logger chose actions based on a variable you did not record, the conditional outcome given recorded state and action is not the causal effect of the action.
Partial observation	A policy of the observation is not a policy of the state. Values can be intransitive and the Markov property can fail.
Offline claim	The honest claim is about actions the logger took, unless you state an extrapolation model.

202.2 Theory

Offline reinforcement learning wants a new policy from data collected by an old one. The statistical problem is support. The causal problem is confounding. Both are easy to miss if you only look at a training loss that imitates the logger. Imitation learns the logger. It does not learn the return of a different policy. Q-learning on the log, without coverage, fills unseen actions with optimistic or stale values and then chooses them.

Partial observability means the agent does not see S_t. A recurrent state, a belief, or a window of observations is an attempt to rebuild a state. A feedforward map from one observation can be insufficient even when a map from the true state would be trivial. The aliased MDP in the first chapter is the unit test: if your algorithm claims a high value from the observation alone, it is averaging two worlds.

The project-level rule is to label the claim. "Value of the logger's actions on recorded states" is estimable by a regression. "Value of a new policy" needs overlap or a model. "Optimal policy of the hidden state" needs the hidden state or a belief that is good enough. Do not use one phrase for all three.

202.3 Derivation

In a contextual bandit, the mean reward of a new policy π is $\mathbb{E}[\pi(A \mid X)/\pi_b(A \mid X) \cdot R]$ when $\pi_b(a \mid x) = 0$ implies $\pi(a \mid x) = 0$, the expectations exist, and the logged action was drawn from π_b independent of the potential rewards given X. If π_b can be zero where π is not, the weight is undefined and the expectation identity does not give you an estimator. If a confounder U influenced A and R and you only condition on a non-sufficient X, the identity with weights built from X estimates the wrong functional. Overlap and unconfoundedness are assumptions, not properties of the neural net.

202.4 Code

```
import numpy as np

def importance_weight(pi_new, pi_behavior, floor=1e-3):
    pi_behavior = np.asarray(pi_behavior, dtype=np.float64)
    if np.any(pi_behavior <= 0):
        raise ValueError("behavior probability is zero; the new policy is off support")
    w = np.asarray(pi_new, dtype=np.float64) / np.maximum(pi_behavior, floor)
    return w
```

202.5 Experiment

Hypothesis. On a contextual bandit where the logger never chooses action 1 in context 0, an importance-weighted estimate of a policy that does choose it should raise, not return a confident finite number. On a fully supported logger, the weighted reward should approach the true value of the new policy as n grows.

Protocol. Two contexts, two actions, known reward means. Logger with full support versus logger with a hole. New policy that steps into the hole.

Metric. Whether the code raises, and the error of the weighted estimator in the full-support case over 100 trials.

Decision rule. The hole must be an error, not a clipped weight that pretends the probability was `floor` without a huge variance warning. If you use a floor, the note must say you changed the target.

202.6 Project

Deliverable. The bandit simulation, the aliased observation from the MDP chapter evaluated with Q-learning on observations, and a paragraph that labels your claim as regression, off-policy evaluation, or belief-state control.

Constraints. A zero behavior probability is not clipped silently.

Rubric. You can state one deployment policy you refuse to evaluate from the log you have.

202.7 Depth tracks

Beginner. Compute an importance weight for probabilities 0.8 and 0.4, and see the code reject 0.

University. Run both logger regimes and the aliased Q-learning comparison.

Research. Describe a belief-state update for a two-state hidden Markov model with a discrete observation, and why the belief is Markov even though the observation is not.

202.8 Failure modes

- Behavioral cloning reported as an improved decision policy without a return comparison.
- Clipped importance weights with no variance diagnostic.
- A partially observed problem trained and evaluated as if the observation were the state, with no aliasing check.

202.9 See also

Volume 05 for inverse propensity as a statistical instrument. Volume 16 for policies that optimize a proxy.

203 Project: a tabular agent with ablations

204 Project: a tabular agent with ablations

Reference question. Can you show, on a problem you can solve exactly, which RL ingredient changed the return?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

204.1 Reference card

Item	Statement
Environment	A tabular MDP with a dynamic-programming optimum.
Agents	At least two of: Monte Carlo, Q-learning, SARSA, policy gradient.
Ablations	Exploration rate, discount, and behavior-policy support.
Metric	Greedy or learned-policy return on fresh evaluation episodes, versus the optimal value.
Failure	One run with a state alias or a no-exploration behavior policy.

204.2 Theory

This project is the debug checklist of Volume 10 applied to agents. The exact solution is the known case. An agent that cannot approach it on a four-state MDP will not become correct because the state is replaced by a neural embedding. The ablation is the experiment. Exploration, discount, and support are factors with predicted directions. The evaluation protocol is frozen: same start distribution, same horizon, no exploration noise in the reported return unless the deployed policy includes it.

Write the reward contract at the top. If you add a shaping term, it is a different reward. Show the unshaped return too, or you will ship a policy that farms the shaping.

204.3 Derivation

Let $V^\star(s_0)$ be the optimal value at the start. An evaluation estimate $\hat{V}$ from n greedy episodes has a standard error you can compute from the episode returns. A claim that an agent "solved" the MDP means $\hat{V}$ is close to $V^\star$ relative to that standard error and to a practical margin. A claim that exploration mattered means the $\epsilon = 0$ ablation is worse by more than that noise. The dynamic-programming number is not an evaluation episode. It is the target those episodes estimate.

204.4 Code

```
import numpy as np

def evaluate_episodes(returns):
    r = np.asarray(returns, dtype=np.float64)
    se = float(np.std(r, ddof=1) / np.sqrt(r.size)) if r.size > 1 else float("nan")
    return {"mean": float(np.mean(r)), "se": se, "n": int(r.size)}
```

204.5 Experiment

Hypothesis. Q-learning with $\epsilon = 0.2$ approaches $V^\star$ from the start state, and $\epsilon = 0$ from a policy that misses the rewarding action does not. Changing γ from the value used in dynamic programming changes the target, so agreement with the old $V^\star$ should fail.

Protocol. The same MDP as the earlier chapters. Evaluation episodes are greedy. Report `evaluate_episodes`.

Metric. Mean return versus $V^\star$, and the $\epsilon = 0$ gap.

Decision rule. The exploring agent matches within a few standard errors or you debug before adding a second algorithm. The γ mismatch must be visible. If it is not, the horizon is too short for discounting to matter; lengthen it.

204.6 Project

Deliverable. The solved MDP, two agents, the exploration and discount ablations, the aliasing failure, and a card that states the reward, γ, and the evaluation policy.

Constraints. No neural network until the tabular match succeeds. If you add one afterward, it is a separate, labeled experiment on the same MDP with a one-hot state encoding, and it must still match $V^\star$.

Rubric. The optimal value is in the table. A learning curve without that line is incomplete.

204.7 Depth tracks

Beginner. Evaluate a random policy and the optimal policy with `evaluate_episodes`.

University. Complete the ablations and the alias failure.

Research. Add an offline log from a narrow behavior policy and show a new policy you refuse to score. Include the importance-weight error from the previous chapter.

204.8 Failure modes

- Reporting the best episode.
- Evaluating with exploration left on.
- A shaped reward in training and an unshaped claim in the abstract of the note.

204.9 See also

Volume 15 for deploying a policy as a system. Volume 17 if the policy is a prompted model rather than a tabular agent.

Part XV

Volume 15. Systems and production

205 Volume 15. Systems and production

206 Volume 15. Systems and production

Stage. University to research

Audience. A reader who can train a model and can state a data contract. This volume is an original systems guide. The Harvard CS249r volumes are the external deep reference; read them under their license, unmodified.

Question this volume answers. What has to be true of the data, the job, the serving path, and the monitor before a model is a system?

This volume is a reference, not a short tour. Each chapter opens with a card you can look up later, then gives the theory, a reference implementation, an experiment with a decision rule, and a project with a rubric. Read in order the first time. After that, enter through the reference card of the chapter that matches the problem in front of you.

206.1 How to use the four layers

Layer	What you should be able to do when you finish it
Theory	State the definition, the assumptions, and the result, including what the result does not say.
Code	Implement the core object in a few dozen lines and check it against a known case.
Experiment	Change one factor, record a metric with uncertainty, and say what outcome would change your mind.
Project	Produce an artifact someone else can rerun: code, figure, table, and a one-page lab note.

206.2 Depth tracks

Every chapter ends with three tracks.

- **Beginner.** Do the reference card, the code, and the smallest experiment. Skip optional proofs.
- **University.** Do the derivation, the full experiment, and the project rubric.

- **Research.** Read the failure modes, change an assumption, and write what would have to be true for the method to fail on a new distribution.

206.3 Chapters

No.	Chapter	File
01	The lifecycle as a system	`vol15/01_lifecycle.qmd`
02	Compute, memory, and batching	`vol15/02_compute_memory.qmd`
03	Data pipelines	`vol15/03_data_pipelines.qmd`
04	Serving and monitoring	`vol15/04_serving_monitoring.qmd`
05	Reproducibility in production	`vol15/05_reproducibility.qmd`
06	Project: a batch-scoring service	`vol15/06_system_project.qmd`

206.4 Lab note

Keep one markdown file per experiment, `notes/YYYY-MM-DD-<slug>.md`, with the hypothesis, the command you ran, the metric, and the decision. The series is a reference only if the experiments you run are attached to it.

207 The lifecycle as a system

208 The lifecycle as a system

Reference question. Which artifacts exist between a notebook and a system another person can operate?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

208.1 Reference card

Item	Statement
Artifacts	Dataset snapshot, config hash, code version, metrics, model weights, card, and a rollback target.
Boundary	Training code and serving code must implement the same feature function, or they are two models.
Owner	A named person or rotation for the metric, the data, and the rollback.
Loop	Train, evaluate on a locked slice, deploy, monitor, and decide to keep or revert.
External	CS249r Volumes I–IV cover single-node systems through physical AI in depth. This chapter does not replace them.

208.2 Theory

A model in a notebook is a function on a tensor that happened to be in memory. A system is that function plus the contract that produces the tensor, the job that refreshes the weights, the service that applies them, and the monitor that says the world has changed. Most production incidents are contract mismatches: a feature computed one way in training and another way in serving, a label that arrives late, or a model that cannot be named precisely enough to roll back.

This volume stays at the level of artifacts and checks you can run in a small lab. It is the bridge from the earlier experimental folders to an operable loop. When you need hardware

rooflines, distributed training, or serving stacks in full detail, read the CS249r volumes as themselves. Do not paste them into this book. Their textbook text is CC BY-NC-ND.

The config hash from Volume 03 is the identity of a training run. The serving system must record which hash is live. A dashboard that says "the model" without a hash is not a reference.

208.3 Derivation

Let f_{train} be the feature function in the training job and f_{serve} the feature function in the service. The deployed predictor is $g \circ f_{\text{serve}}$. The evaluated predictor was $g \circ f_{\text{train}}$. These are equal as functions only when $f_{\text{serve}} = f_{\text{train}}$ on the serving input domain. A unit test that runs both functions on a fixed batch and compares outputs to a tolerance is the empirical form of that equality. A mismatch larger than the tolerance blocks deployment. It is not a warning.

208.4 Code

```
import hashlib, json

def artifact_id(payload: dict) -> str:
    raw = json.dumps(payload, sort_keys=True, separators=(",", ":")).encode()
    return hashlib.sha256(raw).hexdigest()[:16]
```

208.5 Experiment

Hypothesis. Two feature functions that differ by a unit conversion produce different predictions from the same weights, and `artifact_id` changes when the conversion flag flips.

Protocol. A linear model $y = 2x$ trained on meters. Serving passes centimeters unless a flag converts them. Compare predictions on a fixed batch. Hash both configs.

Metric. Max absolute prediction gap, and whether the hashes differ.

Decision rule. The gap should be large and the hashes different. If the hashes match, the flag was not in the payload. A serving test that compares predictions is the deployment gate.

208.6 Project

Deliverable. A one-page system card for a model you have already trained: feature function, label time, artifact id, owner, metric, and rollback id. Include the unit-mismatch test.

Constraints. The card names one thing this volume does not measure, such as GPU throughput, and points to CS249r for that measurement rather than inventing a number.

Rubric. A teammate can tell which bytes are live and which bytes to restore.

208.7 Depth tracks

Beginner. Hash two configs that differ by one key.

University. Complete the unit-mismatch gate and the system card.

Research. Write the timeline of a label that arrives a day later. Say which training rows would have been illegal to use at decision time, in the sense of Volume 03's leakage definition.

208.8 Failure modes

- A notebook path and a serving path with no shared test.
- "Latest model" as an identifier.
- Copying CS249r chapters into this folder.

208.9 See also

Volume 03 for experiment folders. The upstream chapter of this series for how to read CS249r beside these notes.

209 Compute, memory, and batching

210 Compute, memory, and batching

Reference question. Which resource is binding: arithmetic, memory movement, or latency?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

210.1 Reference card

Item	Statement
Arithmetic	FLOPs or integer ops. A matmul's count is a function of the shapes.
Memory	Bytes moved. A model can be slow because the weights do not fit in cache or RAM, not because the FLOP count is high.
Batch	Larger batches improve arithmetic intensity and worsen the latency of the last example in the batch.
Latency	The time until one decision. Throughput is decisions per second. They trade off.
Measure	A timer around the actual call, with a warmup, beats a FLOP formula you did not check.

210.2 Theory

Systems problems in this series start with a budget. A reference model that misses the budget is not "more accurate" in the sense the product means. The earlier volumes produced a function. This chapter asks what it costs to evaluate it.

Arithmetic intensity is operations per byte moved. Small batches and memory-heavy layers are often bandwidth bound. A formula that counts only FLOPs will rank a convolution and an attention block incorrectly if one of them is waiting on memory. Attention's quadratic score matrix, from Volume 12, is a memory object as well as a FLOP object. At long T it may not fit.

Batching raises throughput and delays the first response until the batch is full or a timeout fires. Interactive systems need a timeout. Offline scoring can wait. The serving card says which one you are. A benchmark that uses batch 128 does not describe a batch-1 robot or a batch-1 reviewer tool.

210.3 Derivation

A dense layer $y = xW$ with $x \in \mathbb{R}^{b \times d}$ and $W \in \mathbb{R}^{d \times m}$ performs about $2bdm$ FLOPs. It reads $bd + dm$ numbers and writes bm numbers. Intensity is roughly $2bdm/(bd + dm + bm)$ operations per element moved, up to the constant for the dtype size. When b is small and d, m are large, the dm term can dominate: you reload the weights for a few outputs. Increasing b amortizes the weight read. The latency of an example that arrived just after a batch closed includes the wait for the next batch. That wait is not in the FLOP count.

210.4 Code

```
def dense_flops(batch, d_in, d_out):
    return 2 * batch * d_in * d_out

def attention_score_elements(batch, heads, seq, dim):
    # Elements in the score tensor Q K^T, not a full systems model.
    return batch * heads * seq * seq
```

210.5 Experiment

Hypothesis. Predicted dense FLOPs scale linearly in batch size, while a measured NumPy matmul on a small layer may not, because of overhead. Attention score elements grow quadratically in sequence length.

Protocol. Time `X @ W` for batches 1, 16, and 128 on a layer of shape $(256, 256)$, with warmup. Compute `attention_score_elements` at sequence lengths 128 and 1024.

Metric. Measured time per example and the quadratic ratio $(1024/128)^2 = 64$ for the score elements.

Decision rule. The FLOP prediction must scale exactly in the formula. The timer may not. The note should say “overhead” rather than “the formula is false” when the constant term dominates. The quadratic ratio must be exact in the element count.

210.6 Project

Deliverable. A cost card for one model you trained: parameter count, a FLOP or element estimate for one forward pass, a measured latency at the batch you will deploy, and a sentence on whether you are latency bound or throughput bound.

Constraints. The batch size on the card is the deployment batch, not the largest batch that looked good.

Rubric. A reader can see the quadratic attention term if the model has one, or a statement that it does not.

210.7 Depth tracks

Beginner. Compute dense FLOPs at two batch sizes.

University. Time the matmul and write the overhead caveat.

Research. Explain, without a vendor number you cannot measure, the difference between a weight that lives in RAM and a weight that must be read from disk for every request. Name the artifact that would make the second case visible: time that scales with cold start.

210.8 Failure modes

- A throughput benchmark used as a latency promise.
- Ignoring dtype: float64 and int8 are not the same bytes.
- An attention model designed at $T = 128$ and deployed at $T = 4096$ without a new measurement.

210.9 See also

CS249r Volume I for hardware-level intensity and Volume II for scale. Volume 12 for the attention definition.

211 Data pipelines

212 Data pipelines

Reference question. What does a pipeline guarantee about time, schema, and backfill?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

212.1 Reference card

Item	Statement
Snapshot	Training reads a named snapshot, not whatever happens to be in the table now.
Schema	A column addition, a unit change, or a null policy is a contract change with a new hash.
Backfill	Recomputing a feature with today's code over yesterday's keys. It can create a feature the past decision did not have.
Late labels	A label that arrives after the decision may be used for a later training snapshot and must not be used as a feature.
Test	A golden batch with expected features, checked in the pipeline, not only in a notebook.

212.2 Theory

Pipelines fail by being helpful. A backfill that uses a better definition of a feature improves the training matrix and silently diverges from what serving computed last year. A snapshot freezes the bytes you trained on. A new definition is a new feature and a new model, even if the column name is unchanged.

Late labels are the supervised version of leakage. If the decision at time t could not have seen the label, the label is a target, not an input. Aggregates that include events after t are also inputs the decision did not have. Point-in-time joins are the implementation. A random row from a feature store is not a point-in-time join.

The golden batch is the known case for data. It is small, hand-checked, and run whenever the pipeline code changes. A distributional monitor, later in this volume, catches drift. It does not catch a wrong join that has been wrong since the first day, if the wrongness is stable.

212.3 Derivation

Let a feature at decision time t be $\phi_t = \Phi(\text{events}_{\le t})$. A backfill that computes $\Phi'(\text{events}_{\le T})$ for $T > t$ using a new function Φ' is a different random variable. Training on Φ' and serving Φ is the unit-mismatch bug with a schema. The equality test is the same: compute both on the golden keys and compare. A null policy is part of Φ. Filling nulls with zero in training and dropping nulls in serving changes the population as well as the number.

212.4 Code

```
def point_in_time(events, decision_time):
    return [e for e in events if e["time"] <= decision_time]
```

212.5 Experiment

Hypothesis. A feature equal to the count of events up to t differs from a count that includes a late event, and a golden batch catches the difference.

Protocol. Three events, one after the decision. Compare `point_in_time` counts with a full count.

Metric. The two counts.

Decision rule. They must differ. A pipeline test that uses the full count as the expected value of the decision-time feature encodes the bug as the specification. The expected value must be the decision-time count.

212.6 Project

Deliverable. A tiny pipeline: raw events, a point-in-time feature, a snapshot id, and a golden test. Include one backfill that you refuse to mix into the old snapshot.

Constraints. The snapshot id changes when the feature code changes.

Rubric. The golden test fails if a future event leaks into the feature.

212.7 Depth tracks

Beginner. Run the three-event example by hand.

University. Add the golden test and the snapshot id.

Research. Describe a training-serving skew that comes from a library default, such as a categorical encoder's handling of unseen levels, and the golden row that would catch it.

212.8 Failure modes

- Training on a live view that moves during the job.
- A backfill that overwrites the only copy of the old feature.
- Nulls filled differently in two languages.

212.9 See also

Volume 03 for contracts. Volume 07 for leakage. CS249r for production data systems at scale.

213 Serving and monitoring

214 Serving and monitoring

Reference question. What should a service check before it returns a prediction, and what should a monitor check after?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

214.1 Reference card

Item	Statement
Schema gate	Reject or default when the input fails the contract. Do not silently score a NaN.
Version	Every response can be tied to a model hash.
Timeout	A latency budget with a defined fallback.
Monitor	Input distribution, output distribution, and, when labels arrive, a delayed outcome metric.
Alert	A threshold on a predeclared statistic, with a human action. An alert with no action is a log.

214.2 Theory

Serving is the feature function plus the gates. A gate that fails open and returns a prediction for a malformed input will look available and be wrong. A gate that fails closed must have a fallback the product understands: a previous model, a constant decision, or an error. The fallback is a second policy and needs its own metric.

Monitoring without labels can still see input shift and output shift. It cannot see that the mapping from inputs to outcomes broke while both margins look normal. When labels arrive late, schedule a delayed job that joins predictions to outcomes on the decision id. That job is the production form of the test set. It is biased if unlabeled cases are dropped without a reason.

Dashboards follow Volume 03's reporting rules. A panel without a denominator, a baseline, or a window is not an alert source. The alert threshold is chosen from a quiet period or from a loss budget, not from the first bad day you want to dramatize.

214.3 Derivation

Let p_t be the fraction of requests whose features fail the schema in hour t, and let $\hat{y}_t$ be the mean prediction among accepted requests. A change in p_t means the input contract is breaking. A change in $\hat{y}_t$ with stable inputs means the model or the feature code changed, or a real shift in the accepted population occurred. These are different incidents. An alert on prediction mean alone conflates them. Log p_t beside $\hat{y}_t$. When outcomes y arrive, the delayed loss is an average over the joinable subset. If joinability depends on y, the delayed loss is biased. Check the join rate.

214.4 Code

```
def gate(features, required):
    missing = [k for k in required if k not in features or features[k] is None]
    if missing:
        return {"ok": False, "fallback": "constant", "missing": missing}
    return {"ok": True, "missing": []}
```

214.5 Experiment

Hypothesis. A request missing a required key takes the fallback and is counted in the failure fraction, and it does not produce a model score. A complete request does.

Protocol. Two requests and a constant fallback value you record.

Metric. Gate decision, whether a score was emitted, and the hourly failure fraction in a toy log of 100 requests with 5 missing keys.

Decision rule. The failure fraction should be 0.05. If malformed requests emit scores, the gate is advisory and does not meet this chapter's contract.

214.6 Project

Deliverable. A serving function with a gate, a version hash in the response, a fallback, and a monitor table: failure fraction, mean output, and a delayed loss on a synthetic label that arrives later for only some rows. State the join rate.

Constraints. The fallback is a constant or a previous hash, not an unlogged exception.

Rubric. You can reconstruct which hash served a given request id.

214.7 Depth tracks

Beginner. Implement `gate` and a constant fallback.

University. Add the monitor table and the delayed join.

Research. Define a latency budget at the 99th percentile and a timeout policy. Explain why the mean latency can look fine while the budget is missed.

214.8 Failure modes

- Scoring NaNs as if they were numbers.
- A monitor that only exists the day after an incident.
- Dropping unjoined labels and reporting a loss as if it were the population loss.

214.9 See also

Volume 07 for the metric. Volume 16 for when the monitor's metric is the wrong harm.

215 Reproducibility in production

216 Reproducibility in production

Reference question. What would you need to rebuild today's prediction tomorrow?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

216.1 Reference card

Item	Statement
Code	A version identifier. A branch name that moves is not an identifier.
Data	A snapshot id the training job recorded.
Numbers	Seeds or a statement that the training is nondeterministic and the metric band you accept.
Environment	Library versions that affect numerics, when you depend on them.
Proof	A rerun that meets a tolerance, or an honest statement that you cannot rerun.

216.2 Theory

Reproducibility is the ability to get the same decision from the same inputs, or to explain the band of decisions you accept. Bitwise equality is sometimes available for a small NumPy model and sometimes not available for a distributed job. The card should not claim bitwise equality you did not test.

The production question is narrower than the research question. Can you name the bytes that served this request, and can you train a successor from a named snapshot? If the snapshot was overwritten, you can still have logs of predictions and no ability to explain them. Logging the features for a sample of requests is expensive and is the only way to debug a feature that is gone.

A tolerance belongs in the golden test. Floating-point reductions across different batch sizes can change a prediction in the last bits and should not flip a high-stakes threshold without warning. If your decision is a threshold on a probability, test stability around the threshold, not only the mean squared difference of logits.

216.3 Derivation

Let $y = g_h(x)$ be the prediction of hash h at input x. A rerun produces $g_{h'}(x)$. The reproducibility claim on a golden set G is $\max_{x \in G} |g_h(x) - g_{h'}(x)| \leq \tau$ when h is supposed to equal h'. If the hashes differ, you are comparing different models and the tolerance is the wrong tool. Log the hash first. Then compare. A threshold decision adds the check that $g_h(x) - t$ and $g_{h'}(x) - t$ have the same sign for x near the boundary. A small τ can still flip a sign if $|g(x) - t| < \tau$.

216.4 Code

```
def decisions_match(scores_a, scores_b, threshold, tol):
    for a, b in zip(scores_a, scores_b):
        if abs(a - b) > tol:
            return False
        if (a >= threshold) != (b >= threshold) and min(abs(a - threshold), abs(b - thresh
            return False
    return True
```

216.5 Experiment

Hypothesis. Two identical score lists match. A perturbation smaller than `tol` that crosses the threshold is flagged by the sign check even when you might have accepted the absolute error.

Protocol. Scores `[0.50, 0.80]`, threshold 0.5, tolerance 0.02, and a second list `[0.49, 0.80]`.

Metric. The boolean from `decisions_match`.

Decision rule. The identical list matches. The perturbed list must not match under this function if you include the sign check as written. If you decide threshold flips inside `tol` are acceptable, change the function and the card together. Do not leave them in conflict.

216.6 Project

Deliverable. A rerun note for one small training job: code version, snapshot id, seed, metric band, and a golden serving test with a tolerance and a threshold check.

Constraints. If you cannot rerun, the note says what was discarded, not a fake hash.

Rubric. The serving test fails when the unit conversion from the lifecycle chapter is reintroduced.

216.7 Depth tracks

Beginner. Run the threshold stability example.

University. Document a real rerun of a Volume 03 experiment folder.

Research. List the nondeterministic sources you accept in a GPU training job and the metric band that replaces bitwise equality. Be concrete: data order, atomic reductions, or library versions.

216.8 Failure modes

- A floating branch name as a version.
- A tolerance so wide that a unit bug passes.
- Deleting the snapshot after the paper or the launch.

216.9 See also

Volume 18 for research reproducibility, which adds the question of whether someone else can rerun the argument, not only the service.

217 Project: a batch-scoring service

218 Project: a batch-scoring service

Reference question. Can you wrap one trained model in a snapshot, a gate, a versioned score file, and a monitor?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

218.1 Reference card

Item	Statement
Input	A snapshot of features with a schema.
Score	A file or response that includes the model hash and the fallback flag.
Gate	Malformed rows take the fallback and are counted.
Monitor	Failure fraction, score distribution, and a delayed metric if you have labels.
Rollback	A previous hash you can select without retraining.

218.2 Theory

The project is intentionally batch, not a networked service, so you can finish it on a laptop. The artifacts are the same ones a service would log. A batch job that overwrites yesterday's scores without a hash is the anti-pattern. Write a new file per hash.

Use a model you understand from an earlier volume, even a linear one. The systems point will be clearer than if you spend the week training. Include the unit-mismatch gate from the first chapter as a test that must fail closed.

The delayed metric can be synthetic: hide the label until a second script joins it. That script is the monitor. If you skip it, the project is a scorer, not a system.

218.3 Derivation

A batch run defines a function from a snapshot id and a model hash to a score file. Rollback is selecting a previous pair $(snapshot, hash)$ and re-emitting scores, or selecting a previous score file if the inputs match. If the feature snapshot has changed, rescoring the old model on new features is a new experiment, not a rollback. The run book must say which of the two you mean. The project note picks one and tests it.

218.4 Code

```
import json
from pathlib import Path

def write_scores(path, model_hash, rows):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps({"model_hash": model_hash, "rows": rows}, indent=2), encodi
```

218.5 Experiment

Hypothesis. Scoring the same snapshot with two hashes produces two files, and a rollback procedure that copies the previous file restores the previous decisions. Scoring a changed snapshot with the old hash does not restore the old decisions when the features changed.

Protocol. A two-point linear model, a unit change in the second snapshot, and `write_scores`.

Metric. Whether the restored file matches the first file byte for byte, and whether the changed snapshot changes a prediction.

Decision rule. Both parts of the hypothesis should hold. A rollback implementation that retrains is a different procedure and must be labeled as retraining.

218.6 Project

Deliverable. The batch scorer, schema gate, hashed score files, monitor script, rollback test, and a one-page run book.

Constraints. The run book states what this system does not do, including online latency and distributed training, and points to CS249r for those topics.

Rubric. A classmate can run the rollback test and the golden feature test without asking you which file is live.

218.7 Depth tracks

Beginner. Write and read a score file with a hash.

University. Complete the scorer, monitor, and rollback test.

Research. Add a canary: score 10 percent of rows with a new hash, compare delayed loss, and define the rule that promotes or rejects the canary. Use synthetic labels.

218.8 Failure modes

- Overwriting the only score file.
- A fallback that is not logged.
- Retraining and calling it rollback.

218.9 See also

Volume 16 before this system is aimed at people. Volume 18 if the run book is the basis of a paper's reproducibility appendix.

Part XVI

Volume 16. Reliability, privacy, and harm

219 Volume 16. Reliability, privacy, and harm

220 Volume 16. Reliability, privacy, and harm

Stage. University to research

Audience. A reader who can evaluate a model and describe a deployment. This volume is about failure of the decision, not about a new architecture.

Question this volume answers. When the data distribution, the adversary, the privacy constraint, or the harm differs from the average metric, what do you measure instead?

This volume is a reference, not a short tour. Each chapter opens with a card you can look up later, then gives the theory, a reference implementation, an experiment with a decision rule, and a project with a rubric. Read in order the first time. After that, enter through the reference card of the chapter that matches the problem in front of you.

220.1 How to use the four layers

Layer	What you should be able to do when you finish it
Theory	State the definition, the assumptions, and the result, including what the result does not say.
Code	Implement the core object in a few dozen lines and check it against a known case.
Experiment	Change one factor, record a metric with uncertainty, and say what outcome would change your mind.
Project	Produce an artifact someone else can rerun: code, figure, table, and a one-page lab note.

220.2 Depth tracks

Every chapter ends with three tracks.

- **Beginner.** Do the reference card, the code, and the smallest experiment. Skip optional proofs.
- **University.** Do the derivation, the full experiment, and the project rubric.

- **Research.** Read the failure modes, change an assumption, and write what would have to be true for the method to fail on a new distribution.

220.3 Chapters

No.	Chapter	File
01	Distribution shift	`vol16/01_shift.qmd`
02	Robustness and adversarial examples	`vol16/02_robustness.qmd`
03	Privacy	`vol16/03_privacy.qmd`
04	Fairness and measurement	`vol16/04_fairness.qmd`
05	Safety cases	`vol16/05_safety.qmd`
06	Project: a shift and harm audit	`vol16/06_audit_project.qmd`

220.4 Lab note

Keep one markdown file per experiment, `notes/YYYY-MM-DD-<slug>.md`, with the hypothesis, the command you ran, the metric, and the decision. The series is a reference only if the experiments you run are attached to it.

221 Distribution shift

222 Distribution shift

Reference question. What exactly changed between training and deployment, and which metric moves?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

222.1 Reference card

Item	Statement
Covariate	The law of X changes. The conditional $P(Y \mid X)$ stays. A reweighted test can represent the new X if you know the weights.
Label	The conditional $P(Y \mid X)$ changes. Reweighting X is not enough.
Concept	Often used for a changing conditional. Say which conditional, of Y given X, or of a human label given the event.
Slice	A shift can be a new mixture of slices you already measured.
Action	Collect data, reweight, restrict the domain, or refuse to deploy. The word retrain is not specific.

222.2 Theory

Shift is a change in a named law. Without the name, "it shifted" is not a diagnosis. Covariate shift preserves the conditional you learned and changes where you evaluate it. If the new region of X was absent from training, you are extrapolating, and the conditional estimate there is a hope. Label shift and concept change alter the relationship the model was fit to. A monitor on X alone can miss them, which is why Volume 15 asked for delayed outcomes.

The practical measurement is a test set drawn from the deployment law, or a reweighting of an old test set that you can justify. A synthetic shift, such as brightness in Volume 11, is a

unit test of the procedure. It is not a certificate about a new hospital or a new year. Those need their own data.

Report the old metric and the new metric side by side. A model that is best on the old test set and worst on the new one has not "generalized." The selection rule has to see the deployment distribution or you are optimizing the wrong risk.

222.3 Derivation

Under covariate shift, $p_{\text{new}}(x, y) = p_{\text{new}}(x)p_{\text{old}}(y \mid x)$. The new risk is $\mathbb{E}_{\text{old}}[w(X)\ell(f(X), Y)]$ with $w(x) = p_{\text{new}}(x)/p_{\text{old}}(x)$ when the support condition holds. An estimated w that is wrong in the tails dominates the weighted average. Clipping w changes the target risk. If the conditional changed, no $w(x)$ recovers the new risk from old labeled data, because the old y values are from the wrong conditional.

222.4 Code

```
import numpy as np

def weighted_mean(losses, weights):
    losses = np.asarray(losses, dtype=np.float64)
    weights = np.asarray(weights, dtype=np.float64)
    if np.any(weights < 0) or np.sum(weights) <= 0:
        raise ValueError("weights must be nonnegative and not all zero")
    return float(np.sum(weights * losses) / np.sum(weights))
```

222.5 Experiment

Hypothesis. On data where $Y = X + \varepsilon$ and the test X is shifted to a region with the same conditional, a linear model trained on the old region still has small loss if it captured the line, while a nearest-neighbor model degrades. A change in the slope at deployment hurts both, and reweighting old data does not create the new slope.

Protocol. Train on $X \in [-1, 0]$. Covariate test: $X \in [0, 1]$ with the same slope. Concept test: slope flips sign. Include a constant baseline.

Metric. Test MSE under both shifts.

Decision rule. The concept shift should be worse for the linear model than a mere covariate shift into a neighboring interval, if the linear fit is accurate. If nearest neighbors beat the line on the original interval and collapse outside it, that is the extrapolation lesson. Record what you actually measure.

222.6 Project

Deliverable. A shift note for one model: the named change, the old metric, the new metric, the constant baseline under the new law, and a decision to reweight, collect, restrict, or refuse.

Constraints. Do not retune on the shifted test set and then call the number a measure of the unshifted procedure.

Rubric. The note says whether the conditional was assumed constant, and what data would falsify that assumption.

222.7 Depth tracks

Beginner. Compute a weighted mean and show a large weight dominating it.

University. Run the covariate versus slope-change comparison.

Research. State a support failure: $w(x)$ undefined when the new x has zero density under the old law. Give a feature that makes your model's support fail, such as a new category level.

222.8 Failure modes

- Calling every failure "shift" without a law.
- Reweighting with unbounded weights and no effective sample size.
- Selecting the model on the deployment test set.

222.9 See also

Volume 11 for image shifts. Volume 15 for monitors that see only X.

223 Robustness and adversarial examples

224 Robustness and adversarial examples

Reference question. Robust to what perturbation, under what threat model?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

224.1 Reference card

Item	Statement
Threat	Who can change the input, by how much, and whether they see the model.
Ball	A set of allowed perturbations, such as an ℓ_∞ budget ε on pixels or tokens.
Attack	A search for a perturbation in the ball that changes the decision. It is a lower bound on badness: failure to find an example is not a proof of robustness.
Defense	A training or detection method. It must be tested against an attack that knows the defense, or the result is incomplete.
Ordinary noise	A random perturbation is a different experiment from an adversary.

224.2 Theory

Robustness is a property of a pair: a model and a set of perturbations. A model can be robust to sensor noise and fragile to a one-pixel change an adversary chose. Reporting "robust" without the set is the same error as reporting accuracy without the base rate.

An attack that fails tells you that this search did not find a bad input. A stronger search might. The honest table is the attack's success rate at a stated budget, plus the clean accuracy. A defense that drops clean accuracy below the useful line has a cost. Put the cost in the table.

Random augmentation during training, from Volume 10, is not an adversarial evaluation. It may or may not improve the adversarial number. Measure it. Defenses that obfuscate

gradients can look strong against gradient attacks and fail against a simpler search. Include at least one attack that does not rely on the gradient if you claim a defense.

224.3 Derivation

For a linear score $w^\top x + b$, the perturbation δ with $\|\delta\|_\infty \le \varepsilon$ that most increases the score in a chosen direction is $\delta_j = \varepsilon \operatorname{sign}(w_j)$ when you want to increase $w^\top x$. The change in the score is $\varepsilon \|w\|_1$. A small ε flips the decision when the clean margin is smaller than that change. This is the threat model "bounded ℓ_∞, full knowledge of w." A different norm gives a different worst δ. The formula is the unit test for any attack code you write on a linear model before you attack a network.

224.4 Code

```
import numpy as np

def linf_attack_linear(w, margin, eps):
    direction = eps * np.sign(w)
    new_margin = margin - eps * np.sum(np.abs(w))
    return direction, float(new_margin)
```

224.5 Experiment

Hypothesis. On a linear classifier, the ℓ_∞ step reduces the margin by $\varepsilon \|w\|_1$, and a random perturbation of the same budget reduces it less on average.

Protocol. A random w in $\mathbb{R}^{20}$, several margins, $\varepsilon = 0.1$. Compare the analytic change with a Monte Carlo average of random δ in $[-\varepsilon, \varepsilon]^d$.

Metric. Margin change under the attack and under noise.

Decision rule. The attack must match the formula and must be worse than the average noise. If your network attack cannot beat random noise on a linear model, the attack code is wrong.

224.6 Project

Deliverable. The linear unit test, plus one attack on a small network or a statement that you only claim random-noise robustness, with clean accuracy beside the robust number.

Constraints. The threat model is in the first paragraph. The attack budget is not chosen by looking at which budget makes the paper exciting. Pick it from a sensor or a product constraint, or label it as arbitrary.

Rubric. You do not claim robustness beyond the attacks you ran.

224.7 Depth tracks

Beginner. Compute the linear ℓ_∞ margin change by hand for a two-dimensional w.

University. Compare the attack with random noise.

Research. Explain gradient masking in a paragraph: a defense that makes ∇_x useless while a decision-based or transfer attack still succeeds. You do not need to break a published defense; describe the measurement that would reveal masking.

224.8 Failure modes

- Robust accuracy with no clean accuracy.
- An attack budget that includes perturbations a camera cannot produce, presented as a physical threat, or the reverse.
- Testing only the attack the defense was designed on, if you had access to the defense's internals during tuning, without saying so.

224.9 See also

Volume 10 for augmentation. Volume 15 for a monitor that might detect an attack only if the attack leaves the input distribution.

225 Privacy

226 Privacy

Reference question. What can a released model or statistic reveal about a training row, and what promise would prevent it?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

226.1 Reference card

Item	Statement
Leak	A membership inference: an attacker's guess that a particular row was in the training set, better than chance.
Memorization	A model that can be prompted or queried to emit a training string.
Aggregate	A mean or a model is still a function of the private rows.
Differential privacy	A promise about the output distribution of a mechanism: a single row's participation changes probabilities by a bounded factor.
Cost	Noise or clipping that enforces the promise usually hurts accuracy. The curve is the result.

226.2 Theory

Privacy in this volume is about what the released artifact reveals, not about the virtue of the team. A model that memorizes a rare sentence can reveal it to someone who queries carefully. A statistic of a small group can reveal a member. Removing names is not a privacy analysis if the remaining features identify the person.

Differential privacy is a precise promise with a parameter budget. It is not a synonym for "we added noise." The promise is about neighboring datasets that differ by one row. A mechanism that satisfies it can still be useless if the noise is huge, and a mechanism that feels private can fail it. If you do not implement a mechanism with a known proof, do not claim

the guarantee. You may still measure memorization empirically and call it an empirical attack, which is a lower bound on leakage.

The project-scale question is whether you should release the model, a coarser aggregate, or nothing. Accuracy on a held-out set does not answer it.

226.3 Derivation

A randomized mechanism M is (ε, δ)-differentially private if for neighboring datasets D, D' and measurable sets S,

$$P(M(D) \in S) \leq e^{\varepsilon} P(M(D') \in S) + \delta.$$

The inequality is a constraint on the mechanism, not on a single run's output. The Laplace mechanism adds noise to a function with a known sensitivity Δ: scale Δ/ε for pure ε-differential privacy in the scalar case, under the usual definition. A function whose sensitivity you do not know, such as an unbounded neural training run, does not become private because you add a little noise at the end. Clipping is how some training mechanisms bound sensitivity. Without the clip, the theory does not apply.

226.4 Code

```
import numpy as np

def laplace_mech(value, sensitivity, epsilon, rng):
    if epsilon <= 0 or sensitivity < 0:
        raise ValueError("epsilon and sensitivity must be valid")
    scale = sensitivity / epsilon
    return float(value + rng.laplace(0.0, scale))
```

226.5 Experiment

Hypothesis. On a counting query, the Laplace mechanism's outputs spread by about the scale Δ/ε, and a tiny noise that ignores sensitivity fails a simple neighboring-dataset distinguishability check you can simulate.

Protocol. True count 100, sensitivity 1, $\varepsilon = 0.5$, many draws. Also a broken mechanism that adds $\mathcal{N}(0, 10^{-6})$ noise. Compare the distributions on counts 100 and 101 only descriptively: plot or summarize overlap. Do not claim a formal test you did not calibrate.

Metric. Empirical standard deviation versus the Laplace scale $\sqrt{2}\,\Delta/\varepsilon$, and a qualitative note on the broken mechanism.

Decision rule. The Laplace spread should match the formula's order. The broken mechanism's outputs on 100 and 101 should be easy to tell apart. If you cannot see that, the simulation is wrong. The point of the broken mechanism is that "some noise" is not the definition.

226.6 Project

Deliverable. A privacy note for one artifact you might release: what a training row contains, who the attacker is, an empirical extraction or membership check if it is safe and ethical on synthetic data, and either a correctly scoped Laplace mechanism on a simple statistic or an explicit refusal to claim differential privacy.

Constraints. Synthetic rows only, unless you have the right to use the real ones for this attack. Do not attack a real person's record as a demonstration.

Rubric. The note's claim matches the mechanism. "We care about privacy" is not a result.

226.7 Depth tracks

Beginner. Add Laplace noise to a count and see the scale change with ε.

University. Compare the broken tiny-noise mechanism on neighboring counts.

Research. Describe membership inference at the level of an attacker's score and a true-positive rate at a low false-positive rate. Say why accuracy of the attacker is the wrong summary when most people are non-members.

226.8 Failure modes

- Hashing emails and calling the model private.
- Adding Gaussian noise with no sensitivity and no budget.
- A public model card that includes unique text from the training set.

226.9 See also

Volume 13 for memorization distance. Volume 18 for what not to put in a paper's appendix.

227 Fairness and measurement

228 Fairness and measurement

Reference question. Which disparity did you measure, on which groups, and what can the number not decide?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

228.1 Reference card

Item	Statement
Group	A set of people or cases defined by a feature you are willing to use in the audit. The definition is an ethical and a measurement choice.
Base rate	Groups can have different outcome rates. A metric that ignores this will be misread.
Error rates	False-negative and false-positive rates can differ even when accuracy matches, and the reverse.
Threshold	A single threshold can create disparate error rates. Group-specific thresholds are a different policy with their own costs.
Limit	A parity number is not a proof of justice. It is one measurement of one definition.

228.2 Theory

Fairness metrics conflict. Equalizing accuracy, equalizing false-negative rates, and equalizing predictive values cannot all hold in general when base rates differ and the predictor is not perfect. A reference note picks the disparity that matches the harm, reports the others as context, and does not claim a theorem that all of them improved.

Groups are not discovered by clustering the model's errors and naming the clusters. They are specified by a question: who bears the error? Proxy attributes are dangerous. A postal code can stand in for a protected attribute and for many other things. If you audit only the proxy, say so.

The data that define group membership may be missing, wrong, or illegal to use. An audit that requires collecting a sensitive attribute can itself be the harm. The note should consider that cost. Sometimes the right measurement is a process constraint you can check without new sensitive data, such as "this feature is not an input," which is weaker and clearer.

A model card from Volume 07 grows a disparity table here. The table has denominators. A gap on a group of size 5 is a case review, not a rate.

228.3 Derivation

Let group g have base rate p_g and false-positive and false-negative rates f_g and m_g. Precision is $p_g(1-m_g)/(p_g(1-m_g)+(1-p_g)f_g)$. Two groups with equal f and m and unequal p have unequal precision. Two groups with equal precision and unequal p generally have unequal error rates. This is algebra, not a moral ranking. It is why a memo that says "we equalized accuracy" has not said what happened to the false-negative rate. Write the table of p_g, f_g, m_g and the policy threshold.

228.4 Code

```
import numpy as np

def group_rates(y, pred, group, g):
    mask = np.asarray(group) == g
    y = np.asarray(y).astype(bool)[mask]
    pred = np.asarray(pred).astype(bool)[mask]
    if y.size == 0:
        raise ValueError("empty group")
    pos = y.sum()
    neg = y.size - pos
    fp = np.sum(pred & ~y)
    fn = np.sum(~pred & y)
    return {
        "n": int(y.size),
        "base_rate": float(y.mean()),
        "fpr": float(fp / neg) if neg else float("nan"),
        "fnr": float(fn / pos) if pos else float("nan"),
    }
```

228.5 Experiment

Hypothesis. Two groups with the same score distribution within the positive class and within the negative class, but different base rates, show equal true and false positive rates at one threshold and unequal precision.

Protocol. Simulate scores that depend on the label and not on the group. Vary the base rate. Apply one threshold.

Metric. FPR, FNR, and precision by group.

Decision rule. Error rates should match up to sampling noise. Precision should not. If error rates differ, the score generator accidentally depends on the group. This simulation is the algebraic point, not a claim that real errors are independent of group.

228.6 Project

Deliverable. A disparity table for a synthetic or a permitted dataset, with base rates, error rates, sizes, and a paragraph on which disparity you would optimize and which you cannot also satisfy. Include a feature you refuse to use as an input, with the reason.

Constraints. Do not invent real demographic claims from a toy generator. Label the data as synthetic when they are.

Rubric. The table includes n per group. The prose does not say the model is fair. It says which measured gap remains.

228.7 Depth tracks

Beginner. Compute precision from the formula for two base rates and equal error rates.

University. Run the simulation and write the table.

Research. Discuss a threshold policy that equalizes false-negative rates and the incentive it creates if people can choose which group label is recorded. Keep it concrete and short.

228.8 Failure modes

- A fairness claim from accuracy alone.
- Groups of size 10 reported as percentages without n.
- Using a protected attribute as a model input in order to "fix" a disparity, without a policy analysis.

228.9 See also

Volume 07 for metrics. Volume 15 for a monitor that should break down by the slices you decided to audit, where that is legitimate.

229 Safety cases

230 Safety cases

Reference question. What argument would justify running this system, and what evidence is still missing?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

230.1 Reference card

Item	Statement
Hazard	A concrete way the system can cause a loss: a wrong action, a missed case, a leak, a feedback loop.
Control	A mechanism that reduces that hazard: a threshold, a human review, a fallback, a limit on actions.
Evidence	A measurement that the control works under a stated condition.
Assumption	A condition the argument needs and that you have not tested.
Stop	A condition under which the system is withdrawn.

230.2 Theory

A safety case is a structured argument, not a list of hopes. For each hazard you name a control and the evidence that the control is effective, and you list the assumptions. An assumption that is false breaks the case even if the in-domain accuracy is high. Volume 15's fallback, Volume 14's reward, and this volume's shift and privacy notes are inputs to the case.

Feedback loops are a hazard the earlier metrics miss. A model that denies loans changes who comes back with a label. A recommender changes what is clicked. The logged data then follow the system's own policy. Off-policy evaluation from Volume 14 is relevant, and a static test set is not the world after deployment.

The case should be readable by someone who can stop the launch. If every hazard ends with "the model is state of the art," there is no case. State of the art is not a control.

230.3 Derivation

Let H be a hazard and C a control that is claimed to reduce $P(H)$ below a bound b when assumption A holds. The evidence is an experiment whose outcome is informative about $P(H)$ under A. If the experiment was run only when A is true by construction, the case must not cite it for worlds where A is false. Write A beside the number. A fallback control has its own hazard: the fallback decision can be harmful. Include it as a row, not as an implementation detail.

230.4 Code

```
def safety_row(hazard, control, evidence, assumption, withdraw_if):
    fields = [hazard, control, evidence, assumption, withdraw_if]
    if any(not str(x).strip() for x in fields):
        raise ValueError("a safety row cannot be empty")
    return {
        "hazard": hazard,
        "control": control,
        "evidence": evidence,
        "assumption": assumption,
        "withdraw_if": withdraw_if,
    }
```

230.5 Experiment

Hypothesis. A row with an empty assumption fails the helper, and a completed row for the batch scorer's unit mismatch names a withdraw condition you can actually test.

Protocol. Call `safety_row` with a blank assumption and with a full row whose evidence is the golden feature test.

Metric. Whether the blank row raises, and whether the full row's `withdraw_if` is a check rather than a slogan.

Decision rule. "Withdraw if users complain" is weaker than "withdraw if the golden feature test fails or the delayed loss exceeds a set budget." Prefer the check. If you keep a human signal, say who reviews it and how fast.

230.6 Project

Deliverable. A safety case with at least three hazards for a system you described in Volume 15 or for a purely advisory predictor. Each row has evidence you have or an explicit "not yet measured."

Constraints. One hazard must be a feedback loop or a fallback harm. One assumption must be marked untested.

Rubric. A reader can see what would force withdrawal. The case does not conclude "safe." It concludes "acceptable under these assumptions" or "not acceptable yet."

230.7 Depth tracks

Beginner. Fill one row for a schema-gate failure.

University. Complete three rows tied to measurements from earlier volumes.

Research. Describe a closed-loop data collection problem in which the logging policy is the deployed policy, and the static accuracy number no longer estimates the next period's risk.

230.8 Failure modes

- Hazards written as "model error" with no mechanism.
- Evidence cited from a different threat model.
- A withdraw condition nobody is assigned to watch.

230.9 See also

Volume 14 for reward hacking as a hazard. Volume 18 for the difference between a safety case and a paper.

231 Project: a shift and harm audit

232 Project: a shift and harm audit

Reference question. Can you audit one model for shift, a robustness or privacy limit, and a disparity or safety row without hiding behind the average metric?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

232.1 Reference card

Item	Statement
Average	The in-domain primary metric, with n and a baseline.
Shift	One named shift and the metric under it.
Limit	One robustness budget or one privacy statement that does not over-claim.
Harm	One disparity table or one safety row, chosen because it matches the decision.
Decision	Ship, restrict, collect data, or stop.

232.2 Theory

The audit is the last applied project in the series before the research volumes. It uses a model you already understand so the new work is the measurement of failure. An audit that only restates the best test number is a brochure. The decision paragraph must be able to say stop.

Use synthetic data if real data would involve people you should not instrument for a class project. Label the synthetic world as synthetic. The skill is the structure of the audit, and it transfers. The numbers do not transfer to a real population you did not measure.

If two checks conflict, leave the conflict visible. A robust model that worsens a disparity, or a private mechanism that misses the accuracy budget, is a result. The decision names which constraint wins and who is exposed if you are wrong.

232.3 Derivation

The audit estimates several functionals: in-domain risk, shifted risk, an attack success rate or a privacy parameter, and a group-specific error. These estimators have different sample sizes. A decision rule that treats a gap of 0.01 in every column as equal is ignoring the standard errors and the harms. Write the rule in advance: for example, refuse deployment if the shifted risk exceeds a bound or if a group of size at least n_0 has a false-negative gap above a bound. Applying the rule after seeing which bound is convenient is the peeking problem from Volume 05.

232.4 Code

```
def audit_decision(shifted_risk, shift_budget, disparity_gap, disparity_budget, group_n, m
    if group_n < min_n:
        disparity_flag = "insufficient_n"
    else:
        disparity_flag = "exceeds" if disparity_gap > disparity_budget else "within"
    shift_flag = "exceeds" if shifted_risk > shift_budget else "within"
    return {"shift": shift_flag, "disparity": disparity_flag}
```

232.5 Experiment

Hypothesis. The decision helper distinguishes a small group, which should not produce a rate-based exceedance, from a large group with a large gap, and it flags a shifted risk above the budget independently.

Protocol. Three calls: small n, large gap with large n, and a shifted risk above budget with disparity within budget.

Metric. The returned flags.

Decision rule. The flags must match the written rule. If your prose recommends "ship" while a flag says exceeds, the prose loses.

232.6 Project

Deliverable. An audit memo for one model: average metric, shift, robustness or privacy limit, disparity or safety row, the prewritten decision rule, and the resulting decision.

Constraints. Synthetic or permitted data. Claims match the evidence. No borrowed demographic story.

Rubric. The memo can say stop. A memo that cannot say stop is not an audit.

232.7 Depth tracks

Beginner. Run the decision helper on the three cases.

University. Write the full memo around a model from an earlier project.

Research. Add a feedback-loop assumption to the memo and a measurement you would need after deployment to test it. Do not pretend the static audit already did.

232.8 Failure modes

- An audit written after choosing the decision.
- Private or sensitive real records used as toy examples.
- A conclusion of "fair and robust" from an average accuracy.

232.9 See also

Volume 17 for audits of models you did not train and cannot fully inspect. Volume 18 for turning an audit into a reproducible note.

Part XVII

Volume 17. Foundation models

233 Volume 17. Foundation models

234 Volume 17. Foundation models

Stage. Research

Audience. A reader who has trained small models, evaluated generators, and written an audit. The claims in this volume are narrower than the marketing around large models.

Question this volume answers. What can you responsibly conclude from a model trained by someone else on data you did not see?

This volume is a reference, not a short tour. Each chapter opens with a card you can look up later, then gives the theory, a reference implementation, an experiment with a decision rule, and a project with a rubric. Read in order the first time. After that, enter through the reference card of the chapter that matches the problem in front of you.

234.1 How to use the four layers

Layer	What you should be able to do when you finish it
Theory	State the definition, the assumptions, and the result, including what the result does not say.
Code	Implement the core object in a few dozen lines and check it against a known case.
Experiment	Change one factor, record a metric with uncertainty, and say what outcome would change your mind.
Project	Produce an artifact someone else can rerun: code, figure, table, and a one-page lab note.

234.2 Depth tracks

Every chapter ends with three tracks.

- **Beginner.** Do the reference card, the code, and the smallest experiment. Skip optional proofs.
- **University.** Do the derivation, the full experiment, and the project rubric.

- **Research.** Read the failure modes, change an assumption, and write what would have to be true for the method to fail on a new distribution.

234.3 Chapters

No.	Chapter	File
01	Scale, data, and claimed emergence	`vol17/01_scale.qmd`
02	Adaptation	`vol17/02_adaptation.qmd`
03	Preferences and alignment	`vol17/03_alignment.qmd`
04	Evaluating foundation models	`vol17/04_eval_foundation.qmd`
05	Agents as systems	`vol17/05_agents.qmd`
06	Project: an evaluation harness	`vol17/06_foundation_project.qmd`

234.4 Lab note

Keep one markdown file per experiment, `notes/YYYY-MM-DD-<slug>.md`, with the hypothesis, the command you ran, the metric, and the decision. The series is a reference only if the experiments you run are attached to it.

235 Scale, data, and claimed emergence

236 Scale, data, and claimed emergence

Reference question. What does a scaling curve show, and what does a jump in a metric not show?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

236.1 Reference card

Item	Statement
Scale	Parameters, tokens, and compute. A curve that varies only one of them confounds the others.
Smooth	Many error curves fall smoothly on a log compute axis. A plot can look like a jump because the metric is a thresholded accuracy.
Emergence	A claim that a capability appears suddenly. Check the metric, the resolution of the x-axis, and the task selection.
Data	The mixture, the cutoff date, and the deduplication policy are part of the model.
Your run	A small model you train is the place to learn the measurement. A downloaded model is the place to apply it, not to skip it.

236.2 Theory

Large pretrained models are empirical objects. Scaling laws, when they hold, are fits of loss against compute on a family of training runs. They predict loss, not a product requirement, and they do not transfer to a new data mixture without a new fit. A figure with three points is not a law.

Threshold metrics create apparent emergence. If the underlying per-token loss improves smoothly, an exact-match accuracy can stay at zero until the loss crosses a cliff and then jump. Report the underlying loss or a partial-credit metric before you announce a phase

change. Also beware task selection: a paper that shows the tasks that jumped, and not the tasks that did not, is a selected sample.

You usually cannot reproduce the pretraining. You can still demand the model id, the date, the evaluation protocol, and a baseline a smaller or simpler system can run. Volume 12's bag-of-tokens baseline remains a legitimate comparison on classification. Being embarrassed by it is informative.

236.3 Derivation

Suppose a continuous score s improves as $s = a - b \log c$ with compute c, and a reported accuracy is $1\{s > \tau\}$. The indicator jumps at one compute threshold even though s is smooth. Changing τ moves the jump. A metric with partial credit, such as mean token accuracy, tracks s more closely. When you see a step-function plot, ask which threshold created it and whether the x-axis is coarse. This does not prove every sudden capability is a metric artifact. It is the first alternative to rule out.

236.4 Code

```
import numpy as np

def threshold_curve(log_compute, tau):
    score = 1.0 - 0.15 * np.asarray(log_compute, dtype=np.float64)
    return (score > tau).astype(np.float64), score
```

236.5 Experiment

Hypothesis. A smooth score and a thresholded version of the same score tell different stories about suddenness.

Protocol. Evaluate `threshold_curve` on a fine grid and on three coarse x positions, at two thresholds.

Metric. Where the indicator flips, and the smoothness of the raw score.

Decision rule. The raw score must not jump. The indicator must. A write-up of this toy that calls the indicator an emergent capability has made the mistake the chapter is about. If your real model's curve remains a jump on a graded metric at high x-resolution, the toy does not explain it away. Say that.

236.6 Project

Deliverable. A one-page reading of a public scaling or capability plot you can access: what was held constant, what metric was thresholded, what baseline is missing, and what you would need to reproduce even one point.

Constraints. Do not invent numbers you did not read. Quote the axis labels.

Rubric. The page distinguishes a loss curve from a capability claim.

236.7 Depth tracks

Beginner. Plot or tabulate the toy threshold curve.

University. Write the reading note on a real public figure.

Research. Separate in-distribution loss scaling from out-of-distribution task performance. Give one reason a model with a better pretraining loss can be worse on your deployment shift.

236.8 Failure modes

- Three noisy points fit with a law.
- A benchmark suite reported only where the new model wins.
- Compute estimated so loosely that two runs are not comparable.

236.9 See also

Volume 05 for fitting a curve you are not willing to validate. Volume 13 for likelihood versus sample quality, which also diverge.

237 Adaptation

238 Adaptation

Reference question. Which adaptation method matches the amount of labeled data and the access you have?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

238.1 Reference card

Item	Statement
Prompt	Inputs only. Fragile to wording. No gradient. Version the model and the prompt.
Probe	A small head on frozen features. The Volume 08 protocol.
Low-rank update	A constrained Δ, as in the language volume. Needs gradients and weights or an adapter API.
Full fine-tune	Highest capacity to forget the original model and to memorize a small set.
Selection	The adaptation method is chosen on validation. The test set scores the choice once.

238.2 Theory

Adaptation is supervised learning on top of a representation you did not choose feature by feature. The same split, baseline, and paired-gap rules apply. A prompt that was edited twenty times on the test examples is a test-set search. Log the attempts or hold out a validation slice of prompts.

Frozen probes tell you whether the representation is linearly readable for your label. If the probe ties a bag-of-tokens model, the foundation model is not earning its cost on this task. That is a successful negative result.

Full fine-tuning on a tiny set needs the regularization chapter: small step, early stopping, and a check that training loss is not the only number that moved. Catastrophic forgetting of

a capability you still need is a shift in the model's behavior on a previous task. If you care about that task, measure it. Do not assume the original checkpoint's card still describes the fine-tuned weights.

238.3 Derivation

Let $R_{\text{val}}(m)$ be the validation risk of adaptation method m after its internal knobs are chosen. You pick $m^\star = \arg\min_m R_{\text{val}}(m)$. The test risk of $m^\star$ is the number you may publish as the risk of the selected adaptation, if the test set did not enter the prompts, the early stopping, or the method choice. The gap $R_{\text{val}}(\text{probe}) - R_{\text{val}}(\text{full})$ estimates whether the extra capacity helped on new draws from the same split. It does not estimate performance on a new domain. Run the shift audit if the domain is the question.

238.4 Code

```
def adaptation_record(method, val_metric, n_train, model_id):
    if not model_id or n_train < 0:
        raise ValueError("model id and n_train are required")
    return {
        "method": method,
        "val_metric": float(val_metric),
        "n_train": int(n_train),
        "model_id": model_id,
    }
```

238.5 Experiment

Hypothesis. On a synthetic labeling task that is a keyword, a bag-of-tokens probe matches or beats a method that only sees a paraphrased prompt you deliberately make vague. On a task that needs a long-range phrase, the keyword method fails and a sequence method can win.

Protocol. Reuse the Volume 12 generators if you still have them. Treat "prompt" as a fixed lookup table you wrote before seeing the test split: a simulated prompted classifier with a known error rate is acceptable if you do not have local weights. The point is the record and the comparison, not a particular vendor.

Metric. Validation metric for each method in `adaptation_record`.

Decision rule. The keyword task should not crown an opaque model over the bag of tokens. If it does, the bag baseline is underfit or the test leaked.

238.6 Project

Deliverable. A comparison card: majority baseline, bag of tokens or linear probe, and one heavier adaptation if you have legal access to it. Model id, date, validation metric, and a contamination caveat.

Constraints. No test-set prompt editing. If you only have an API, store the date and do not claim you fine-tuned.

Rubric. A negative result that keeps the simpler method is an acceptable outcome.

238.7 Depth tracks

Beginner. Fill `adaptation_record` for a majority baseline and a keyword rule.

University. Complete the comparison on the two synthetic tasks.

Research. Define a forgetting check: a small set of previous-task examples scored before and after fine-tuning. State the budget at which you would stop fine-tuning.

238.8 Failure modes

- Prompt iteration on the test set.
- A fine-tune with no simple baseline.
- Reporting the original model's benchmark numbers as if they were your fine-tuned model's numbers.

238.9 See also

Volume 12 for fine-tuning and prompts. Volume 08 for probes.

239 Preferences and alignment

240 Preferences and alignment

Reference question. Whose preference does a post-training procedure encode, and what did it trade away?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

240.1 Reference card

Item	Statement
Preference data	Pairs or ratings from a process you should be able to name: who labeled, under what instructions.
Objective	A loss that increases the probability of preferred responses and decreases others. It is not a truth certificate.
Reward model	A learned proxy for the preferences. It can be gamed by the policy.
Tradeoff	Helpfulness, harmlessness, and task accuracy move differently. Report the ones you care about.
Not included	A moral certificate. A refusal behavior is a policy, measured on a test suite you specify.

240.2 Theory

Post-training that uses human or model-written preferences changes the conditional distribution of answers. The change is as good as the preference process. If the labelers disagree, a single reward model averages them and hides the disagreement. If the labelers share a blind spot, the model inherits it. The audit of a preference dataset is a data contract: who is absent, which instructions were ambiguous, and which behaviors were never rated.

A policy trained against a reward model can find a high reward that a person would not endorse, the reward-hacking pattern from the reinforcement-learning volume. The evaluation has to include checks the reward model does not grade: factuality on a keyed set, refusal on

a hazard set, and usefulness on the original task. Optimizing the reward and reporting the reward is circular.

This chapter does not teach you to bypass safety policies. It teaches you to measure a stated policy. A test suite of disallowed requests is scored by whether the system refuses, not by whether you can trick it for sport. If you are building the system, the safety case from Volume 16 names the hazards the preference data were supposed to cover and the ones they do not.

240.3 Derivation

A simple preference loss on a pair (y^+, y^-) given a prompt x increases a score $s_\theta(x, y^+)$ relative to $s_\theta(x, y^-)$. The implied target is "rank preferred outputs higher under s_θ," not "match the world." If s_θ is a reward model fit to the same pairs, a policy that maximizes s_θ can drift off the distribution of y the reward model saw. The KL penalty toward a reference policy is a constraint that limits that drift. Removing it changes the objective. Report whether it was present and the coefficient.

240.4 Code

```
import numpy as np

def pairwise_accuracy(score_pos, score_neg):
    score_pos = np.asarray(score_pos, dtype=np.float64)
    score_neg = np.asarray(score_neg, dtype=np.float64)
    return float(np.mean(score_pos > score_neg))
```

240.5 Experiment

Hypothesis. A score that copies a preference label achieves high pairwise accuracy on those pairs and can still rank a rewarded but empty answer above a correct answer on a keyed factual item you did not put in the preference set.

Protocol. Build ten synthetic pairs where the preferred answer is longer, and five factual items where the correct answer is short and a long empty answer scores higher on "length." Compute pairwise accuracy on the preference pairs and accuracy on the factual items under the length score.

Metric. The two numbers.

Decision rule. High preference accuracy with low factual accuracy is the illustration. A note that reports only pairwise accuracy has not evaluated the reward hack. Do not use this toy as evidence about any real company's model. It is a measurement pattern.

240.6 Project

Deliverable. A measurement note: the preference process you are willing to describe, the pairwise metric, a task metric that the preference model does not define, and a hazard the preference data do not cover.

Constraints. Do not collect real people's private judgments for a toy. Synthetic preferences are enough. Do not write exploits.

Rubric. The note says whose preference was modeled and what was not asked of the labelers.

240.7 Depth tracks

Beginner. Compute pairwise accuracy for a perfect ranking and for a reversed one.

University. Complete the length-versus-factuality toy.

Research. State the KL-to-reference term and the qualitative effect of setting its coefficient to zero. Connect it to the offline RL support problem: the policy can leave the region where the reward model is meaningful.

240.8 Failure modes

- Calling a preference-tuned model truthful because labelers preferred confident prose.
- Reporting the reward on the training pairs as evaluation.
- A refusal test you iterated on until the demo looked strict, with no held-out hazards.

240.9 See also

Volume 14 for reward hacking. Volume 16 for the safety case that should list uncovered hazards.

241 Evaluating foundation models

242 Evaluating foundation models

Reference question. How do you evaluate a model when you do not control training and the test may already have been seen?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

242.1 Reference card

Item	Statement
Contamination	Overlap between the evaluation items and the pretraining or post-training data. Exact match is a lower bound on leakage.
Freshness	Items written after the model's known data cutoff, when you know the cutoff.
Protocol	Temperature, system prompt, number of samples, and parser. These change the score.
Baseline	A simpler system on the same items, with the same parser.
Slice	The average hides the item type where the model systematically fails.

242.2 Theory

Leaderboards saturate and leak. A model can score well because the items were in its training mixture. Exact-string overlap catches the obvious cases and misses paraphrases. A cutoff date helps only if you trust it and the items are actually new. When you cannot rule contamination out, the card says "contamination unknown," which lowers the claim from "the model learned this skill" to "the model scored this number under this protocol."

The protocol is part of the estimator. A chain-of-thought prompt, a tool, and a retry loop are different systems from a single sample. Compare them as different systems. Parsing a free-form answer into a label introduces errors. Check the parser on a sample of outputs. A wrong parse can invent a gain or a loss.

Slices matter more than a new average. If every failure is an arithmetic item or a negation, the next experiment is about that slice, not about a larger model. Volume 07's mass calculation applies to item types.

242.3 Derivation

Let an item be contaminated if a normalized copy appears in the training mixture. The observed accuracy a is a mixture $a = \rho a_{\text{seen}} + (1 - \rho) a_{\text{unseen}}$ when you could partition items, with ρ the contaminated fraction. If you only have a lower bound $\rho_{\min}$ from exact match, you cannot recover a_{unseen} without more assumptions. You can report a and $\rho_{\min}$. Dropping contaminated items and reporting the rest estimates accuracy on the residual set, which is a different, possibly harder or easier set. Say which one you did.

242.4 Code

```
def exact_overlap(eval_strings, train_strings):
    train = {s.strip().lower() for s in train_strings}
    flags = [s.strip().lower() in train for s in eval_strings]
    rho = sum(flags) / len(flags) if flags else float("nan")
    return {"n": len(flags), "rho_min": rho, "flags": flags}
```

242.5 Experiment

Hypothesis. Exact overlap detects copied items and misses a paraphrase. A reported accuracy that includes copies is higher than accuracy on the residual set when copies are answered correctly and new items are not.

Protocol. Five eval strings, two copied into the train set, one paraphrase, two new. Assign correct on copies and incorrect on the rest, including the paraphrase.

Metric. `rho_min` and accuracy with and without the flagged copies.

Decision rule. The paraphrase must not be flagged by exact overlap. The two accuracies must differ. A contamination statement that says "we deduplicated" because $\rho_{\min} = 0$ on a paraphrased set overclaims the check.

242.6 Project

Deliverable. An evaluation protocol for a model you can query or for a small local model: prompt text, decode settings, parser, baseline, exact-overlap check against the data you have, and a slice table.

Constraints. If you cannot check overlap with pretraining data, write "unknown" in that cell.

Rubric. The protocol is specific enough that a rerun would not quietly change temperature or prompt.

242.7 Depth tracks

Beginner. Run `exact_overlap` on the toy strings.

University. Write the full protocol around a real or synthetic item set.

Research. Describe a cutoff test: items dated after the model freeze, and the selection bias if you only keep items the authors thought were interesting after seeing model behavior.

242.8 Failure modes

- A leaderboard number with no protocol.
- Deleting hard items after the run.
- A parser that strips the correct answer into the wrong bucket on a slice you never checked.

242.9 See also

Volume 12 for generation metrics. Volume 16 for harm slices that should appear in the same table when the items concern people.

243 Agents as systems

244 Agents as systems

Reference question. What changes when a model can call tools and take a sequence of actions?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

244.1 Reference card

Item	Statement
Loop	Observe, propose an action, execute under a policy, observe again.
Tool	A function with a contract, a permission, and a failure mode. The model does not get every tool.
State	The transcript is not automatically a Markov state of the world.
Stop	A budget on steps, money, and side effects. A loop without a stop is not a design.
Eval	Task success on held-out goals, not the model's confidence, and not a demo transcript you edited.

244.2 Theory

An agent is a system from Volume 15 wrapped around a policy from Volume 14, with a foundation model as one possible policy class. The new hazards are tool side effects and long horizons. A wrong email, a deleted file, or a purchased item is not a classification error you can average away. The safety case needs a permission boundary: which tools exist in the evaluation environment, which exist in production, and what the fallback does when the model proposes a forbidden tool.

The transcript can hide state. If a tool's effect is not written back into the observation, the next step acts on a stale belief. That is the partial-observability bug. Log the tool arguments and the tool results. A transcript that stores only the model's prose cannot be debugged.

Evaluate on a fixed suite of tasks with a checker. A sandbox that does not match production will miss the failures that depend on real permissions. Say which environment you measured. Held-out tasks are the test set. Editing a task until the agent succeeds is training on the test set.

244.3 Derivation

Let a task have a checker $c(h) \in \{0, 1\}$ on a terminal transcript h, and let the policy interact for at most T steps. The success rate is the expectation of c under the policy and the environment. A step limit T is part of the policy-environment pair. Raising T after seeing failures changes the system. Tool permissions enter the transition kernel: a forbidden tool call should transition to an error observation, not to the side effect. If the evaluation harness allows the side effect and production does not, or the reverse, the measured success is for the harness.

244.4 Code

```
def tool_dispatch(name, args, allow):
    if name not in allow:
        return {"ok": False, "error": "forbidden", "name": name}
    if name == "add":
        return {"ok": True, "value": float(args["a"]) + float(args["b"])}
    return {"ok": False, "error": "unknown", "name": name}
```

244.5 Experiment

Hypothesis. A dispatch allow-list blocks a tool that is not permitted and executes a permitted pure function. A checker that looks only at the model's final sentence can mark success when the tool was never called.

Protocol. Call `add` when it is allowed and when it is not. Define a toy task "compute 2+2" with a checker on the tool result and a weaker checker on a sentence that says "4" without a tool call.

Metric. Whether the forbidden call is refused, and whether the two checkers disagree on a transcript that never called the tool.

Decision rule. The allow-list must refuse. The two checkers must be able to disagree. Use the tool-result checker for the project if the task is a tool task. The sentence checker is the leaky metric.

244.6 Project

Deliverable. A sandboxed agent loop with one or two pure tools, a step budget, a task suite with a checker, a baseline that does not use tools, and a safety row for a forbidden tool.

Constraints. No tool with an irreversible external side effect. No attempt to defeat a real system's safety policy.

Rubric. The success rate is computed by the checker on held-out tasks. A transcript in the note is an illustration, not the result.

244.7 Depth tracks

Beginner. Implement the allow-list and the two checkers.

University. Complete the loop on a few arithmetic or lookup tasks with a step budget.

Research. Describe a feedback loop in which the agent's earlier action changes the next observation's distribution, and why the offline log of a different policy does not score the new one. This is Volume 14 inside the tool environment.

244.8 Failure modes

- An edited demo as the evaluation.
- A tool with production credentials in a notebook.
- A checker that grades the model's self-assessment.

244.9 See also

Volume 14 for the policy and the offline gap. Volume 16 for the safety case. Volume 15 for logs and rollback of the tool-using system.

245 Project: an evaluation harness

246 Project: an evaluation harness

Reference question. Can you evaluate a large or simulated foundation model with a harness that would catch contamination, a weak baseline, and a protocol change?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

246.1 Reference card

Item	Statement
Items	A fixed set with ids, slices, and gold answers or checkers.
Systems	At least two: a simple baseline and the model under test.
Protocol file	Prompt, decode settings, model id, date.
Contamination	Exact overlap against the corpora you have, or the explicit status unknown.
Report	Overall metric, slice table, and the decision the number supports.

246.2 Theory

The harness is the research instrument. It should be boring. Given an item file and a system, it writes a score file with a hash of the protocol. Changing the prompt without changing the hash is the reproducibility failure from Volume 15.

You may not have the compute to train a foundation model. You do not need to. A local small model or a deterministic simulated system still exercises the harness. If you query a real model, the harness stores the raw outputs so a parser change can be rerun without a new query, and you respect the provider's terms.

The decision is a claim about this item set under this protocol. It is not a claim that the model understands a domain. Volume 18 will ask you to write that limitation in a form a reviewer can see.

246.3 Derivation

The score file is a function of the item ids, the protocol hash, and the system id. A comparison between two systems is valid when the item ids and the checker match and the protocols are the ones you intended. If one system was allowed three retries and the other was not, the protocol hashes should differ and the comparison is confounded. The harness can still run it, and the report must not call it a matched comparison. This is the ablation rule from Volume 10 applied to evaluation.

246.4 Code

```
import hashlib, json

def protocol_hash(protocol: dict) -> str:
    raw = json.dumps(protocol, sort_keys=True, separators=(",", ":")).encode()
    return hashlib.sha256(raw).hexdigest()[:16]
```

246.5 Experiment

Hypothesis. Changing the temperature in the protocol changes the hash. Reordering JSON keys does not. A simulated system that always returns the gold answer scores perfectly and is rejected as a serious system by a note that also runs a baseline with a known error rate.

Protocol. Hash two equivalent dicts and one temperature change. Score a perfect stub and a baseline that misses one slice.

Metric. Hash equality, and slice metrics.

Decision rule. Key order must not affect the hash. The perfect stub must not be the only system in the report. If the baseline is absent, the harness run is incomplete.

246.6 Project

Deliverable. The harness, an item file with at least two slices, two systems, a contamination cell, a protocol hash, and a short decision.

Constraints. Raw outputs are saved. The checker is versioned. No test-set tuning presented as a final number.

Rubric. A second person can rerun the scorer from the saved outputs without calling the model again.

246.7 Depth tracks

Beginner. Hash two protocols and explain a mismatch.

University. Complete the harness on synthetic items.

Research. Add a shift slice whose items use a new format. Show that overall accuracy can hide a collapse on that slice, using the mass or the slice rate from Volume 07.

246.8 Failure modes

- A harness that only stores the final accuracy.
- A protocol that lives in an untitled chat rather than in the hashed file.
- A claim about "the model" when only one prompt was tried.

246.9 See also

Volume 18 for writing this harness up as a research note. Volume 16 if the items can harm people when mishandled.

Part XVIII

Volume 18. Research practice

247 Volume 18. Research practice

248 Volume 18. Research practice

Stage. Research

Audience. A reader who has finished the projects they cite. This volume is about making a claim that someone else can check.

Question this volume answers. How do you read, reproduce, and write a result without laundering a guess into a finding?

This volume is a reference, not a short tour. Each chapter opens with a card you can look up later, then gives the theory, a reference implementation, an experiment with a decision rule, and a project with a rubric. Read in order the first time. After that, enter through the reference card of the chapter that matches the problem in front of you.

248.1 How to use the four layers

Layer	What you should be able to do when you finish it
Theory	State the definition, the assumptions, and the result, including what the result does not say.
Code	Implement the core object in a few dozen lines and check it against a known case.
Experiment	Change one factor, record a metric with uncertainty, and say what outcome would change your mind.
Project	Produce an artifact someone else can rerun: code, figure, table, and a one-page lab note.

248.2 Depth tracks

Every chapter ends with three tracks.

- **Beginner.** Do the reference card, the code, and the smallest experiment. Skip optional proofs.
- **University.** Do the derivation, the full experiment, and the project rubric.

- **Research.** Read the failure modes, change an assumption, and write what would have to be true for the method to fail on a new distribution.

248.3 Chapters

No.	Chapter	File
01	Reading a paper into this reference	`vol18/01_reading.qmd`
02	Reproducing and revising a result	`vol18/02_reproducibility.qmd`
03	Writing a research note	`vol18/03_writing.qmd`
04	A map of open problems	`vol18/04_open_problems.qmd`
05	Upstream books and how to use them	`vol18/05_upstream.qmd`
06	Capstone: a checkable study	`vol18/06_capstone.qmd`

248.4 Lab note

Keep one markdown file per experiment, `notes/YYYY-MM-DD-<slug>.md`, with the hypothesis, the command you ran, the metric, and the decision. The series is a reference only if the experiments you run are attached to it.

249 Reading a paper into this reference

250 Reading a paper into this reference

Reference question. What do you extract from a paper so it becomes a usable, skeptical reference note?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

250.1 Reference card

Item	Statement
Claim	One sentence with a quantity, a population, and a comparison.
Assumptions	What must be true for the claim to transfer, in the language of earlier volumes.
Protocol	Data split, metric, baselines, compute, and selection of the reported number.
Threats	The two or three ways the claim would fail. Leakage, shift, and a weak baseline are common.
Place	Which chapter of this series the method extends, and which project of yours would change if the claim is true.

250.2 Theory

Reading for a reference is not summarizing the abstract. The abstract is the authors' framing. Your note restates the estimand. A paper that says a method "works" has not named a risk. Translate it into a metric, a split, and a baseline. If you cannot, the paper is not yet usable, and the note says that.

Place the method in the map of this series. A new optimizer belongs next to Volume 10's ablation rules. A new robustness paper belongs next to a threat model. This prevents a technique from floating free of the assumptions that make it meaningful.

Write the threats before you write the praise. A threat is specific: the test set shares users with the training set, the baseline was not tuned, the metric is accuracy at a 99 percent base rate. "More research is needed" is not a threat.

250.3 Derivation

A paper reports $\hat{\Delta}$, an estimated gap. Your note should say which expectation Δ is supposed to be, the unit of randomization, and whether $\hat{\Delta}$ was chosen among many gaps. If the paper's table has ten metrics and the abstract quotes one, the selection is part of the estimator. The note records the selection. Recomputing a confidence interval as if the metric were predeclared would be a new, false precision. Volume 05's peeking warning is the formal version.

250.4 Code

```
def paper_note(claim, estimand, baseline, threat):
    for part in (claim, estimand, baseline, threat):
        if not str(part).strip():
            raise ValueError("paper notes need a claim, estimand, baseline, and threat")
    return {"claim": claim, "estimand": estimand, "baseline": baseline, "threat": threat}
```

250.5 Experiment

Hypothesis. A note that omits the baseline fails the helper, and a filled note for a hypothetical accuracy improvement of 2 points on an unbalanced test names a threat a Volume 07 reader would recognize.

Protocol. Reject an empty baseline. Accept a note whose threat is the base rate or a shared-entity split.

Metric. Whether the helper enforces the fields, and whether the threat is specific.

Decision rule. A threat that only says "bias" is incomplete. Require a mechanism. This is a writing standard you can apply to the next paper you read.

250.6 Project

Deliverable. Two paper notes in this format, on papers or technical reports you can access, each placed in a volume of this series. One threat per note must be something you could check if you had the code, even if you do not run it yet.

Constraints. Do not invent experimental details. If the paper does not state the split, the note says "split not stated."

Rubric. A classmate can tell what would change their project if the claim is true, and what would make the claim inapplicable.

250.7 Depth tracks

Beginner. Fill one note for the least-squares project in Volume 01, as if it were a paper, and list a real threat.

University. Write the two external paper notes.

Research. Add a what-I-would-rerun-first line: the smallest experiment that could falsify the claim in your setting.

250.8 Failure modes

- A summary that copies the abstract's adjectives.
- A threat list that could be attached to every paper unchanged.
- Placing a method in the series without naming the assumption it inherits.

250.9 See also

The reproducibility chapter in this volume. Volume 07 if the paper's number chose the metric after the fact.

251 Reproducing and revising a result

252 Reproducing and revising a result

Reference question. What counts as a reproduction, and what do you owe a negative result?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

252.1 Reference card

Item	Statement
Reproduction	The same claim, the same protocol, within a tolerance, on data you are allowed to use.
Replication	The same claim under a justified change: new sample, new seed, new implementation.
Negative	A failed reproduction is a result if the protocol was followed. It is not an embarrassment to hide.
Tolerance	Predeclare how close is close enough. Do not shrink the tolerance after you see the gap.
Credit	The original claim keeps its citation. Your note adds what happened when you ran it.

252.2 Theory

Reproduction is how a reference stays honest. If you cannot rerun a result that your later chapter depends on, the later chapter depends on a rumor. A small rerun with a stated tolerance is enough for a teaching reference. A failed rerun should stay in the note, with the difference in environment you could identify.

Change one thing when you revise. A new dataset and a new metric and a new architecture is a new project, not a reproduction that "didn't work." Volume 10's ablation discipline is the research discipline.

Negative results need the same artifacts as positive ones: config, metric, and the known-case test that shows your harness works. A buggy rerun is not a negative result about the claim. The debug checklist has to pass on a control problem before the negative result counts.

252.3 Derivation

Let $\hat{\theta}$ be the published number and $\tilde{\theta}$ yours. You predeclare a tolerance τ in the units of $\hat{\theta}$. The reproduction succeeds if $|\tilde{\theta} - \hat{\theta}| \le \tau$ and the protocol differences are empty or listed as irrelevant. If you list a protocol difference that could move the metric by more than τ, you do not have a reproduction. You have a confounded comparison. Record it as such. The decision to adopt the method then rests on your protocol, not on the paper's number.

252.4 Code

```
def reproduction_status(published, ours, tol, protocol_differences):
    gap = abs(float(published) - float(ours))
    if protocol_differences:
        return {"status": "not_a_clean_reproduction", "gap": gap, "differences": list(prot
    status = "reproduced" if gap <= tol else "not_reproduced"
    return {"status": status, "gap": gap, "tol": tol}
```

252.5 Experiment

Hypothesis. A gap inside the tolerance with no protocol differences is marked reproduced. The same gap with a different split is marked not clean. A gap above the tolerance is not reproduced, and lowering the tolerance after the fact is a protocol change you can represent by calling the function twice.

Protocol. Published value 0.80, yours 0.83, tolerance 0.05 and then 0.01.

Metric. The status field.

Decision rule. The statuses must match the rule. A lab note that says "roughly reproduced" when the function says not reproduced has to quote the gap and the tolerance instead of the adverb.

252.6 Project

Deliverable. A reproduction or a negative-result note for one small project from this series: rerun your own least-squares or training experiment from the folder alone, with a tolerance, after a week or on a second seed. If you change a knob, list it.

Constraints. The known-case test for the harness passes. A failed known case means you do not interpret the scientific gap.

Rubric. The status word matches `reproduction_status`. The folder is enough for someone else to rerun.

252.7 Depth tracks

Beginner. Call the helper on a successful and a failed toy pair.

University. Rerun one of your own experiment folders.

Research. Write what you would publish if a third party failed to reproduce your result: the log you would ask for, and the claim you would withdraw if the known-case test passed on their side.

252.8 Failure modes

- Changing seeds until the number matches.
- A negative result from a harness that cannot fit the known case.
- Citing a paper as reproduced because you reimplemented something inspired by it.

252.9 See also

Volume 15 for production reruns. Volume 03 for the folder layout that makes this chapter possible.

253 Writing a research note

254 Writing a research note

Reference question. What is the smallest document that carries a claim, the evidence, and the limits?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

254.1 Reference card

Item	Statement
Question	The decision or estimand, before the method.
Method	Enough to rerun, including the baseline and the selection rule.
Result	The number, the uncertainty, the n, and the comparison.
Limit	The population and the threat model you did not cover.
Claim strength	A verb that matches the evidence: estimates, suggests, or does not show.

254.2 Theory

A research note in this series is a long-form reference entry with a single claim. It is not a short textbook chapter and not a pitch. The reader should be able to stop at the result table and know the comparison, then read the method to see whether they believe it.

Order the document by the logic of the argument, not by the order you suffered in. The question and the predeclared metric come first. The method follows. Exploratory plots are labeled exploratory. A surprise you found after looking is allowed in a section called exploratory. It is not allowed to replace the predeclared result in the first paragraph.

Verbs are part of the science. "Causes" requires a design that identifies a causal effect. "Predicts" requires a test set that matches the prediction claim. "Correlates" is available when that is all you measured. Volume 16's audit language is the right tone for limits. Do not end with a limitation paragraph that withdraws the claim you just overstated. Match them.

254.3 Derivation

Let the predeclared comparison be Δ and the exploratory comparisons be Δ_j for $j = 1, \dots, m$. The note's lead result is $\hat{\Delta}$ with its interval. The exploratory section can show $\hat{\Delta}_j$ without a family-wise guarantee. If you promote one $\hat{\Delta}_j$ into the lead because it was significant, the lead is false. A revision of the note after a failed Δ is honest when it says the predeclared comparison was null and the new question is future work, or when it labels the new lead as selected from m looks.

254.4 Code

```
def lead_sentence(estimand, comparison, n, limit):
    text = (
        f"On n={n}, we estimate {estimand} for {comparison}. "
        f"This does not establish the result beyond {limit}."
    )
    if n <= 0 or not limit.strip():
        raise ValueError("a lead sentence needs n and a limit")
    return text
```

254.5 Experiment

Hypothesis. A lead sentence that includes a limit is accepted, and a sentence you draft with the word "proves" can be rewritten into the helper's pattern without losing the number.

Protocol. Write one overclaimed sentence about a small experiment you actually ran, then rewrite it with `lead_sentence` or the same structure.

Metric. Whether the rewritten sentence names n, the comparison, and a limit.

Decision rule. If the rewritten sentence is still broader than the experiment, the limit clause is decorative. Tighten it. This is a writing experiment with a visible artifact.

254.6 Project

Deliverable. A research note of at most a few pages on one project from this series: question, predeclared metric, method, result, exploratory section if needed, and limits. Use the lead-sentence discipline.

Constraints. The note links the experiment folder. It does not introduce a metric that was not computed.

Rubric. A reader who disagrees with you can point to the comparison and the limit, not to a mood.

254.7 Depth tracks

Beginner. Rewrite one overclaimed sentence about the constant baseline.

University. Write the full note on a project you completed.

Research. Add an explicit not-claimed list: causal effect, fairness, privacy, or robustness, unless you actually measured them. Refer to the volume that would be required.

254.8 Failure modes

- A lead result chosen after seeing several metrics.
- A method section that omits the baseline.
- Limits that contradict the first paragraph.

254.9 See also

Volume 03 for figures that support a sentence. Volume 16 for limits that are about people.

255 A map of open problems

256 A map of open problems

Reference question. Which questions in this series are still open in a way that a project could bite on?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

256.1 Reference card

Item	Statement
Open	A question with a measurable gap, not a topic label.
Bit	The smallest experiment that would reduce the gap.
Dependence	Which earlier result the question needs, and which failure of that result would dissolve it.
Non-goal	A question that is too broad to finish, recorded as a program rather than a project.
Living	This map should be edited when you close a question. A static list of buzzwords is not a map.

256.2 Theory

An open problem in a reference is a hole you can point to. “Understand intelligence” is not a hole you can point to. “Measure whether this representation helps at a fixed label budget on this shift” is a hole. The map in this chapter is a template you fill with your own holes after the projects you have run. The examples below are illustrations of the grain, not an assignment of the field’s priorities.

Each entry names the volume that already constrains it. A robustness question that does not name a threat model is sent back to Volume 16. A systems question that does not name a binding resource is sent back to Volume 15. This is how the series stays one reference rather than a pile of courses.

When you close a problem, leave the entry and add the result. Deleting the question erases the path. A reference reader wants to see what was answered.

256.3 Derivation

Treat the map as a set of estimands that lack an accepted measurement in your notes. For an entry q, a bite is an experiment whose outcome would change the status of q from open to answered or to ill-posed. If no outcome would change the status, q is a topic, not a problem. Write the outcome that would change your mind before you spend the data. This is the same predeclaration rule as everywhere else in the series, applied to the research agenda.

256.4 Code

```
def problem(question, bite, volume, dissolve_if):
    if not all(str(x).strip() for x in (question, bite, volume, dissolve_if)):
        raise ValueError("a problem needs a bite and a dissolution condition")
    return {
        "question": question,
        "bite": bite,
        "volume": volume,
        "dissolve_if": dissolve_if,
    }
```

256.5 Experiment

Hypothesis. An entry without a bite is rejected, and a concrete entry about evaluation under shift can be dissolved if the conditional $P(Y \mid X)$ changed and you only reweighted X.

Protocol. Try to construct `problem` with an empty bite. Then write one full problem whose `dissolve_if` refers to a violated assumption from Volume 16.

Metric. Whether the helper enforces the bite, and whether the dissolution condition is an assumption rather than a mood.

Decision rule. Keep the problem only if a negative bite result would teach you something named in `dissolve_if` or in the question. Otherwise the experiment cannot fail informatively.

256.6 Project

Deliverable. Five open problems in your own words, each with a bite you could run with the skills in this series, and one non-goal you explicitly refuse to list as a project.

Constraints. At least one problem must be a negative-result-friendly measurement, such as a baseline that might win. At least one must name an ethical or safety constraint that would stop the bite.

Rubric. Another reader can tell what data the bite needs and what result would close the question.

256.7 Depth tracks

Beginner. Turn one failed experiment of yours into a problem entry with a bite.

University. Write the five entries across different volumes.

Research. Add a dependence between two entries: a representation-learning question that is ill-posed until an evaluation question is answered. Order them.

256.8 Failure modes

- A list of model names.
- A bite that is "train a bigger model" with no metric.
- A dissolution condition that cannot actually occur.

256.9 See also

Every earlier volume. The upstream chapter for problems that are already treated in a book you should read rather than rederive.

257 Upstream books and how to use them

258 Upstream books and how to use them

Reference question. Which external books should you read beside this series, and what are you not allowed to do with their text?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

258.1 Reference card

Item	Statement
This series	Original notes. They do not contain those books' chapters.
CS249r	Machine Learning Systems, four volumes. Textbook text is CC BY-NC-ND 4.0. Read it, share it unmodified, do not fold edited text into this book.
D2L	Dive into Deep Learning. Text is CC BY-SA 4.0. You may adapt it if you keep attribution and the share-alike duty. This series does not copy it.
Think books	Think Python, Think Stats, Think Bayes are CC BY-NC-SA. Non-commercial adaptations require share-alike. We point at them for foundations.
Hugging Face course	Apache 2.0. You may reuse it with attribution and a notice of changes. The language volume here is original and smaller.
Not open	A free PDF is not permission to republish. Famous textbooks you can read online are not therefore upstream you can integrate.

258.2 Theory

The series was built so you can learn the path without pasting someone else's book into your repository. The external books are better than these notes on their own ground: D2L for a

broad implemented deep-learning course, the Think books for computational foundations, the Hugging Face course for a library-centered language pipeline, and CS249r for systems from the chip to physical AI. Read them at the point in the path where the volume guide sends you.

Integration has a legal meaning. Unmodified CS249r volumes can sit beside your notes as a separate work with their license intact. Editing them into your chapters is a derivative the ND license does not grant. D2L and the Think books can be adapted only if you accept their share-alike terms for the adapted material. The clean design, and the one this repository uses, is citation and a reading note in your own words, plus your own experiments.

A reading note uses the paper-note format from this volume. It does not reproduce long passages. If a proof is short and you need it, rederive it. If you cannot rederive it, say you are taking the result as a citation and point to the source. That is more honest than a close paraphrase.

258.3 Derivation

Suppose you adapt a CC BY-SA chapter and mix it into a note you want to keep under a different license. The adapted portion remains under BY-SA, and if the mixture is one work rather than a separable collection, the share-alike duty can extend further than you intended. The practical rule used here is separation: external text stays in the external repository; your notes stay here; the link is a citation. No derivation in this chapter weakens an upstream license. If you need a combined work, read the license text before you merge repositories.

258.4 Code

```
UPSTREAM = [
    {"id": "cs249r", "use": "read unmodified", "license": "CC BY-NC-ND 4.0 textbook text"}
    {"id": "d2l", "use": "read; adapt only with BY-SA obligations", "license": "CC BY-SA 4
    {"id": "think", "use": "read; non-commercial adaptation is share-alike", "license": "C
    {"id": "hf-course", "use": "reuse with attribution and change notices", "license": "Ap
]
```

258.5 Experiment

Hypothesis. You can state, without opening an upstream PDF and copying, which book you would read for a systems question and which you would read for a first deep-learning implementation, and you can name the license constraint that stops you from pasting either into this repository.

Protocol. Answer in a note of ten lines, then check your answer against the upstream page of this series.

Metric. Whether the systems pointer is CS249r, the implementation pointer can be D2L, and both license constraints are present.

Decision rule. If the note says "I'll merge the PDFs and fix the wording later," it fails. Separation is the requirement.

258.6 Project

Deliverable. A personal reading order: which upstream book accompanies which volume you have already studied, plus one paper note in your own words on a chapter you actually read. No pasted paragraphs.

Constraints. The note includes the license and the access date or edition.

Rubric. A reader can follow your path without receiving a copied chapter they would not have the right to redistribute.

258.7 Depth tracks

Beginner. List the four upstream works and their licenses from memory, then check.

University. Write the reading order tied to volumes you have done.

Research. Pick one result you are relying on and rederive it. If you cannot, mark the dependence as a citation with an unreviewed proof, which is a real status, not a failure of character.

258.8 Failure modes

- Pasting ND text because the use is non-commercial.
- A paraphrase that tracks the source sentence by sentence.
- Treating a free download as a license.

258.9 See also

The upstream page at the start of this book, which repeats the pointers without the legal reasoning, and Volume 01 for how a citation should enter your notes.

259 Capstone: a checkable study

260 Capstone: a checkable study

Reference question. Can you produce one study that uses the series as a reference and could be checked by someone who does not have your chat history?

Use the card to look the result up. Use the rest of the chapter when you need to derive it, implement it, or decide whether the result still applies.

260.1 Reference card

Item	Statement
Question	One estimand, predeclared.
Path	The volumes you actually used, not every volume in the table of contents.
Baseline	The simplest method that could answer the question.
Artifacts	An experiment folder, a model or audit card if a model is deployed in the study, and a research note.
Stop	A result that includes the possibility that the baseline won or that the audit says not to ship.

260.2 Theory

The capstone is the series in one folder. It is a reference use, not a tour. You pick a question that needs theory, code, an experiment, and a written limit. You do not demonstrate every algorithm. A study that tries to show every volume usually shows none of them carefully.

The checker's standard is unkind and fair. They have the folder and the note. They can rerun the known case. They can see the split. They can find the baseline. They can tell what you are not claiming. If the answer is no, the missing piece is the next edit, not a longer introduction.

Choose a question from your open-problem map so the capstone closes a bite you already defined. Changing the question after the result is the last peeking temptation in the book. Resist it in the note's first paragraph, where the predeclared question is quoted from the config.

260.3 Derivation

The study estimates a gap Δ between a chosen method and a baseline under a named distribution, with a tolerance for reproduction and a limit on interpretation. Everything else is appendix. A successful capstone is one in which $\hat{\Delta}$ is computed as specified, the known case passes, and the prose matches the table. Whether $\hat{\Delta}$ has the sign you hoped is not the success criterion. A reference that only contains hoped-for signs is advertising.

260.4 Code

```
def capstone_manifest(question, baseline, folder, known_case_passed):
    if not known_case_passed:
        raise ValueError("do not write the claim until the known case passes")
    if not folder or not baseline or not question:
        raise ValueError("question, baseline, and folder are required")
    return {"question": question, "baseline": baseline, "folder": folder}
```

260.5 Experiment

Hypothesis. The manifest helper blocks a claim without a known case, and a completed manifest points at a real folder you produced earlier in the series.

Protocol. Call it once with `known_case_passed=False` and once with a real folder path after you rerun that folder's check.

Metric. The exception and the written manifest.

Decision rule. No manifest, no capstone claim. A folder that only contains a draft note without metrics is not the folder this helper is for.

260.6 Project

Deliverable. The manifest, the research note, the experiment folder, and a short "path through the series" that cites the specific chapters you used for the split, the baseline, the uncertainty, and the limit.

Constraints. One question. One primary metric. Upstream books are cited, not copied.

Rubric. A checker who starts from the manifest can rerun the known case and can state your claim in one sentence that you would accept.

260.7 Depth tracks

Beginner. Assemble the manifest for the Volume 01 least-squares project, including a limit.

University. Write the capstone on the best project you have, with a fresh known-case rerun.

Research. Add a reproduction status and an audit or safety row if the study recommends a deployment. If it does not recommend one, say so in the lead sentence.

260.8 Failure modes

- A capstone that is a syllabus rather than a measurement.
- A claim whose folder is missing the baseline.
- A pasted chapter from an upstream book filling the method section.

260.9 See also

Volume 01 for the standard of a known case. The rest of the series for the pieces the path paragraph should cite by name.